

Trabajo de Fin de Grado - Curso 2025/ 2026

Métodos de aproximación y aplicaciones

Autor/a: **Pablo Ramón Muro**

Tutor/a: Dionisio F. Yáñez



Grado en Matemáticas

Índice general

1. Introducción **5**
- 1.1. El desafío geométrico: limitaciones de los métodos con malla . . . 5
- 1.2. La alternativa de los métodos sin malla y sus aplicaciones . . . 6
- 1.3. Planteamiento del problema y estructura del trabajo . . . 7

2. Interpolación con RBF global **9**
- 2.1. Puntos de Halton . . . 10
- 2.2. Algunas definiciones y resultados previos al método RBF . . . 10
- 2.3. Interpolación con matrices distancia . . . 15
  - 2.3.1. La invertibilidad de la matriz distancia y otros resultados matriciales . . . 16
  - 2.3.2. Ejemplo práctico de RBF con matrices distancia . . . 22
- 2.4. RBF gaussiano . . . 26
  - 2.4.1. Invertibilidad de la matriz del sistema para RBF gaussiano . . . 26
  - 2.4.2. Distancia de llenado y orden . . . 28
  - 2.4.3. Condicionamiento: amplificación del error y cálculos en Matlab . . . 30
  - 2.4.4. EL RBF gaussiano en la práctica . . . 33
  - 2.4.5. Convergencia del RBF . . . 38

3. Métodos de partición de la unidad **41**
- 3.1. Condiciones sobre el recubrimiento . . . 42
  - 3.1.1. Construcción explícita de un recubrimiento regular . . . 43
- 3.2. Condiciones analíticas sobre las funciones involucradas y espacios nativos . . . 45
  - 3.2.1. Teoría de espacios nativos . . . 45
- 3.3. Convergencia de los MPU . . . 48
  - 3.3.1. Algunos resultados sobre los interpolantes locales del teorema 3.11 . . . 48
  - 3.3.2. Convergencia para el caso $\Phi(x) = ||x||^p$ (p impar) . . . 52
  - 3.3.3. Convergencia para el caso Φ gaussiano . . . 57
- 3.4. Escalabilidad de los MPU . . . 62
  - 3.4.1. Recubrimiento fijo . . . 63
  - 3.4.2. Recubrimiento variable con N_j acotado . . . 64

3.5. Condicionamiento del MPU con parches variables y N_j acotado 64
3.5.1. Caso gaussiano 65
3.5.2. Caso $\Phi(x) = \|x\|^p$ 66

4. Experimentos numéricos **69**
4.1. Construcción del recubrimiento y descripción del método 70
4.1.1. Condiciones sobre el recubrimiento y elección de M 70
4.1.2. Implementación del método 72
4.2. Experimentos numéricos del MPU y RBF global 73
4.2.1. Función de Franke 73
4.2.2. Experimento numérico para un polinomio de grado dos 78

5. Conclusiones **83**

Bibliografía **87**

A. Funciones en $\mathcal{N}_\Phi$, con Φ gaussiana **91**
A.1. Algunos resultados teóricos sobre $\mathcal{N}_\Phi$, con Φ gaussiana 91
A.2. Funciones de los experimentos numéricos 93
A.2.1. Función de Franke 93
A.2.2. Polinomio de grado dos 96

B. k-estabilidad de las particiones de la unidad de Shepard-Wendland **99**

C. Códigos de MATLAB **111**

Capítulo 1

Introducción

El problema de la interpolación y aproximación de datos ha constituido uno de los pilares fundamentales del análisis numérico desde sus orígenes. Clásicamente, la reconstrucción de una función desconocida $f : \Omega \to \mathbb{R}$ a partir de un conjunto discreto de datos u observaciones ha sido abordada mediante el uso de polinomios globales, como las formas de Lagrange o de Hermite [5,23]. Sin embargo, es bien conocido que la interpolación polinómica global de alto grado es susceptible a inestabilidades oscilatorias entre nodos, sobre todo cerca de los bordes del dominio, un comportamiento formalizado a principios del siglo XX por el matemático Carl David Tolmé Runge, y que es conocido como el "fenómeno de Runge" [7].

Con el objetivo de solventar estas inestabilidades, el análisis numérico evolucionó hacia enfoques locales, dando lugar a los métodos de interpolación polinómica a trozos. Estos se caracterizan por usar polinomios locales de bajo grado para evitar oscilaciones y empalmarlos a lo largo de todo el dominio. Algunos ejemplos de estos métodos son Lagrange segmentario, splines o Hermite segmentario [2, 6, 20]. Esta estrategia de fraccionar el dominio en subdominios más pequeños, sentó las bases para herramientas tan potentes como el Método de los Elementos Finitos (FEM, por sus siglas en inglés) [4,24]. No obstante, la transición de dominios unidimensionales a dominios multidimensionales ($\Omega \subset \mathbb{R}^s$ con $s \geq 2$) fue lo que introdujo el desafío topológico de construir una malla estructural con ciertas propiedades de regularidad, que analizamos a continuación.

1.1. El desafío geométrico: limitaciones de los métodos con malla

Los métodos clásicos de aproximación local nombrados anteriormente exigen una partición geométrica estricta del dominio original en subdominios elementales (como triángulos o tetraedros), lo que comúnmente se denomina malla o triangulación. Esta red topológica debe ser tal que los elementos adyacentes compartan caras, aristas o vértices de manera exacta, sin presentar solapamientos ni dejar huecos espaciales.

La rigidez de este requerimiento topológico sobre nuestra malla se hace especialmente crítica cuando se trabaja en dominios con geometrías complejas o irregulares. Una técnica estándar y eficiente para la generación de nodos en un dominio arbitrario Ω consiste en embeber dicho dominio geométrico dentro del hiper-rectángulo más pequeño que lo contenga. En este enfoque, los puntos se generan primero en el hiper-rectángulo coordenado $\prod_{i=1}^{s}[a_i, b_i]$ mediante transformaciones afines de la forma $x_i' = a_i + x_i(b_i - a_i)$, donde x_i pertenece al cubo unidad estándar, procediendo finalmente a descartar todos aquellos puntos que caigan fuera de Ω.

A pesar de la simplicidad algorítmica del método geométrico planteado, su aplicación directa sobre métodos basados en malla resulta inviable. Al descartar los elementos rectangulares o tetraédricos que intersectan el exterior de Ω, la aproximación de la frontera resultante sufre el denominado "efecto de escalera", es decir, una discretización dentada de la frontera de nuestro dominio. Esto produce un mal condicionamiento a la hora de resolver ciertos problemas, como ecuaciones diferenciales con ciertas condiciones sobre la frontera.

Para solventar esta deformidad en la frontera, las técnicas de generación de malla obligan a desplazar los nodos perimetrales para que se ajusten a la curvatura real de $\partial\Omega$. Sin embargo, esta adaptación geométrica fuerza la creación de elementos degenerados, como triángulos o tetraedros severamente aplastados o con ángulos muy agudos. Desde un punto de vista computacional, basar los cálculos en una geometría tan distorsionada empeora la estabilidad del método: los pequeños errores numéricos de redondeo se amplifican drásticamente, reduciendo la precisión de la solución o haciendo que el algoritmo colapse por completo. En dimensiones superiores ($s \geq 3$), implementar métodos basados en malla se vuelve una tarea topológica muy compleja, ya que requiere establecer y mantener la conectividad mediante líneas y caras entre todos los nodos. Este es un obstáculo que los métodos sin malla evitan de forma natural al trabajar únicamente con nodos dispersos [3].

1.2. La alternativa de los métodos sin malla y sus aplicaciones

Es precisamente en la superación de estas limitaciones topológicas donde reside la motivación fundamental de los métodos sin malla, o *meshless methods* en inglés. Esta familia de algoritmos prescinde de la conectividad estructurada entre nodos, pues se fundamenta exclusivamente en relaciones de distancias entre nodos, por lo que la distribución de los puntos no requiere formar un mallado regular del espacio, como el que se exigía para los métodos con malla [25].

Ahora bien, la técnica del hiper-rectángulo delimitador para dominios Ω arbitrarios, descrita anteriormente, se aplica de manera natural a estos métodos sin malla, obteniendo una nube de puntos recortada a la silueta del dominio, tan complejo como sea, sobre la cual aplicar el método. De esta forma, los métodos sin malla garantizan adaptabilidad y suavidad a lo largo de las fronteras curvas de Ω [12], eludiendo por completo los riesgos de mal condicionamiento asociados a la degeneración de elementos [3].

Dentro de este conjunto de métodos destacan distintas herramientas, algunas de las cuales se

usarán en este trabajo, como son la interpolación por Funciones de Base Radial (RBF), el método de Mínimos Cuadrados Móviles (MLS) o los Métodos de Partición de la Unidad (MPU). Estas nos ofrecen aproximaciones de orden elevado y, algunas de ellas, con una alta estabilidad y un coste computacional que crece linealmente con los nodos.

Debido a las ventajas de estos métodos: gran flexibilidad geométrica y escalabilidad algorítmica, los enfoques sin malla han experimentado un auge notable en la literatura científica. Históricamente, una de sus aplicaciones fundamentales ha sido la resolución numérica de Ecuaciones en Derivadas Parciales (EDPs) proporcionando una alternativa robusta a los métodos de elementos finitos clásicos [11]. En la actualidad son muy empleados para aproximar soluciones de EDPs en problemas físicos donde los métodos basados en malla imponen graves cuellos de botella, tales como fronteras móviles o dominios que se deforman fuertemente con el tiempo. Esto ha impulsado su uso generalizado en campos como la dinámica de fluidos computacional (simulaciones mediante hidrodinámica de partículas suavizadas), el procesamiento de geometrías en gráficos 3D, la astrofísica y la geofísica predictiva [13].

Más recientemente, la inteligencia artificial ha abierto nuevas fronteras de investigación. En particular, enfoques punteros como las Redes Neuronales Informadas por la Física (PINNs, por sus siglas en inglés) aprovechan la evaluación de residuos sobre nubes de puntos dispersas, sin requerir una topología subyacente, para inferir y resolver ecuaciones en derivadas parciales complejas, consolidando el paradigma de los métodos sin malla en la vanguardia del cálculo científico moderno [17].

Además de su integración con la inteligencia artificial, los Métodos de Partición de la Unidad (MPU) han adquirido un gran protagonismo en las simulaciones físicas actuales. Recientemente, se ha demostrado que esta técnica es excepcionalmente útil para resolver problemas donde interactúan varios fenómenos físicos a la vez. En estos casos, cada fenómeno se simula utilizando su propia nube de puntos y, al generarse de forma independiente, las coordenadas espaciales de una nube no suelen coincidir con las de la otra. El MPU actúa como un puente matemático: toma los datos de la primera nube y construye una aproximación continua que permite evaluar y transferir esa información de manera exacta a las coordenadas de la segunda. Al aprovechar que el método divide el espacio en "parches"locales independientes, los superordenadores pueden repartir este enorme cálculo y resolver cada zona en paralelo. Este avance ha logrado que el intercambio masivo de datos sea muchísimo más rápido, eliminando uno de los mayores cuellos de botella computacionales en la ingeniería moderna [19].

1.3. Planteamiento del problema y estructura del trabajo

Una vez motivados los métodos sin malla e introducidas algunas de sus aplicaciones, en el caso que nos ocupa, este Trabajo de Fin de Grado se enmarca en el contexto de los métodos numéricos sin malla, mediante los cuales se pretenderá abordar el problema de interpolación de datos dispersos en $\mathbb{R}^s$ con el objetivo de estudiar matemáticamente su fundamentación, formulación, convergencia y estabilidad.

Por tanto, nuestro cometido será aproximar una función f desconocida, con $f : \Omega \to \mathbb{R}$, donde

$\Omega \subseteq \mathbb{R}^s$ es conexo. Para ello, se eligen una serie de puntos de muestreo $x_1, ..., x_N$, con $x_j \in \Omega$, $1 \leq j \leq N$, y denotaremos $\chi \coloneqq \{x_j : 1 \leq j \leq N\}$. Notemos que los puntos elegidos no tienen por qué formar un mallado uniforme del dominio. A continuación, para cada uno de esos puntos x_j, se obtiene un valor de medida $y_j \in \mathbb{R}$, que cumple que $f(x_j) = y_j$, $1 \leq j \leq N$. Finalmente, construimos nuestra función interpoladora P_f, que pretende aproximar a f, y que se caracteriza por $P_f(x_j) = y_j$, $1 \leq j \leq N$. Este proceso es el que se conoce como interpolación de datos dispersos en $\mathbb{R}^s$ y, en él, asumiremos que nuestras funciones f y P_f son, al menos, continuas.

Con este objetivo en mente, a lo largo de este Trabajo de Fin de Grado se abordan dos métodos sin malla: la interpolación por RBF global y los MPU, que son locales. El primero de ellos, que se estudia en el capítulo 2, nos proporciona, como veremos, una buena tasa de convergencia, pero suele sufrir problemas de condicionamiento y una mala escalabilidad computacional. Para resolver estos dos problemas sin empeorar el orden de aproximación, se desarrollan, a lo largo del capítulo 3, los MPU. En el apartado de experimentos numéricos 4, se pone a prueba la teoría estudiada a lo largo de los dos capítulos anteriores para estos dos métodos y, finalmente, el capítulo de conclusiones 5 recoge los principales resultados extraídos a lo largo de este trabajo.

Capítulo 2

Interpolación con RBF global

En este capítulo, en el que utilizaremos la notación y resultados de [7], realizaremos nuestra primera búsqueda de un interpolante basado en un método sin malla, más concretamente un método global sin malla: la interpolación mediante Funciones de Base Radial (RBF). Sin embargo, antes de abordar el método, deberemos contextualizar e introducir algunos resultados y definiciones que resultarán claves en nuestro desarrollo.

En primer lugar, asumiremos que

$$P_f(x) = \sum_{k=1}^{N} c_k B_k(x), \forall x \in \Omega,$$

donde $\{B_1, ..., B_N\}$ es la base de funciones de nuestro problema que, como veremos más adelante, pueden depender de los elementos de χ. Así, sean

$$c = (c_1, ..., c_N)^T,\ A \in \mathbb{R}^{N\times N},\ \text{con } A_{jk} = B_k(x_j),\ \text{e } y = (y_1, \ldots, y_N)^T,$$

entonces resolviendo el sistema lineal $Ac = y$, obtendremos los coeficientes c_k, con $1 \leq k \leq N$. Se dice que el problema está bien puesto, es decir, que tiene solución única, si y solo si A es invertible.

Una vez tenemos el contexto general, iremos planteando cada uno de los problemas que van surgiendo a la hora de abordar el problema de interpolación por RBF.

En primer lugar, trataremos el problema de generar puntos "aleatorios" dispersos, para lo que trabajaremos siempre, por simplicidad, en $\Omega = [0, 1]^s$ ya que, como vimos en la introducción, estos puntos son fácilmente reescalables en el hiper-rectángulo más pequeño que contiene a un dominio arbitrario, eliminándose luego los que cayeran fuera de dicho dominio.

Para generar estos puntos aleatorios, recurriremos a los denominados puntos de Halton.

2.1. Puntos de Halton

En esta sección trataremos los puntos de Halton, que serán nuestros nodos aleatorios en $[0,1]^s$ a lo largo de todo este trabajo, y cuya caracterización e introducción original se remontan a los trabajos de Halton [10] y Wong *et al.* [26]. Sin embargo, un resumen de sus propiedades y su construcción a partir de las secuencias de van der Corput, en la forma aquí empleada, puede consultarse en [7, Apéndice A].

El siguiente lema nos permitirá definir las sucesiones de van der Corput, que resultan esenciales en la definición de los puntos de Halton.

Lema 2.1. Sabemos que, sea p un número primo y $n \in \mathbb{N}$, podemos escribir $n = \sum_{i=0}^{k} a_i p^i$ de forma única, para algún entero $k \geq 0$, con $0 \leq a_i < p$ y $a_i \in \mathbb{N} \cup \{0\}$. Ahora, si definimos

$$h_p(n) = \sum_{i=0}^{k} \frac{a_i}{p^{i+1}},$$

se tiene que $h_p(n) < 1$.

Demostración.

$$h_p(n) = \sum_{i=0}^{k} \frac{a_i}{p^{i+1}} = \frac{a_0}{p} + \frac{a_1}{p^2} + \cdots + \frac{a_k}{p^{k+1}} = \frac{a_0 p^k + a_1 p^{k-1} + \cdots + a_k}{p^{k+1}} \leq \frac{(p-1)p^k + (p-1)p^{k-1} + \ldots}{p^{k+1}} +$$

$$+ \frac{(p-1)p + (p-1)}{p^{k+1}} = \frac{p-1}{p^{k+1}} \sum_{i=0}^{k} p^i = \frac{1-p^{k+1}}{1-p} \cdot \frac{p-1}{p^{k+1}} = \frac{p^{k+1}-1}{p^{k+1}} < 1.$$

□

Definición 2.2. A partir de la llamada sucesión de van der Corput $h_{p,N} := \{h_p(n) : n = 0, 1, \ldots, N\}$, construimos el conjunto de $N+1$ puntos de Halton en $(0,1)^s \cup \{0\}$:

$$H_{s,N} := \{(h_{p_1}(n), \ldots, h_{p_s}(n)) : n = 0, 1, \ldots, N\},$$

con $p_1, \ldots, p_s$ primos, normalmente distintos.

Una vez definidos, en base a [7, Apéndice A], se sabe que, para s pequeñas ($s \leq 6$) y para N no excesivamente grandes, la sucesión multidimensional de Halton rápidamente llena el cubo unidad con puntos bien distribuidos.

Una vez establecidos cómo generamos los puntos dispersos en nuestro dominio, pasamos a tratar una serie de resultados que nos permitirán abordar el problema de la base de funciones en cualquier dimensión.

2.2. Algunas definiciones y resultados previos al método RBF

Comenzaremos esta sección con una definición y una serie de resultados que nos ayudarán a entender mejor algunas propiedades que tiene la base de funciones que forman nuestro interpolante, tal y como explicamos al inicio de este capítulo.

Definición 2.3. Sea $\mathcal{B} \subset \mathcal{C}(\Omega)$ un espacio lineal de funciones de dimensión N (es un espacio vectorial), con base $\{B_1, ..., B_N\}$, entonces $\mathcal{B}$ se dice espacio de Haar si

$$\det(A) \neq 0, \forall \{x_1, ..., x_N\} \subset \Omega : x_i \neq x_j, 1 \leq i < j \leq N,$$

donde $A \in \mathbb{R}^{N \times N}$, con $A_{jk} = B_k(x_j)$.
Nota 2.4. Si $\mathcal{B}$ es espacio de Haar, entonces el problema de interpolación de datos dispersos usando una base de este espacio estará bien puesto.

Esta definición nos lleva a plantearnos si ser espacio de Haar es independiente de la base elegida. Veamos probando la siguiente proposición que es así.
Proposición 2.5 (Invarianza de la condición de Haar)**.** Sea $\mathcal{B} \subset \mathcal{C}(\Omega)$ un espacio de funciones de dimensión N. La propiedad de ser un espacio de Haar es independiente de la base elegida.

Demostración. Sean $\{B_1, \ldots, B_N\}$ y $\{\tilde{B}_1, \ldots, \tilde{B}_N\}$ dos bases cualesquiera del espacio $\mathcal{B}$, existe una matriz de cambio de base $P \in \mathbb{R}^{N \times N}$ invertible tal que:

$$\tilde{B}_k = \sum_{m=1}^{N} P_{mk} B_m, \quad \forall k = 1, \ldots, N.$$

Consideremos ahora un conjunto de N puntos distintos $\{x_1, \ldots, x_N\} \subset \Omega$. Definimos las matrices A y $\tilde{A}$ asociadas a cada base como $A_{jk} = B_k(x_j)$ y $\tilde{A}_{jk} = \tilde{B}_k(x_j)$ respectivamente. La relación entre ellas es la siguiente:

$$\tilde{A}_{jk} = \tilde{B}_k(x_j) = \sum_{m=1}^{N} P_{mk} B_m(x_j) = \sum_{m=1}^{N} P_{mk} A_{jm} = \sum_{m=1}^{N} A_{jm} P_{mk}.$$

Por tanto, podemos escribir:

$$\tilde{A} = AP.$$

Aplicando la propiedad del determinante para el producto de matrices, se tiene que:

$$\det(\tilde{A}) = \det(A) \det(P).$$

Dado que P es invertible, se sigue que $\det(\tilde{A}) \neq 0$ si y solo si $\det(A) \neq 0$.
Por tanto, se concluye que ser espacio de Haar no depende de la base escogida. □

Un ejemplo de espacio Haar para $s = 1$ es el espacio de polinomios de grado $N - 1$, tal y como vemos en la siguiente proposición.
Proposición 2.6. El espacio de polinomios de grado $N-1$ sobre $\mathbb{R}$ es un espacio de Haar de dimensión N.

Demostración. Sea $\mathcal{B}$ el espacio de polinomios de grado $N - 1$, elegimos la base $\{1, x, ..., x^{N-1}\}$. Así, para un conjunto de puntos $\{x_1, ..., x_N\} \subset \Omega : x_i \neq x_j, 1 \leq i < j \leq N$, A es la matriz de Vandermonde, con $A_{ij} = x_i^{j-1}$ y se sabe que $\det(A) = \prod_{1 \leq i < j \leq N} (x_j - x_i) \neq 0$.

Por tanto, concluimos que $\mathcal{B}$ es espacio de Haar. □

Veamos qué ocurre para $s \geq 2$, para lo que necesitaremos del siguiente teorema, que nos ayudará a intuir cómo debemos escoger nuestra base de funciones.

Teorema 2.7 (Teorema de Mairhuber-Curtis)**.** Si $\Omega \subseteq \mathbb{R}^s$, con $s \geq 2$ y con $\text{int}(\Omega) \neq \varnothing$, entonces no existen espacios de Haar de funciones continuas, salvo los de dimensión 1.

Demostración. Procederemos por R.A. Supongamos $s \geq 2$, $\mathcal{B}$ espacio de Haar con base $\{B_1, \ldots, B_N\}$, $N \geq 2$.

Como $\text{int}(\Omega) \neq \varnothing$, sea $x \in \text{int}(\Omega)$, sabemos que $\exists \varepsilon > 0$ tal que $\overline{B}(x, \varepsilon) \subset \Omega$, con $x = (x^1, x^2, \ldots, x^s)$.

Elegimos $x_1, \ldots, x_N$ puntos, que son una partición equiespaciada del eje X en la $\overline{B}(x, \varepsilon)$.

Es decir, los puntos anteriores se eligen de la forma $x_i = (x_i^1, x^2, \ldots, x^s), \forall i \in \{1, \ldots, N\}$, con $x^1 - \varepsilon = x_1^1 < x_2^1 < \ldots < x_{N-1}^1 < x_N^1 = x^1 + \varepsilon$ puntos equiespaciados.

Definimos $M := \frac{x_1+x_2}{2} = \left(\frac{x_1^1+x_2^1}{2}, x^2, \ldots, x^s\right)$, $r := \frac{\|x_1-x_2\|_2}{2} = \left|\frac{x_1^1-x_2^1}{2}\right| = \frac{x_2^1-x_1^1}{2} = \frac{\varepsilon}{N-1}$, $\gamma_k : [0,1] \to \mathbb{R}^s, \gamma_k \in C([0,1]), \forall k \in \{1, \ldots, N\}$, donde

$$\begin{cases} \gamma_1(t) = M - r(\cos(\pi t), \sin(\pi t), 0, \ldots, 0), \\ \gamma_2(t) = M + r(\cos(\pi t), \sin(\pi t), 0, \ldots, 0), \\ \gamma_k(t) = x_k \quad , \quad 3 \leq k \leq N. \end{cases}$$

De hecho, $\gamma_k(t) \in \overline{B}(x, \varepsilon) \subset \Omega, \forall t \in [0,1], \forall k \in \{1, \ldots, N\}$, que es obvio para $k \geq 3$. Ahora, sea $1 \leq k \leq 2$, se tiene que $\|\gamma_k(t) - x\|_2 \leq \|\gamma_k(t) - M\|_2 + \|M - x\|_2 = r + \left|\frac{x_1^1+x_2^1}{2} - x^1\right| = r + \left|\frac{-x_1^1+x_2^1}{2} - \varepsilon\right| = r + (N-2)r = (N-1)r = \varepsilon$.

Además, se observa que, tanto γ_1 como γ_2 cumplen $\gamma_1(t) \neq x_i \neq \gamma_2(t) \neq \gamma_1(t), \forall t \in [0,1], 3 \leq i \leq N$ y $\gamma_k(t) \in \Omega, \forall t \in [0,1], 1 \leq k \leq N$. Por otro lado, $\gamma_1(0) = x_1$, $\gamma_1(1) = x_2$, $\gamma_2(0) = x_2$, $\gamma_2(1) = x_1$.

Así, consideramos $\{\gamma_1(t), \ldots, \gamma_N(t)\} \subset \Omega$ y $A(t)$ tal que $A_{jk}(t) := B_k(\gamma_j(t)), \forall t \in [0,1], 1 \leq j \leq N, 1 \leq k \leq N$. Donde $0 \neq \det(A(0)) = -\det(A(1))$, porque hemos cambiado dos filas y sabemos que es distinto de cero por hipótesis (hemos asumido $\mathcal{B}$ espacio de Haar).

Observemos que $\phi(t) := \det(A(t))$ es una función continua en $[0,1]$ (es continua porque es una combinación lineal de $B_k(\gamma_j(t))$, que son continuas en $[0,1]$, por ser composición de continuas). Como $\phi(0) \cdot \phi(1) < 0$, por el Teorema de Bolzano $\exists t_1 \in (0,1)$ tal que $\phi(t_1) = 0$.

Pero $A(t_1)$ cumple que $A_{jk}(t_1) := B_k(\gamma_j(t_1))$, con $\{\gamma_1(t_1), \ldots, \gamma_N(t_1)\} \subset \Omega$, y con $\gamma_i(t_1) \neq \gamma_j(t_1), 1 \leq i < j \leq N$, por lo que $\phi(t_1) = \det(A(t_1)) \neq 0$, por ser $\mathcal{B}$ espacio de Haar.

Así, hemos llegado a una contradicción que viene de suponer $\mathcal{B}$ espacio de Haar con base $\{B_1, \ldots, B_N\}$,

$N \geq 2$ y $s \geq 2$.

□

Analizando las consecuencias de lo anterior, por la proposición 2.6, se sabe que, para $s = 1$, con $x_1, ..., x_N$ fijos, P_f es un polinomio de grado $N-1$. No obstante, para N grande, los polinomios oscilan demasiado y para solventarlo, se suelen buscar otras soluciones. Algunas de ellas son que la función interpoladora sea un spline lineal continuo a trozos de la forma $P_f(x) = \sum_{k=1}^{N} c_k|x - x_k|$, o usar interpolación segmentaria.

Sin embargo, el Teorema de Mairhuber-Curtis (teorema 2.7) plantea un problema para dimensiones mayores o iguales que dos, y es que no podemos fijar la base $\{B_1, \ldots, B_N\}$ y pretender que para cualesquiera $\{x_1, \ldots, x_N\} \subseteq \Omega$ distintos entre sí, el problema de interpolación de datos dispersos esté bien puesto. Como veremos más adelante, la solución a este problema, para $s \geq 2$, consistirá en hacer depender nuestra base de funciones del conjunto de datos $\{x_1, \ldots, x_N\}$ escogido en algunos casos buscando una solución parecida a la de los splines lineales para $s = 1$.

Siguiendo este hilo del teorema 2.7, presentamos un primer ejemplo en el que vemos que intentar generalizar el espacio de polinomios a dos dimensiones no nos va a dar un espacio de Haar.

Ejemplo 2.8. A modo de ejemplo ilustrativo para $s = 2$, consideremos el espacio de polinomios de grado 1 en dos variables, cuya base natural fija es $\mathcal{B} = \{1, x, y\}$. En este caso, la dimensión del espacio es $N = 3$. Si la propiedad de Haar se cumpliera, el problema de interpolación debería estar bien puesto para cualquier conjunto de 3 puntos distintos. Sin embargo, si elegimos tres puntos alineados en el plano, por ejemplo, $x_1 = (0, 0)$, $x_2 = (1, 0)$ y $x_3 = (2, 0)$, la matriz de interpolación A dada por $A_{jk} = B_k(x_j)$ resulta ser:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 2 & 0 \end{pmatrix}$$

Como el valor de y es cero para todos los puntos, la tercera columna de la matriz está compuesta únicamente por ceros, lo que implica directamente que $\det(A) = 0$. Al existir configuraciones de puntos distintos donde la matriz se vuelve singular, se evidencia que no podemos fijar la base a priori, ilustrando así la limitación que plantea el Teorema de Mairhuber-Curtis.

Ahora, antes de ver cómo será la forma general de nuestra base de funciones cuando $s \geq 2$, debemos introducir una definición esencial en nuestro desarrollo: las funciones radiales.

Definición 2.9. Una función $\Phi : \mathbb{R}^s \to \mathbb{R}$ es radial (pura) si existe una función de una variable $\varphi : [0, \infty) \to \mathbb{R}$ tal que: $\Phi(x) = \varphi(r)$, con $r = \|x\|$, donde $\|\cdot\|$ es una norma en $\mathbb{R}^s$ (habitualmente la euclídea).

Definición 2.10. Una función $\Phi : \mathbb{R}^s \to \mathbb{R}$ es radial desplazada respecto a un centro y si existe una función de una variable $\varphi : [0, \infty) \to \mathbb{R}$ tal que $\Phi(x) = \varphi(\|x - y\|)$.

A partir de estas definiciones, conviene establecer varias convenciones y propiedades que serán fundamentales para nuestro desarrollo. En primer lugar, es evidente que una función radial pura no es más que una función radial desplazada con centro en el origen. Además, de ahora en adelante y salvo que se especifique lo contrario, cuando hablemos de norma asumiremos siempre que nos referimos a la norma euclídea.

Geométricamente, las funciones radiales presentan simetría esférica respecto a su centro y. Es decir, si tomamos dos puntos $x_1, x_2 \in \mathbb{R}^s$ que equidistan del centro ($||x_1 - y|| = ||x_2 - y||$), entonces sus imágenes coinciden, $\Phi(x_1) = \Phi(x_2)$.

En el contexto de nuestro problema de interpolación, supondremos que todas las funciones de la base $\{B_k\}_{k=1}^N$ son funciones radiales desplazadas respecto a los nodos de muestreo x_k, y que todas ellas se obtienen a partir de una misma función básica $\varphi : [0, \infty) \to \mathbb{R}$. Esto significa que $B_k(x) = \varphi(||x - x_k||)$ para $1 \leq k \leq N$. De esta forma, nuestra función interpoladora adopta la siguiente estructura:

$$P_f(x) = \sum_{k=1}^{N} c_k \varphi(||x - x_k||).$$

Una de las grandes ventajas de esta formulación es que la evaluación de la función básica φ es independiente de la dimensión del espacio s. Sin embargo, como contrapartida, este enfoque nos obligará a estudiar detenidamente bajo qué condiciones el problema de interpolación está bien puesto, es decir, cuándo la matriz del sistema resultante es efectivamente invertible.

Otra ventaja es, tal y como ilustra el siguiente lema, que el interpolante considerado es invariante por transformaciones euclídeas.

Lema 2.11. Los interpoladores dados por una base de funciones radiales para un problema bien puesto son invariantes por transformaciones euclídeas.

Demostración. Sea $\chi = \{x_1, \ldots, x_N\}$, con $x_i \in \mathbb{R}^s$ para $1 \leq i \leq N$. Sea $y = [y_1, \ldots, y_N]^T$ y sea $E : \mathbb{R}^s \to \mathbb{R}^s$ una transformación euclídea definida como $E(x) = Qx + b$, donde $Q \in \mathbb{R}^{s \times s}$ es una matriz ortogonal y $b \in \mathbb{R}^s$.

En primer lugar, vemos que:

$$\|E(x) - E(z)\|_2^2 = \|Q(x - z)\|_2^2 = (x - z)^T Q^T Q(x - z) = \|x - z\|_2^2, \quad \forall x, z \in \mathbb{R}^s.$$

Así, tenemos que $\|E(x) - E(z)\|_2 = \|x - z\|_2$.

Ahora, veamos que $s(x) = s'(E(x))$, donde:

$$s(x) = \sum_{j=1}^{N} c_j \varphi(\|x - x_j\|_2),$$

$$s'(x) = \sum_{j=1}^{N} c'_j \varphi(\|x - E(x_j)\|_2).$$

Comprobemos primero que que $c_j = c'_j$:

$$A'_{ij} = \varphi(\|E(x_i) - E(x_j)\|_2) = \varphi(\|x_i - x_j\|_2) = A_{ij}.$$

Por tanto, las matrices son idénticas ($A' = A$) y los sistemas de ecuaciones resultantes son:

$$Ac = y,$$

$$Ac' = y$$

(Nota: los datos del vector y no se trasladan).

Si asumimos que A es invertible (problema bien puesto), entonces $c = c'$.

Finalmente:

$$s'(E(x)) = \sum_{j=1}^{N} c'_j \varphi(\|E(x) - E(x_j)\|_2) = \sum_{j=1}^{N} c_j \varphi(\|x - x_j\|_2) = s(x).$$

□

Tras confirmar que el interpolante es invariante por transformaciones euclídeas, podríamos preguntarnos qué función φ podríamos escoger para que, de acuerdo a la definición 2.10, construyamos nuestro interpolante. El caso más sencillo por el que podemos empezar es elegir $\varphi = \mathrm{id}$, que se inspira en el caso de splines en 1D, y esto es lo que haremos en la siguiente sección.

2.3. Interpolación con matrices distancia

Como ya introdujimos al final de la sección anterior, para nuestro primer ejemplo de interpolación en una base de funciones radiales (RBF), escogemos $\varphi = \mathrm{id}$, por lo que $P_f(x) = \sum_{k=1}^{N} c_k ||x - x_k||_2, \forall x \in \Omega$ (recordemos que $\Omega = [0,1]^s$), que queda con una forma similar a los splines en una dimensión, como ya anticipamos cuando hablamos de las consecuencias del teorema 2.7. Para obtener los coeficientes $c_1, \ldots, c_N$, debemos resolver el sistema lineal $Ac = y$, con

$$A = \begin{bmatrix} \|x_1 - x_1\|_2 & \ldots & \|x_1 - x_N\|_2 \\ \vdots & \ddots & \vdots \\ \|x_N - x_1\|_2 & \ldots & \|x_N - x_N\|_2 \end{bmatrix},$$

donde A es la matriz distancia. Ahora, en primer lugar, veamos que el problema está bien puesto, para lo que demostraremos en la siguiente subsección que la matriz distancia basada en la distancia euclídea entre un conjunto de puntos distintos en $\mathbb{R}^s$ es invertible.

2.3.1. La invertibilidad de la matriz distancia y otros resultados matriciales

Con el objetivo de demostrar la invertibilidad de la matriz distancia, desarrollamos en esta subsección una serie de resultados que nos llevarán a esta conclusión.

Teorema 2.12 (Teorema de Courant-Fischer)**.** Sea $A \in \mathbb{R}^{N\times N}$ una matriz simétrica con autovalores $\lambda_1 \geq \cdots \geq \lambda_N$. Entonces, para cualquier $k \in \{1, \ldots, N\}$, se cumple que:

$$(1)\ \lambda_k = \max_{\dim V = k} \min_{\substack{x\in V \\ \|x\|=1}} x^T A x \qquad (2)\ \lambda_k = \min_{\dim V = N-k+1} \max_{\substack{x\in V \\ \|x\|=1}} x^T A x.$$

Demostración. Comenzaremos nuestra demostración apoyándonos en el teorema espectral. Puesto que $A \in \mathbb{R}^{N\times N}$ es una matriz simétrica, sabemos que posee autovalores reales ordenados $\lambda_1 \geq \cdots \geq \lambda_N$ y una base ortonormal de autovectores $\{u_1, \ldots, u_N\}$, de forma que $Au_i = \lambda_i u_i$ para todo $1 \leq i \leq N$.

Fijemos un índice $k \in \{1, \ldots, N\}$. Nuestro primer objetivo será demostrar la fórmula (1). Para ello, dividiremos el problema en dos partes, buscando acotar el valor de λ_k por debajo y por encima. Primero, veamos que $\lambda_k \leq \max_{\dim V=k} \min_{\substack{x\in V \\ \|x\|=1}} x^T A x$.

Para lograr esta cota superior, busquemos un subespacio que funcione. Consideremos el subespacio vectorial generado por los primeros k autovectores, $S_k := \langle u_1, \ldots, u_k\rangle \leq \mathbb{R}^N$. Si tomamos cualquier vector unitario en este subespacio, es decir, $x \in S_k$ con $\|x\| = 1$, lo podemos expresar como $x = \sum_{i=1}^k c_i u_i$. Evaluando la forma cuadrática en este vector, y dado que todos los autovalores involucrados son mayores o iguales a λ_k, obtenemos:

$$x^T A x = \sum_{i=1}^k \lambda_i c_i^2 \geq \sum_{i=1}^k \lambda_k c_i^2 = \lambda_k \|x\|_2^2 = \lambda_k.$$

De este modo, hemos descubierto que, para todo $x \in S_k$, el valor de $x^T Ax$ es al menos λ_k. En particular, si tomamos el propio autovector $x = u_k$, se alcanza exactamente la igualdad $u_k^T A u_k = \lambda_k$. Luego, el mínimo evaluado exclusivamente sobre nuestro candidato S_k es exactamente λ_k:

$$\min_{\substack{x\in S_k \\ \|x\|=1}} x^T A x = \lambda_k.$$

Dado que el máximo sobre todos los subespacios posibles de dimensión k tiene que ser, al menos, tan grande como el valor que nos da un subespacio particular como S_k, deducimos nuestra primera desigualdad:

$$\max_{\dim V=k} \min_{\substack{x\in V \\ \|x\|=1}} x^T A x \geq \lambda_k.$$

Abordemos ahora la desigualdad contraria, es decir, veamos que

$$\lambda_k \geq \max_{\dim V=k} \min_{\substack{x\in V \\ \|x\|=1}} x^T A x.$$

Imaginemos un subespacio arbitrario $S \leq \mathbb{R}^N$ cualquiera de dimensión $\dim S = k$ y construyamos $W := \langle u_k, \ldots, u_N \rangle \leq \mathbb{R}^N$, cuya dimensión es $\dim W = N - k + 1$. Al ser S y W subespacios vectoriales de dimensión finita dentro de $\mathbb{R}^N$, podemos usar la siguiente fórmula para analizar su intersección:

$$\dim(S \cap W) = \dim(S) + \dim(W) - \dim(S + W).$$

Dado que $S + W \leq \mathbb{R}^N$, su dimensión no puede superar N. Sustituyendo en la fórmula, obtenemos que $\dim(S \cap W) \geq k + N - k + 1 - N = 1$. Esto significa que la intersección no es trivial, por tanto, existe un vector unitario $x_0 \in S \cap W$, con $\|x_0\| = 1$.

Por pertenecer a W, este vector se escribe como $x_0 = \sum_{i=k}^{N} c_i u_i$. Al evaluar la forma cuadrática, como ahora todos los autovalores involucrados cumplen que $\lambda_i \leq \lambda_k$, resulta que:

$$x_0^T A x_0 = \sum_{i=k}^{N} \lambda_i c_i^2 \leq \lambda_k \sum_{i=k}^{N} c_i^2 = \lambda_k \|x_0\|_2^2 = \lambda_k.$$

En consecuencia, el mínimo evaluado sobre todo el subespacio S no puede superar el valor que nos da este vector concreto x_0, lo que implica que

$$\min_{\substack{x \in S \\ \|x\|=1}} x^T A x \leq x_0^T A x_0 \leq \lambda_k.$$

Como lo anterior es cierto para cualquier subespacio S de dimensión k, el máximo de todos esos mínimos posibles tampoco lo superará, obteniendo:

$$\max_{\dim V = k} \min_{\substack{x \in V \\ \|x\|=1}} x^T A x \leq \lambda_k.$$

Al reunir ambas desigualdades, la fórmula (1) queda completamente demostrada.

Pasemos ahora a demostrar la segunda identidad del teorema. En este caso, debemos definir la matriz opuesta $B = -A$. Si los autovalores de A son $\lambda_1 \geq \cdots \geq \lambda_N$, los de B invertirán su orden y su signo. Así, si llamamos $\mu_1 \geq \cdots \geq \mu_N$ a los autovalores ordenados de B, la relación entre ambos espectros es claramente $\mu_j = -\lambda_{N-j+1}$ para $1 \leq j \leq N$.

Fijando de nuevo nuestro índice k, definimos su índice complementario como $m := N - k + 1$. Al evaluar nuestra relación de autovalores para este índice específico, comprobamos que $\mu_m = \mu_{N-k+1} = -\lambda_k$. Aplicando a la matriz B y al índice m la primera parte del teorema que ya acabamos de demostrar, sabemos que:

$$\mu_m = \max_{\dim V = m} \left(\min_{\substack{x \in V \\ \|x\|=1}} x^T B x \right).$$

Sustituyendo nuestras equivalencias conocidas ($\mu_m = -\lambda_k$, $m = N - k + 1$ y $B = -A$), podemos reescribir el problema en términos de nuestra matriz original:

$$-\lambda_k = \max_{\dim V = N-k+1} \left(\min_{\substack{x \in V \\ \|x\|=1}} (-x^T A x) \right).$$

Llegados a este punto, debemos extraer con cuidado los signos negativos empleando propiedades básicas de los números reales. En primer lugar, sabemos que, para cualquier conjunto $S \subseteq \mathbb{R}$, el mínimo de los valores negativos es el negativo del máximo de los positivos, es decir, $\min(S) = -\max(-S)$. Para aplicar esto, fijamos un subespacio V de dimensión $N-k+1$ y definimos el conjunto $S_V := \{-x^T Ax : x \in V, \|x\| = 1\} \subseteq \mathbb{R}$. Usando la propiedad descrita, obtenemos $\min(S_V) = -\max(-S_V)$, lo que nos permite sacar el primer signo negativo del interior del paréntesis:

$$-\lambda_k = \max_{\dim V = N-k+1} \left(- \max_{\substack{x \in V \\ \|x\|=1}} x^T Ax \right).$$

Finalmente, para extraer el segundo signo negativo, consideramos G_{N-k+1}, el conjunto de todos los subespacios vectoriales de $\mathbb{R}^N$ de dimensión $N-k+1$. Sobre él, definimos la función

$$\varphi : G_{N-k+1} \to \mathbb{R}, \text{ con } \varphi(V) = \max_{\substack{x \in V \\ \|x\|=1}} (x^T Ax).$$

Si agrupamos todos los valores que toma esta función en un conjunto $R = \{\varphi(V) : V \in G_{N-k+1}\} \subseteq \mathbb{R}$, podemos volver a usar nuestra regla de signos sobre los extremos de un conjunto: $\max(-R) = -\min(R)$. Aplicando esto a nuestra ecuación completa, el signo sale definitivamente de la maximización exterior:

$$-\lambda_k = - \min_{V \in G_{N-k+1}} \left(\max_{\substack{x \in V \\ \|x\|=1}} x^T Ax \right).$$

Cancelando signos, concluimos con éxito la demostración de la segunda identidad:

$$\lambda_k = \min_{\dim V = N-k+1} \max_{\substack{x \in V \\ \|x\|=1}} x^T Ax.$$

□

Una vez demostrado el teorema anterior, continuemos con las siguientes definiciones, que resultarán de gran utilidad a lo largo de todo este trabajo.

Definición 2.13. Sea $\chi = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^s$ un conjunto de N puntos distintos y sea Π^s_{m-1} el espacio de polinomios en s variables de grado menor o igual a $m-1$. Dada una matriz $A \in \mathbb{R}^{N \times N}$ simétrica, se dice que A es estrictamente condicionalmente definida positiva (ECDP) de orden m con respecto a χ si

$$c^T Ac > 0, \forall c = (c_1, \ldots, c_N)^T \in \mathbb{R}^N - \{0\} \text{ tal que } \sum_{i=1}^{N} c_i p(x_i) = 0, \text{ para todo } p \in \Pi^s_{m-1}.$$

Observamos que, tanto para el caso de $m = 0$ ($\Pi^s_{-1} = \{0\}$) como $m = 1$, la definición no depende del conjunto χ escogido. En particular, se dice que A es ECDP de orden uno si $c^T Ac > 0$ para todo $c \in \mathbb{R}^N - \{0\}$ tal que $\sum_{i=1}^{N} c_i = 0$.

Definición 2.14. Sea $A \in \mathbb{R}^{N\times N}$ simétrica. Se dice que A es estrictamente definida positiva (EDP) si A es ECDP de orden cero, es decir, si $x^T Ax > 0$, $\forall x = (x_1, \dots, x_N) \in \mathbb{R}^N$ si y solo si $x \neq 0$.
Definición 2.15. Sea $\Phi : \mathbb{R}^s \to \mathbb{R}$ una función par. Se dice que Φ es estrictamente condicionalmente definida positiva (ECDP) de orden m si $\forall\{x_1, \dots, x_N\} \subseteq \mathbb{R}^s$, con $x_i \neq x_j$ si $i \neq j$, que definen una matriz $A \in \mathbb{R}^{N\times N}$ dada por $A_{ij} = \Phi(x_i - x_j)$, se tiene que A es ECDP de orden m con respecto a dicho conjunto de puntos.

En particular, se dice que Φ es ECDP de orden uno si la matriz A asociada resulta ser ECDP de orden uno.
Definición 2.16. Sea $\Phi : \mathbb{R}^s \to \mathbb{R}$ una función par. Se dice que Φ es estrictamente definida positiva (EDP) si es ECDP de orden cero, es decir, si

$$\sum_{j=1}^{N}\sum_{k=1}^{N} c_j c_k \Phi(x_j - x_k) > 0, \forall\{x_1, \dots, x_N\} \subseteq \mathbb{R}^s \text{ y } c = [c_1, \dots, c_N]^T \in \mathbb{R}^N \text{ si y solo si } c \neq 0.$$

Tras estas definiciones, el siguiente corolario, que es una consecuencia inmediata del teorema 2.12, supone un pequeño inciso en esta lista de resultados matriciales, ya que, aunque no lo utilizaremos en esta sección, resultará importante más adelante en este trabajo.
Corolario 2.17 (Caracterización espectral de las normas y del número de condición espectral de la matriz A, $\kappa_2(A)$)**.** Sea $A \in \mathbb{R}^{N\times N}$ simétrica con autovalores $\lambda_{\text{máx}} = \lambda_1 \geq \cdots \geq \lambda_N = \lambda_{\text{mín}}$ y base ortonormal de autovectores $\{u_i\}$ ($Au_i = \lambda_i u_i$). Entonces los autovalores extremos vienen caracterizados por

$$\lambda_{\text{máx}} = \max_{\|x\|_2=1} x^T Ax, \qquad \lambda_{\text{mín}} = \min_{\|x\|_2=1} x^T Ax,$$

y la norma matricial verifica $\|A\|_2 = \text{máx}_i\, |\lambda_i|$. Supongamos, además, que A es invertible. Entonces A^{-1} es simétrica y

$$\|A\|_2 = \text{máx}\, |\lambda_i|, \qquad \|A^{-1}\|_2 = \frac{1}{\text{mín}\, |\lambda_i|}, \qquad \kappa_2(A) := \|A\|_2\, \|A^{-1}\|_2 = \frac{\text{máx}\, |\lambda_i|}{\text{mín}\, |\lambda_i|}.$$

Por último, si A es, además, EDP se tiene trivialmente que $\kappa_2(A) = \frac{\lambda_{\text{máx}}}{\lambda_{\text{mín}}}$.

Demostración. Partimos de los casos extremos del teorema 2.12, donde por su primera fórmula, $\lambda_k = \text{máx}_{\dim V=k}\, \text{mín}_{x\in V,\, \|x\|=1}\, x^T Ax$. Así, si tomamos $k = 1$, un subespacio de dimensión 1 es una recta $V = \langle u\rangle$, que contiene un único vector unitario salvo signo y, como $(-x)^T A(-x) = x^T Ax$, ambos producen el mismo valor. De esta forma, el mínimo del interior de la fórmula se reduce entonces a ese único valor y desaparece, de modo que elegir la recta V equivale a elegir su vector unitario x y, por tanto, queda que $\lambda_{\text{máx}} = \text{máx}_{\|x\|=1}\, x^T Ax$. Análogamente, tomando $k = N$ en la segunda fórmula del teorema 2.12, con $N - k + 1 = 1$, se obtiene $\lambda_{\text{mín}} = \text{mín}_{\|x\|=1}\, x^T Ax$.

Una vez tenemos las igualdades para los valores propios, probemos las igualdades para las normas matriciales. Así, por ser A simétrica,

$$\|Ax\|_2^2 = (Ax)^T(Ax) = x^T A^T Ax = x^T A^2 x.$$

Por otro lado, tenemos que

$$\|A\|_2^2 := (\max_{\|x\|=1} \|Ax\|_2)^2 = \max_{\|x\|=1} \|Ax\|_2^2 = \max_{\|x\|=1} x^T A^2 x = \lambda_{\max}(A^2) = (\max_i |\lambda_i|)^2.$$

La última igualdad se debe a que, como $A^2 u_i = \lambda_i^2 u_i$, los autovalores de A^2 son los λ_i^2 y el mayor de ellos es $\lambda_{\max}(A^2) = \max_i \lambda_i^2 = (\max_i |\lambda_i|)^2$.

Por su parte, para A^{-1}, si A es invertible, como A^{-1} es simétrica ($(A^{-1})^T = (A^T)^{-1} = A^{-1}$) y con autovalores $1/\lambda_i$, pues de $Au_i = \lambda_i u_i$ se sigue que $A^{-1}u_i = \lambda_i^{-1} u_i$. Aplicándole lo ya probado, $\|A^{-1}\|_2 = \max_i(1/|\lambda_i|) = 1/(\min_i |\lambda_i|)$. Multiplicando ambas normas, $\kappa_2(A) = \frac{\max |\lambda_i|}{\min |\lambda_i|}$.

Ahora, si A es EDP, todos los λ_i son positivos y $\max_i |\lambda_i| = \lambda_{\max}$, $\max_i(1/\lambda_i) = 1/\lambda_{\min}$. Por tanto, se concluye que $\kappa_2(A) = \lambda_{\max}/\lambda_{\min}$. □

Tras este inciso, continuamos con la serie de resultados que nos permitirán demostrar la invertibilidad de la matriz distancia. Los siguientes dos teoremas basan su importancia en demostrar que, primero, cierto tipo de matrices, son invertibles; y, segundo, que funciones ECDP de orden uno y no positivas en 0, definen el tipo de matriz que las hace invertibles.

Teorema 2.18. Sea $A \in \mathbb{R}^{N\times N}$ simétrica, ECDP de orden uno y con traza no positiva, entonces A tiene un autovalor negativo y $N-1$ autovalores positivos.

Demostración. Definimos $L := \{c = (c_1, \ldots, c_N) : \sum_{i=1}^N c_i = 0\}$, que es un $\mathbb{R}$-subespacio vectorial de $\mathbb{R}^N$, ya que $0_{\mathbb{R}^N} \in L$ y $\alpha x + \beta y \in L, \forall \alpha, \beta \in \mathbb{R}, x, y \in L$, puesto que $\sum_{i=1}^N \alpha x_i + \beta y_i = 0$. Además, $\dim L = N-1$, veámoslo. Si definimos la transformación lineal $T : \mathbb{R}^N \to \mathbb{R}$, dada por:

$$T(c_1, \ldots, c_N) = c_1 + \cdots + c_N$$

Como $\dim(\mathrm{Im}(T)) = 1$ y $L = \ker(T)$, tenemos que $\dim(L) = \dim(\ker(T)) = \dim(\mathbb{R}^N) - \dim(\mathrm{Im}(T)) = N-1$ y ya lo tenemos.

Ahora, sea $A \in \mathbb{R}^{N\times N}$ la de nuestra hipótesis, sean $\lambda_1 \geq \cdots \geq \lambda_N$ autovalores de A, por el Teorema de Courant-Fischer (teorema 2.12) se tiene que:

$$\lambda_{N-1} = \max_{\dim V = N-1} \min_{\substack{x\in V \\ \|x\|=1}} x^T A x \geq \min_{\substack{c\in L \\ \|c\|=1}} c^T A c = \min_{c\in W} c^T A c \overset{(*)}{>} 0,$$

donde $W := \left\{x \in \mathbb{R}^N \mid \sum_{i=1}^N x_i = 0,\ \|x\| = 1\right\}$. La desigualdad (*) viene por ser A ECDP de orden uno.

Así, $\lambda_1 \geq \cdots \geq \lambda_{N-1} > 0$. Como $0 \geq \mathrm{Tr}(A) = \sum_{i=1}^N \lambda_i$, se concluye que $\lambda_N < 0$. □

Teorema 2.19 (Interpolación)**.** Sea $\Phi : \mathbb{R}^s \to \mathbb{R}$ ECDP de orden 1, con $\Phi(0) \leq 0$. Entonces $\forall x_1, \ldots, x_N \in \mathbb{R}^s$, la matriz $A \in \mathbb{R}^{N\times N}$, dada por $A_{ij} = \Phi(x_i - x_j)$ tiene $N-1$ autovalores positivos y uno negativo y es, por tanto, invertible.

Demostración. Por ser Φ ECDP, sea $\{x_1, \ldots, x_N\} \subseteq \mathbb{R}^s$, si definimos $A_{ij} = \Phi(x_i - x_j)$, entonces se tiene que $c^T A c > 0, \forall c \in \mathbb{R}^N - \{0\}$ tal que $\sum c_i = 0$. Además, $\mathrm{Tr}(A) = N \cdot \Phi(0) \leq 0$.

Así, se tiene que A es ECDP de orden uno con traza no positiva y, por el teorema 2.18, se llega a que A tiene $N-1$ autovalores positivos y uno negativo y, por tanto, es invertible. □

Llegados a este punto, notemos que, si demostramos que la función que genera nuestra matriz distancia (A) cumple las hipótesis del teorema 2.19, esta será invertible. Más concretamente, vamos a demostrar que $\Phi(x) = -||x||$ verifica dichas hipótesis y genera $-A$. Con este objetivo en mente, introducimos una definición previa, un lema y una proposición y, concluiremos la demostración con un corolario.

Definición 2.20. Una función $g \in C^\infty(0, \infty)$ se dice completamente monótona si

$$(-1)^\ell \, g^{(\ell)}(t) \geq 0 \qquad \text{para todo } \ell \in \mathbb{N}_0 \text{ y todo } t > 0.$$

A continuación, se enuncia un lema que se deja sin probar, porque excede los objetivos de este trabajo, y cuya caracterización se remonta a Micchelli [15]. Sin embargo, el enunciado en la forma estricta aquí empleada, junto con su demostración, puede consultarse en [25, Cap. 8] o [7, Cap. 8].

Lema 2.21 (Caracterización por monotonía completa)**.** Sean $\varphi \in C[0, \infty) \cap C^\infty(0, \infty)$ y $m \in \mathbb{N}$. Si $(-1)^m \varphi^{(m)}$ es completamente monótona y no constante en $(0, \infty)$, entonces la función radial

$$\Phi(x) := \varphi\big(\|x\|_2^2\big), \qquad x \in \mathbb{R}^s,$$

es ECDP de orden m en $\mathbb{R}^s$ para todo $s \geq 1$.

Proposición 2.22 (Potencias impares)**.** Sea p un entero impar y sea $m := \lceil p/2 \rceil = \frac{p+1}{2}$. Entonces la función radial

$$\Phi(x) = (-1)^m \, \|x\|_2^p, \qquad x \in \mathbb{R}^s,$$

es ECDP de orden m en $\mathbb{R}^s$ para todo $s \geq 1$. En consecuencia, dados nodos distintos $x_1, \ldots, x_N \in \mathbb{R}^s$, la matriz $A_{ij} = \Phi(x_i - x_j)$ verifica $c^T A c > 0$ para todo $c \in \mathbb{R}^N \setminus \{0\}$ con $\sum_{i=1}^N c_i \, q(x_i) = 0$ para todo $q \in \Pi_{m-1}^s$.

Demostración. Escribimos $p = 2m - 1$ con $m \geq 1$ y consideramos

$$\varphi(t) = (-1)^m \, t^{\,m - \frac{1}{2}}, \qquad t \geq 0,$$

que es continuo en $[0, \infty)$, de clase C^∞ en $(0, \infty)$ y satisface $\varphi(\|x\|_2^2) = (-1)^m \|x\|_2^{2m-1} = (-1)^m \|x\|_2^p = \Phi(x)$. Veamos ahora que $(-1)^m \varphi^{(m)}$ es completamente monótona y no constante en $(0, \infty)$. Derivando m veces, con $\alpha := m - \frac{1}{2}$ y usando $\frac{d}{dt} t^\alpha = \alpha t^{\alpha - 1}$,

$$\varphi^{(m)}(t) = (-1)^m \Big[\prod_{j=0}^{m-1} \big(m - \tfrac{1}{2} - j\big) \Big] \, t^{\alpha - m} = (-1)^m \, \tfrac{(2m-1)!!}{2^m} \, t^{-1/2},$$

pues los factores $m-\frac{1}{2}, m-\frac{3}{2}, \ldots, \frac{1}{2}$ tienen producto $\frac{(2m-1)!!}{2^m}$[1] y $\alpha - m = -\frac{1}{2}$. Así

$$g(t) := (-1)^m \varphi^{(m)}(t) = \frac{(2m-1)!!}{2^m}\, t^{-1/2} > 0, \qquad t > 0.$$

Para todo $\ell \in \mathbb{N}_0$ (con el convenio usual $(-1)!! = 1$, que reproduce el caso $\ell = 0$),

$$(-1)^\ell g^{(\ell)}(t) = \frac{(2m-1)!!}{2^m} \cdot \frac{(2\ell-1)!!}{2^\ell}\, t^{-\frac{1}{2}-\ell} > 0, \forall t > 0$$

de modo que g es completamente monótona y, además, es no constante. Por tanto, tenemos todas las hipótesis para aplicar el lema 2.21, de forma que concluimos que $\Phi(x) := \varphi(\|x\|_2^2)$, $x \in \mathbb{R}^s$ es ECDP de orden m en $\mathbb{R}^s, \forall s \geq 1$. Directamente de la definición 2.15 se deriva la afirmación sobre la matriz A del enunciado de nuestra proposición. □

Corolario 2.23. La matriz distancia definida al principio de esta sección como $A_{ij} = ||x_i - x_j||_2$, para un conjunto de nodos $\chi = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^s$, es invertible y, por tanto, el problema de interpolación de datos dispersos está bien puesto.

Demostración. Sea $\Phi(x) = -||x||_2$, por la proposición 2.22 aplicada con $p = 1$, sabemos que Φ es ECDP de orden uno. Además, claramente, $\Phi(0) = 0 \leq 0$. Así, por el teorema 2.19, la matriz dada por $B_{ij} = \Phi(x_i - x_j)$ es invertible y, por tanto, la matriz del enunciado, $A = -B$ es invertible. □

Con este último corolario concluimos esta subsección afirmando que la matriz distancia es, efectivamente, invertible y comenzamos otro apartado en el que, una vez concluida la teoría necesaria sobre el método, procedemos a hacer experimentos numéricos con él.

2.3.2. Ejemplo práctico de RBF con matrices distancia

Una vez probado que nuestro problema de interpolación para el caso de RBF por matrices distancia está bien puesto, procedemos a aplicarlo en un primer ejemplo, con el objetivo de comprobar la calidad de la aproximación y el error cometido por este método. Así, lo aplicamos a la función test

$$f_s(x) = 4^s \prod_{i=1}^{s} x^i(1-x^i), \text{ con } x = (x^1, \ldots, x^s) \in \Omega = [0,1]^s,$$

generada en Matlab por la función 4. Cabe destacar que todos los códigos empleados en este trabajo se encuentran recogidos en el apéndice C. Para su implementación, se han tomado como referencia diversas rutinas del libro [7], complementadas con el uso de herramientas de inteligencia artificial.

Cabe recordar que los puntos de muestreo serán los N puntos de Halton, donde $N = (2^k+1)^s$, con $k \in \mathbb{N}$. Dichos puntos de Halton se construyen con la función 1 y la matriz distancia a la que estos dan lugar se crea con la función 3. A su vez, crearemos un mallado equiespaciado del cubo de dimensión s (función 2), que nos servirá para evaluar los errores cometidos por la función interpoladora respecto a

[1] El símbolo !! denota el doble factorial. Para un número natural impar n, se define como el producto de todos los enteros impares desde 1 hasta n, es decir, $n!! = n(n-2)\cdots 3 \cdot 1$.

nuestra función test, que pretendemos aproximar. Lo anterior se integra en el script 9, que resuelve el problema de interpolación para nuestra función test para $s = 1, 2, 3, 4, 5, 6$, donde el número de puntos (M) del mallado del cubo unidad donde se evalúan los errores cometidos son 1000, 1600, 1000, 256, 1024 y 4096, respectivamente.

Ahora bien, el error mediremos sobre el método, y que se recoge en la tabla 2.1, es la raíz del error cuadrático medio (RMSE), que se define de la siguiente forma:

$$\text{RMSE} = \sqrt{\frac{1}{M}\sum_{i=1}^{M}[P_f(\epsilon_i) - f(\epsilon_i)]^2},$$

donde $\epsilon_i, 1 \leq i \leq M$ son los puntos de evaluación del mallado equiespaciado del cubo.

	1D		2D		3D		4D		5D		6D	
k	N	RMS-error	N	RMS-error	N	RMS-error	N	RMS-error	N	RMS-error	N	RMS-error
1	3	7.205502e-01	9	1.832166e-01	27	1.055678e-01	81	1.235563e-01	243	9.143136e-02	729	4.976642e-02
2	5	2.737481e-01	25	6.514899e-02	125	6.044121e-02	625	6.536983e-02	3125	3.108212e-02		
3	9	9.998035e-02	81	2.451219e-02	729	2.650678e-02						
4	17	3.604459e-02	289	9.902839e-03								
5	33	1.297877e-02	1089	4.007503e-03								
6	65	4.710927e-03	4225	1.636056e-03								
7	129	1.744186e-03										
8	257	6.706735e-04										
9	513	2.748063e-04										
10	1025	1.235284e-04										
11	2049	6.176348e-05										
12	4097	3.088165e-05										

Tabla 2.1: Raíz del error cuadrático medio en interpolación de la matriz distancia en los puntos de Halton para dimensiones de 1 a 6.

Con el objetivo de extraer más conclusiones de este primer ejemplo, graficamos, a continuación, la función interpoladora y su error respecto a la función que pretende aproximar.

Si miramos las gráficas 2.1a y 2.1b, vemos que, en una dimensión, la función interpoladora aproxima bastante bien a la función, para la poca cantidad de puntos de Halton empleados (esto es lo que hace que la función interpoladora tenga tan poca suavidad), salvo cerca del borde, donde el error se dispara.

Igualmente, este patrón se repite en 2D (gráficas 2.1c y 2.1d), donde la función interpoladora logra bastante más suavidad, debido a la cantidad de puntos de Halton empleados, pero que, igualmente, dispara su error cerca de los bordes del cubo unidad.

Ahora, si observamos las gráficas 2.2a y 2.2b, en las que se proyectan los valores de la función interpoladora y de los errores sobre los planos de corte, respectivamente, se observa nuevamente un aumento del error cerca de los bordes.

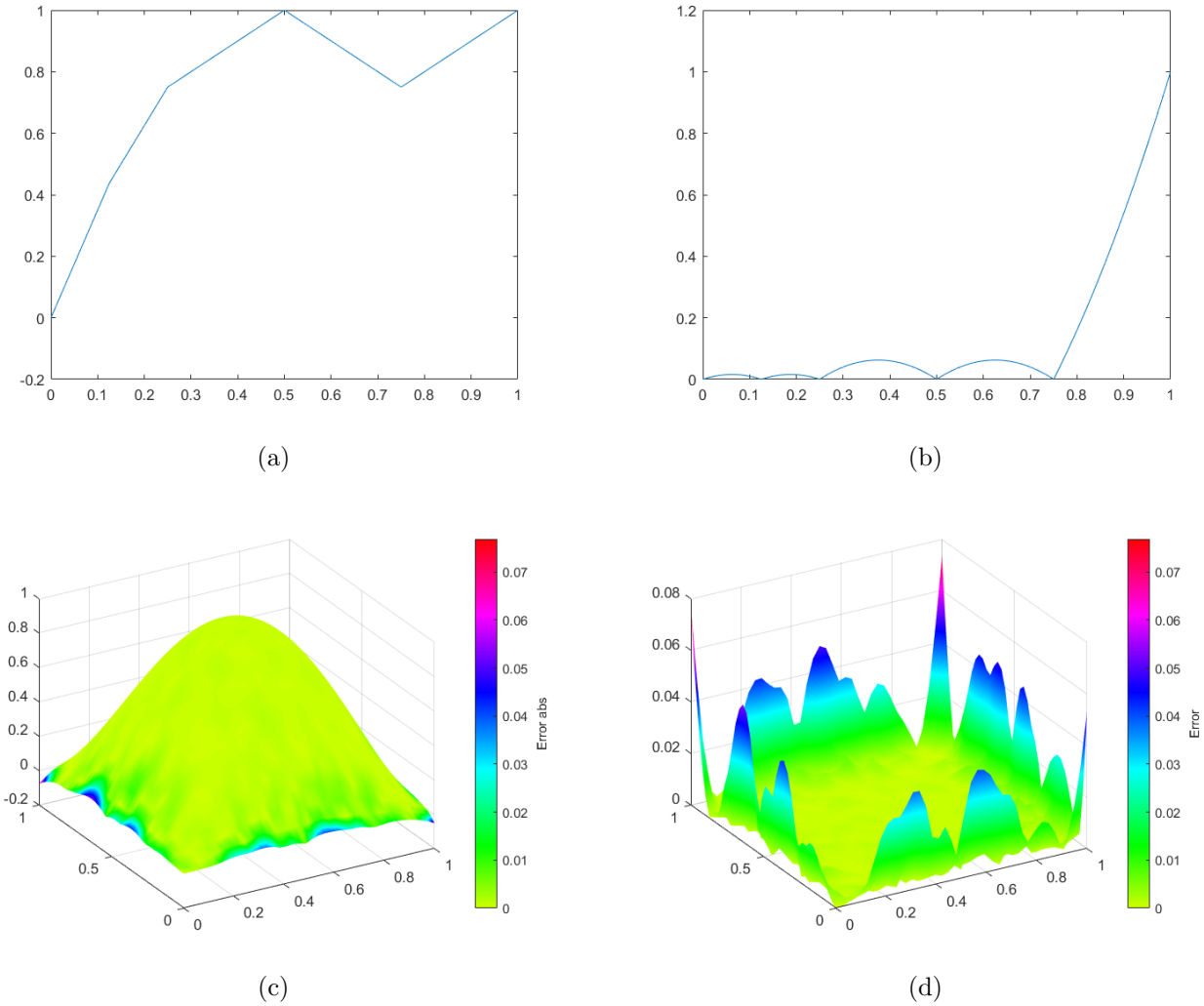
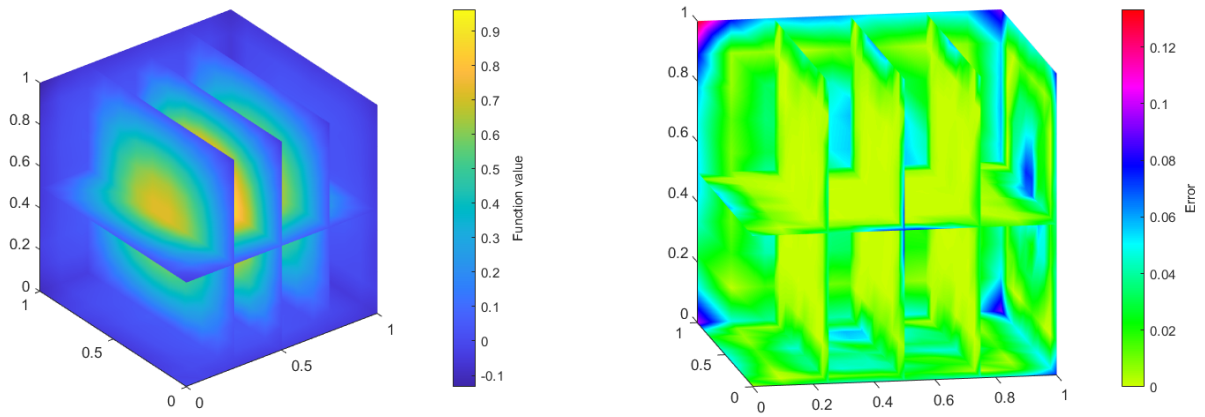


Figura 2.1: Gráficas 1D y 2D del ejemplo de interpolación de matrices distancia para la función f_s. (a) Función interpoladora y (b) error absoluto cometido para interpolación con 5 puntos de Halton en 1D. (c) Función interpoladora y (d) error absoluto cometido para interpolación con 289 puntos de Halton en 2D. Los colores determinados por la barra de errores se refieren al error absoluto.



(a) (b)

Figura 2.2: Gráficas 3D del ejemplo de interpolación de matrices distancia para la función f_s. (a) Función interpoladora y (b) error absoluto cometido para interpolación con 729 puntos de Halton en 3D. Los colores determinados por la barra de errores se refieren al error absoluto.

Así es que se concluye que el error absoluto es mucho mayor cerca de los bordes del cubo que en su interior, lo cual puede deberse, en parte, a que, mientras que los puntos de Halton solo se generan en el interior de este y en el cero (único punto de la frontera en que hay un nodo de Halton), los del mallado se generan también en los bordes y, además, estos aumentan conforme aumentamos la dimensión.

Por otro lado, como se observa en la tabla 2.1, al aumentar el número de puntos (N), el error disminuye en un factor aproximado de dos. A modo de anticipo, y aunque este fenómeno se analizará en detalle más adelante, esto indica que el error en función de la distancia de llenado (concepto que se definirá posteriormente) tiene orden uno.

En cuanto a las ventajas de este primer método, la principal es que, como ya anticipamos en la introducción, al tratarse de un método sin malla, es muy fácil de implementar en dimensiones mayores, pues genera puntos dispersos, que no tienen que estar "unidos" por líneas generando mallados regulares.

Sin embargo, existe el problema de que el método debe resolver un sistema con una matriz $N \times N$ densa, lo que lo hace muy costoso. Además, estas matrices suelen estar mal condicionadas para N grande (lo veremos), por lo que en dimensiones grandes, solo se pueden presentar resultados para conjuntos de datos pequeños, lo que, como hemos visto, reduce notablemente la precisión del método.

Así que, debemos encontrar alternativas que resuelvan los problemas de densidad de la matriz, mal condicionamiento, suavidad limitada del interpolador y precisión. Comencemos por intentar resolver el último de estos, eligiendo una φ que mejore la aproximación de nuestro interpolante y, aunque podríamos tratar de elegir otra φ polinómica, como veremos más tarde en sucesivos experimentos

numéricos, en temas de convergencia, es mejor elegir $\varphi(r) = e^{-(\varepsilon r)^2}$, en lo que se conoce como RBF gaussiano y que analizaremos en la siguiente sección.

2.4. RBF gaussiano

Como ya hemos introducido, en esta sección, y con el fin de obtener un aproximante más preciso, analizaremos el problema de interpolación mediante base de funciones radiales $P_f(x) = \sum_{k=1}^{N} c_k \varphi(||x - x_k||)$, en el caso en el que $\varphi(r) = e^{-(\varepsilon r)^2}$, donde ε es el parámetro de forma. Este tipo de interpolación recibe el nombre de RBF gaussiano.

Ahora bien, cabe preguntarnos, igual que hicimos en el caso del RBF por matrices distancia, si el problema está bien puesto, algo que analizaremos en el siguiente apartado.

2.4.1. Invertibilidad de la matriz del sistema para RBF gaussiano

Para ver que la matriz de nuestro sistema es invertible, está claro que, si demostramos que $\Phi(x) = \varphi(||x||)$ es EDP, como es una función real, la matriz del sistema lineal que surge para obtener los coeficientes que acompañan a las funciones radiales de la base, será EDP y, por tanto, todos sus valores propios positivos, luego la matriz será invertible.

Así, para demostrar esto, recurrimos al siguiente teorema, cuya demostración no abordaremos aquí porque requeriría entrar en profundidad en teoría de la medida pero que, si quiere consultarse, se puede hacer en el libro de Wendland [25].

Teorema 2.24. Sea $\varphi : [0, \infty) \to \mathbb{R}$ continua tal que $r \mapsto r^{s-1}\varphi(r) \in L_1[0, \infty)$. Si definimos $\Phi = \varphi(\|\cdot\|)$, esta es EDP y radial en $\mathbb{R}^s$ si y solo si su transformada de Fourier s-dimensional es no negativa y no idénticamente cero.

Como consecuencia de este teorema tenemos el corolario que nos asegura que el RBF gaussiano está bien puesto.

Corolario 2.25. Φ es EDP y radial en $\mathbb{R}^s$ y, por tanto, el problema de interpolación para RBF gaussiano está bien puesto.

Demostración. Veamos en primer lugar que, para el caso de RBF gaussiano, nuestra $\varphi(r) = e^{-(\varepsilon r)^2}$ verifica las hipótesis del teorema 2.24. Claramente es una función continua, luego falta comprobar que $r^{s-1}e^{-(\varepsilon r)^2}$ es $L^1[0, \infty)$.

La función $r \mapsto r^{s-1}\varphi(r) = r^{s-1}e^{-\varepsilon^2 r^2}$ es continua en $[0, \infty)$, por ser producto de funciones continuas, y no negativa. Al ser no negativa y medible, por el Teorema de Tonelli-Hobson basta acotar su integral en $\mathbb{R}^+$ para que $r^{s-1}e^{-(\varepsilon r)^2} \in L^1[0, \infty)$. Separamos el dominio en $[0, 1]$ y $[1, \infty)$:

$$\int_0^\infty r^{s-1}e^{-\varepsilon^2 r^2}\, dr = \int_0^1 r^{s-1}e^{-\varepsilon^2 r^2}\, dr + \int_1^\infty r^{s-1}e^{-\varepsilon^2 r^2}\, dr.$$

En $[0,1]$, el integrando es continuo sobre un compacto, luego acotado, por lo que definiendo $M_1 := \max_{r\in[0,1]} r^{s-1}e^{-\varepsilon^2 r^2} < \infty$, se tiene que

$$\int_0^1 r^{s-1}e^{-\varepsilon^2 r^2}\,dr \le M_1 < \infty.$$

Para acotar la integral en $[1,\infty)$, fijamos $N \in \mathbb{N}$ con $2N > s$. Del desarrollo en serie de la exponencial, cuyos términos son todos positivos, se sigue $e^{\varepsilon^2 r^2} \ge \dfrac{(\varepsilon^2 r^2)^N}{N!}$, de donde

$$r^{s-1}e^{-\varepsilon^2 r^2} \le r^{s-1}\,\frac{N!}{(\varepsilon^2)^N r^{2N}} = \frac{N!}{\varepsilon^{2N}}\,r^{\,s-1-2N}, \qquad r \ge 1.$$

Sea $a := -(s-1-2N)$, sabemos que $\int_1^\infty r^{-a}\,dr < \infty$ para $a > 1$. Ahora, como $a = -(s-1-2N) > 1$ (pues $2N > s$), existe $M_2 \ge 0$ tal que:

$$\int_1^\infty r^{s-1}e^{-\varepsilon^2 r^2}\,dr \le \frac{N!}{\varepsilon^{2N}}\int_1^\infty r^{\,s-1-2N}\,dr \le M_2 < \infty.$$

Sumando ambas cotas, $\int_0^\infty r^{s-1}e^{-\varepsilon^2 r^2}\,dr \le M_1 + M_2 < \infty$, luego por Tonelli-Hobson $r \mapsto r^{s-1}\varphi(r) \in L^1[0,\infty)$, y se cumple la hipótesis del Teorema 2.24.

Ahora, veamos que la transformada de Fourier de Φ es positiva. Para ello, elegimos la siguiente convención para la transformada de Fourier: $\widehat{g}(\xi) = \frac{1}{\sqrt{2\pi}}\int_{\mathbb{R}} g(t)\,e^{-i\xi t}\,dt$, y deberemos tener en cuenta que la gaussiana factoriza como $\Phi(x) = \prod_{j=1}^s e^{-\varepsilon^2 x_j^2}$, de modo que su transformada s-dimensional es el producto de las unidimensionales. Para $g(t) = e^{-\varepsilon^2 t^2}$, completando el cuadrado en el exponente,

$$\widehat{g}(\xi) = \frac{1}{\sqrt{2\pi}}\int_{\mathbb{R}} e^{-\varepsilon^2 t^2}e^{-i\xi t}\,dt = \frac{1}{\sqrt{2\pi}}\,e^{-\xi^2/(4\varepsilon^2)}\int_{\mathbb{R}} e^{-\varepsilon^2 z^2}\,dz = \frac{1}{\sqrt{2}\,\varepsilon}\,e^{-\xi^2/(4\varepsilon^2)},$$

donde se ha usado $\int_{\mathbb{R}} e^{-\varepsilon^2 z^2}\,dz = \sqrt{\pi}/\varepsilon$, de modo que $\frac{1}{\sqrt{2\pi}}\cdot\frac{\sqrt{\pi}}{\varepsilon} = \frac{1}{\sqrt{2}\,\varepsilon}$. Multiplicando sobre las s coordenadas,

$$\widehat{\Phi}(\omega) = \prod_{j=1}^s \widehat{g}(\omega_j) = \frac{1}{(\sqrt{2}\,\varepsilon)^s}\,e^{-\|\omega\|_2^2/(4\varepsilon^2)} > 0.$$

Por tanto, por el teorema 2.24, Φ es EDP y radial en $\mathbb{R}^s$ y, por tanto, el problema de interpolación para RBF gaussiano está bien puesto. □

Tras este corolario, ya tenemos garantizado que el problema para RBF gaussiano está bien puesto. Ahora bien, antes de seguir explicando este método será necesario introducir una serie de conceptos clave, algo que haremos a continuación.

2.4.2. Distancia de llenado y orden

Una vez garantizado que el problema está bien puesto, es importante diferenciar que en el RBF hay dos tipos de aproximación: la no estacionaria y la estacionaria. En la primera, que será la única que luego adaptemos al caso de MPU, para un parámetro de forma fijo, vamos aumentando los puntos de muestreo. Por su parte, en la segunda, se va variando el parámetro de forma en función de la distancia entre los puntos de muestreo, que se mide a partir de la distancia de llenado, que definimos justo debajo.

Definición 2.26. Se define la distancia de llenado para un dominio Ω y un conjunto de puntos χ dados, como el radio de la bola abierta más grande que podemos poner en algún punto de nuestro dominio sin que contenga ningún dato de χ. Es decir, $h := h_{\chi,\Omega} = \sup_{x\in\Omega}(\text{ínf}_{x_j\in\chi}\,||x - x_j||)$.

Para calcular la distancia de llenado en Matlab, un primer procedimiento, aunque no el más eficiente, sería aproximar que $h \approx \text{máx}(\text{mín}(B'))$, con $B \in \mathbb{R}^{M\times N}$ la matriz distancia dada por $B_{ij} = ||z_i - x_j||, 1 \leq i \leq M, 1 \leq j \leq N$, con $x_j \in \chi$ y z_j los puntos del mallado equiespaciado en el que también evaluamos los errores.

Así, continuando con lo anterior, en la aproximación estacionaria, para distancias de llenado pequeñas se usan parámetros de forma grandes (funciones base localizadas o de pico agudo), mientras que, para distancias de llenado mayores, se usan parámetros de forma pequeños (funciones base planas).

Ahora, antes de ver todo esto en un experimento, debemos hablar del concepto de orden, que será importante a la hora de analizar los errores cometidos por los métodos.

Definición 2.27. Sean $f : (0, a) \to \mathbb{R}$ y $p > 0$. Se dice que $f(h)$ es de orden al menos p, lo que se denota como $f(h) = \mathcal{O}(h^p)$, si existen constantes $C > 0$ y $\delta \in (0, a)$ tales que

$$|f(h)| \leq C\, h^p \qquad \text{para todo } 0 < h < \delta.$$

Definición 2.28. Sean $f : (0, a) \to \mathbb{R}$ y $p > 0$. Se dice que $f(h)$ es de orden exactamente p, lo que se denota como $f(h) = \Theta(h^p)$, si

$$\lim_{h\to 0} \frac{|f(h)|}{h^p} = C, \qquad 0 < C < \infty.$$

A raíz de estas definiciones, denotemos $E(h)$ una medida del error de aproximación (por ejemplo, el error cuadrático medio RMS o el error máximo en norma L^∞) cometido por el método de interpolación, en función de la distancia de llenado, para dimensión $s \leq 6$ fija.
Supongamos que el número de puntos de Halton es de la forma $N_k = (2^k + 1)^s$, con $k \in \mathbb{N}$ y $E(h) = \Theta(h^p)$, entonces vamos a ver cómo calcular p en la práctica.

Sea $k_0 \in \mathbb{N}$ suficientemente grande para que $2^{k_0} + 1 \approx 2^{k_0}$, y para que $N_0 = (2^{k_0} + 1)^s$ sea suficientemente grande para que, de acuerdo a la sección 2.1 ($s \leq 6$), los puntos de Halton estén bien

distribuidos. Sea $k \geq k_0$, $N = (2^k + 1)^s$. Ahora, recordemos la definición de distancia de llenado: $h = \sup_{x \in \Omega} \left(\text{mín}_{x_j \in \chi}(\|x - x_j\|_2) \right)$. Por tanto,

$$\Omega \subseteq \bigcup_{j=1}^{N} \overline{B}(x_j, h), \text{ pues sea } x \in \Omega, \exists x_j \in \chi : \|x - x_j\| \leq h.$$

Además, sabemos que $\text{Vol}(\overline{B}(x_j, h)) = C_s h^s$. De lo anterior se sigue que

$$\text{Vol}(\Omega) \leq \sum_{j=1}^{N} \text{Vol}(\overline{B}(x_j, h)) = N C_s h^s, \text{ por lo que, } h^s \geq \frac{\text{Vol}(\Omega)}{C_s \cdot N}.$$

Si definimos $C_1 := \left(\frac{\text{Vol}(\Omega)}{C_s} \right)^{1/s}$, llegamos a que $h \geq C_1 \cdot N^{-1/s}$. Ahora, como en nuestro contexto $\Omega = [0, 1]^s$, lo dividimos en una cuadrícula de subcubos idénticos, dividiendo cada eje en $M = N^{1/s}$ (M es natural por la definición de N) partes iguales. Así, se obtienen M^s subcubos de volumen $\frac{1}{M^s}$) y diagonal $d = \sqrt{s}\frac{1}{M}$.

A continuación, observamos que el número de puntos de Halton es igual al número de subcubos: $N = M^s$. Además, hemos asumido que N era suficientemente grande para que los puntos de Halton se distribuyeran bien, luego podemos asumir que hay uno de ellos por cada subcubo. Como la máxima distancia entre los dos puntos más alejados entre sí pertenecientes a dos subcubos contiguos es dos veces la diagonal, tenemos que, si definimos $C_2 := 2\sqrt{s}$, se tiene que $h \leq C_2 N^{-1/s}$ (nos curamos en salud y nos ponemos en caso de tener un punto del dominio en un vértice y el nodo más cercano en la esquina opuesta no del cubo de al lado, sino del que va en diagonal). Así, por lo anterior, llegamos a que

$$C_1 N^{-1/s} \leq h \leq C_2 N^{-1/s}, \text{ para el } k \geq k_0 \text{ fijado al principio.}$$

Por tanto,

$$C_1 N_k^{-1/s} \leq h_k \leq C_2 N_k^{-1/s}, \forall k \geq k_0, \text{ con } N_k = (2^k + 1)^s.$$

Luego, ya sabemos a qué ritmo decrece h con N. Ahora, aunque a priori $h_k N^{1/s}$ podría estar oscilando permanentemente entre C_1 y C_2, asumiremos que si k es suficientemente grande, esta oscilación se estabiliza y, por tanto, $h \approx C \cdot N^{(-1/s)}$. Así, sea $k_1 \geq k_0$, con $k_1 \in \mathbb{N}$ suficientemente grande tal que se cumple lo anterior, se tiene que $h_k \approx C \cdot N_k^{(-1/s)} = C\frac{1}{2^k+1} \approx C\frac{1}{2^k}, \forall k \geq k_1$. Por tanto, se tiene que $h_{k+1} \approx \frac{1}{2} h_k \forall k \geq k_1$.

La otra hipótesis que habíamos supuesto es que $E(h) = \Theta(h^p)$, es decir, que existe $L \in \mathbb{R}$ con $0 < L < \infty$ tal que

$$\lim_{h \to 0} \frac{E(h)}{h^p} = L.$$

Por definición de límite, para cada $\varepsilon > 0$ existe $h_\varepsilon > 0$ tal que $\left| E(h)/h^p - L \right| < \varepsilon$ para $0 < h < h_\varepsilon$. Toamos ε suficientemente pequeño para poder considerar que $E(h) \approx L\, h^p$ para $0 < h < h_\varepsilon$. Elegimos ese ε y ese h_ε.

Por tanto, sea $k_2 \geq k_1$, $k_2 \in \mathbb{N}$, tal que $h_k < h_\varepsilon$ para todo $k \geq k_2$ (existe, pues $h_k \approx C/2^k \to 0$), se razona que $E(h_k) \approx L \cdot h_k^p, \forall k \geq k_2$. Finalmente, llegamos a que

$$E(h_{k+1}) \approx L \cdot h_{k+1}^p \approx \frac{1}{2^p} L \cdot h_k^p \approx \frac{1}{2^p} E(h_k), \qquad \forall k \geq k_2.$$

Y, por tanto, concluimos que

$$p \approx \frac{\ln \frac{E(h_{k+1})}{E(h_k)}}{\ln(1/2)} = \log_2 \frac{E(h_k)}{E(h_{k+1})}. \tag{2.1}$$

2.4.3. Condicionamiento: amplificación del error y cálculos en Matlab

En este apartado veremos cómo se amplifican los errores, la dependencia con el número de condición y cómo afecta esto a los experimentos numéricos en Matlab.

Empecemos diciendo que, la noción de matriz "computacionalmente singular", que más adelante, a lo largo de esta subsección, definiremos rigurosamente, se cuantifica mediante el número de condición espectral $\kappa_2(A) = \|A\|_2 \, \|A^{-1}\|_2$, del que ya hablamos anteriormente y que, para una matriz simétrica e invertible, coincide, por el corolario 2.17, con el cociente de autovalores $\frac{\text{máx}\,|\lambda_i|}{\text{mín}\,|\lambda_i|}$.

El interés de $\kappa_2(A)$ es que mide cuánto se amplifica el error al resolver el sistema $Ac = y$. Ahora, si planteamos el sistema perturbado $(A+\delta A)(c+\delta c) = y+\delta y$, se llega a que $Ac+A\delta c+\delta Ac+\delta A\delta c = y+\delta y$. Quitando perturbaciones a segundo orden y empleando que $Ac = y$ obtenemos que $A\delta c + \delta Ac \approx \delta y$, por lo que $\delta c \approx A^{-1}\delta y - A^{-1}\delta A\, c$. Ahora, tomando normas y aplicando la desigualdad triangular y que $\|A^{-1}\delta A\, c\| \leq \|A^{-1}\| \, \|\delta A\| \, \|c\|$, llegamos a $\|\delta c\| \leq \|A^{-1}\| \, \|\delta y\| + \|A^{-1}\| \, \|\delta A\| \, \|c\|$. Finalmente, dividiendo por $\|c\|$:

$$\frac{\|\delta c\|}{\|c\|} \leq \|A\| \, \|A^{-1}\| \left(\frac{\|\delta y\|}{\|A\| \, \|c\|} + \frac{\|\delta A\|}{\|A\|} \right) \leq \kappa_2(A) \left(\frac{\|\delta y\|}{\|y\|} + \frac{\|\delta A\|}{\|A\|} \right).$$

El razonamiento anterior nos dice que el error relativo de la solución puede llegar a ser $\kappa_2(A)$ veces la suma de los errores relativos de la matriz y los datos. Notemos que, mientras los datos suelen estar sometidos a errores de redondeo y de ruido (este suele ser el mayor), la matriz solo está sujeta a errores de redondeo, por lo que muchas veces este último término se desprecia, quedando: $\frac{\|\delta c\|}{\|c\|} \leq \kappa_2(A) \frac{\|\delta y\|}{\|y\|}$.

Ahora, si recordamos, el error de la solución (despreciando el término δA) es $\delta c = A^{-1}\delta y$, luego la dirección donde este más crece es la del autovalor más pequeño: en ella A^{-1} amplifica por el factor $1/\,\text{mín}\,|\lambda_i|$. Si $\text{mín}\,|\lambda_i|$ es minúsculo, ese factor es enorme y una perturbación diminuta del dato con componente en esa dirección aumenta desmesuradamente en la solución. Y este es precisamente el escenario de la interpolación gaussiana con ε pequeño: la columna k-ésima de la matriz es $(\Phi(x_i - x_k))_i$, y con ε pequeño la gaussiana $e^{-\varepsilon^2 r^2}$ es muy plana, de modo que, si dos nodos x_k y $x_{k'}$ están próximos, las distancias $x_i - x_k$ y $x_i - x_{k'}$ son casi iguales para todo i y las dos columnas correspondientes resultan casi idénticas. Columnas casi linealmente dependientes significan que existe un vector unitario v con $Av \approx 0$, esto es, un autovector de autovalor casi nulo: $\lambda_{\text{mín}} \to 0$. Por otra parte, como A tiene

diagonal unidad ($\Phi(0) = 1$) y es definida positiva, su mayor autovalor está acotado por la traza, $\lambda_{\text{máx}} \leq \sum_i \lambda_i = \text{tr}(A) = N$, con N el número de nodos. Con el numerador acotado por N, se tiene que $\lambda_{\text{mín}} \to 0$ si y solo si el cociente $\kappa_2 = \lambda_{\text{máx}}/\lambda_{\text{mín}} \to \infty$: en el caso gaussiano, un $\lambda_{\text{mín}}$ minúsculo equivale a un número de condición enorme. De esta forma, hemos reexpresado en términos de autovalores lo que ya comentamos sobre el condicionamiento en función del valor de ε.

En el extremo opuesto, cuando todos los autovalores son parecidos y en particular $\lambda_{\text{máx}} \approx \lambda_{\text{mín}}$ (cociente próximo a 1), la matriz escala casi por igual en todas las direcciones. De hecho, como su traza vale N y reparte ese valor entre N autovalores casi iguales, todos ellos valen aproximadamente uno, de manera que A es casi un múltiplo de la identidad, $A \approx I$. Así, en este caso, el elipsoide es casi una esfera y ni A ni A^{-1} poseen una dirección donde amplifiquen sin control. Así es que, concluimos que el caso de mejor condicionamiento posible es $\kappa_2 \approx 1$.

Todo lo que hemos dicho hasta ahora es aplicable, con algunos matices, también para otros casos de matrices de la forma $A_{ij} = \varphi(||x_i - x_j||)$, con φ de la forma de la definición 2.10, siempre que, de acuerdo al corolario 2.17, dichas matrices sean simétricas reales e invertibles.

Debemos observar que, aunque ahora no hay parámetro de forma, igualmente, si dos nodos están "muy" juntos (la cercanía o lejanía de los nodos es relativa y depende de la forma de φ), mín $|\lambda_i| \to 0$. Sin embargo, aquí no podemos controlar máx $|\lambda_i|$ con la traza (no tenemos garantizado que todas los autovalores tengan el mismo signo). No obstante, si de alguna forma somos capaces de imponer que máx $|\lambda_i|$ esté acotado conforme crece N (esto es lo que ocurre en los subsistemas locales del MPU con $\Phi = r^p$, como veremos llegado el momento), entonces igual que pasaba en el caso gaussiano, $\lambda_{\text{mín}} \to 0$ si y solo si el cociente $\kappa_2 = \lambda_{\text{máx}}/\lambda_{\text{mín}} \to \infty$. Por otro lado y, aunque a priori para el caso de $\varphi(r) = r^p$ en RBF global no tenemos ninguna cota sobre los autovalores, numéricamente se observa que el condicionamiento empeora conforme aparecen nodos más juntos (lo veremos).

Una vez entendido cómo afecta el número de condición a la propagación de errores, estudiemos las consecuencias en aritmética de coma flotante. En doble precisión cada número se almacena con unas 16 cifras significativas: lo que se mantiene fijo no es el error absoluto sino el relativo, del orden de 10^{-16}, con independencia de la magnitud del número. Así, tanto al representar el término independiente y como al construir la matriz A, cuyas entradas $\Phi(x_i - x_j)$ se redondean, se introducen errores relativos $\|\delta y\|_2/\|y\|_2 \approx \|\delta A\|_2/\|A\|_2 \approx 10^{-16}$. La cota de perturbación a primer orden, como hemos visto, es

$$\frac{\|\delta c\|_2}{\|c\|_2} \;\leq\; \kappa_2(A)\left(\frac{\|\delta y\|_2}{\|y\|_2} + \frac{\|\delta A\|_2}{\|A\|_2}\right) \;\approx\; \kappa_2(A)\cdot 10^{-16}.$$

Ahora, puesto que el número de cifras significativas correctas de una cantidad es aproximadamente $-\log_{10}$ de su error relativo (± 1), se sigue la regla práctica de que se pierden aproximadamente $\log_{10}\kappa_2(A)$ cifras: si $\kappa_2(A) \approx 10^k$, quedan unas $16 - k$ cifras fiables.

Llegados a este punto, conviene precisar en qué sentido usamos el término "computacionalmente singular": lo aplicamos a dos situaciones que, a efectos prácticos, consideraremos equivalentes, porque en ninguna de ellas obtenemos una solución aprovechable. La primera es que la solución carezca de

cifras fiables aun cuando el solver devuelva un vector finito. En la segunda, nos referimos al caso en el que el algoritmo de resolución falla directamente, devolviendo `Inf` o `NaN`, o el aviso de singularidad de MATLAB. No son fenómenos distintos, sino dos manifestaciones de un mismo deterioro: para nuestra matriz, que es definida positiva, la cadena de equivalencias

$$\text{nodos próximos y } \varepsilon \to 0 \text{ (en caso gaussiano) / columnas casi iguales} \iff \text{mín}\,|\lambda_i| \to 0^+ \iff$$

$$\iff \kappa_2 = \frac{\text{máx}\,|\lambda_i|}{\text{mín}\,|\lambda_i|} \to \infty \iff \text{cifras fiables} \to 0,$$

válida en los casos donde el numerador $\text{máx}\,|\lambda_i|$ está acotado (como en el método gaussiano o en los subsistemas locales del MPU), lo que muestra que perder cifras y acercarse a la singularidad son el mismo proceso visto por sus dos extremos. Teóricamente, el parámetro $\text{mín}\,|\lambda_i|$ varía de forma continua con los nodos y con ε y, en aritmética exacta, nunca alcanza el cero. Ahora bien, el comportamiento en el ordenador no hereda esa continuidad, pues la aritmética de coma flotante es discreta: dos nodos distintos pero muy próximos pueden, al redondearse sus distancias, producir columnas idénticas y un pivote nulo, de modo que la matriz se vuelva singular a partir de una cierta precisión en las distancias entre estos nodos. Habitualmente esto sucede al final de un tránsito gradual cuya cota es $\kappa_2(A) \approx 10^{16}$, donde no queda ninguna cifra fiable. En cualquiera de los dos casos anteriores, la solución resulta inútil y la matriz debe descartarse.

Veamos, por último, cómo detectamos esta situación en la práctica. MATLAB no calcula κ_2 directamente, sino una estimación de su inverso: $\texttt{RCOND}(A) \approx 1/\kappa_2(A) \in [0, 1]$. Así, al estimar $1/\kappa_2$, `RCOND` cuantifica el condicionamiento de la matriz, es decir, las cifras fiables que conserva la solución: $16 + \log_{10} \texttt{RCOND}(A)$; luego no es, en sentido estricto, un test de invertibilidad, pues una matriz invertible pero mal condicionada tiene `RCOND` pequeño aunque estrictamente positivo. Sin embargo, por la cadena de equivalencias anterior, en nuestra matriz definida positiva la pérdida de cifras y la proximidad a la singularidad coinciden: ambas equivalen a $\text{mín}\,|\lambda_i| \to 0^+$, de modo que un `RCOND` próximo a 0 señala a la vez las dos cosas. Se emplea el inverso de κ_2 por comodidad: una matriz singular tendría $\kappa_2 = \infty$, no representable, mientras que su inverso es 0, perfectamente representable, y así el indicador decae con suavidad hacia 0 al aproximarse la matriz a ese régimen.

En base a lo anterior, notemos que la condición que emplearemos en nuestros scripts, $\texttt{RCOND}(A) < 10^{-13}$, descarta las matrices con $\kappa_2 \gtrsim 10^{13}$, esto es, aquellas en las que quedarían menos de unas tres cifras fiables. Es decir, se elige el umbral 10^{-13}, y no el límite teórico 10^{-16}, como margen de seguridad y como definición de la precisión que consideramos aceptable: unas 3 cifras significativas. Además, de esta forma, como el régimen casi singular se intercepta antes de alcanzarse el exactamente singular, evitamos llegar al caso de matrices singulares en el ordenador. Para el caso gaussiano, esta es la traducción operativa de lo que se llama el rango "seguro"/"inseguro" del parámetro de forma: el ε "seguro" más pequeño es el menor ε para el que `RCOND` aún no ha cruzado dicho umbral.

Ahora que ya sabemos cómo medir la estabilidad y condicionamiento de nuestro método, hacemos un experimento numérico para ver qué tal funciona en la práctica.

2.4.4. EL RBF gaussiano en la práctica

Realizamos un primer experimento en el que compararemos el RBF gaussiano en el caso no estacionario ($\varepsilon = 21.1$) con la interpolación por matrices distancia en el caso 2D, eligiendo el número de puntos de Halton $N = (2^k + 1)^2, k \in \mathbb{N}$ y la función test de Franke en su formulación como combinación lineal de gaussianas, habitual en la literatura de interpolación con funciones de base radial (véase, p. ej., [16] o [7]). Esta variante difiere de la definición original de [8] en el segundo sumando, donde la dependencia en y es aquí gaussiana, $e^{-\frac{1}{10}(9y+1)^2}$, en lugar del término lineal $e^{-\frac{1}{10}(9y+1)}$ de la formulación original. De esta forma nuestra función de Franke es

$$f(x,y) = \frac{3}{4}e^{(-1/4)((9x-2)^2+(9y-2)^2)} + \frac{3}{4}e^{(-1/49)(9x+1)^2-(1/10)(9y+1)^2} + \frac{1}{2}e^{(-1/4)((9x-7)^2+(9y-3)^2)} - \frac{1}{5}e^{-(9x-4)^2-(9y-7)^2}.$$

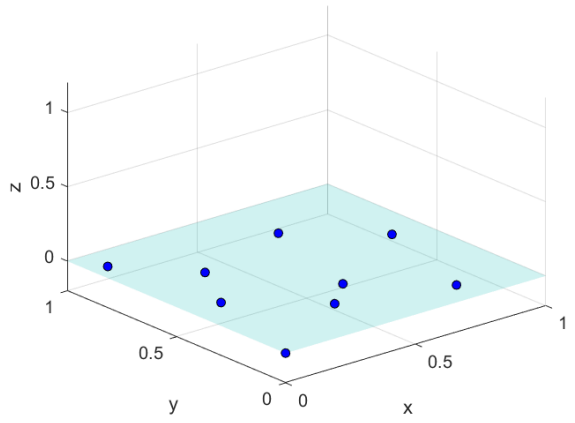
Así, mediante el script 10, se obtiene la tabla 2.2, en la que como podemos observar, se ha pretendido calcular el orden del error cometido por el método de interpolación, usando la ecuación (2.1). También se han obtenido, por medio de este script, las gráficas que se sitúan bajo dicha tabla (figuras 2.3 y 2.4).

		gaussiano			Matrices distancia		
k	N	RMS-error	p	max-error	RMS-error	p	max-error
1	9	3.631580e-01		1.039682e+00	1.228070e-01		4.025445e-01
2	25	3.166165e-01	0.1979	9.670980e-01	5.673937e-02	1.1140	2.542878e-01
3	81	2.104427e-01	0.5893	8.455161e-01	1.323935e-02	2.0995	6.608256e-02
4	289	6.475114e-02	1.7004	4.934133e-01	3.705096e-03	1.8372	3.090488e-02
5	1089	7.000750e-03	3.2093	8.231831e-02	1.072176e-03	1.7890	1.463254e-02
6	4225	3.426699e-05	7.6745	5.000767e-04	3.435092e-04	1.6421	5.130539e-03
7	16641	3.874205e-07	6.4668	1.396545e-05	1.270952e-04	1.4344	1.802775e-03

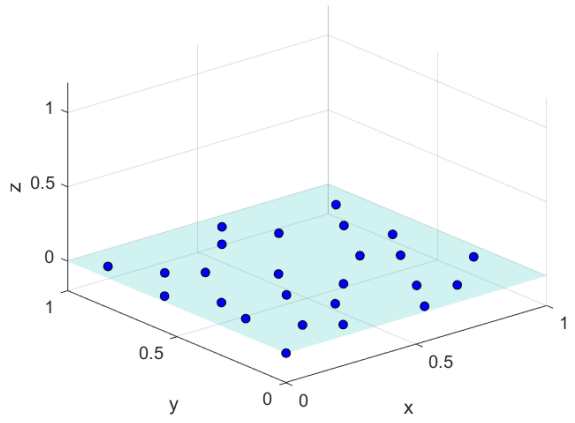
Tabla 2.2: RMSE, error máximo y orden p en función del número de puntos de Halton (N) para el RBF gaussiano no estacionario ($\varepsilon = 21.1$) e interpolación por matrices distancia para la función test de Franke.

Es importante notar que se requiere ε grande cuando N es grande para que el problema esté bien condicionado. Por eso hemos elegido en nuestro ejemplo un parámetro de forma de 21.1. Sin embargo, con este parámetro de forma tan grande, para pocos puntos Halton, la aproximación es bastante pobre, como vemos, principalmente, en las gráficas 2.3c y 2.3d, donde se puede observar que las gaussianas de la base radial caen justo sobre los puntos de Halton generados (gráficas 2.3a y 2.3b), pero debido a lo estrechas que son (varianza pequeña), la función interpoladora no es capaz de aproximar bien la función de Franke en todo el dominio.

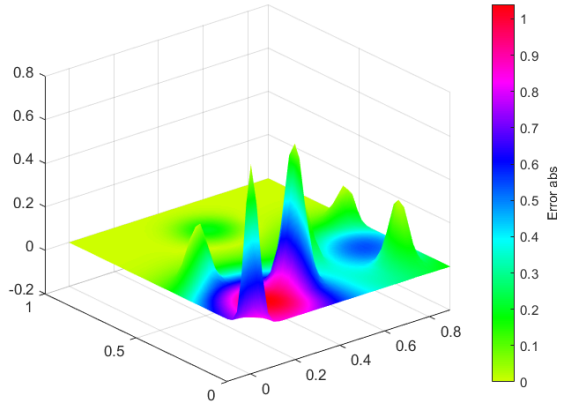
Conforme aumentamos el número de puntos de Halton, esto se va corrigiendo y, aunque en un



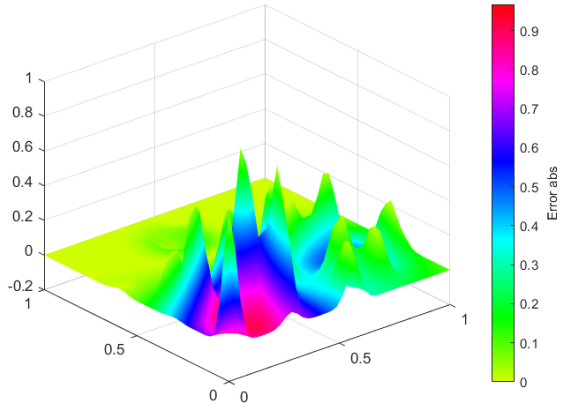
(a) Distribución de puntos de Halton para $k = 1$, $N = 9$.



(b) Distribución de puntos de Halton para $k = 2$, $N = 25$.

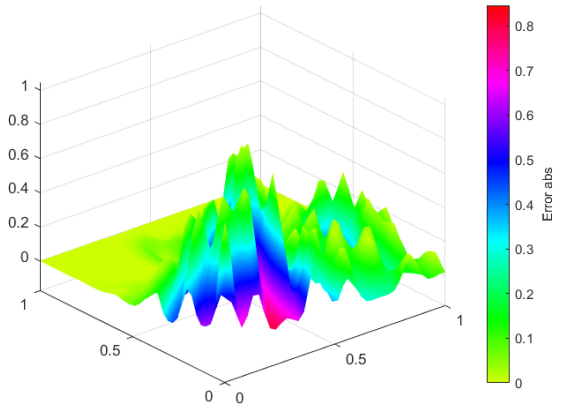


(c) RBF gaussiano en función del error absoluto ($N = 9$).

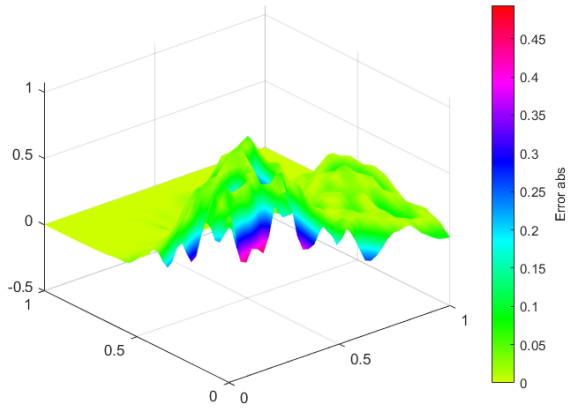


(d) RBF gaussiano en función del error absoluto ($N = 25$).

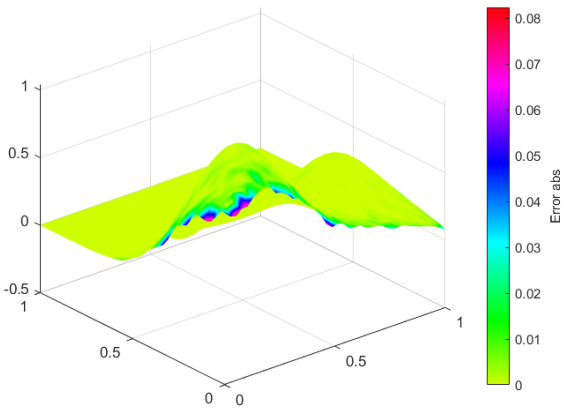
Figura 2.3: Puntos de Halton y error absoluto de la interpolación con RBF gaussiano no estacionario ($\varepsilon = 21.1$) para la función de Franke con $N = 9$ y $N = 25$.



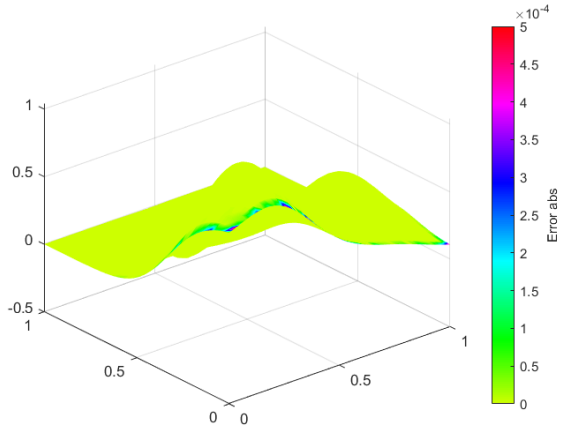
(a) RBF gaussiano en función del error absoluto ($N = 81$).



(b) RBF gaussiano en función del error absoluto ($N = 289$).



(c) RBF gaussiano en función del error absoluto ($N = 1089$).



(d) RBF gaussiano en función del error absoluto ($N = 4225$).

Figura 2.4: Error absoluto en la interpolación con RBF gaussiano no estacionario ($\varepsilon = 21.1$) sobre la función de Franke al aumentar el número de puntos de Halton.

primer momento la interpolación por matrices distancia funcionaba mejor, al aumentar el número de puntos observamos que esta diferencia se va reduciendo (ver tabla 2.2).

Por otro lado, abordamos también un ejemplo de RBF gaussiano 2D en aproximación estacionaria, para la misma función de Franke de antes, con $N = (2^k + 1)^2$, nuevamente. Para cada k, buscamos el "mejor" valor para nuestro parámetro de forma, aplicando el siguiente criterio.

De acuerdo con lo que explicamos en 2.4.3, elegimos el valor de ε "seguro" más pequeño posible, es decir, el más pequeño para el que el problema en Matlab no es mal condicionado. A modo de recordatorio, por debajo de este valor se dice que estamos en el rango "inseguro", mientras que por encima de él, estaremos en el rango "seguro". Este mal condicionamiento es debido a que, si N es muy grande y ε pequeño, las funciones de nuestra base radial son muy planas y nuestros puntos de Halton están muy juntos, por lo que la matriz del sistema pasa a tener columnas muy parecidas entre sí y, debido a la precisión del ordenador, por debajo de cierto valor de ε, aunque la matriz sea invertible, se convierte en computacionalmente singular. Esto en Matlab se mide con la función `RCOND` de nuestra matriz y, para valores de esta cercanos a la precisión computacional, se considera que la matriz es singular.

Todo lo anterior está implementado en las dos últimas partes del script que usamos también para el RBF gaussiano no estacionario (10), dando lugar a la tabla 2.3 y a las gráficas de la figura 2.5. En ella se observa que el RMS-error en función del parámetro de forma no es una función monótona, sino que tiene un mínimo y, además, cuanto mayor es el número de puntos de Halton, más grande debe ser ε para lograr estabilidad.

k	N	ε	RMS-error	p	max-error
1	9	0.04	3.636553e-01		2.324805e+00
2	25	0.49	3.614354e-01	0.0088	2.651237e+00
3	81	2.16	7.907560e-02	2.1924	7.220760e-01
4	289	5.48	3.889537e-04	7.6675	6.727017e-03
5	1089	11.81	1.701057e-04	1.1932	3.141179e-03
6	4225	23.50	1.305652e-04	0.3817	2.003594e-03

Tabla 2.3: Valor del ε "seguro" más pequeño, junto con sus errores y el orden (p) del método en el caso de interpolación por RBF gaussiano estacionario a la función de Franke.

Ahora, si comparamos las tablas obtenidas por aproximación no estacionaria 2.2 y estacionaria 2.3, se observa, en primer lugar, que en la segunda, los errores que nos da el método no se van mucho de los errores dados por matrices distancia (máximo un orden arriba o abajo). Además, en cuanto a los órdenes del error RMS, mientras que en el caso de matrices distancia p parece tender a uno y en el caso de RBF gaussiano no estacionario parece crecer sin límite, en el caso estacionario no está claro qué ocurre con el orden (esto tiene que ver con que aquí ε no está fijo, sino que va variando).

Una vez comentado este experimento numérico vamos a introducir un último apartado en este capítulo que nos ayudará a entender mejor los órdenes de convergencia de los métodos que hemos

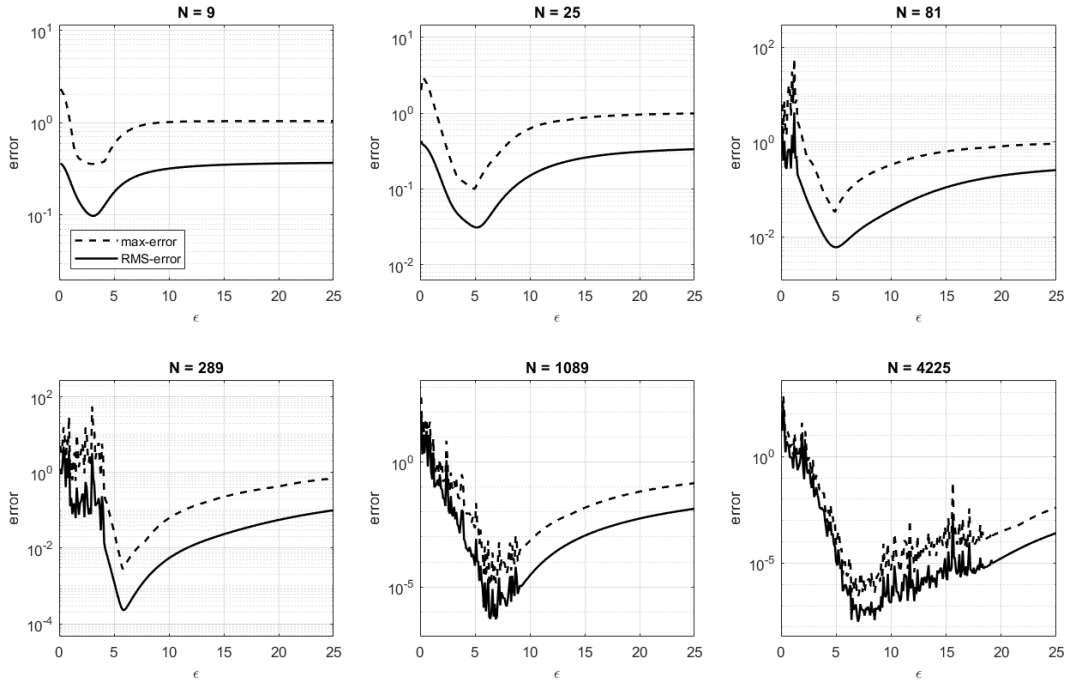


Figura 2.5: RMS-error y error máximo en función del parámetro de forma (ε) para N puntos de Halton en el RBF gaussiano estacionario aplicado a la función de Franke.

estudiado hasta ahora: tanto en el caso de matrices distancia como en el de gaussiano.

2.4.5. Convergencia del RBF

Con el objetivo de establecer unas garantías teóricas de convergencia para nuestros métodos, se recogen dos resultados que nos ayudarán a comprender mejor el comportamiento del error en la interpolación por RBF gaussiano y por matrices de distancias, sirviendo como base teórica para las comparativas posteriores. Sus demostraciones formales se omiten, dado que su desarrollo exigiría la introducción de Espacios de Sobolev, la Teoría de Espacios Nativos y el uso de Transformadas de Fourier multivariables, lo cual, aunque se introducirá brevemente más adelante, en su mayoría escapa al alcance de este trabajo.

Para enunciar estos teoremas se requiere una definición previa.
Definición 2.29 (Frontera Lipschitz)**.** Sea $\Omega \subset \mathbb{R}^s$ abierto y acotado, se dice que tiene frontera Lipschitz si para cada $x_0 \in \partial\Omega$ existen un entorno U de x_0, un sistema ortonormal (y', y_s) y una función $\gamma\colon \mathbb{R}^{s-1} \to \mathbb{R}$ Lipschitz, $|\gamma(y') - \gamma(z')| \leq L\|y' - z'\|$, tales que en esas coordenadas $U \cap \Omega = \{y \in U : y_s < \gamma(y')\}$ y $U \cap \partial\Omega = \{y \in U : y_s = \gamma(y')\}$.

A partir de ahora, exigiremos, en esta sección, que Ω tenga frontera Lipschitz. La definición 2.29 es sencilla de verificar para los dominios habituales en interpolación. En particular, tienen frontera Lipschitz:

- todo dominio convexo acotado [9, Corolario 1.2.2.3]
- todo polígono o poliedro acotado, aun cuando presente esquinas o aristas vivas (ángulo bien definido) [14, Capítulo 3]
- todo dominio de clase C^1, como la bola o el elipsoide [1, Capítulo 4]
- uniones, intersecciones y diferencias finitas de los anteriores, siempre que no se generen cúspides (intersecciones con ángulo cero) ni puntos en los que el dominio se autotoque (como una pajarita, en la que en un punto se intersecan dos cuadrados) [9, Sección 1.2].

La condición Lipschitz admite, por tanto, esquinas y aristas (pendientes acotadas) pero excluye cúspides (pendiente infinita) [1, Sección 4.8], grietas o cortes en los que el dominio se acerque a su propia frontera por dos lados [9, Sección 1.7], y configuraciones en las que dos componentes se peguen únicamente por un vértice. Como contraejemplos clásicos sirven el dominio con cúspide $\{(x, y) : 0 < y < x^2,\ 0 < x < 1\}$ en el origen, el disco con corte $B(0, 1) \setminus \{(x, 0) : 0 \leq x < 1\}$ y la figura en forma de pajarita formada por dos cuadrados unidos en un único vértice.

En este trabajo los dominios considerados (bolas, cajas, polígonos acotados) satisfacen automáticamente esta hipótesis. Una vez introducido este concepto, introducimos los dos resultados que estábamos

buscando.

Teorema 2.30 (Cota de error para matrices de distancia)**.** [25, Teo. 11.16] Sea $\Omega \subseteq \mathbb{R}^s$ un dominio acotado con frontera Lipschitz y h la distancia de llenado de los nodos. Interpolamos f mediante $\Phi(r) = ||\cdot||^p$ con p impar. Si $f \in C^{\lceil (p+s)/2 \rceil}(\overline{\Omega})$, el error máximo está acotado por

$$\|f - P_f\|_{L^\infty(\Omega)} \;\leq\; C\, h^{p/2},$$

donde $C = C(f) > 0$ es independiente de h, agrupa los factores geométricos del dominio y de los nodos, y depende de f a través de sus derivadas hasta el orden $\lceil p + s/2 \rceil$.

Teorema 2.31 (Cota de error para el RBF gaussiano)**.** [25, Teo. 11.22] Sea $\Omega \subseteq \mathbb{R}^s$ un dominio acotado con frontera Lipschitz y h la distancia de llenado de los nodos. Para f que admite una extensión entera $f : \mathbb{C}^s \to \mathbb{C}$ de tipo exponencial, es decir, tal que $|f(z)| \leq A\, e^{\gamma \|\operatorname{Im} z\|}$ para ciertas constantes $A, \gamma > 0$ y todo $z \in \mathbb{C}^s$, existe un umbral $h_0 > 0$ tal que, para $h < h_0$, el error máximo de la interpolación RBF gaussiana no estacionaria decae exponencialmente,

$$\|f - P_f\|_{L^\infty(\Omega)} \leq C\, e^{-c/h},$$

donde $c > 0$ está inversamente ligada al parámetro de forma ε, y $C = C(f) > 0$ es independiente de h, agrupa factores geométricos y depende de f a través de toda su sucesión de derivadas.

Para la correcta aplicación en la práctica de los dos resultados anteriores, debemos introducir la siguiente proposición y una nota posterior.

Proposición 2.32. Cuando la convergencia no es al menos de orden exponencial (como la del teorema 2.31), sino que se asume exactamente exponencial, el orden observado, calculado de acuerdo a la ecuación (2.1) (esta asumía orden polinómico), diverge a infinito.

Demostración. Asumiendo $h_{k+1} = h_k/2$ y un error $E(h) = Ce^{-c/h}$ (con $c > 0$), evaluamos p_{obs}:

$$\begin{aligned} p_{\text{obs}} &= \log_2\left(\frac{e^{-c/h_k}}{e^{-c/(h_k/2)}}\right) = \log_2\left(\frac{e^{-c/h_k}}{e^{-2c/h_k}}\right) \\ &= \log_2\left(e^{c/h_k}\right) = \frac{c}{h_k \ln 2} \end{aligned}$$

Conforme el paso de malla se hace infinitamente pequeño ($h_k \to 0$):

$$\lim_{h_k \to 0} p_{\text{obs}} = \lim_{h_k \to 0} \frac{c}{h_k \ln 2} = +\infty$$

Así, cuando la convergencia es exponencial, el orden observado diverge a infinito. □

Nota 2.33 (El orden de convergencia para el RMS-error)**.** Cabe notar que, aunque los teoremas 2.30 y 2.31 acotan el error en la norma del máximo, $\|f - P_f\|_{L^\infty(\Omega)}$, en los experimentos numéricos hemos calculado el error cuadrático medio (RMSE) sobre un conjunto de puntos de evaluación $\{y_1, \ldots, y_M\}$,

$$\text{RMSE} = \left(\tfrac{1}{M}\textstyle\sum_{i=1}^{M} |e(y_i)|^2\right)^{1/2}, \qquad e := f - P_f.$$

Ahora bien, como $|e(y_i)|^2 \leq \|e\|^2_{L^\infty(\Omega)}$,

$$\text{RMSE}^2 = \frac{1}{M}\sum_{i=1}^{M}|e(y_i)|^2 \;\leq\; \frac{1}{M}\cdot M \cdot \|e\|^2_{L^\infty(\Omega)} = \|e\|^2_{L^\infty(\Omega)},$$

por lo que RMSE $\leq \|e\|_{L^\infty(\Omega)}$ y, consecuentemente, las cotas de los teoremas anteriores se aplican también sobre el RMSE.

Ahora bien, aunque no está garantizado que el error máximo y el RMS-error tengan el mismo orden de convergencia, si experimentalmente vemos que el orden de convergencia del RMS-error es exactamente el dado por la cota del teorema, entonces este será exactamente igual al del error máximo

Tras estas apreciaciones, observemos que, tal y como hemos obtenido en los experimentos numéricos para la función de Franke (es C^∞), el teorema 2.30 nos dice que la interpolación mediante matrices de distancias presenta una tasa de convergencia puramente polinomial acotada por el grado p de $\Phi = ||\cdot||^p$, con p impar.

Por su parte, si consideramos el denominado espacio nativo de Φ, que definiremos más adelante en 3.10, las hipótesis del teorema 2.31 pueden rebajarse a que f pertenezca a este espacio nativo (ver A.6) y, como la función de Franke está en él (por el corolario A.9), tenemos el orden de convergencia garantizado por el teorema. Además, si asumimos que la convergencia que nos da el teorema es exactamente exponencial (no es solo una cota, sino aproximadamente igual), por la proposición 2.32, el orden observado crece indefinidamente.

Para finalizar, y más allá de esta convergencia más rápida del RBF gaussiano, este también tiene sus desventajas, y es que, dado que la constante c exige valores de $\varepsilon \to 0$ para que el método converja lo antes posible, esto genera problemas de mal condicionamiento en sistemas globales. Esto puede corregirse usando otros métodos RBF más estables y que mantienen o mejoran la precisión de la aproximación. Otra desventaja común a todos los métodos RBF globales es que requieren la resolución de una matriz $N \times N$ densa, por lo que el coste del método es proporcional a N^3, lo que hace que su escalabilidad sea mala.

Esto es lo que nos lleva a la búsqueda de métodos que mejoren el condicionamiento y la escalabilidad de los de interpolación mediante RBF, conservando o mejorando su convergencia. Una posible solución se basaría en tratar de encontrar un método sin malla que utilice interpolantes locales de forma similar a los del RBF, con un número acotado de nodos para construir cada interpolante local, limitando de esta forma el coste de la resolución de los sistemas lineales, que es lo más costoso del método. Además, nos gustaría que este método estuviese lo mejor condicionado posible y tuviera un buen orden de convergencia. Es así como nacen los Métodos de Partición de la Unidad (MPU), que trataremos en el siguiente capítulo.

Capítulo 3

Métodos de partición de la unidad

Con el objetivo de superar las desventajas de los métodos globales discutidas anteriormente, introducimos el marco metodológico que constituirá el eje principal de este trabajo: los Métodos de Partición de la Unidad (MPU). Su propósito fundamental es conservar la alta precisión y rápida convergencia de las Funciones de Base Radial (del caso gaussiano, principalmente), pero interpolando en entornos locales, de forma que el método resulte matemáticamente estable y computacionalmente escalable.

A nivel histórico, estos métodos surgen formalmente en la década de los noventa en el contexto de la búsqueda de aproximaciones robustas para la resolución de ecuaciones en derivadas parciales. Sin embargo, investigadores pioneros como Richard Franke, ya nombrado en secciones anteriores, habían propuesto décadas antes algoritmos de naturaleza muy similar aplicados al problema específico del ajuste de datos bidimensionales dispersos. Hoy en día, los MPU se han consolidado como una herramienta altamente versátil dentro de la familia de los métodos sin malla.

La premisa estructural de los MPU se fundamenta en un enfoque de ensamblaje local-global. En lugar de intentar resolver un único problema global susceptible a inestabilidades, el proceso comienza tomando un dominio abierto y acotado $\Omega \subset \mathbb{R}^s$, el cual está recubierto por M subdominios locales solapados (comúnmente denominados parches) Ω_j. Para que estos parches formen una cubierta válida, deben cubrir por completo el dominio original:

$$\Omega \subseteq \bigcup_{j=1}^{M} \Omega_j.$$

Es un requisito geométrico fundamental del método que estos subdominios se solapen, garantizando zonas de transición donde la intersección no es vacía.

Como veremos más adelante, la potencia de esta formulación radica en la independencia computacional de cada parche, en el que construimos un aproximante local $u_j(x)$ que se ajusta a los datos de esa región. La elección de este método local es libre, pudiéndose emplear, por ejemplo, una interpolación mediante RBF a pequeña escala, garantizando una altísima precisión sin riesgo de colapsar el sistema.

Posteriormente, estas soluciones locales independientes deben ensamblarse de forma continua. Para ello, se asocia a la cubierta una familia de funciones de peso $\{w_j\}_{j=1}^{M}$ que deben ser continuas, no negativas y poseer soporte compacto estrictamente en $\overline{\Omega}_j$. Para que esta familia constituya una partición de la unidad debe cumplir:

$$\sum_{j=1}^{M} w_j(x) = 1, \quad \forall x \in \Omega.$$

Finalmente, el aproximante global continuo se construye mediante la superposición de las aproximaciones locales ponderadas por sus respectivas funciones de la partición de la unidad:

$$P_f(x) = \sum_{j=1}^{M} u_j(x) w_j(x), \quad x \in \Omega.$$

A continuación, nuestro objetivo será cuantificar el error cometido por los MPU y, para ello, disponemos una serie de definiciones y resultados, principalmente relacionados con los recubrimientos, el análisis matemático, los espacios de Hilbert y las series de Fourier, que nos permitirán alcanzar nuestro objetivo.

3.1. Condiciones sobre el recubrimiento

Para que nuestra teoría posterior de convergencia sobre los MPU sea válida, se requieren una serie de condiciones sobre el recubrimiento, que se introducen en esta sección.

Definición 3.1. Se dice que $\chi = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^s$ es d-unisolvente si el único polinomio de grado como máximo d, que vale 0 evaluado en los elementos de χ, es el polinomio cero.

Definición 3.2 (Condición de cono interior)**.** $V \subseteq \mathbb{R}^s$ satisface una condición de cono interior si existen un ángulo $\theta \in (0, \pi/2)$ y un radio $r > 0$ tales que para cada $x \in V$ existe un vector unitario $\xi(x)$ con

$$C(x, \xi(x), \theta, r) = \left\{ x + \lambda y : \ \|y\|_2 = 1, \ y^{\mathsf{T}} \xi(x) \geq \cos\theta, \ \lambda \in [0, r] \right\} \subseteq V.$$

Es decir, existen $r > 0$ y $\theta \in (0, \pi/2)$ fijos tales que para todo $x \in V$, existe un cono con vértice en x, ángulo θ y radio r, dentro de V

Definición 3.3 (Recubrimiento regular [25])**.** Una familia de abiertos acotados $\{\Omega_j\}$ es un recubrimiento regular de (Ω, χ) para Π_{m-1}^s si cubre Ω y cumple las siguientes condiciones.

1. Existe K tal que cada $x \in \Omega$ pertenece a lo sumo a K de los Ω_j (solapamiento acotado).

2. Cada Ω_j satisface una condición de cono interior, según la definición 3.2, con ángulo y radio uniformes.

3. Las distancias de llenado locales h_{χ_j,Ω_j}, con $\chi_j = \chi \cap \Omega_j$, están uniformemente acotadas por la distancia de llenado global $h_{\chi,\Omega}$. Notemos que esto implica que $\chi_j := \chi \cap \Omega_j \neq \emptyset, \forall j$, pues si para algún j, la intersección fuese vacía, la distancia de llenado local $h_{\chi_j,\Omega_j} = \sup_{x\in\Omega_j}(\text{ínf}_{x_j\in\chi_j} ||x - x_j||) = +\infty$ y ya no estaría acotada por $h_{\chi,\Omega}$, que es finita. Esto garantiza la 0-unisolvencia de $\chi_j = \chi \cap \Omega_j$. Sin embargo, para un m genérico ($m \geq 2$) se requiere la condición 4.

4. Cada conjunto local $\chi_j = \chi \cap \Omega_j$ es $(m-1)$-unisolvente (definición 3.1). Notemos que, aunque esta última condición no está en la definición original de Wendland [25], aquí se añade para garantizar que el interpolante que posteriormente construiremos reproduce los polinomios en Π^s_{m-1}.

3.1.1. Construcción explícita de un recubrimiento regular

La definición 3.3 de recubrimiento regular resulta algo abstracta. Veamos que, en nuestro caso, $\Omega = (0,1)^s$ y nodos de Halton, admite una construcción particularmente sencilla y que usaremos de ahora en adelante, sobre la que podemos verificar de forma explícita las cuatro condiciones. Se sitúan los centros $\{c_j\}$ de los parches en una rejilla uniforme de paso H (separación entre centros contiguos) y se toma como parche la bola

$$\Omega_j = B(c_j, \rho), \qquad \rho = \alpha\, H, \quad \alpha > \sqrt{s}/2.$$

Todos los parches son, pues, bolas idénticas de radio ρ, regularmente distribuidas.

Claramente el recubrimiento cubre Ω. Esto es así puesto que sea $x \in \Omega$, como todos los centros distan H entre sí, x se halla a distancia a lo sumo $\sqrt{s}H/2$ del centro más cercano y por ser $\rho = \alpha H > H\sqrt{s}/2$, dicho punto pertenece a la bola correspondiente. De esta forma, $\Omega \subseteq \bigcup_j \Omega_j$. Veamos ahora que se cumplen las cuatro condiciones de la definición.

Condición 1 (solapamiento acotado). Un punto x pertenece a Ω_j si y solo si su centro c_j dista de x menos que ρ. Por tanto, el número de parches que contienen a x coincide con el número de centros de la rejilla situados en la bola $B(x,\rho)$. Como la rejilla es uniforme de paso H y $\rho = \alpha H$, el número de centros en la bola es una constante geométrica K que depende solo de α y de la dimensión s, de forma que, si α está acotada uniformemente para todo N, K no dependerá de H ni de N. Por ejemplo, si para todo N, $\sqrt{s}/2 \leq \alpha < \sqrt{s}$, cada bola centrada en un punto de la rejilla solo alcanza a sus vecinos más próximos.

Condición 2 (cono interior uniforme). Cada parche es una bola, que satisface trivialmente la condición de cono interior. En efecto, dado cualquier punto $x \in B(c_j,\rho)$, basta apuntar el cono con vértice en x hacia el centro c_j: por la convexidad de la bola, todo el segmento de x a c_j, y un cono de apertura suficiente en torno a esa dirección (esta ya no es por la convexidad, si no por cómo es la bola), queda contenido en la bola. Además, el ángulo θ y radio r del cono dependen únicamente del

radio ρ de las bolas. Como en nuestra construcción todas las bolas comparten el mismo radio ρ, ese par (θ, r) es idéntico para todos los parches o, lo que es lo mismo, uniforme en j, que es exactamente lo que la condición exige.

Condición 3 (distancias de llenado locales uniformemente acotadas). La condición exige que las distancias de llenado locales estén acotadas uniformemente por la global, es decir, que exista una constante C, común a todos los parches, con $h_{\chi_j,\Omega_j} \leq C\, h_{\chi,\Omega}$ para todo j, donde recordemos que $h_{\chi_j,\Omega_j} = \sup_{x\in\Omega_j} \min_{x_k\in\chi_j} \|x - x_k\|$ mide el mayor hueco sin nodos dentro del parche. No basta, por tanto, con que cada parche contenga algún nodo ($\chi_j \neq \emptyset$, que solo asegura $h_{\chi_j,\Omega_j} < \infty$): se requiere que los nodos estén bien repartidos en cada parche, de modo que su distancia de llenado no sea mucho mayor que el global. Para ello se requiere que el parche sea suficientemente grande frente al espaciado de los nodos, $\rho \geq c\, h_{\chi,\Omega}$ con c algo mayor que 1, para que contenga no uno sino numerosos nodos. Así, la buena distribución de los puntos de Halton, que vimos en la sección 2.1, garantiza que ese conjunto de nodos se reparta de forma homogénea dentro del parche, sin dejar subregiones vacías. En resumen, para un conjunto cuasiuniforme la distancia de llenado se mantiene del mismo orden que la total, y el hueco local resulta comparable al global. Como, además, todos los parches comparten el mismo radio ρ y los nodos son homogéneos, la constante c es la misma para todos ellos, esto es, uniforme en j, según exige la condición. En el caso gaussiano en el que, como veremos más adelante $m = 0$, esta es la única restricción sobre la población de nodos. Sin embargo, para el caso polinómico, la condición 4 la refuerza para $m \geq 2$.

Condición 4 (unisolvencia local, $m \geq 2$). En este caso, la exigencia es sensiblemente más fuerte: cada conjunto local $\chi_j = \chi \cap \Omega_j$ debe ser $(m-1)$-unisolvente, es decir, el único polinomio de grado $\leq m-1$ que se anula en todos sus nodos debe ser el nulo. Esto requiere dos cosas. La primera es que hacen falta al menos $L = \dim \Pi^s_{m-1}$ nodos, pues con menos nodos que la dimensión del espacio siempre existe un polinomio no nulo que se anula en ellos. Como la densidad de los nodos es del orden de N (número de nodos en el cubo unidad) por unidad de volumen y un parche tiene volumen $\lambda_s(B(0,1))\rho^s$ (donde λ_s denota la medida de Lebesgue, que corresponde simplemente a la noción habitual de volumen s-dimensional), el número de nodos por parche es $N_j \approx N\lambda_s(B(0,1))\rho^s$, de modo que la unisolvencia impone la cota inferior

$$\rho \;\geq\; \left(\frac{L}{\lambda_s(B(0,1))N}\right)^{1/s},$$

del mismo carácter que la de la condición 3, aunque algo más restrictiva. Lo segundo que se requiere tiene que ver con la posición de los nodos: los nodos no deben presentar una configuración singular o degenerada (alineados), que, dentro del parche, está garantizado de nuevo por la buena distribución de los puntos de Halton, que llena el cubo de forma uniforme sin generar alineamientos ni acumulaciones sobre subvariedades. En la práctica basta, pues, elegir el paso H de la rejilla de modo que cada parche contenga holgadamente más de L nodos.

A modo de resumen, conviene observar que las condiciones 3 y 4 operan ambas como cotas inferiores sobre el radio ρ, haciendo parches no demasiado pequeños frente al espaciado de los nodos: la condición

3 exige $\rho \geq c\, h_{\chi,\Omega}$ para que el hueco local quede controlado, y la condición 4, algo más restrictiva,

$$\rho \;\geq\; \left(\tfrac{L}{\lambda_s(B(0,1))N}\right)^{1/s}$$

para capturar del orden de L nodos en posición no degenerada (desapareciendo en el caso gaussiano). Frente a estas condiciones, tenemos el deseo de parches pequeños que, como veremos más adelante, favorecen la localidad y el condicionamiento, lo que actúa como cota superior, de manera que el radio ρ se elige en el compromiso entre ambas exigencias.

3.2. Condiciones analíticas sobre las funciones involucradas y espacios nativos

Seguidamente, daremos una serie de definiciones y resultados relacionados con la suavidad de las funciones involucradas.

Definición 3.4 (Norma C^m sobre un cerrado)**.** Sean $\Omega \subseteq \mathbb{R}^s$ abierto y acotado y $m \in \mathbb{N}_0$. Se denota por $C^m(\overline{\Omega})$ el espacio de las $f \in C^m(\Omega)$ tales que toda $D^\alpha f$ con $|\alpha| \leq m$ es acotada y uniformemente continua en Ω, y admite por tanto extensión continua a $\overline{\Omega}$. Sobre él se considera la norma $\|f\|_{C^m(\overline{\Omega})} = \max_{|\alpha|\leq m} \sup_{x\in\overline{\Omega}} |D^\alpha f(x)|$.

Nota 3.5. A diferencia de $C^m(\Omega)$, esta definición exige control uniforme de las derivadas hasta la frontera. En particular $C^m(\overline{\Omega}) \subsetneq C^m(\Omega)$, y es la finitud de $\|f\|_{C^m(\overline{\Omega})}$, y no la mera continuidad en el abierto, la que permitirá acotar la norma del espacio nativo en el lema 3.16.

Definición 3.6 (Partición de la unidad k-estable)**.** Sean $m \geq 0$, $\chi \subseteq \Omega$ y sea $\{\Omega_j\}$ un recubrimiento regular de (Ω, χ) para Π^s_{m-1}. Entonces, una familia $\{w_j\}$ subordinada a $\{\Omega_j\}$ es una partición de la unidad k-estable si $\exists U \subseteq \mathbb{R}^s$ abierto tal que $\overline{\Omega} \subseteq U$, con $w_j \in C^k(U)$, con soporte compacto $\bar{\Omega}_j$, satisfaciendo $\sum_j w_j = 1$ en Ω, y existe $C_\alpha > 0$, independiente de j, con $\|D^\alpha w_j\|_{L^\infty} \leq C_\alpha\, \delta_j^{-|\alpha|}$ para todo $|\alpha| \leq k$, siendo $\delta_j = \operatorname{diam}\Omega_j$.

En particular, una familia ∞-estable es k-estable para todo $k \in \mathbb{N}$, con el mismo abierto U para todos los órdenes.

Nota 3.7. En base a la definición anterior, es trivial notar que sea una familia $\{w_j\}$ una partición de la unidad k-estable subordinada a $\{\Omega_j\}$, un recubrimiento regular, entonces $\{w_j\}$ es una partición de la unidad l-estable, para todo $l \in \{0, \ldots, k\}$.

Definición 3.8 (El espacio C^k_β)**.** Para $k \in \mathbb{N}_0$ y $\beta \in [0,1]$, $C^k_\beta(\mathbb{R}^s)$ es el espacio de las $f \in C^k(\mathbb{R}^s)$ cuyas derivadas de orden $|\alpha| = k$ satisfacen $D^\alpha f(x) = \mathcal{O}(\|x\|_2^\beta)$ cuando $\|x\|_2 \to 0$.

3.2.1. Teoría de espacios nativos

Aquí abordaremos brevemente un concepto que será necesario para enunciar el teorema general que cuantifica el error cometido por los MPU: los espacios nativos. El siguiente teorema supone nuestro punto de partida en este desarrollo.

Teorema 3.9 (Mercer)**.** [7, Cap. 13] Sea $\Omega \subset \mathbb{R}^s$ acotado y $\Phi : \mathbb{R}^s \to \mathbb{R}$ una función par y continua tal que $(x, z) \mapsto \Phi(x - z)$ es estrictamente definida positiva sobre $\overline{\Omega}$. Considérese el operador integral

asociado

$$T_\Phi \colon L_2(\Omega) \to L_2(\Omega), \qquad (T_\Phi f)(x) = \int_\Omega \Phi(x-z)\, f(z)\, dz.$$

Entonces T_Φ posee una sucesión de autovalores $\lambda_1 \geq \lambda_2 \geq \cdots > 0$ con $\lambda_k \to 0$, y autofunciones asociadas $\{\gamma_k\}_{k\geq 1}$ continuas que forman un sistema ortonormal completo de $L_2(\Omega)$, es decir, $T_\Phi \gamma_k = \lambda_k \gamma_k$, de modo que

$$\Phi(x-z) = \sum_{k=1}^{\infty} \lambda_k\, \gamma_k(x)\, \gamma_k(z).$$

Una vez dado el teorema, definimos, ahora sí, el espacio nativo de una función Φ EDP.

Definición 3.10 (Espacio nativo)**.** Sea $\Phi : \mathbb{R}^s \to \mathbb{R}$ una función par, continua y EDP (recordemos la definición 2.16), y sea $\Omega \subset \mathbb{R}^s$ acotado. Sean $(\lambda_k, \gamma_k)_{k\geq 1}$ los autovalores y autofunciones del operador integral T_Φ proporcionados por el teorema de Mercer (teorema 3.9). El espacio nativo de Φ sobre Ω es el subespacio de $L_2(\Omega)$

$$N_\Phi(\Omega) = \Big\{ f \in L_2(\Omega) : \sum_{k=1}^{\infty} \frac{1}{\lambda_k} \big| \langle f, \gamma_k \rangle_{L_2(\Omega)} \big|^2 < \infty \Big\},$$

dotado del producto escalar

$$\langle f, g \rangle_{N_\Phi} = \sum_{k=1}^{\infty} \frac{1}{\lambda_k} \langle f, \gamma_k \rangle_{L_2(\Omega)} \langle g, \gamma_k \rangle_{L_2(\Omega)}, \qquad f, g \in N_\Phi(\Omega),$$

y de la norma inducida

$$\|f\|_{N_\Phi(\Omega)} = \Big(\sum_{k=1}^{\infty} \frac{1}{\lambda_k} \big| \langle f, \gamma_k \rangle_{L_2(\Omega)} \big|^2 \Big)^{1/2}.$$

Ahora bien, sea $f \in N_\Phi(\Omega) \subseteq L_2(\Omega)$, como el $\{\gamma_k\}$ dado por el Teorema de Mercer 3.9 es un sistema ortonormal completo de $L_2(\Omega)$, los números $c_k = \langle f, \gamma_k \rangle_{L_2(\Omega)}$ son las coordenadas de f en dicha base, de modo que $f = \sum_k c_k \gamma_k$ y, por la identidad de Parseval, $\|f\|^2_{L_2(\Omega)} = \sum_k |c_k|^2$. Por su parte, la norma del espacio nativo es esa misma suma con cada coeficiente ponderado por $1/\lambda_k$:

$$\|f\|^2_{N_\Phi(\Omega)} = \sum_{k=1}^{\infty} \frac{1}{\lambda_k} |c_k|^2.$$

Ahora, como $\lambda_k \to 0$ de forma decreciente, los pesos $1/\lambda_k$ crecen con k, luego $\|f\|_{N_\Phi(\Omega)} \geq \lambda_1^{-1/2} \|f\|_{L_2(\Omega)}$, lo que nos dice que la norma nativa es más restrictiva que la de L_2 y, por tanto, pertenecer a $N_\Phi(\Omega)$ obliga a que los coeficientes c_k decaigan más deprisa de lo que exige la mera pertenencia a $L_2(\Omega)$.

Relación entre norma y oscilación en el espacio nativo

Finalmente y antes de dar por finalizado este apartado, quedémonos con que la norma $\|f\|_{N_\Phi(\Omega)}$ nos da una idea intuitiva de las oscilaciones y regularidad de f. Así, veamos esto último mediante una prueba algo heurística, sin entrar en profundidad en materia, pues no es nuestro objetivo aquí.

Partimos de la definición del operador integral T_Φ para una autofunción γ_k:

$$(T_\Phi \gamma_k)(x) = \int_\Omega \Phi(x-z)\gamma_k(z)dz = \lambda_k \gamma_k(x).$$

Dado que el teorema de Mercer nos asegura que las $\{\gamma_k\}$ forman una base ortonormal en $L_2(\Omega)$, podemos calcular el autovalor λ_k proyectando la ecuación anterior sobre la propia γ_k:

$$\lambda_k = \langle \lambda_k \gamma_k, \gamma_k \rangle_{L_2(\Omega)} = \langle T_\Phi \gamma_k, \gamma_k \rangle_{L_2(\Omega)}$$

Si escribimos esta proyección explícitamente en forma de integral doble, obtenemos:

$$\lambda_k = \int_\Omega \left(\int_\Omega \Phi(x-z)\gamma_k(z)dz \right) \gamma_k(x)dx$$

Sabemos por la hipótesis del teorema 3.9 que $\Phi(x-z)$ es una función continua, luego no presentan saltos bruscos, sino que varía suavemente. Así, tenemos dos casos:

1. Si γ_k oscila poco (Baja frecuencia): La función $\gamma_k(z)$ mantiene el mismo signo en regiones grandes de Ω. Al multiplicarla por la función suave $\Phi(x-z)$, el producto se acumula constructivamente en la integral, dando como resultado un valor de λ_k grande.

2. γ_k es altamente oscilante (Alta frecuencia): La función $\gamma_k(z)$ cambia de signo de positivo a negativo muy rápidamente. Como $\Phi(x-z)$ es continua y cambia lentamente, en un intervalo pequeño donde γ_k hace una oscilación completa, $\Phi(x-z)$ es prácticamente constante (digamos, C). Entonces, la integral local se ve como $C\int \gamma_k(z)dz \approx 0$ y, por tanto, la integral global en Ω, que se aproximaría por una suma de integrales locales es aproximadamente cero.

Este fenómeno se denomina cancelación y, en él, a medida que la "frecuencia" de oscilación de γ_k aumenta, las áreas positivas y negativas de la integral se anulan entre sí cada vez más rápido. De esta forma, se asocian valores de λ_k pequeños (grandes) a γ_k muy oscilantes (poco oscilantes).

Ahora bien, recordemos que

$$\|f\|^2_{N_\Phi(\Omega)} = \sum_{k=1}^{\infty} \frac{1}{\lambda_k} |c_k|^2.$$

Así, si $\|f\|_{N_\Phi(\Omega)}$ es finita, quiere decir que los coeficientes $|c_k|^2$ decaen mucho más rápido de lo que lo hace λ_k. Por tanto, al reconstruir la función como combinación lineal de la base, $f = \sum_{k=1}^{\infty} c_k \gamma_k$, las autofunciones γ_k muy oscilantes tienen un peso c_k casi nulo en la serie, mientras que las poco oscilantes dominan la suma. En consecuencia, f resulta ser una función poco oscilante y muy regular.

Por su parte, si $\|f\|_{N_\Phi(\Omega)}$ diverge, por el mismo razonamiento, f tiende a oscilar más y ser más irregular.

Todo lo hecho hasta ahora en este capítulo ha sido, como ya hemos dicho, con el objetivo de alcanzar ciertos resultados teóricos sobre convergencia de los MPU y, llegados a este punto, ya nos encontramos en condiciones de introducirlos, algo que haremos en la siguiente sección.

3.3. Convergencia de los MPU

Ahora que ya está claro el concepto de espacio nativo, tenemos todos los conceptos necesarios para abordar, en su forma más general, el teorema que acota el error de los MPU.

Teorema 3.11 (Cota de error para la interpolación por partición de la unidad [25])**.** Sea $\Omega \subseteq \mathbb{R}^s$ abierto y acotado, y sea $\Phi \in C^k_\beta(\mathbb{R}^s)$ ECDP de orden m. Para cada multiíndice con $|\alpha| \leq k/2$ existe una constante $C_\alpha > 0$, independiente de f y del conjunto de nodos, tal que se cumple lo siguiente. Sean $\chi = \{x_1, \ldots, x_N\} \subseteq \Omega$ un conjunto finito de nodos, $\{\Omega_j\}$ un recubrimiento regular de (Ω, χ) para Π^s_{m-1}, $\{w_j\}$ una partición de la unidad k-estable subordinada a $\{\Omega_j\}$ y $f \in N_\Phi(\Omega)$. Entonces el interpolante por partición de la unidad

$$P_f = \sum_j w_j \, u_j, \qquad u_j \in \operatorname{span}\{\Phi(\cdot - x) : x \in \chi \cap \Omega_j\} + \Pi^s_{m-1},$$

donde cada u_j interpola a f en $\chi \cap \Omega_j$, verifica, para todo $x \in \Omega$,

$$\left|D^\alpha f(x) - D^\alpha P_f(x)\right| \leq C_\alpha \, h_{\chi,\Omega}^{\frac{k+\beta}{2} - |\alpha|} \, \|f\|_{N_\Phi(\Omega)},$$

donde $h_{\chi,\Omega}$ es la distancia de llenado de χ en Ω.

Cabe notar que, aunque nosotros hemos definido el espacio nativo para funciones EDP, existe una versión más general que abarca a las funciones ECDP de orden m a las que se refiere el teorema anterior; no obstante, esta definición no es relevante para nuestro desarrollo y, por eso, prescindiremos de ella. A continuación, presentamos una serie de apreciaciones y resultados sobre el teorema 3.11.

3.3.1. Algunos resultados sobre los interpolantes locales del teorema 3.11

Buscar una función $u \in \operatorname{span}\{\Phi(\cdot - x_i) : 1 \leq i \leq N\} + \Pi^s_{m-1}$,

$$u(x) = \sum_{i=1}^{N} \lambda_i \, \Phi(x - x_i) + \sum_{l=1}^{L} c_l \, q_l(x), \text{ con } \{q_1, \ldots, q_L\} \text{ base de } \Pi^s_{m-1},$$

que interpole los datos $u(x_j) = f(x_j)$, $1 \le j \le N$, equivale a imponer

$$A\boldsymbol{\lambda} + P\mathbf{c} = \mathbf{f}, \text{ con } A \in \mathbb{R}^{N\times N}, A_{ik} = \Phi(x_i - x_k), P \in \mathbb{R}^{N\times L}, P_{il} = q_l(x_i) \text{ y } \mathbf{f} = (f(x_j))_j.$$

Lo anterior resulta en N ecuaciones sobre las $N + L$ incógnitas $(\boldsymbol{\lambda}, \mathbf{c})$. El problema tiene, por tanto, L grados de libertad: la sola pertenencia a $\operatorname{span}\{\Phi\} + \Pi^s_{m-1}$ junto con la interpolación no determina u.

El interpolante se define cerrando el sistema con las L condiciones de momentos:

$$\sum_{i=1}^{N} \lambda_i\, q(x_i) = 0, \text{ para todo } q \in \Pi^s_{m-1}, \text{ es decir, } P^T\boldsymbol{\lambda} = 0.$$

Estas L nuevas restricciones no son una elección arbitraria, sino que imponen que el vector $\boldsymbol{\lambda}$ quede confinado al subespacio

$$V = \left\{\boldsymbol{\lambda} \in \mathbb{R}^N : \ \textstyle\sum_i \lambda_i\, q(x_i) = 0 \ \ \forall q \in \Pi^s_{m-1}\right\} = \ker P^T,$$

que es precisamente aquel sobre el cual el hecho de que A sea ECDP de orden m (por serlo Φ, según la definición 2.13) garantiza $\boldsymbol{\lambda}^T A\boldsymbol{\lambda} > 0$ para todo $\boldsymbol{\lambda} \neq 0$ en V. Es esta positividad, junto con la $(m-1)$-unisolvencia de los nodos (definición 3.1), la que, como veremos a continuación en el lema 3.12, vuelve invertible el sistema cuadrado de orden $N + L$, obtenido al reunir las condiciones de interpolación y de momentos. Con otra elección de L condiciones, dicha invertibilidad no estaría garantizada.

Lema 3.12 (Interpolación RBF con aumento polinómico)**.** Sea $\Phi : \mathbb{R}^s \to \mathbb{R}$ par y ECDP de orden m, y sea $\chi = \{x_1, \ldots, x_N\} \subset \mathbb{R}^s$ un conjunto de nodos distintos $(m-1)$-unisolvente. Fijada una base $\{q_1, \ldots, q_L\}$ de Π^s_{m-1} ($L = \dim \Pi^s_{m-1}$), para cualesquiera valores $f(x_1), \ldots, f(x_N)$ existe un único par $(\boldsymbol{\lambda}, \mathbf{c}) \in \mathbb{R}^N \times \mathbb{R}^L$ tal que la función

$$u(x) = \sum_{i=1}^{N} \lambda_i\, \Phi(x - x_i) + \sum_{l=1}^{L} c_l\, q_l(x)$$

verifica las condiciones de interpolación $u(x_j) = f(x_j)$, $1 \le j \le N$, y la condición de momentos $\sum_{i=1}^N \lambda_i\, q(x_i) = 0$ para todo $q \in \Pi^s_{m-1}$. En particular, u está unívocamente determinada. Además, para todo escalar $\sigma \neq 0$, el interpolante construido sustituyendo Φ por $\sigma\Phi$ (con la misma condición de momentos) coincide, como función, con u.

Demostración. Escribamos

$$A \in \mathbb{R}^{N\times N}, A_{ik} = \Phi(x_i - x_k), \text{ y } P \in \mathbb{R}^{N\times L}, P_{il} = q_l(x_i).$$

Las condiciones de interpolación equivalen a $A\boldsymbol{\lambda} + P\mathbf{c} = \mathbf{f}$, con $\mathbf{f} = (f(x_j))_j$, y la condición de momentos a $P^T\boldsymbol{\lambda} = 0$. Reuniendo ambas familias de ecuaciones, el par $(\boldsymbol{\lambda}, \mathbf{c})$ es solución del sistema aumentado

$$\begin{pmatrix} A & P \\ P^T & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{c} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ 0 \end{pmatrix},$$

cuya fila superior recoge la interpolación y la inferior la condición de momentos. Ahora, definimos $M := \begin{pmatrix} A & P \\ P^T & 0 \end{pmatrix}$ y basta, pues, ver que M es invertible.

Sea $\left(\begin{smallmatrix} \mathbf{u} \\ \mathbf{v} \end{smallmatrix}\right) \in \ker M$. Entonces

$$A\mathbf{u} + P\mathbf{v} = 0, \qquad P^T\mathbf{u} = 0.$$

La segunda igualdad dice que $\sum_i u_i\, q(x_i) = 0$ para todo $q \in \Pi^s_{m-1}$; es decir, $\mathbf{u}$ pertenece al subespacio sobre el que actúa la condición de ECDP de orden m. Multiplicando la primera igualdad por $\mathbf{u}^T$ y usando $\mathbf{u}^T P\mathbf{v} = (P^T\mathbf{u})^T\mathbf{v} = 0$, se obtiene

$$\mathbf{u}^T A\mathbf{u} = 0.$$

Por ser Φ ECDP de orden m, se tiene $\mathbf{w}^T A\mathbf{w} > 0$ para todo $\mathbf{w} \neq 0$ con $P^T\mathbf{w} = 0$, luego $\mathbf{u}^T A\mathbf{u} = 0$ junto con $P^T\mathbf{u} = 0$ implica que $\mathbf{u} = 0$. La primera igualdad queda entonces $P\mathbf{v} = 0$, esto es, el polinomio $\sum_l v_l\, q_l \in \Pi^s_{m-1}$ se anula en todo χ; pero, por ser χ $(m-1)$-unisolvente, dicho polinomio es el nulo y, al ser $\{q_l\}$ base, $\mathbf{v} = 0$. Así $\ker M = \{0\}$, M es invertible y el sistema posee solución única.

Para la última afirmación, sea $\sigma \neq 0$. El sistema asociado a $\sigma\Phi$ tiene matriz

$$M_\sigma = \left(\begin{smallmatrix} \sigma A & P \\ P^T & 0 \end{smallmatrix}\right).$$

Pero, el argumento anterior prueba igualmente que M_σ es invertible, siendo el único cambio que de $\sigma\, \mathbf{u}^T A\mathbf{u} = 0$ se sigue $\mathbf{u}^T A\mathbf{u} = 0$ por ser $\sigma \neq 0$. Luego su interpolante existe y es único. Ahora bien, si $(\boldsymbol{\lambda}, \mathbf{c})$ son los coeficientes de u, el par $(\sigma^{-1}\boldsymbol{\lambda}, \mathbf{c})$ resuelve M_σ, pues

$$\sigma A(\sigma^{-1}\boldsymbol{\lambda}) + P\mathbf{c} = A\boldsymbol{\lambda} + P\mathbf{c} = \mathbf{f} \text{ y } P^T(\sigma^{-1}\boldsymbol{\lambda}) = \sigma^{-1}P^T\boldsymbol{\lambda} = 0,$$

y genera la misma función:

$$\sum_i \sigma^{-1}\lambda_i\, (\sigma\Phi)(x - x_i) + \sum_l c_l\, q_l(x) = \sum_i \lambda_i\, \Phi(x - x_i) + \sum_l c_l\, q_l(x) = u(x).$$

Por la unicidad para M_σ, el interpolante asociado a $\sigma\Phi$ coincide con u. □

Ahora bien, cabría preguntarse si, al aplicar las condiciones de interpolación y momentos, el interpolante obtenido depende de la base escogida para el espacio de polinomios. La respuesta que esperaríamos es que no es así; veámoslo en el siguiente lema.

Lema 3.13 (Independencia del interpolante respecto de la base de Π^s_{m-1})**.** Bajo las hipótesis del Lema 3.12 (nodos $(m-1)$-unisolventes y Φ ECDP de orden m), el interpolante local

$$u = \sum_{i=1}^{N_j} \lambda_i\, \Phi(\cdot - x_i) + \pi, \qquad \pi \in \Pi^s_{m-1},$$

determinado por $u(x_i) = f_i$ y $\sum_i \lambda_i\, q(x_i) = 0$ para todo $q \in \Pi^s_{m-1}$, es independiente de la base elegida de Π^s_{m-1}: el vector $\boldsymbol{\lambda}$ y la función π (y por tanto u) son las mismas para cualesquiera dos bases, solo cambian las coordenadas de π.

Demostración. Sean $\{q_l\}_{l=1}^L$ y $\{\tilde{q}_l\}_{l=1}^L$ dos bases de Π_{m-1}^s relacionadas por $\tilde{q}_l = \sum_r G_{rl}\, q_r$ con G matriz real invertible de dimensión L, y sea $\pi = \sum_r c_r q_r$, querremos ver que $\boldsymbol{\lambda} = \tilde{\boldsymbol{\lambda}}$ y que $\pi = \sum_l \tilde{c}_l \tilde{q}_l$. Para ello, en primer lugar, escribimos las matrices que evalúan la base en los nodos, que son $(P)_{il} = q_l(x_i)$, $(\tilde{P})_{il} = \tilde{q}_l(x_i)$, y cumplen $\tilde{P} = PG$. Así, los dos sistemas quedan

$$\begin{pmatrix} A & P \\ P^\top & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{c} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ 0 \end{pmatrix}, \qquad \begin{pmatrix} A & \tilde{P} \\ \tilde{P}^\top & 0 \end{pmatrix} \begin{pmatrix} \tilde{\boldsymbol{\lambda}} \\ \tilde{\mathbf{c}} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ 0 \end{pmatrix}.$$

El segundo equivale a $A\tilde{\boldsymbol{\lambda}} + P(G\tilde{\mathbf{c}}) = \mathbf{f}$ y $G^\top P^\top \tilde{\boldsymbol{\lambda}} = 0$. Como G es invertible, $G^\top P^\top \tilde{\boldsymbol{\lambda}} = 0 \iff P^\top \tilde{\boldsymbol{\lambda}} = 0$. Luego $(\tilde{\boldsymbol{\lambda}},\, G\tilde{\mathbf{c}})$ satisface el mismo sistema que $(\boldsymbol{\lambda}, \mathbf{c})$ y, por la unicidad del lema 3.12, $\tilde{\boldsymbol{\lambda}} = \boldsymbol{\lambda}$ y $G\tilde{\mathbf{c}} = \mathbf{c}$, es decir, $\tilde{\mathbf{c}} = G^{-1}\mathbf{c}$. Por tanto, la parte polinómica es la misma función, como vemos en el siguiente cálculo,

$$\sum_l \tilde{c}_l \tilde{q}_l = \sum_l (G^{-1}\mathbf{c})_l \sum_r G_{rl} q_r = \sum_r \Big(\sum_l G_{rl} (G^{-1}\mathbf{c})_l \Big) q_r = \sum_r (GG^{-1}\mathbf{c})_r q_r = \sum_r c_r q_r = \pi.$$

Además, como $\tilde{\boldsymbol{\lambda}} = \boldsymbol{\lambda}$, el interpolante u coincide. Solo cambian las coordenadas, $\tilde{\mathbf{c}} = G^{-1}\mathbf{c}$. □

Otra pregunta que podríamos hacernos es si nuestros interpolantes locales u y, por tanto, nuestro interpolante global tienen la propiedad de reproducción polinómica.

Corolario 3.14 (Reproducción de polinomios del interpolante)**.** Sea Φ una función par y ECDP de orden m, y sea $\chi = \{x_1, \ldots, x_N\} \subset \mathbb{R}^s$ un conjunto $m-1$-unisolvente. Fijada una base $\{q_1, \ldots, q_L\}$ de Π_{m-1}^s (con $L = \dim \Pi_{m-1}^s = \binom{s+m-1}{s}$), por el lema 3.12 el sistema

$$\begin{pmatrix} A & P \\ P^\mathsf{T} & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{c} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ 0 \end{pmatrix}, \qquad A_{ik} = \Phi(x_i - x_k), \quad P_{il} = q_l(x_i),$$

posee solución única $(\boldsymbol{\lambda}, \mathbf{c}) \in \mathbb{R}^N \times \mathbb{R}^L$ para todo dato $\mathbf{f} = (f_1, \ldots, f_N)^\mathsf{T}$, y con ella se forma el interpolante

$$u(x) = \sum_{k=1}^N \lambda_k\, \Phi(x - x_k) + \sum_{l=1}^L c_l\, q_l(x).$$

Entonces u reproduce exactamente Π_{m-1}^s: si la función a interpolar es un polinomio $g \in \Pi_{m-1}^s$, es decir, $f_i = g(x_i)$ para $1 \le i \le N$, se tiene

$$u = g \qquad \text{en todo } \mathbb{R}^s.$$

Demostración. Como $\{q_1, \ldots, q_L\}$ es base de Π_{m-1}^s y $g \in \Pi_{m-1}^s$, existe un único vector de coordenadas $\mathbf{c}^g = (c_1^g, \ldots, c_L^g)^\mathsf{T} \in \mathbb{R}^L$ tal que $g = \sum_{l=1}^L c_l^g\, q_l$. Comprobamos que el par $(\boldsymbol{\lambda}, \mathbf{c}) = (0, \mathbf{c}^g)$ satisface el sistema.

Para la primera ecuación, ocurre que, para cada $1 \le i \le N$,

$$\big(A \cdot 0 + P\, \mathbf{c}^g\big)_i = \sum_{l=1}^L P_{il}\, c_l^g = \sum_{l=1}^L q_l(x_i)\, c_l^g = g(x_i) = f_i,$$

donde la penúltima igualdad es la definición de $\mathbf{c}^g$ y la última es la hipótesis sobre el dato. Por tanto $A\,0 + P\,\mathbf{c}^g = \mathbf{f}$.

En cuanto a la segunda ecuación, $P^{\mathsf{T}} \cdot 0 = 0$.

Así, $(0, \mathbf{c}^g)$ es solución del sistema y, por la unicidad de solución que garantiza el lema 3.12, esta es la solución. Sustituyendo en la expresión de u, la parte radial desaparece por completo y queda

$$u(x) = \sum_{k=1}^{N} 0 \cdot \Phi(x - x_k) + \sum_{l=1}^{L} c_l^g \, q_l(x) = g(x), \qquad \forall x \in \mathbb{R}^s.$$

□

Una vez vista la reproducción polinómica localmente, veamos que el interpolante global conserva dicha propiedad.

Corolario 3.15 (Reproducción de polinomios del MPU)**.** Considérese un recubrimiento regular y el aproximante por partición de la unidad $P_f = \sum_j w_j \, u_j$, donde $\{w_j\}$ es una partición de la unidad subordinada al recubrimiento ($\sum_j w_j = 1$ en Ω) y cada u_j es el interpolante local del corolario 3.14 sobre $\chi \cap \Omega_j$. Si en cada parche el conjunto de nodos $\chi \cap \Omega_j$ es $m-1$-unisolvente (condición (4) de recubrimiento regular), entonces el MPU reproduce exactamente Π_{m-1}^s: para todo $g \in \Pi_{m-1}^s$,

$$P_g \equiv g \qquad \text{en } \Omega.$$

Demostración. Sea $g \in \Pi_{m-1}^s$, por el corolario 3.14, aplicado en cada parche, $u_j = g$. Entonces, para todo $x \in \Omega$,

$$P_g(x) = \sum_j w_j(x) \, u_j(x) = \sum_j w_j(x) \, g(x) = g(x) \sum_j w_j(x) = g(x),$$

donde en la suma solo intervienen los índices j con $x \in \Omega_j$ (los únicos con $w_j(x) \neq 0$) y se ha usado $\sum_j w_j = 1$. □

Una vez vista esta serie de apreciaciones sobre la versión general del teorema 3.11, vamos a particularizarlo a los dos casos principales que nos ocupan: para $\Phi(x) = \|x\|^p$, con p impar, o para Φ gaussiana. Empecemos por el primer caso.

3.3.2. Convergencia para el caso $\Phi(x) = ||x||^p$ (p impar)

Con el objetivo de estudiar la convergencia en el caso de $\Phi(x) = ||x||^p$, con p impar, debemos introducir primero una serie de definiciones y resultados previos.

Lema 3.16. Sea $\Omega \subset \mathbb{R}^s$ un dominio acotado con frontera Lipschitz, sean p impar y

$$\Phi(x) = \varepsilon_p \, \|x\|_2^p, \qquad \varepsilon_p := (-1)^{\lceil p/2 \rceil},$$

ECDP de orden $m = \lceil p/2 \rceil$, por la proposición 2.22. Sea $\tau = (s+p)/2$. Si $f \in C^{\lceil \tau \rceil}(\overline{\Omega})$, entonces $f \in N_\Phi(\Omega)$ y existe una constante $C > 0$, dependiente solo de s, p y Ω, tal que

$$|f|_{N_\Phi(\Omega)} \;\leq\; C\,\|f\|_{C^{\lceil \tau \rceil}(\overline{\Omega})}.$$

La demostración se basa en teoría de Espacios de Sobolev, que exceden el alcance de este trabajo. Esta puede consultarse en [25, Cap. 10] y [21].

A continuación, introducimos el concepto de funciones positivamente homogéneas y disponemos algunos resultados sobre este tipo de funciones, que nos harán falta más adelante.

Definición 3.17 (Función positivamente homogénea)**.** Sea $d \in \mathbb{R}$. Una función $g : \mathbb{R}^s \setminus \{0\} \to \mathbb{R}$ se dice positivamente homogénea de grado d si

$$g(tx) = t^d\, g(x) \qquad \text{para todo } t > 0 \text{ y todo } x \in \mathbb{R}^s \setminus \{0\}.$$

Lema 3.18 (Derivación de funciones positivamente homogéneas)**.** Sea $g : \mathbb{R}^s \setminus \{0\} \to \mathbb{R}$ de clase C^1 y positivamente homogénea de grado d. Entonces, para cada $i \in \{1, \ldots, s\}$, la derivada parcial $\partial_i g$ es positivamente homogénea de grado $d-1$. En consecuencia, toda derivada $D^\alpha g$ de orden $1 \leq |\alpha| = j \leq d$ es positivamente homogénea de grado $d - j$, con $g \in C^j$ en $\mathbb{R}^s \setminus \{0\}$.

Demostración. Fijado $t > 0$, derivamos la identidad de homogeneidad $g(tx) = t^d g(x)$ respecto a x_i, tratando t como parámetro. Por la regla de la cadena, el primer miembro da $t\,(\partial_i g)(tx)$, y el segundo $t^d\,(\partial_i g)(x)$; igualando,

$$t\,(\partial_i g)(tx) = t^d\,(\partial_i g)(x) \quad \Longrightarrow \quad (\partial_i g)(tx) = t^{\,d-1}(\partial_i g)(x),$$

para todo $t > 0$ y todo $x \in \mathbb{R}^s \setminus \{0\}$, esto es, $\partial_i g$ es positivamente homogénea de grado $d-1$. Aplicando esto reiteradamente a las sucesivas derivadas parciales se obtiene que $D^\alpha g$ es positivamente homogénea de grado $d - j$ para $|\alpha| = j \geq 1$. □

Lema 3.19. Sea $G : \mathbb{R}^s \setminus \{0\} \to \mathbb{R}$ continua y positivamente homogénea de grado 0, esto es, $G(tx) = G(x)$ para todo $t > 0$ y todo $x \neq 0$. Si G no es constante sobre la esfera unidad S^{s-1}, entonces $\lim_{x \to 0} G(x)$ no existe. En particular, G no admite extensión continua al origen.

Demostración. Por hipótesis existen $\theta_1, \theta_2 \in S^{s-1}$ con $G(\theta_1) \neq G(\theta_2)$. Consideremos los caminos radiales $x = r\theta_1$ y $x = r\theta_2$, que tienden a 0 cuando $r \to 0^+$. Por homogeneidad de grado 0, $G(r\theta_k) = G(\theta_k)$ para todo $r > 0$, $1 \leq k \leq 2$, de modo que los límites a lo largo de cada camino son, respectivamente, $G(\theta_1)$ y $G(\theta_2)$, distintos entre sí. Si existiera $\lim_{x \to 0} G(x) = L$, todo límite direccional debería coincidir con L, pero al ser los dos anteriores distintos, tal límite no existe. En consecuencia, G no se extiende con continuidad al origen. □

Tras los resultados anteriores, en el siguiente lema, veremos que, nuestra función $\Phi(x) = \|x\|_2^p$ es positivamente homogénea de grado p y que, en base a los resultados que hemos visto ya para este tipo de funciones, una de sus parciales p-ésimas no es constante sobre la esfera S^{s-1}. Este resultado será importante, pues lo aplicaremos directamente en la demostración del teorema central de este apartado.

Lema 3.20. Sea p impar y $\Phi(x) = \|x\|_2^p$ en $\mathbb{R}^s$. La derivada pura $\partial_1^p \Phi$, esto es $D^{\alpha_0}\Phi$ con $\alpha_0 = (p, 0, \ldots, 0)$, no es constante sobre S^{s-1}, sino que toma el valor $p!$ en $e_1 = (1, 0, \ldots, 0)$ y $-p!$ en $-e_1 = (-1, 0, \ldots, 0)$.

Demostración. Notemos primeramente que la función $\Phi(x) = \|x\|_2^p$ es, claramente, positivamente homogénea de grado p y es de clase C^∞ en $\mathbb{R}^s \setminus \{0\}$, por ser composición de $x \mapsto \|x\|_2^2$ (polinómica) con $t \mapsto t^{p/2}$ (C^∞ en $t > 0$). Ahora, como por el lema 3.18 derivar p veces una función homogénea de grado p y clase $C^p(\mathbb{R}^s - \{0\})$ produce una homogénea de grado 0, la derivada $\partial_1^p \Phi$ es positivamente homogénea de grado 0. En consecuencia, su valor sobre S^{s-1} a lo largo del eje x_1 coincide con su valor en cualquier punto no nulo de dicho eje: $\partial_1^p \Phi(\pm e_1) = \partial_1^p \Phi(t, 0, \ldots, 0)$ para todo $t > 0$ (signo $+$) ó $t < 0$ (signo $-$).

Por otro lado, la derivada pura ∂_1^p deriva p veces respecto a x^1, manteniendo fijas las coordenadas $x^2, \ldots, x^s$. Sobre el eje x^1 estas valen 0 y no se ven afectadas por dicha derivación, de modo que $\partial_1^p \Phi(t, 0, \ldots, 0)$ coincide con la derivada p-ésima de la función de una variable obtenida al fijar $x^2 = \cdots = x^s = 0$, esto es, $t \mapsto \Phi(t, 0, \ldots, 0) = |t|^p$. Por tanto,

$$\partial_1^p \Phi(t, 0, \ldots, 0) = \frac{d^p}{dt^p} |t|^p, \qquad t \neq 0.$$

Como p es impar, $|t|^p = \operatorname{sgn}(t)\, t^p$ y se tiene $\frac{d^p}{dt^p}|t|^p = p! \operatorname{sgn}(t)$. Por tanto $\partial_1^p \Phi(t, 0, \ldots, 0) = p! \operatorname{sgn}(t)$, de donde $\partial_1^p \Phi(e_1) = p!$ y $\partial_1^p \Phi(-e_1) = -p!$. Por ser distintos, $\partial_1^p \Phi$ no es constante sobre S^{s-1}. □

Finalmente, se introduce un último resultado previo sobre la extensión de las derivadas de una función en un entorno del cero, que también aplicaremos en el teorema que concluye esta subsección.
Lema 3.21. Sea $j \geq 1$ y $f : \mathbb{R}^s \to \mathbb{R}$, con $f \in C^{j-1}$ en un entorno de 0. Sea $|\alpha| = j$, si $D^\alpha f$ existe y es continua en el entorno sin el cero, y si $\lim_{x \to 0} D^\alpha f(x) = L$ existe, entonces $D^\alpha f(0)$ existe y vale L.

Demostración. Como $|\alpha| = j \geq 1$, alguna componente de α es positiva. Fijemos una dirección $i \in \{1, \ldots, s\}$ con $\alpha_i \geq 1$, y sea α' el multiíndice obtenido al restar 1 a α en la componente i, de modo que $|\alpha'| = j - 1$. Por construcción,

$$D^\alpha f = \partial_i \big(D^{\alpha'} f \big).$$

Escribamos $g := D^{\alpha'} f$, de manera que $D^\alpha f = \partial_i g$. Por hipótesis $f \in C^{j-1}$ en un entorno de 0, luego $g = D^{\alpha'} f$ está bien definida y es continua en dicho entorno. Además, por hipótesis, $\partial_i g = D^\alpha f$ existe y es continua en el entorno salvo en el origen, y

$$\lim_{x \to 0} \partial_i g(x) = L. \tag{3.1}$$

Nótese que (3.1) es una afirmación sobre el límite de $\partial_i g$ al acercarse al origen; no presupone que $\partial_i g$ esté definida ni sea continua en 0. Nuestro objetivo es probar que la derivada parcial $\partial_i g(0)$ existe y que vale L, ya que, como $D^\alpha f = \partial_i g$, esto equivale a $D^\alpha f(0) = L$. Recordemos que la derivada parcial en el origen se define como el límite del cociente incremental

$$\partial_i g(0) := \lim_{h \to 0} \frac{g(h e_i) - g(0)}{h},$$

donde e_i es el i-ésimo vector de la base canónica. Veamos que dicho límite existe y vale L. Fijemos $h \neq 0$ suficientemente pequeño para que el segmento de extremos 0 y he_i esté contenido en el entorno. La función de una variable $t \mapsto g(te_i)$ es continua en el segmento cerrado de extremos 0 y h (por la continuidad de g) y derivable en su interior, con derivada $\partial_i g(te_i)$, que existe en esos puntos, por ser distintos del cero. Por el teorema del valor medio, existe ξ_h estrictamente entre 0 y h tal que

$$\frac{g(he_i) - g(0)}{h} = \partial_i g(\xi_h\, e_i).$$

Tomemos ahora $h \to 0$. Como ξ_h está entre 0 y h, se tiene $\xi_h \to 0$, y por tanto el punto $\xi_h e_i \to 0$, siempre distinto de cero. Aplicando la ecuación (3.1) a esta aproximación al origen,

$$\partial_i g(\xi_h e_i) \xrightarrow[h\to 0]{} L.$$

En consecuencia, el miembro izquierdo de la igualdad anterior también converge, es decir, el cociente incremental tiene límite cuando $h \to 0$ y

$$\partial_i g(0) = \lim_{h\to 0} \frac{g(he_i) - g(0)}{h} = L.$$

Por definición, esto significa que $\partial_i g(0)$ existe y vale L. Finalmente, recordando que $D^\alpha f = \partial_i g$ como funciones, al evaluar en el origen se obtiene $D^\alpha f(0) = \partial_i g(0) = L$. □

Llegados a este punto ya tenemos todo lo necesario para lograr nuestra meta: adaptar el teorema 3.11 al caso de $\varphi(r) = r^p$, con p impar.

Corolario 3.22 (Convergencia para $\varphi(r) = r^p$, p impar)**.** Sea $\Omega \subset \mathbb{R}^s$ abierto acotado con frontera Lipschitz, $\varphi(r) = r^p$ con p impar, $m = \frac{p+1}{2}$ y $l \in \mathbb{N}_0$. Si definimos $\kappa := \min\{(p-1), l\}$, para cada multiíndice con $|\alpha| \leq \kappa/2$ existen constantes $C_\alpha, D_\alpha > 0$, independientes de f y del conjunto de nodos, tales que se cumple lo siguiente. Sean $\chi \subset \Omega$ un conjunto finito de nodos, $\{\Omega_j\}$ un recubrimiento regular de (Ω, χ) para Π^s_{m-1} y $\{w_j\}$ una partición de la unidad l-estable subordinada a $\{\Omega_j\}$, y sea $f \in C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})$. Entonces el interpolante por partición de la unidad

$$P_f = \sum_j w_j\, u_j, \qquad u_j \in \operatorname{span}\big\{ \|\cdot - x\|_2^p : x \in \chi \cap \Omega_j \big\} + \Pi^s_{m-1},$$

donde cada u_j interpola a f en $\chi \cap \Omega_j$ con la condición de momentos, explicada al principio de esta subsección, verifica, para todo $x \in \Omega$,

$$\big|D^\alpha f(x) - D^\alpha P_f(x)\big| \leq C_\alpha\, h_{\chi,\Omega}^{(\kappa+1)/2-|\alpha|}\, |f|_{N_\Phi(\Omega)} \leq D_\alpha\, h_{\chi,\Omega}^{(\kappa+1)/2-|\alpha|}\, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}.$$

En particular, $\|f - P_f\|_{L^\infty(\Omega)} \leq D_0\, h_{\chi,\Omega}^{(\kappa+1)/2}\, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}$, donde $\Phi(x) = \varepsilon_p \|x\|_2^p$, $\varepsilon_p = (-1)^{\lceil p/2 \rceil}$ y $N_\Phi(\Omega)$ su espacio nativo.

Demostración. Sea $\Phi(x) = \varepsilon_p \|x\|_2^p$. Verificamos las hipótesis del Teorema 3.11 para esta función y comprobamos que el interpolante del enunciado (el de φ), coincide con el de Φ.

(i) Interpolante bien definido. El interpolante generado por partición de la unidad $P_f = \sum_j w_j u_j$ exige que cada interpolante local

$$u_j \in \operatorname{span}\{\Phi(\cdot - x) : x \in \chi \cap \Omega_j\} + \Pi^s_{m-1}$$

esté bien definido. Por la Proposición 2.22, Φ es ECDP de orden $m = \lceil p/2 \rceil = (p+1)/2$. Por otro lado, cada conjunto local $\chi_j = \chi \cap \Omega_j$ es $(m-1)$-unisolvente por la condición (4) de la Definición 3.3.

Así, por el lema 3.12, el interpolante de f en χ_j construido a partir de Φ (y el espacio de polinomios) existe y es único. Además, tomando $\sigma = \varepsilon_p$, de forma que $\sigma\Phi = \varepsilon_p^2 \|\cdot\|_2^p = \|\cdot\|_2^p$, por la última afirmación del lema 3.12, dicho interpolante coincide con el construido a partir de $\operatorname{span}\{\|\cdot - x\|_2^p : x \in \chi_j\} + \Pi^s_{m-1}$, es decir, con el u_j del enunciado. Por tanto cada u_j está bien definido y $P_f = \sum_j w_j u_j$ coincide con el interpolante por partición de la unidad asociado a Φ, al que se aplica el Teorema 3.11.

(ii) Regularidad $\Phi \in C^k_\beta$. Recordemos que, por la definición 3.8, $\Phi \in C^k_\beta(\mathbb{R}^s)$ significa que $\Phi \in C^k(\mathbb{R}^s)$ y que sus derivadas de orden $|\alpha| = k$ satisfacen $D^\alpha\Phi(x) = \mathcal{O}(\|x\|_2^\beta)$ cuando $\|x\|_2 \to 0$. Como $|\varepsilon_p| = 1$, la regularidad de $\Phi = \varepsilon_p \|\cdot\|_2^p$ coincide con la de $\|\cdot\|_2^p$; trabajamos con esta.

En $\mathbb{R}^s \setminus \{0\}$, la función $\Phi(x) = \|x\|_2^p$ es de clase C^∞, por ser composición de $x \mapsto \|x\|_2^2$ (polinómica) con $t \mapsto t^{p/2}$ (C^∞ en $t > 0$). Así, debemos centrarnos en el comportamiento de la función en un entorno del origen.

Sabemos que Φ es positivamente homogénea de grado p y $C^\infty(\mathbb{R}^s \setminus \{0\})$, luego por el lema 3.18, cada derivada $D^\alpha\Phi$ de orden $|\alpha| = j \le p$ es positivamente homogénea de grado $e = p - j \ge 0$ en $\mathbb{R}^s \setminus \{0\}$. Por tanto, $D^\alpha\Phi(t\theta) = t^e D^\alpha\Phi(\theta)$ para todo $t > 0$ y todo θ. Ahora, ya sin derivar, evaluamos en $t = \|x\|_2$ y $\theta = x/\|x\|_2$,

$$D^\alpha\Phi(x) = \|x\|_2^{p-j}\, D^\alpha\Phi\big(x/\|x\|_2\big), \qquad x/\|x\|_2 \in S^{s-1}.$$

Observamos que la función $D^\alpha\Phi$ es continua sobre el compacto $S^{s-1} \subseteq \mathbb{R}^s \setminus \{0\}$, luego acotada, por lo que existe $M_\alpha \ge 0$ tal que $|D^\alpha\Phi(\theta)| \le M_\alpha, \forall\theta \in S^{s-1}$. Analicemos ahora el comportamiento en el origen según el grado e.

- Si $e > 0$ (esto es, $j \le p-1$): $|D^\alpha\Phi(x)| \le M_\alpha \|x\|_2^e \to 0$ cuando $x \to 0$. Lo anterior muestra que, para $1 \le |\alpha| \le p-1$, $D^\alpha\Phi$ tiene límite 0 en el origen. Ahora, como Φ es continua en $\mathbb{R}^s$, aplicando recursivamente el lema 3.21, se tiene que la derivada existe en cero y vale lo mismo que dicho límite, es decir, cero. Por tanto, $\Phi \in C^{p-1}(\mathbb{R}^s)$.

- Si $e = 0$ (esto es, $j = p$): cada derivada de orden p de Φ es positivamente homogénea de grado 0. Por el lema 3.20, existe un multiíndice α_0 con $|\alpha_0| = p$ tal que $D^{\alpha_0}\Phi$ no es constante sobre S^{s-1}. Como $D^{\alpha_0}\Phi$ es continua en $\mathbb{R}^s \setminus \{0\}$ (de hecho Φ es de clase C^∞ en $\mathbb{R}^s \setminus \{0\}$) y positivamente homogénea de grado 0, el lema 3.19 garantiza que $\lim_{x\to 0} D^{\alpha_0}\Phi(x)$ no existe. Como resultado de lo anterior, $D^{\alpha_0}\Phi$ no se extiende de forma continua al origen y $\Phi \notin C^p(\mathbb{R}^s)$.

Por tanto, $\Phi \in C^{p-1}(\mathbb{R}^s)$ y $\Phi \notin C^p(\mathbb{R}^s)$, y elegimos $k = \kappa \leq p-1$.

Tomamos $|\alpha| = \kappa \leq p-1$, donde el grado es $e = p - \kappa \geq 1 > 0$. Por la identidad $D^\alpha \Phi(x) = \|x\|_2^{p-\kappa} D^\alpha \Phi(x/\|x\|_2)$ y la acotación $|D^\alpha \Phi(\theta)| \leq M_\alpha$ sobre S^{s-1}, se tiene

$$|D^\alpha \Phi(x)| \leq M_\alpha \, \|x\|_2^{p-\kappa},$$

con $p - \kappa \geq 1$, luego $D^\alpha \Phi(x) = \mathcal{O}(\|x\|_2)$ cuando $\|x\|_2 \to 0$. Así, eligiendo $k = \kappa$ y $\beta = 1$, llegamos a que $\Phi \in C^k_\beta$.

Por otro lado, notemos que, como $\kappa \leq l$, entonces, por la nota 3.7, la partición de la unidad considerada es κ-estable.

Considerando todo lo anterior, concluimos que el exponente del teorema 3.11 es $(k+\beta)/2 - |\alpha| = (\kappa+1)/2 - |\alpha|$, válido para $|\alpha| \leq k/2 = \kappa/2$.

(iii) Pertenencia de f al espacio nativo. Como por hipótesis $f \in C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})$ y Ω tiene frontera Lipschitz, por el lema 3.16 se sigue que $f \in N_\Phi(\Omega)$ y, además, $|f|_{N_\Phi(\Omega)} \leq C_1 \, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}$. Así, queda verificada la hipótesis del teorema 3.11 sobre f.

Reunidas las tres comprobaciones, el teorema 3.11 con $k = \kappa$ y $\beta = 1$ proporciona, para cada $|\alpha| \leq (\kappa)/2$ una constante $C_\alpha > 0$ tal que, para todo $x \in \Omega$,

$$\left| D^\alpha f(x) - D^\alpha P_f(x) \right| \leq C_\alpha \, h_{\chi,\Omega}^{(\kappa+1)/2-|\alpha|} \, |f|_{N_\Phi(\Omega)}.$$

Combinando con la cota del lema 3.16, $|f|_{N_\Phi(\Omega)} \leq C_1 \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}$, y poniendo $D_\alpha := C_\alpha C_1$, se obtiene la segunda desigualdad. Las constantes C_α y D_α heredan de los enunciados del teorema 3.11 y del lema 3.16 su independencia tanto de f como del conjunto de nodos χ. Para $\alpha = 0$, como el segundo miembro no depende de x, tomando supremo en $x \in \Omega$ resulta $\|f - P_f\|_{L^\infty(\Omega)} \leq D_0 \, h_{\chi,\Omega}^{(\kappa+1)/2} \, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}$. □

3.3.3. Convergencia para el caso Φ gaussiano

Una vez estudiado el caso de $\varphi(r) = r^p$, con p impar, pasamos ahora a particularizar el teorema 3.11 para el caso de Φ gaussiana, tratando primero un lema previo.

Lema 3.23 (Cota puntual del error para interpolante global)**.** Sea $\{w_j\}$ una familia de funciones subordinada a un recubrimiento $\{\Omega_j\}$ de Ω, con $w_j \geq 0$ y $\sum_j w_j = 1$ en Ω. Sean $u_j : \Omega_j \to \mathbb{R}$ funciones cualesquiera y $P_f := \sum_j w_j u_j$. Entonces, para todo $x \in \Omega$, definimos $J(x) := \{j : x \in \Omega_j\}$ y se cumple

$$|f(x) - P_f(x)| \leq \max_{j \in J(x)} |f(x) - u_j(x)|.$$

Demostración. Sean $x \in \Omega$, $J(x) := \{j : x \in \Omega_j\}$, el conjunto de índices cuyo parche contiene a x, que es finito y no vacío por ser $\{\Omega_j\}$ un recubrimiento. Como el soporte$(w_j) \subseteq \overline{\Omega_j}$, se tiene $w_j(x) = 0$

para $j \notin J(x)$, de modo que $\sum_j w_j(x) = \sum_{j\in J(x)} w_j(x) = 1$. Usando esta identidad,

$$f(x) - P_f(x) \;=\; f(x)\sum_{j\in J(x)} w_j(x) \;-\; \sum_{j\in J(x)} w_j(x)u_j(x) \;=\; \sum_{j\in J(x)} w_j(x)\big(f(x)-u_j(x)\big).$$

Tomando valor absoluto, aplicando la desigualdad triangular y usando $w_j(x) \geq 0$,

$$|f(x)-P_f(x)| \;\leq\; \sum_{j\in J(x)} w_j(x)\,|f(x)-u_j(x)| \;\leq\; \Big(\max_{j\in J(x)} |f(x)-u_j(x)|\Big)\sum_{j\in J(x)} w_j(x) \;=\; \max_{j\in J(x)} |f(x)-u_j(x)|,$$

donde la última igualdad vuelve a usar $\sum_{j\in J(x)} w_j(x) = 1$. □

Una vez introducido el lema anterior, disponemos de todas las herramientas para abordar la particularización del teorema 3.11 para el caso gaussiano, que se traduce en el siguiente corolario.

Corolario 3.24 (Convergencia para Φ gaussiana)**.** Sea $\Omega \subset \mathbb{R}^s$ abierto acotado y $\varphi(r) = e^{-\varepsilon^2 r^2}$ ($\varepsilon > 0$), con $\Phi(x) = e^{-\varepsilon^2\|x\|_2^2}$. Para cada $k \in \mathbb{N}$ y cada multiíndice con $|\alpha| \leq k/2$ existe una constante $C_{k,\alpha} > 0$, independiente de f y del conjunto de nodos, tal que se cumple lo siguiente. Sean $\chi \subset \Omega$ un conjunto finito de nodos, $\{\Omega_j\}$ un recubrimiento regular de (Ω, χ) y $\{w_j\}$ una partición de la unidad k-estable subordinada a $\{\Omega_j\}$, y sea $f \in N_\Phi(\Omega)$. Entonces el interpolante por partición de la unidad

$$P_f = \sum_j w_j\, u_j, \qquad u_j \in \operatorname{span}\{\Phi(\cdot - x) : x \in \chi \cap \Omega_j\},$$

donde cada u_j interpola a f en $\chi \cap \Omega_j$, verifica, para todo $x \in \Omega$,

$$\big|D^\alpha f(x) - D^\alpha P_f(x)\big| \leq C_{k,\alpha}\, h_{\chi,\Omega}^{k/2-|\alpha|}\, \|f\|_{N_\Phi(\Omega)}.$$

En particular, para $\alpha = 0$,

$$\|f - P_f\|_{L^\infty(\Omega)} \leq C_{k,0}\, h_{\chi,\Omega}^{k/2}\, \|f\|_{N_\Phi(\Omega)},$$

de modo que, siendo k arbitrario, la convergencia es algebraica de orden arbitrariamente alto.

Más aún, si el recubrimiento $\{\Omega_j\}$ se mantiene fijo al refinar χ y los Ω_j tienen frontera Lipschitz, existen constantes $C = C(f) > 0$ y $c > 0$, esta última independiente de f, y un umbral $h_0 > 0$, todos ellos independientes de χ, tales que, para $h_{\chi,\Omega} < h_0$, la cota mejora a un decaimiento exponencial,

$$\|f - P_f\|_{L^\infty(\Omega)} \;\leq\; C\, e^{-c/h_{\chi,\Omega}},$$

con $c > 0$ inversamente proporcional a ε y dependiente de s y del recubrimiento fijo $\{\Omega_j\}$.

Demostración. Sea $\Phi(x) = e^{-\varepsilon^2\|x\|_2^2}$. Fijado $k \in \mathbb{N}$, verificamos las hipótesis del teorema 3.11.

(i) Interpolante bien definido. Si recordamos, por el corolario 2.25, teníamos garantizado que la gaussiana es EDP. Por tanto Φ es ECDP de orden $m = 0$, caso en que la interpolación no requiere aumento polinómico: $\Pi_{-1}^s = \{0\}$, de modo que desaparecen la parte polinómica y la condición de

momentos, y la $(m-1)$-unisolvencia se cumple trivialmente. El Lema 3.12, con $m=0$, garantiza entonces que, en cada parche $\chi_j = \chi \cap \Omega_j$ (no vacío, como vimos en la definición de recubrimiento regular 3.3), el interpolante local $u_j = \sum_{x\in\chi_j} \lambda_x \Phi(\cdot - x)$ existe y es único. Luego $P_f = \sum_j w_j u_j$ está bien definido y se cumple una de las hipótesis del Teorema 3.11.

(ii) Regularidad $\Phi \in C^k_\beta$. La gaussiana cumple que $\Phi \in C^\infty(\mathbb{R}^s)$, por lo que todas sus derivadas son continuas y acotadas en cualquier entorno acotado del origen. Por tanto, para todo $k \in \mathbb{N}$, $\Phi \in C^k(\mathbb{R}^s)$ y sus derivadas de orden k, de acuerdo con la definición 2.27, satisfacen $D^\alpha\Phi(x) = \mathcal{O}(1) = \mathcal{O}(\|x\|_2^0)$ cuando $\|x\|_2 \to 0$. De esta forma, tenemos que $\Phi \in C^k_0(\mathbb{R}^s)$ con $\beta = 0$.

Cabe notar que este valor $\beta = 0$ no tiene por qué ser el óptimo, es decir, el mayor $\beta \in [0,1]$ para el que $\Phi \in C^k_\beta$. No obstante, dicha optimalidad es irrelevante para nuestros fines. Esto es así puesto que, en el Teorema 3.11, el parámetro β interviene en el exponente de la distancia de llenado únicamente a través de la suma $k+\beta$. Como la gaussiana es de clase C^k para todo $k \in \mathbb{N}$, podemos tomar k arbitrariamente grande (con la única exigencia de que la partición de la unidad sea k-estable, a costa de modificar la constante $C_{k,\alpha}$). Por tanto, cualquier incremento del orden que se obtuviera mejorando β se alcanza igualmente aumentando k, de modo que basta trabajar con $\beta = 0$.

Aplicando el teorema 3.11 con $k \geq 0$ fijo y $\beta = 0$, para cada $|\alpha| \leq k/2$ y todo $x \in \Omega$,

$$\left|D^\alpha f(x) - D^\alpha P_f(x)\right| \leq C_{k,\alpha}\, h_{\chi,\Omega}^{(k+\beta)/2-|\alpha|}\, \|f\|_{N_\Phi(\Omega)} = C_{k,\alpha}\, h_{\chi,\Omega}^{k/2-|\alpha|}\, \|f\|_{N_\Phi(\Omega)},$$

con $C_{k,\alpha}$ independiente de f y de χ (Teorema 3.11). En particular, para $\alpha = 0$, como el segundo miembro no depende de x, tomando supremo en $x \in \Omega$ se obtiene

$$\|f - P_f\|_{L^\infty(\Omega)} = \sup_{x\in\Omega} |f(x) - P_f(x)| \leq C_{k,0}\, h_{\chi,\Omega}^{k/2}\, \|f\|_{N_\Phi(\Omega)}.$$

Puesto que k es arbitrario, el orden de convergencia $k/2$ puede tomarse tan alto como se quiera. Ahora bien, la cota correspondiente exige una partición k-estable y una constante $C_{k,\alpha}$ que depende de k. En particular, no se dispone de una única constante válida simultáneamente para todos los valores de k. Por ello, lo que se obtiene no es un único orden de convergencia, sino una familia de cotas, una por cada k, cada una con su propio exponente $k/2$ y su propia constante.

(iii) Refinamiento exponencial, con recubrimiento fijo de parches. Añadimos ahora la hipótesis de que el recubrimiento $\{\Omega_j\}$ permanezca fijo al refinar χ, es decir, que el refinamiento incorpora nodos dentro de los parches ya existentes, sin subdividirlos ni multiplicarlos. Veamos que existen constantes $C > 0$, $\tilde{c} > 0$, independientes de χ, y un umbral $h_0 > 0$, tales que, para $h_{\chi,\Omega} < h_0$,

$$\|f - P_f\|_{L^\infty(\Omega)} \;\leq\; C\, e^{-\tilde{c}/h_{\chi,\Omega}}.$$

Para ello, como por hipótesis, $w_j \geq 0$ y $\sum_j w_j = 1$ en Ω, por el lema 3.23, para todo $x \in \Omega$ se tiene

$$|f(x) - P_f(x)| \;\leq\; \max_{j\,:\,x\in\Omega_j} |f(x) - u_j(x)| \;\leq\; \max_{1\leq j\leq M} \sup_{y\in\Omega_j} |f(y) - u_j(y)|.$$

Basta, por tanto, acotar exponencialmente cada error local $\sup_{\Omega_j} |f - u_j|$, con constantes uniformes en j.

Ahora bien, cada u_j es, por construcción, el interpolante RBF gaussiano global de f sobre el subdominio Ω_j con los nodos $\chi_j = \chi \cap \Omega_j$, donde, por hipótesis, cada Ω_j es acotado con frontera Lipschitz, de modo que el teorema 2.31 se aplica directamente a cada parche. Además, por hipótesis, $f \in N_\Phi(\Omega)$ y, como comentamos al final del apartado 2.4.5, esto es suficiente para aplicar el teorema 2.31, para el cual se verifican todas las hipótesis localmente en cada parche. Por tanto, para cada $j \in \{1, \ldots, M\}$, existen $h_{0,j} > 0$, $C_j, c_j > 0$ tales que, si $h_{\chi_j,\Omega_j} < h_{0,j}$,

$$\sup_{\Omega_j} |f - u_j| \;\leq\; C_j \, e^{-c_j / h_{\chi_j,\Omega_j}}.$$

Por ser M finito, definimos

$$C := \max_{1 \leq j \leq M} C_j \;<\; \infty, \qquad h_{0,*} := \min_{1 \leq j \leq M} h_{0,j} \;>\; 0,$$

que son cotas válidas para los M parches simultáneamente. En cuanto a c_j, este depende únicamente del parámetro de forma ε, no de f ni del dominio y, como ε es común a todos los interpolantes locales por hipótesis del corolario, $c_* := c_j$ para todo j. Así, si $h_{\chi_j,\Omega_j} < h_{0,*}$,

$$\sup_{\Omega_j} |f - u_j| \;\leq\; C \, e^{-c_* / h_{\chi_j,\Omega_j}}.$$

Por la condición 3 de la definición de recubrimiento regular 3.3, existe $C_0 > 0$ tal que

$$h_{\chi_j,\Omega_j} \;\leq\; C_0 \, h_{\chi,\Omega} \qquad \text{para todo } j.$$

Ahora, sea $h_0 := h_{0,*}/C_0$, observamos que si $h_{\chi,\Omega} < h_0$, entonces $h_{\chi_j,\Omega_j} \leq C_0 \, h_{\chi,\Omega} < h_{0,*}$ para todo j. Además, como $t \mapsto e^{-c_*/t}$ es creciente para $t > 0$, tenemos que, para $h_{\chi,\Omega} < h_0$,

$$\sup_{\Omega_j} |f - u_j| \;\leq\; C \, e^{-c_* / h_{\chi_j,\Omega_j}} \;\leq\; C \, e^{-c_* / (C_0 \, h_{\chi,\Omega})} \;=\; C \, e^{-\tilde{c}/h_{\chi,\Omega}}, \qquad \tilde{c} := \frac{c_*}{C_0} > 0,$$

Por tanto,

$$|f(x) - P_f(x)| \;\leq\; \max_{1 \leq j \leq M} \sup_{\Omega_j} |f - u_j| \;\leq\; C_* \, e^{-\tilde{c}/h_{\chi,\Omega}}, \forall x \in \Omega, h_{\chi,\Omega} < h_0$$

y tomando supremo en $x \in \Omega$, se obtiene la desigualdad buscada. □

Nota 3.25 (Sobre la convergencia exponencial del caso gaussiano)**.** Es importante señalar que el decaimiento exponencial deducido en el apartado (iii) de la demostración del corolario 3.24 se ha obtenido bajo dos restricciones que conviene tener en cuenta. La primera es la hipótesis de recubrimiento fijo: el argumento cubre el refinamiento mediante la adición de nodos dentro de los parches existentes, pero no el caso en que el propio recubrimiento se refina a la vez, manteniendo el número de nodos por parche fijo. La segunda es que el argumento es para $\alpha = 0$, pero no se extiende a las cotas sobre derivadas, con $|\alpha| \geq 1$.

Señalemos antes de seguir que, si buscamos algún tipo de caracterización o criterio para identificar qué funciones están en $\mathcal{N}_\Phi$, para poder aplicarles el corolario 3.24, este puede encontrarse en el desarrollo realizado en el apéndice A.

Ahora bien, una vez tenemos una cota teórica para el orden de convergencia, conviene precisar que esto no nos garantiza que el error decaiga exactamente con ese orden. Por este motivo, usaremos la ecuación (2.1), no para confirmar un orden previamente conocido, sino como herramienta para estimar el orden de convergencia observado. Así, si la cantidad

$$p_{\text{obs}} = \log_2 \frac{E(h_k)}{E(h_{k+1})}$$

tiende a un número, experimentalmente podremos afirmar que la convergencia es algebraica de ese orden exacto; mientras que, si p_{obs} diverge a infinito, el orden exacto de convergencia será superalgebraico (mayor que algebraico). Por último, tengamos en cuenta que, si p parece estar continuamente oscilando, es posible que hayamos alcanzado el rango de mal condicionamiento del método. Antes de analizar este concepto para los MPU, comparemos nuestros resultados de convergencia con los que obtuvimos para la interpolación por RBF global.

Comparación de los resultados de convergencia para RBF global y MPU

Los corolarios 3.22 y 3.24 reproducen, en el marco de la partición de la unidad, el mismo orden de convergencia que las cotas de interpolación global de los RBF obtenidas en los teoremas 2.30 y 2.31, a pesar de que el método de partición de la unidad reemplaza la resolución de un único sistema con todos los nodos por el ensamblaje de interpolantes locales. De hecho, las cotas no solo comparten el orden, sino la estructura: la dependencia en f, que en dichos teoremas queda absorbida en la constante $C = C(f)$, aparece en los corolarios desglosada como un factor explícito, una norma de f, multiplicado por una constante puramente geométrica. Conviene subrayar que esta correspondencia es estructural y de orden, no de igualdad numérica: al tratarse de métodos distintos (interpolación global frente a partición de la unidad), que producen interpolantes diferentes, las constantes geométricas involucradas no coinciden en valor, aunque sí el orden de convergencia y el tipo de norma de f que interviene.

1. *Caso* $\varphi(r) = r^p$. Asumiendo que la k-estabilidad de los pesos es tal que $k \geq p-1$, el corolario 3.22 proporciona, para $f \in C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})$, la cota

   $$\|f - P_f\|_{L^\infty(\Omega)} \leq D_0 \, h_{\chi,\Omega}^{p/2} \, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})},$$

   cuyo orden $p/2$ coincide exactamente con el del teorema 2.30 para la interpolación global. En ambos casos el orden es finito y queda fijado por la regularidad de $\Phi \in C_1^{p-1}$, reflejo de la suavidad finita de r^p en el origen. La coincidencia se da también en la forma misma de la cota: la constante $C(f)$ del teorema 2.30 se corresponde con el producto $D_0 \, \|f\|_{C^{\lceil (s+p)/2 \rceil}(\overline{\Omega})}$ del corolario, donde D_0 es independiente de f y la dependencia en f se concentra en la norma $C^{\lceil (s+p)/2 \rceil}$, es decir, en sus derivadas hasta el orden $\lceil (s+p)/2 \rceil$.

2. *Caso gaussiano.* El corolario 3.24 proporciona, para nodos suficientemente densos y parches fijos, un decaimiento exponencial

$$\|f - P_f\|_{L^\infty(\Omega)} \leq C\, e^{-c/h_{\chi,\Omega}}\, \|f\|_{N_\Phi(\Omega)},$$

del mismo tipo que el del teorema 2.31 para la interpolación global. Además, como ya adelantamos en su momento, la condición impuesta en el teorema 2.31 sobre la extensión de tipo exponencial de f implica, por el corolario A.6, $f \in N_\Phi(\Omega)$. De hecho la hipótesis del teorema 2.31 se podría rebajar exigiendo únicamente pertenencia al espacio nativo, igual que requerimos para el corolario 3.24. En cualquier caso, la constante $C(f)$ del teorema global se corresponde con el producto de una constante geométrica por la norma nativa $\|f\|_{N_\Phi(\Omega)}$, que recoge la regularidad de f, al igual que la dependencia en "toda la sucesión de derivadas" señalada en el teorema 2.31. De esta forma se concluye que, en este caso, el decaimiento es más rápido que cualquier potencia, y la constante $c > 0$ está, en ambos casos, inversamente ligada al parámetro de forma ε.

Esta coincidencia de órdenes pone de manifiesto que la localización del método de partición de la unidad no degrada la velocidad de convergencia respecto a la interpolación global, salvo en el caso gaussiano con parches variables: orden algebraico finito para r^p frente a decaimiento exponencial para la gaussiana en el caso de parches fijos.

3.4. Escalabilidad de los MPU

El epígrafe 3.3.3 zanjó la cuestión de la convergencia: la localización propia de los MPU no degrada el orden respecto a la interpolación global, salvo en el caso gaussiano con parches variables, como ya vimos en los corolarios 3.22 y 3.24. Estudiamos ahora la escalabilidad en los mismos dos escenarios en los que estudiamos la convergencia: parches fijos y parches variables.

Tanto en el caso gaussiano, como en el asociado a $\Phi(x) = \|x\|^p$ con p impar, el sistema local del parche j-ésimo tiene la forma del sistema aumentado

$$M_j = \begin{pmatrix} A_j & P_j \\ P_j^T & 0 \end{pmatrix} \in \mathbb{R}^{(N_j+L)\times(N_j+L)}, \qquad A_j \in \mathbb{R}^{N_j\times N_j}, \quad P_j \in \mathbb{R}^{N_j\times L},$$

donde A_j es la matriz de interpolación sobre los N_j nodos del parche y P_j el bloque polinómico, con $L = \dim \Pi^s_{m-1} = \binom{s+m-1}{s}$ columnas. El orden m depende del núcleo: recordemos que, para $\Phi(x) = \|x\|^p$ es $m = \frac{p+1}{2}$, mientras que en el caso gaussiano $m = 0$, $L = 0$ y P_j es vacía, de modo que $M_j = A_j$ tiene tamaño N_j. Escribiremos $\widetilde{N}_j := N_j + L$ para el tamaño del sistema local, de forma que las cotas que siguen cubren ambos casos sin más que tomar $L = 0$ en el gaussiano. Como s y p son fijos, L es una constante independiente de N y del parche.

Antes de entrar en el cálculo, notemos lo siguiente. Como para cualquier $p \geq 1$ la función $f(x) = x^p$ es convexa en $[0, \infty)$, la desigualdad de Jensen aplicada a $N_1, \ldots, N_M \geq 0$ da

$$\frac{1}{M}\sum_{j=1}^{M} f(N_j) \;\geq\; f\Big(\frac{1}{M}\sum_{j=1}^{M} N_j\Big).$$

Sustituyendo f y operando,

$$\sum_{j=1}^{M} N_j^p \geq \frac{1}{M^{p-1}} \Big(\sum_{j=1}^{M} N_j \Big)^p \;\geq\; \frac{N^p}{M^{p-1}},$$

donde hemos usado en el último paso que $\sum_j N_j \geq N$, pues $\{\Omega_j\}$ cubre Ω y $\chi \subset \Omega$, por lo que cada nodo pertenece al menos a un parche.

3.4.1. Recubrimiento fijo

En este caso se mantiene fijo el recubrimiento $\{\Omega_j\}_{j=1}^{M}$, con mismo M y mismo radio ρ, de forma que se refina añadiendo nodos dentro de los parches ya existentes. De este modo, como para un número considerable de nodos de Halton se tiene $N_j \sim N/M$[1], este crece sin cota según $N \to \infty$.

Resolver el sistema local M_j mediante eliminación gaussiana o factorización LU (la vía estándar tras el operador \ de MATLAB) tiene un número de operaciones del orden de $\widetilde{N}_j^3$: existen constantes $c, c''' > 0$, dependientes solo del algoritmo, con

$$c''' \widetilde{N}_j^3 \;\leq\; \text{coste}_j \;\leq\; c\, \widetilde{N}_j^3 \qquad \text{para cada parche } j.$$

Como L es constante y $N_j \to \infty$, $\widetilde{N}_j = N_j + L \sim N_j$, de modo que el bloque polinómico no altera el orden y el argumento es común a los dos núcleos. Sumando la cota superior sobre los M parches, y usando $N_j \leq N$ junto con la condición de solapamiento acotado (condición 1 de la definición 3.3, que garantiza $\sum_j N_j \leq KN$),

$$\text{coste(MPU)} = \sum_{j=1}^{M} \text{coste}_j \;\leq\; c \sum_{j=1}^{M} \widetilde{N}_j^3 \;\leq\; c \Big(\max_j \widetilde{N}_j \Big)^2 \sum_{j=1}^{M} \widetilde{N}_j \;\leq\; c\,(N+L)^2\,(KN+ML),$$

donde en la última igualdad L, M y K son constantes. Sumando la cota inferior, y aplicando la desigualdad preliminar, derivada de la desigualdad de Jensen, con $p = 3$ (junto con $\widetilde{N}_j \geq N_j$),

$$\text{coste(MPU)} \;\geq\; c''' \sum_{j=1}^{M} N_j^3 \;\geq\; \frac{c'''}{M^2}\, N^3.$$

Combinando ambas cotas, con c, c''', K, M, L constantes independientes de N, $\text{coste(MPU)} = \Theta(N^3)$. Por su parte, guardar la matriz densa del parche j-ésimo tiene un coste de $\widetilde{N}_j^2$ y, por el mismo argumento, con la desigualdad preliminar con $p = 2$, da $\text{mem(MPU)} = \Theta(N^2)$.

Es decir, tanto el coste como la memoria recuperan exactamente el mismo orden, cúbico y cuadrático respectivamente, que el método RBF global. La razón es que cada uno de los M bloques crece con N exactamente igual que lo haría el bloque global, y dividir el dominio en M partes fijas solo separa el problema en M copias más pequeñas del mismo problema cúbico, sin alterar su orden de crecimiento. Se pierde, por tanto, la ventaja de coste y memoria que motivaba el MPU.

[1] El símbolo $\sim$ indica que las dos cantidades son del mismo orden de magnitud: su cociente permanece acotado entre dos constantes positivas, independientes de N, cuando $N \to \infty$. No expresa igualdad exacta, sino que ambas cantidades crecen al mismo ritmo.

3.4.2. Recubrimiento variable con N_j acotado

En lugar de mantener fijo el recubrimiento, elegimos ahora el diámetro de los parches de modo que $N_j \leq N_{\text{máx}}$ para todo j, con $N_{\text{máx}}$ una cota independiente de N, dejando crecer en su lugar el número de parches M.

Repetimos el argumento anterior, pero ahora con $\widetilde{N}_j = N_j + L \leq N_{\text{máx}} + L =: \widetilde{N}_{\text{máx}}$ acotado uniformemente. Usando de nuevo el solapamiento acotado ($\sum_j N_j \leq KN$) y que cada parche contiene al menos un nodo (condición 3 de la definición 3.3), de donde $M \leq \sum_j N_j \leq KN$, y empleando $\widetilde{N}_j^3 = \widetilde{N}_j^2\,\widetilde{N}_j \leq \widetilde{N}_{\text{máx}}^2\,\widetilde{N}_j$, se tiene que

$$\begin{aligned}\text{coste(MPU)} &= \sum_{j=1}^{M} \text{coste}_j \leq c \sum_{j=1}^{M} \widetilde{N}_j^3 \leq c\,\widetilde{N}_{\text{máx}}^2 \sum_{j=1}^{M} \widetilde{N}_j \\ &= c\,\widetilde{N}_{\text{máx}}^2 \Big(\sum_j N_j + ML \Big) \leq c\,\widetilde{N}_{\text{máx}}^2\, K(1+L)\, N = \mathcal{O}(N),\end{aligned}$$

Razonando de forma análoga para la memoria, pero con un factor $\widetilde{N}_{\text{máx}}$ menos, $\text{mem(MPU)} = \mathcal{O}(N)$. Estas cotas son además ajustadas. Por otro lado, observamos que el interpolante debe emplear cada uno de los N datos al menos una vez, por lo que el coste es también $\geq c''N$, luego el orden es exactamente N,

$$\text{coste(MPU)} = \Theta(N), \qquad \text{mem(MPU)} = \Theta(N).$$

A diferencia del caso con parches fijos, aquí sí se obtiene un coste y un gasto de memoria de orden lineal con N. En consecuencia, solo el recubrimiento variable con N_j acotado proporciona la escalabilidad lineal en coste y memoria que motivaba el método frente al RBF global. Por este motivo, a partir de ahora, descartaremos el método de parches fijos y nos quedaremos con el de parches variables con N_j acotado, para el que estudiaremos su estabilidad en la siguiente sección.

3.5. Condicionamiento del MPU con parches variables y N_j acotado

Tras analizar la escalabilidad de los MPU, descartar el método con recubrimiento fijo y quedarnos con el de parches variables con N_j acotado, en esta sección estudiamos su condicionamiento.

Comenzamos con un resultado que garantiza que el MPU no está peor condicionado que el RBF global.

Lema 3.26 (Monotonía del número de condición bajo submatrices principales)**.** Sea $A \in \mathbb{R}^{N\times N}$ simétrica y definida positiva, y sea $\mathcal{J} \subseteq \{1,\ldots,N\}$ con $|\mathcal{J}| = n \leq N$. Denotemos por $A_{\mathcal{J}} := (A_{ik})_{i,k\in\mathcal{J}} \in \mathbb{R}^{n\times n}$ la submatriz principal asociada a $\mathcal{J}$. Entonces

$$\lambda_{\text{mín}}(A) \;\leq\; \lambda_{\text{mín}}(A_{\mathcal{J}}) \;\leq\; \lambda_{\text{máx}}(A_{\mathcal{J}}) \;\leq\; \lambda_{\text{máx}}(A),$$

y, en particular, $\kappa_2(A_{\mathcal{J}}) \leq \kappa_2(A)$.

Demostración. Identifiquemos $\mathbb{R}^n$ con el subespacio $E_{\mathcal{J}} = \{x \in \mathbb{R}^N : \ x_i = 0 \ \ \forall i \notin \mathcal{J}\} \leq \mathbb{R}^N$ mediante la extensión por ceros $x \mapsto \tilde{x}$, que es una isometría ($\|\tilde{x}\|_2 = \|x\|_2$) y verifica $x^T A_{\mathcal{J}}\, x = \tilde{x}^T A\, \tilde{x}$ para todo $x \in \mathbb{R}^n$. Por el corolario 2.17, toda matriz simétrica B admite las caracterizaciones

$$\lambda_{\text{máx}}(B) = \max_{\|x\|=1} x^T B x \text{ y } \lambda_{\text{mín}}(B) = \min_{\|x\|=1} x^T B x.$$

Aplicadas a $A_{\mathcal{J}}$ y restringiendo el dominio de optimización a $E_{\mathcal{J}} \subseteq \mathbb{R}^N$,

$$\lambda_{\text{máx}}(A_{\mathcal{J}}) = \max_{\substack{x\in\mathbb{R}^n \\ \|x\|=1}} x^T A_{\mathcal{J}}\, x \ = \max_{\substack{\tilde{x}\in E_{\mathcal{J}} \\ \|\tilde{x}\|=1}} \tilde{x}^T A\, \tilde{x} \ \leq \max_{\|\tilde{x}\|=1} \tilde{x}^T A\, \tilde{x} = \lambda_{\text{máx}}(A),$$

$$\lambda_{\text{mín}}(A_{\mathcal{J}}) = \min_{\substack{x\in\mathbb{R}^n \\ \|x\|=1}} x^T A_{\mathcal{J}}\, x \ = \min_{\substack{\tilde{x}\in E_{\mathcal{J}} \\ \|\tilde{x}\|=1}} \tilde{x}^T A\, \tilde{x} \ \geq \min_{\|\tilde{x}\|=1} \tilde{x}^T A\, \tilde{x} = \lambda_{\text{mín}}(A).$$

Como A es definida positiva, $\lambda_{\text{mín}}(A) > 0$, y combinando ambas desigualdades se obtiene $\kappa_2(A_{\mathcal{J}}) \leq \kappa_2(A)$. □

3.5.1. Caso gaussiano

El lema 3.26 da $\kappa_2(A_j) \leq \kappa_2(A)$, lo que, por lo explicado en la subsección 2.4.3, garantiza que el condicionamiento no es peor que en el caso global; veamos si es mejor. Por sí sola, esa cota no ofrece garantía cuantitativa: acotar $\lambda_{\text{máx}}(A_j) \leq \text{tr}(A_j) = N_j$ solo controla el numerador de κ_2, no el denominador $\lambda_{\text{mín}}$, de donde procede realmente el mal condicionamiento. Para ver si la reducción de N_j se traduce en una separación local mayor recurrimos a un argumento de escala.

El volumen de Ω es fijo. Los M parches, cada uno de volumen $\sim \rho^s$, lo cubren con solapamiento acotado por una constante K (condición 1 de la definición 3.3), de modo que $M \cdot \rho^s \sim |\Omega|$, luego $\rho \sim M^{-1/s}$. Para N grande, los N nodos de Halton se reparten aproximadamente por igual entre los M parches, con N_j fijos en cada uno, de modo que $M \sim N/N_j$. Combinando ambas relaciones,

$$\rho \ \sim \ M^{-1/s} \ \sim \ \left(\frac{N_j}{N}\right)^{1/s} :$$

con N_j fijo y $\rho \to 0$, conforme $N \to \infty$. Como dentro de un parche la separación típica entre nodos es del orden de $\rho/N_j^{1/s}$,

$$\text{separación local} \ \sim \ \frac{\rho}{N_j^{1/s}} \ \sim \ \frac{(N_j/N)^{1/s}}{N_j^{1/s}} \ = \ N^{-1/s},$$

el mismo orden en N que la distancia de llenado global del método global (ver demostración de la ecuación (2.1)). Por tanto, con ε fijo (interpolación no estacionaria), esto reproduce a escala de parche exactamente la misma concurrencia de nodos con dominio acotado que en la subsección 2.4.3 identificamos como causa de $\lambda_{\text{mín}}(A) \to 0$ en el problema global, de modo que $\kappa_2(A_j) \to \infty$. Por tanto, concluimos que mantener N_j fijo no mejora, a priori, el condicionamiento.

La mejora real, cuando existe, es de naturaleza geométrica más que teórica: al contener cada parche solo los nodos de una región reducida, su separación mínima suele ser en la práctica algo mayor que la del conjunto completo, sin que exista garantía teórica de ello. Para nodos de Halton, que se distribuyen de forma cuasiuniforme, esta mejora resulta poco apreciable. En consecuencia, el condicionamiento del caso gaussiano deberá comprobarse numéricamente para cada N del rango de estudio, y no de forma analítica de antemano: bastará comenzar por el N mayor del rango, donde el parche es más pequeño y el riesgo de mal condicionamiento es más alto, y si $\texttt{RCOND}(A_j)$ se mantiene por encima del umbral de seguridad de la subsección 2.4.3, se mantendrá ahí también para todo N menor.

3.5.2. Caso $\Phi(x) = \|x\|^p$

Para $\Phi(x) = \|x\|^p$ la situación es distinta y, aquí, sí podemos garantizar un buen condicionamiento; veámoslo.

En primer lugar, notemos que, si construyéramos el sistema local con $\Phi(x) = \|x\|^p$ y la base estándar de polinomios, el condicionamiento empeoraría al crecer N: las entradas $(A_j)_{kl} = \|x_k - x_l\|^p \to 0$, cuando $\rho \to 0$, luego A pasaría a estar mal condicionada. Además, cuando $\rho \to 0$, los monomios de la base estándar evaluados sobre nodos del parche son casi iguales entre sí, por lo que P_j se vuelve también mal condicionada.

Para evitar ambos problemas, realizamos dos reescalados que, como veremos a continuación, por los lemas de invariancia, no alteran el interpolante local.

En primer lugar, empleamos la versión reescalada del Φ, que definimos como $\Phi_\rho = (\|\cdot\|/\rho)^p$. Aunque Φ_ρ depende de ρ (y por tanto de N), por el lema 3.12 da el mismo interpolante local que $\Phi(x) = \|x\|^p$, que es fijo y al que se aplica el resultado de convergencia del corolario 3.22. Además, escribiendo los nodos en coordenadas referidas al centro del parche y reescaladas $y_k = (x_k - c_j)/\rho$, queda

$$(A_j)_{kl} = \Big(\frac{\|x_k - x_l\|}{\rho}\Big)^p = \|y_k - y_l\|^p,$$

que depende solo de la configuración normalizada $\{y_k\} \subset B(0,1)$ y es independiente de ρ, por lo que hemos logrado que nuestra nueva matriz A, que genera el mismo interpolante que antes, no se convierta en computacionalmente singular conforme aumentamos N.

En segundo lugar, para el bloque polinómico tomamos bases recentradas y reescaladas al parche, en lugar de la base estándar de Π^s_{m-1} (escribimos los casos para las tres primeras p y $s = 2$, que son las que estudiaremos en los experimentos numéricos, aunque, por analogía, se pueden construir para otros parámetros de p y s):

$$\begin{aligned}
B_{j,1} &= \{1\} \quad (p = 1),\\
B_{j,3} &= \Big\{1,\ \tfrac{x-c_j^1}{\rho},\ \tfrac{y-c_j^2}{\rho}\Big\} \quad (p = 3),\\
B_{j,5} &= B_{j,3} \cup \Big\{\big(\tfrac{x-c_j^1}{\rho}\big)^2,\ \tfrac{(x-c_j^1)(y-c_j^2)}{\rho^2},\ \big(\tfrac{y-c_j^2}{\rho}\big)^2\Big\} \quad (p = 5),
\end{aligned}$$

es decir, $q_l(x) = \big((x - c_j)/\rho\big)^\gamma$, donde $\gamma = (\gamma^1, \ldots, \gamma^s)$ es el multiíndice del monomio correspondiente, de grado $|\gamma| = \gamma^1 + \cdots + \gamma^s \leq m - 1$, con la notación $y_i^\gamma = (y_i^1)^{\gamma^1} \cdots (y_i^s)^{\gamma^s}$ para las coordenadas $y_i = (y_i^1, \ldots, y_i^s)$. De este modo, $(P_j)_{il} = q_l(x_i) = y_i^\gamma$, también independiente de ρ. Además, tengamos en cuenta que, por el lema 3.13, el interpolante local es invariante por cambios de base, de modo que el reescalado no altera el resultado.

Con ambos ingredientes, el sistema local

$$M_j = \begin{pmatrix} A_j & P_j \\ P_j^T & 0 \end{pmatrix}, \qquad (A_j)_{kl} = \|y_k - y_l\|^p, \quad (P_j)_{il} = y_i^\gamma,$$

depende únicamente de la nube de nodos normalizada $\{y_k\} \subset B(0,1)$, no de ρ. Como N_j está acotado, los y_k son a lo sumo $N_{\text{máx}}$ puntos de la bola unidad cuya separación mínima $\sim N_j^{-1/s}$ está acotada inferiormente por una constante; por tanto $\kappa_2(M_j)$ queda uniformemente acotado en N.

En definitiva, el MPU con $\Phi(x) = \|x\|^p$ y N_j acotado, además de heredar la escalabilidad lineal de la sección 3.4, se mantiene bien condicionado para todo N, a diferencia del caso gaussiano.

Por tanto y, a modo de conclusión para este capítulo, aunque el MPU con N_j acotado y Φ gaussiana logra, a priori, una convergencia más rápida que el caso de $\Phi = ||\cdot||^p$, es debido al mal condicionamiento que aparece en el primero conforme aumentamos N que, en la práctica puede resultar más ventajoso utilizar el método con $\Phi = ||\cdot||^p$, que sí está bien condicionado para todo N, como ya hemos visto. En el siguiente capítulo comprobaremos mediante experimentos numéricos todo lo visto hasta ahora.

Capítulo 4

Experimentos numéricos

En esta sección veremos cómo se usan en la práctica los MPU que hemos desarrollado en la teoría y realizaremos una serie de experimentos numéricos en los que compararemos el MPU con N_j acotado, con los RBF globales, tanto en los casos en que $\Phi = r^p$, con p impar (concretamente para $p = 1$, $p = 3$ y $p = 5$), como con Φ gaussiano.

En primer lugar, decir que, en lo que respecta al MPU, usaremos parches como los definidos en 3.1.1, para los que deberemos comprobar, numéricamente, como dijimos en su momento, que se cumplen dos condiciones: $\sqrt{s}/2 < \alpha < \sqrt{s}$ y $\rho \geq ch_{\chi,\Omega}$, donde elegiremos $c = \frac{7}{4}$, porque es una constante no muy grande (para que el número de nodos por parche no sea demasiado grande, logrando un mejor condicionamiento y escalabilidad) y, como veremos, para nuestros experimentos numéricos funciona bien. Además, para $\Phi(x) = \|x\|^p$, sea

$$L = \dim \Pi^s_{\frac{p-1}{2}} = \binom{s + \frac{p-1}{2}}{s},$$

requeriremos también, por el punto 4) de 3.1.1, que

$\rho \geq \left(\frac{L}{\lambda_s(B(0,1))N}\right)^{1/s}$ de forma holgada, luego en la práctica exigiremos $\rho \geq \frac{7}{4}\left(\frac{L}{\lambda_s(B(0,1))N}\right)^{1/s}$.

En cuanto a los pesos, recurriremos a las denominadas particiones de la unidad de Shepard,

$$w_j(x) = \frac{\varphi_j(x)}{\sum_{i=1}^M \varphi_i(x)}, \text{ donde } \varphi_i(x) := \varphi\big(1 - \|x - c_i\|/\rho\big),$$

donde $\varphi : \mathbb{R} \to \mathbb{R}^+ \cup \{0\}$.

En todos nuestros experimentos numéricos emplearemos las siguientes tres funciones φ

- $\varphi(t) = t_+^4\,(5-4t)$, denominada Wendland C^2,
- $\varphi(t) = t_+^6\,\big(35t^2 - 88t + 56\big)$, denominada Wendland C^4,
- $\varphi(t) = t_+^8\,\big(-32t^3 + 121t^2 - 154t + 66\big)$, denominada Wendland C^6,

que generan los denominados pesos de Wendland-Shepard.

Ahora bien, si imponemos $2 \leq s \leq 3$, $\Omega = (0,1)^s$, $H = 1/(n-1)$ con $n \geq 2$, y considerando la construcción dada por 3.1.1: centros $\{c_j\}_{j=1}^M$ en la rejilla $H\mathbb{Z}^s \cap \overline{\Omega}$ de paso H, parches $\Omega_j = B(c_j, \rho)$ con

$$\rho = \alpha H, \qquad \frac{\sqrt{s}}{2} < \alpha < 1$$

(intervalo no vacío precisamente porque $s \leq 3$), se sabe (ver corolarios B.11, B.12, B.13) que las familias de pesos consideradas $\{\omega_j\}$ para cada una de las tres φ, son exactamente 3-estables, 5-estables y 7-estables, respectivamente.

Ahora bien, veamos cómo reunimos todas las condiciones planteadas hasta ahora para aplicar el método en la práctica.

4.1. Construcción del recubrimiento y descripción del método

A lo largo de esta sección estableceremos cómo construimos nuestro recubrimiento, qué condiciones es necesario imponer sobre él y cómo se implementa el MPU en la práctica; empecemos por el recubrimiento.

4.1.1. Condiciones sobre el recubrimiento y elección de M

Recapitulando todo lo tratado hasta ahora, recordemos qué condiciones deberemos exigir sobre nuestro recubrimiento y experimento numérico sobre $\Phi(x) = \|x\|^p$ y $\Phi(x) = e^{-\epsilon^2||x||^2}$ gaussiana simultáneamente, y cómo estas determinan la elección del número total de parches M (o, equivalentemente, el número de nodos por parche N_j).

Trabajamos con $2 \leq s \leq 3$ y con la construcción de rejilla desarrollada en 3.1.1: centros en una rejilla de n puntos por eje sobre $\overline{\Omega} = [0,1]^s$, con paso $H = 1/(n-1)$, parches $\Omega_j = B(c_j, \rho)$ de radio $\rho = \alpha H$, $\alpha = (n-1)/n$. Además, como en nuestro método el número de parches es variable, imponemos la siguiente relación: $M = n^s$, que determinará cómo varía ρ con N.

Ahora bien, recordemos que, para N suficientemente grande, los nodos de Halton se distribuyen de forma cuasi uniforme sobre Ω, por lo que se tiene

$$M \;\approx\; \frac{N}{N_j}, \qquad\qquad n \;\approx\; \Big(\frac{N}{N_j}\Big)^{1/s}.$$

Como nuestro método se basa en fijar N_j, M se determina como función de N. Así, toda la discusión se reduce a elegir N_j de modo que se cumplan nuestras condiciones requeridas. Tengamos en cuenta que, en esta construcción, $\rho = 1/n$ y que la distancia de llenado de un conjunto de N nodos de Halton en Ω satisface asintóticamente, por el mismo desarrollo que realizamos en el subapartado 2.4.2 y que nos llevó a la ecuación (2.1), $h_{\chi,\Omega} \geq \frac{1}{\lambda_s(B(0,1))^{1/s}} N^{-1/s}$.

Analizamos las condiciones exigidas sobre el recubrimiento una a una:

(1) Cobertura del dominio: $\sqrt{s}/2 < \alpha < 1$. Es una condición sobre el factor de solapamiento α, no sobre M directamente. En la construcción empleada $\alpha = (n-1)/n$, que cumple $\alpha < 1$ para todo n y $\alpha > \sqrt{s}/2$ a partir de cierto n mínimo. Calculémoslo para $2 \leq s \leq 3$:

$$\frac{\sqrt{s}}{2} < \frac{n-1}{n} \iff \frac{\sqrt{s}}{2} n < n-1 \iff 1 < n\left(1 - \frac{\sqrt{s}}{2}\right) \iff n > \frac{1}{1-\frac{\sqrt{s}}{2}}.$$

Como $n \in \mathbb{N}$, si definimos $n_0(s) := \left\lfloor \frac{1}{1-\frac{\sqrt{s}}{2}} \right\rfloor + 1$, esto se traduce en la condición $n \geq n_0(s)$ y, usando que $n \approx (N/N_j)^{1/s}$, llegamos a

$$\frac{N}{n_0^s(s)} \geq N_j, \text{ donde } n_0(2) = 4.$$

Notemos que, de lo anterior, se deduce que $\alpha \geq \alpha_0 := \frac{n_0(s)-1}{n_0(s)}$.

(2) Separación frente a la distancia de llenado: $\rho \geq \frac{7}{4} h_{\chi,\Omega}$. Esta condición la comprobaremos numéricamente en nuestra implementación.

(3) Unisolvencia local para $\Phi(x) = \|x\|^p$*:* $\rho \gtrsim \frac{7}{4}\big(L/(\lambda_s(B(0,1))\, N)\big)^{1/s}$*, con* $L = \binom{s+(p-1)/2}{s}$. Procediendo igual, con $\rho = 1/n$ y $n \approx (N/N_j)^{1/s}$,

$$\frac{\alpha}{n} \;\geq\; \frac{7}{4}\Big(\frac{L}{\lambda_s(B(0,1))\, N}\Big)^{1/s} \iff \alpha\Big(\frac{N_j}{N}\Big)^{1/s} \;\geq\; \frac{7}{4}\Big(\frac{L}{\lambda_s(B(0,1))\, N}\Big)^{1/s},$$

y de nuevo el factor $N^{-1/s}$ se cancela, dejando una cota inferior constante sobre N_j:

$$N_j \;\geq\; N_j^{(max)}(s) := \frac{(\frac{7}{4})^s\, L}{\lambda_s(B(0,1))\, \alpha_0^s}.$$

En resumen, la condición (3) fija una cota inferior para el número de nodos por parche,

$$N_j \;\geq\; N_j^{(max)}(s),$$

mientras que la condición (1) impone $n \geq n_0(s)$, satisfecha automáticamente para N suficientemente grande.

Ahora, que hemos recapitulado todas las condiciones que debemos imponer sobre el recubrimiento, veamos cómo implementar el método en nuestros experimentos numéricos.

4.1.2. Implementación del método

En este apartado explicamos de forma ordenada cómo implementamos el MPU recogido a lo largo de todo este capítulo para realizar nuestros experimentos numéricos y comparar los resultados obtenidos con la interpolación por RBF global.

Fijado $s = 2$, para el MPU, se toma $N_j(s) = \text{mín}\{N \in \mathbb{N} : N \geq N_j^{(max)}(s)\}$ y el menor k_0 tal que $N = (2^{k_0} + 1)^s$ cumple $(N/N_j)^{1/s} \geq n_0(s)$. A partir de ahí, para cada $N = (2^k + 1)^s$ $(k \geq k_0)$ el número de nodos son N puntos de Halton en $\Omega = [0, 1]^s$, y los centros de los parches forman una rejilla de $n = \text{round}\left((N/N_j)^{1/s}\right)$ puntos por eje, con $M = n^s$ parches $\Omega_j = B(c_j, \rho)$ de radio $\rho = \alpha H$, donde $H = 1/(n-1)$ y $\alpha = (n-1)/n$ (esto es, $\rho = 1/n$). Los errores se miden en una malla regular de evaluación de neval $\times$ neval puntos, fija e independiente de N.

Como pesos empleamos los de Shepard–Wendland,

$$w_j(x) = \frac{\varphi_j(x)}{\sum_i \varphi_i(x)}, \qquad \varphi_j(x) = \varphi\Big(1 - \tfrac{\|x - c_j\|}{\rho}\Big),$$

con φ una Wendland C^2, C^4 o C^6, que forman una partición de la unidad subordinada al recubrimiento. Para cada N y cada peso resolvemos, en cada parche j, el interpolante local u_j en dos casos: Φ gaussiana (sin parte polinómica) y $\Phi(x) = \|x\|^p$, $p \in \{1, 3, 5\}$ (con aumento polinómico).

En el caso gaussiano se elige el menor parámetro de forma ε por encima de la cota $9/\sqrt{2}$ (esta garantiza que la función de Franke está en el espacio nativo, ver A.2) tal que un parche representativo mal condicionado de $N_{\text{máx}}$ (el mayor N del barrido) alcance RCOND $\approx 10^{-13}$; ese ε se reutiliza en todos los N.

En el caso $\Phi(x) = \|x\|^p$, el interpolante local es

$$u_j(x) = \sum_{i=1}^{N_j} \lambda_i \, \Phi(x - x_i) + \sum_{l=1}^{L} c_l \, q_l(x),$$

con $B_j = \{q_1, \ldots, q_L\}$ base de Π^s_{m-1}, $m = \frac{p+1}{2}$, $L = \binom{s+(p-1)/2}{s}$. Por estabilidad no usamos ni la base estándar de Π^s_{m-1} ni el núcleo sin reescalar: tomamos bases recentradas y reescaladas al parche y construimos las submatrices locales a partir del núcleo reescalado $\Phi_\rho = (\|\cdot\|/\rho)^p$. Ambos reescalados dejan invariante el interpolante local (lemas 3.13 y 3.12) y son los que mantienen acotado el condicionamiento al crecer N; los detalles y su justificación se exponen en la subsección 3.5.2.

Resueltos los sistemas locales, el interpolante global es $P_f = \sum_j w_j \, u_j$. Finalmente, este se evalúa en la malla de evaluación y se calculan el error máximo, el RMS y el orden observado $p_{obs} =$

$\log_2\big(E(h_k)/E(h_{k+1})\big)$ (ecuación (2.1)). Estos resultados se compararán con los del método RBF global (gaussiano y $\Phi(x) = \|x\|^p$) aplicado a las mismas funciones. Ahora bien, como este resuelve un único sistema denso de tamaño $N \times N$, cuyo coste crece como $\mathcal{O}(N^3)$ y cuyo condicionamiento empeora con N, el barrido en el caso global se limita a valores de N sensiblemente menores que los alcanzados por el MPU.

Para cuantificar esa diferencia de coste medimos también, con `tic/toc`, el tiempo de cómputo de un ajuste completo para cada N y cada Φ: en el MPU con el peso Wendland C^6 (los tiempos son prácticamente independientes del peso, pues el interpolante local no depende de él) y en el RBF global. Para amortiguar el ruido de medición, cada caso se cronometra varias veces descartando una primera ejecución de calentamiento y tomando la mediana. Conforme al orden lineal establecido en la sección 3.4, el coste del MPU es lineal en el número de parches, $M \propto N$, con un factor logarítmico adicional debido a las búsquedas de vecindad sobre un árbol *kd* (`createns/rangesearch`), que operan con $\mathcal{O}(N \log N)$ [7, Ap. A]; frente al $\mathcal{O}(N^3)$ del sistema denso del RBF global. Presentamos, por tanto, los tiempos $t(N)$ de ambos métodos y los cocientes normalizados $\big(t_{k+1}/t_k\big)\big/\big(N_{k+1}\log N_{k+1}\,/\,N_k \log N_k\big)$ para el MPU y $\big(t_{k+1}/t_k\big)\big/\big(N_{k+1}^3/N_k^3\big)$ para el RBF, que deben tender a 1 si el orden empírico de coste del método coincide con el previsto teóricamente.

4.2. Experimentos numéricos del MPU y RBF global

Una vez explicada la implementación del método realizamos una serie de experimentos numéricos en los que comparar la convergencia, estabilidad y escalabilidad de la interpolación por RBF global y por MPU para $\Phi = r^p$, con p impar y Φ gaussiano.

En primer lugar, realizamos un experimento numérico aquí sobre la conocida función de Franke, por analogía los experimentos realizados en su momento en el apartado 2.4.4, aunque aquí se emplea otro parámetro de forma, con el objetivo de lograr un mínimo de estabilidad. El otro caso que se aborda es el de un polinomio de grado dos, con el objetivo de verificar si se produce reproducción polinómica para alguno de los métodos empleados.

4.2.1. Función de Franke

El primer experimento lo realizamos para la función de Franke, que, recordemos, tiene la forma

$$\begin{aligned} f(x,y) = \frac{3}{4}e^{(-1/4)((9x-2)^2+(9y-2)^2)} + \frac{3}{4}e^{(-1/49)(9x+1)^2-(1/10)(9y+1)^2} \\ + \frac{1}{2}e^{(-1/4)((9x-7)^2+(9y-3)^2)} - \frac{1}{5}e^{-(9x-4)^2-(9y-7)^2}. \end{aligned}$$

Observemos que esta función es C^∞, luego siempre le podremos aplicar los resultados de convergencia para $\Phi(x) = \|x\|^p$, con p impar, tanto en el caso de RBF global, como de MPU. Por otro lado, en base al corolario A.9, como nuestro parámetro de forma se elige para que sea mayor que $\varepsilon_0 = \frac{9}{\sqrt{2}}$, la función de Franke está en el espacio nativo para Φ gaussiano. Ahora realizamos el experimento numérico ejecutando el script 15 y obtenemos los siguientes resultados.

Resultados obtenidos para MPU

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	4.0697e-03		3.7487e-02	1.56e-03	4.1403e-03		4.0849e-02	1.56e-03	4.1910e-03		4.2131e-02	1.56e-03
5	1089	1.4372e-03	1.5017	2.0791e-02	1.31e-03	1.4385e-03	1.5252	2.0791e-02	1.31e-03	1.4472e-03	1.5341	2.0791e-02	1.31e-03
6	4225	6.5046e-04	1.1437	1.5566e-02	1.11e-03	6.5162e-04	1.1424	1.6057e-02	1.11e-03	6.5437e-04	1.1450	1.6296e-02	1.11e-03
7	16641	3.1988e-04	1.0239	7.6658e-03	7.67e-04	3.2043e-04	1.0240	7.7585e-03	7.67e-04	3.2238e-04	1.0214	7.7836e-03	7.67e-04
8	66049	1.3615e-04	1.2324	2.5652e-03	5.23e-04	1.3654e-04	1.2307	2.5625e-03	5.23e-04	1.3745e-04	1.2299	2.5624e-03	5.23e-04
9	263169	7.2123e-05	0.9166	1.6256e-03	6.15e-04	7.2427e-05	0.9147	1.5664e-03	6.15e-04	7.2997e-05	0.9130	1.5151e-03	6.15e-04
10	1050625	3.9932e-05	0.8529	8.6141e-04	7.31e-04	4.0317e-05	0.8451	8.8867e-04	7.31e-04	4.0731e-05	0.8417	9.1239e-04	7.31e-04

Tabla 4.1: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^1$ y pesos de Wendland C^2, C^4 y C^6, sobre la función de Franke.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	9.8395e-04		1.1878e-02	1.26e-05	1.0726e-03		1.5438e-02	1.26e-05	1.1326e-03		1.6839e-02	1.26e-05
5	1089	1.3808e-04	2.8331	2.8044e-03	9.49e-06	1.4221e-04	2.9150	2.8044e-03	9.49e-06	1.4592e-04	2.9564	2.8044e-03	9.49e-06
6	4225	3.1915e-05	2.1132	6.1154e-04	6.85e-06	3.2590e-05	2.1255	6.2980e-04	6.85e-06	3.3234e-05	2.1344	6.4597e-04	6.85e-06
7	16641	8.9368e-06	1.8364	2.4579e-04	4.06e-06	9.0569e-06	1.8473	2.6472e-04	4.06e-06	9.1805e-06	1.8560	2.7710e-04	4.06e-06
8	66049	1.9548e-06	2.1927	6.1034e-05	1.09e-06	1.9723e-06	2.1991	6.1034e-05	1.09e-06	1.9948e-06	2.2023	6.1034e-05	1.09e-06
9	263169	5.4368e-07	1.8462	1.5537e-05	2.77e-06	5.5025e-07	1.8417	1.5851e-05	2.77e-06	5.5735e-07	1.8396	1.6124e-05	2.77e-06
10	1050625	1.4494e-07	1.9073	3.6767e-06	2.98e-06	1.4724e-07	1.9019	3.6767e-06	2.98e-06	1.4916e-07	1.9017	3.6767e-06	2.98e-06

Tabla 4.2: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^3$ y pesos de Wendland C^2, C^4 y C^6, sobre la función de Franke.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	6.5052e-04		1.1550e-02	4.98e-07	6.6914e-04		1.1550e-02	4.98e-07	6.8222e-04		1.1550e-02	4.98e-07
5	1089	4.4314e-05	3.8758	9.3204e-04	2.97e-07	4.5565e-05	3.8763	1.0371e-03	2.97e-07	4.6578e-05	3.8725	1.0760e-03	2.97e-07
6	4225	5.5320e-06	3.0019	2.0306e-04	3.07e-07	5.7667e-06	2.9821	2.2854e-04	3.07e-07	5.9523e-06	2.9682	2.4844e-04	3.07e-07
7	16641	6.0665e-07	3.1889	1.7289e-05	1.49e-07	6.3518e-07	3.1825	1.9541e-05	1.49e-07	6.5652e-07	3.1805	2.1767e-05	1.49e-07
8	66049	7.1946e-08	3.0759	3.3032e-06	2.61e-08	7.4543e-08	3.0910	3.3032e-06	2.61e-08	7.6675e-08	3.0980	3.3032e-06	2.61e-08
9	263169	8.6943e-09	3.0488	2.0129e-07	7.43e-08	8.8691e-09	3.0712	2.1707e-07	7.43e-08	8.9907e-09	3.0923	2.2344e-07	7.43e-08
10	1050625	1.1619e-09	2.9036	2.4253e-08	9.82e-08	1.2112e-09	2.8723	2.5470e-08	9.82e-08	1.2485e-09	2.8482	2.5670e-08	9.82e-08

Tabla 4.3: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^5$ y pesos de Wendland C^2, C^4 y C^6, sobre la función de Franke.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	3.7057e-01		1.0413e+00	1.00e+00	3.7057e-01		1.0413e+00	1.00e+00	3.7057e-01		1.0413e+00	1.00e+00
5	1089	3.4995e-01	0.0826	1.0405e+00	8.64e-01	3.4995e-01	0.0826	1.0405e+00	8.64e-01	3.4995e-01	0.0826	1.0405e+00	8.64e-01
6	4225	2.7056e-01	0.3712	1.0159e+00	1.77e-01	2.7055e-01	0.3713	1.0158e+00	1.77e-01	2.7055e-01	0.3713	1.0158e+00	1.77e-01
7	16641	1.0053e-01	1.4283	7.9834e-01	1.02e-02	1.0048e-01	1.4290	7.9921e-01	1.02e-02	1.0049e-01	1.4288	8.0022e-01	1.02e-02
8	66049	1.1014e-02	3.1902	2.9541e-01	8.20e-06	1.1114e-02	3.1765	2.8748e-01	8.20e-06	1.1191e-02	3.1667	2.8179e-01	8.20e-06
9	263169	9.9001e-04	3.4757	2.7756e-02	8.70e-10	1.0018e-03	3.4716	2.8792e-02	8.70e-10	1.0129e-03	3.4657	2.9691e-02	8.70e-10
10	1050625	6.2393e-05	3.9880	3.4522e-03	1.18e-14	6.2904e-05	3.9934	3.4528e-03	1.18e-14	6.3376e-05	3.9984	3.4528e-03	1.18e-14

Tabla 4.4: RMSE y error máximo frente al número de nodos de Halton N para el MPU con la gaussiana no estacionaria ($\varepsilon = 186.5532$) y pesos de Wendland C^2, C^4 y C^6, sobre la función de Franke.

Conclusiones sobre los resultados para el MPU

Antes de comentar los resultados, conviene recordar qué orden de convergencia garantiza la teoría de MPU en cada caso. Para $\Phi(x) = \|x\|^p$, el corolario 3.22 proporciona el exponente $(\kappa + 1)/2$, con $\kappa = \min\{p-1, l\}$, donde l es el orden de estabilidad de la partición empleada. En base a los corolarios B.11, B.12 y B.13, los pesos de Wendland C^2, C^4 y C^6 son exactamente 3, 5 y 7-estables, de modo que $l = 3, 5, 7$ respectivamente. Por su parte, en el caso gaussiano, el corolario 3.24 da directamente el exponente $k/2$, donde k es el orden de k-estabilidad de los pesos, de modo que el mínimo lo fija la suavidad de la partición: 3/2, 5/2 y 7/2 para C^2, C^4 y C^6.

Recogemos los órdenes teóricos garantizados en la siguiente tabla, para poder compararlos con los obtenidos en los experimentos.

	Wendland C^2 (3-estable)	Wendland C^4 (5-estable)	Wendland C^6 (7-estable)
$\Phi(x) = \|x\|$	1/2	1/2	1/2
$\Phi(x) = \|x\|^3$	3/2	3/2	3/2
$\Phi(x) = \|x\|^5$	2	5/2	5/2
Gaussiana	3/2	5/2	7/2

Tabla 4.5: Orden de convergencia garantizado para el MPU por los corolarios 3.22 y 3.24 para cada Φ y cada peso.

Ahora, comparamos los órdenes de convergencia teóricos, con los obtenidos experimentalmente para el MPU con $\Phi(x) = \|x\|^p$.

En este caso, las tablas 4.1, 4.2 y 4.3 muestran que en los tres casos se respeta la cota de la tabla 4.5, con un margen amplio. Para $\Phi(x) = \|x\|$ el orden observado se estabiliza en torno a 1 (valores entre 0.85 y 1.23 a partir de $N = 4225$), frente al 1/2 garantizado; para $\Phi(x) = \|x\|^3$ ronda el 1.9, frente al 3/2; y para $\Phi(x) = \|x\|^5$ se sitúa cerca de 3 (entre 2.90 y 3.19, también a partir de $N = 4225$), frente al 2 o 5/2 garantizados.

Se observa además que la convergencia mejora al aumentar p; mientras que, en cambio, la suavidad del peso apenas influye en la práctica: los errores con C^2, C^4 y C^6 apenas difieren entre sí. Aunque la distinción teórica entre ellos solo era apreciable para $p = 5$, donde se llegaba a rebajar el orden de 5/2 a 2 para los pesos C^2, esto no llega a manifestarse en el experimento numérico, puesto que el orden observado es mayor que el garantizado por la teoría.

En cuanto al condicionamiento, los valores de `RCOND` confirman lo demostrado en la subsección 3.5.2 (el sistema local M_j es independiente de N), ya que se mantienen prácticamente constantes al variar N. El condicionamiento sí empeora, en cambio, al aumentar p, en unos dos órdenes de magnitud por cada incremento. Esto es así porque los monomios de grado mayor son cada vez más parecidos entre sí sobre la bola unidad, lo que empeora el condicionamiento de P_j. En cualquier caso, todos los

valores quedan muy por encima del umbral 10^{-13}, de modo que los resultados son fiables en todo el barrido de N.

En el caso del MPU gaussiano, la tabla 4.4 presenta un comportamiento muy distinto. El orden observado no se estabiliza, sino que crece de forma sostenida (0.08, 0.37, 1.43, 3.19, 3.48, 3.99), y el error es el peor de los cuatro núcleos: para $N = 1050625$ vale $6.24 \cdot 10^{-5}$, comparable al de $\Phi(x) = \|x\|$ y casi cinco órdenes de magnitud peor que el de $\Phi(x) = \|x\|^5$. Nótese que, aun así, la cota del corolario 3.24 se respeta en las últimas filas, donde el orden empírico se encuentra en torno a 7/2 (garantizado para los pesos C^6) para la penúltima N, ya que el último valor de N tiene un `RCOND` inferior a nuestro umbral, que lo hace poco fiable.

Este comportamiento se explica por el compromiso entre precisión y condicionamiento descrito en la subsección 3.5.1. Como allí se probó, con ε fijo, conforme aumentamos N, $\kappa_2(A_j) \to \infty$. Como alcanzamos un N del orden de 10^6, esto nos fuerza a elegir un valor para el parámetro de forma muy grande ($\varepsilon = 186.5532$), muy por encima de la cota $9/\sqrt{2}$ que exige el corolario A.9. Esta elección es la causante de que las gaussianas de la base sean extremadamente estrechas, por lo que, para los primeros valores de N, la aproximación de los interpolantes locales es unos dos órdenes de magnitud peor que en el caso de $\Phi(x) = \|x\|$, exactamente el fenómeno que ya observamos en las figuras 2.3c y 2.3d para el RBF global. Solo al final del barrido, el método empieza a converger con mayor rapidez, hasta que el sistema entra en el régimen de inestabilidad y los valores obtenidos dejan de ser fiables.

Resultados obtenidos para RBF global

k	N	RMS-error	p	max-error	`RCOND`
1	9	3.7538e-01		1.0413e+00	1.00e+00
2	25	3.6635e-01	0.0351	1.0326e+00	1.00e+00
3	81	3.4471e-01	0.0878	1.0271e+00	9.90e-01
4	289	2.6057e-01	0.4037	1.0064e+00	4.29e-01
5	1089	9.4106e-02	1.4693	7.1284e-01	2.10e-02
6	4225	9.1679e-03	3.3596	1.9004e-01	1.71e-05
7	16641	1.1645e-04	6.2988	4.1781e-03	1.00e-13

Tabla 4.6: RMSE y error máximo frente al número de nodos de Halton N para el RBF global con la gaussiana ($\varepsilon = 47.3902$), sobre la función de Franke.

		$\Phi(x) = \|x\|^1$				$\Phi(x) = \|x\|^3$				$\Phi(x) = \|x\|^5$			
k	N	RMS-error	p	max-error	`RCOND`	RMS-error	p	max-error	`RCOND`	RMS-error	p	max-error	`RCOND`
1	9	1.2248e-01		4.0260e-01	1.92e-02	1.1851e-01		3.5696e-01	1.88e-03	6.0184e-01		3.5703e+00	8.90e-04
2	25	5.5060e-02	1.1535	2.5429e-01	3.35e-03	5.1665e-02	1.1977	4.5548e-01	6.81e-05	1.1970e-01	2.3299	1.1614e+00	5.63e-06
3	81	1.2720e-02	2.1139	6.6121e-02	4.91e-04	4.7587e-03	3.4406	3.2805e-02	2.68e-06	5.4475e-03	4.4577	6.5884e-02	4.27e-08
4	289	3.2827e-03	1.9541	3.1167e-02	6.24e-05	5.5641e-04	3.0963	1.3575e-02	4.59e-08	4.1435e-04	3.7167	1.5606e-02	1.48e-10
5	1089	7.9608e-04	2.0439	1.4633e-02	7.40e-06	7.0977e-05	2.9707	2.9349e-03	1.01e-09	2.3504e-05	4.1399	1.4201e-03	6.76e-13
6	4225	2.2063e-04	1.8513	5.1615e-03	9.01e-07	8.8228e-06	3.0080	3.4200e-04	2.34e-11	1.3068e-06	4.1688	6.1529e-05	4.41e-15
7	16641	7.6521e-05	1.5277	1.8028e-03	1.10e-07	2.0211e-06	2.1261	1.5027e-04	5.37e-13	2.3979e-07	2.4462	2.0609e-05	2.65e-17

Tabla 4.7: RMSE y error máximo frente al número de nodos de Halton N para el RBF global con $\Phi(x) = \|x\|^1$, $\Phi(x) = \|x\|^3$ y $\Phi(x) = \|x\|^5$, sobre la función de Franke.

Conclusiones sobre los resultados para el RBF global

La tabla 4.6 recoge el caso de RBF global gaussiano. Su orden observado no se estabiliza en ningún valor, sino que crece sin cota (0.04, 0.09, 0.40, 1.47, 3.36, 6.30), que es exactamente lo que ocurre en el régimen de convergencia exponencial, según la proposición 2.32. Esto concuerda con la teoría, pues el teorema 2.31 nos garantiza un orden de convergencia exponencial. El barrido se detiene en $N = 16641$ porque `RCOND` alcanza justamente 10^{-13}. Nótese, además, que el error alcanzado, $1.16 \cdot 10^{-4}$, es peor que el que el MPU con $\Phi(x) = \|x\|^5$ logra con ese mismo N. Además, este último puede continuar el barrido dos órdenes de magnitud más allá en N.

La tabla 4.7 muestra el RBF global con $\Phi(x) = \|x\|^p$, cuyo orden teórico es $p/2$ por el teorema 2.30. Para $p = 1$ el orden observado ronda 1.9–2.0, por encima del 1/2 garantizado. Para $p = 3$ el orden observado se estabiliza en 3.10, 2.97 y 3.01, salvo la última iteración, que ya entra en un rango de inestabilidad que no la hace fiable (`RCOND` $\approx 5 \cdot 10^{-13}$), lo que está por encima del 3/2 teórico. Por último, para $p = 5$, el orden experimental se estabiliza en torno a 4, mayor que el 5/2 teórico. Además, se observa que, en este último caso, el problema se mal condiciona: `RCOND` cae hasta $4.41 \cdot 10^{-15}$ y $2.65 \cdot 10^{-17}$ en las dos últimas filas, muy por debajo del umbral de seguridad, de modo que esos valores no son fiables. Esto contrasta con el MPU, donde el reescalado de 3.5.2 mantiene el condicionamiento bajo control y permite llegar a $N = 1050625$ sin degradación, debido en parte también a su mejor escalabilidad.

Resultados obtenidos para los tiempos de cómputo

		MPU (C^6) [s]				RBF global [s]			
k	N	r^1	r^3	r^5	Gauss	r^1	r^3	r^5	Gauss
4	289	3.657e-02	4.193e-02	4.000e-02	2.912e-02	3.253e-02	7.219e-02	7.308e-02	3.514e-02
5	1089	7.281e-02	8.827e-02	7.993e-02	7.202e-02	1.505e-01	3.132e-01	3.092e-01	1.390e-01
6	4225	2.160e-01	2.453e-01	2.389e-01	1.998e-01	1.074e+00	1.856e+00	1.867e+00	9.047e-01
7	16641	8.680e-01	1.026e+00	1.213e+00	9.601e-01	3.270e+01	4.027e+01	4.821e+02	3.010e+01
8	66049	1.089e+01	9.803e+00	1.100e+01	9.850e+00				
9	263169	1.049e+02	1.081e+02	1.174e+02	1.154e+02				
10	1050625	9.991e+02	1.621e+03	4.935e+02	5.167e+02				

Tabla 4.8: Tiempos de cómputo (tic/toc) del MPU (Wendland C^6) y del RBF global, por Φ, sobre la función de Franke.

		MPU: $(t_k/t_{k-1})/(N \log N)$-ratio				RBF: $(t_k/t_{k-1})/N^3$-ratio			
k	N	r^1	r^3	r^5	Gauss	r^1	r^3	r^5	Gauss
4	289								
5	1089	0.4282	0.4528	0.4297	0.5319	0.0864	0.0811	0.0791	0.0739
6	4225	0.6405	0.6000	0.6452	0.5991	0.1223	0.1015	0.1034	0.1115
7	16641	0.8763	0.9117	1.1073	1.0477	0.4981	0.3551	4.2271	0.5445
8	66049	2.7676	2.1092	2.0011	2.2637				
9	263169	2.1508	2.4611	2.3818	2.6147				
10	1050625	2.1472	3.3812	0.9478	1.0096				

Tabla 4.9: Cocientes de tiempos normalizados: MPU por $N \log N$ y RBF global por N^3. Valores ≈ 1 confirman el orden esperado.

Conclusiones sobre los resultados para los tiempos de cómputo

La tabla 4.8 confirma la diferencia de escalabilidad establecida en la sección 3.4. Para $N = 16641$, el MPU resuelve el problema en torno a 1 segundo frente a los 30–40 segundos del RBF global, y a partir de ahí este último ya no es aplicable. El MPU alcanza $N = 1050625$ en tiempos que van de los 500 a los 1600 segundos, según la Φ escogida. La columna de $\Phi(x) = \|x\|^5$ del RBF global para $N = 16641$ ($4.82 \cdot 10^2$ segundos, frente a los 40 de los demás núcleos) constituye una anomalía de medición, atribuible al mal condicionamiento de ese caso.

La tabla 4.9 recoge los cocientes normalizados, que deberían tender a 1 si el orden empírico coincidiera con el previsto. Para el RBF global los valores crecen hacia 1 desde abajo (0.08, 0.12, 0.50) sin llegar a alcanzarlo, pues cuando el problema se vuelve mal condicionado, todavía nos encontramos en un régimen preasintótico de convergencia; la excepción es el 4.23 de $\Phi(x) = \|x\|^5$, que recoge la anomalía ya comentada. Para el MPU los cocientes no convergen a uno, pero tampoco se alejan demasiado, oscilando entre 0.43 y 3.38. La clave aquí es que, a partir de $k = 7$, el método tarda más de 0.5 segundos, por lo que el script realiza una única repetición en el cálculo de los tiempos y la medida resulta menos fiable. En conjunto, ambas columnas son compatibles con los órdenes $\mathcal{O}(N \log N)$ y $\mathcal{O}(N^3)$ predichos por la teoría.

4.2.2. Experimento numérico para un polinomio de grado dos

En este experimento, la función a aproximar es el polinomio de grado dos

$$f(x, y) = 2 - x + 3y + x^2 - 2xy + 4y^2.$$

Observemos que esta función es C^∞, luego siempre le podremos aplicar los resultados de convergencia para $\Phi(x) = \|x\|^p$, con p impar, tanto en el caso de RBF global, como de MPU. Por otro lado, en base al corolario A.11, el polinomio no está en el espacio nativo para Φ gaussiana, por lo que no tenemos ninguna garantía teórica de convergencia sobre él.

Sin embargo, este es un caso que resulta de especial interés a la hora de estudiar la reproducción polinómica que se observa para nuestra Φ polinómica y que pondremos a prueba en el experimento numérico. Para ello, tendremos en cuenta el siguiente resultado teórico.
Proposición 4.1 (Reproducción polinómica del MPU para $\Phi(x) = \|x\|^p$)**.** La función $\Phi(x) = \|x\|^p$ con p impar es condicionalmente definida positiva de orden $m = \frac{p+1}{2}$, y se aumenta con Π^s_{m-1}, para construir el interpolante local. Por el corolario 3.15, el interpolante global reproduce exactamente los polinomios de grado a lo sumo $m - 1 = \frac{p-1}{2}$. En particular, $\Phi(x) = \|x\|^1$ reproduce Π^s_0 (constantes), $\Phi(x) = \|x\|^3$ reproduce Π^s_1 (grado ≤ 1) y $\Phi(x) = \|x\|^5$ reproduce Π^s_2 (grado ≤ 2), en concordancia con la base empleada en la implementación.

Teniendo en cuenta el resultado que acabamos de enunciar, realizamos el experimento numérico ejecutando el script 16 y obtenemos los siguientes resultados.

Resultados obtenidos para MPU

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	2.4620e-02		3.4618e-01	1.56e-03	2.4645e-02		3.4618e-01	1.56e-03	2.4682e-02		3.4618e-01	1.56e-03
5	1089	9.6915e-03	1.3450	2.5274e-01	1.31e-03	9.7020e-03	1.3449	2.5274e-01	1.31e-03	9.7309e-03	1.3428	2.5274e-01	1.31e-03
6	4225	4.2335e-03	1.1949	1.1976e-01	1.11e-03	4.2350e-03	1.1959	1.1976e-01	1.11e-03	4.2451e-03	1.1968	1.1976e-01	1.11e-03
7	16641	1.9439e-03	1.1229	4.8050e-02	7.67e-04	1.9450e-03	1.1226	4.8050e-02	7.67e-04	1.9521e-03	1.1208	4.8050e-02	7.67e-04
8	66049	9.0323e-04	1.1058	2.3653e-02	5.23e-04	9.0366e-04	1.1060	2.3653e-02	5.23e-04	9.0622e-04	1.1071	2.3653e-02	5.23e-04
9	263169	4.6935e-04	0.9444	1.2458e-02	6.15e-04	4.6864e-04	0.9473	1.2458e-02	6.15e-04	4.6972e-04	0.9481	1.2458e-02	6.15e-04
10	1050625	2.4229e-04	0.9539	6.4886e-03	7.31e-04	2.4258e-04	0.9500	6.4886e-03	7.31e-04	2.4351e-04	0.9478	6.4886e-03	7.31e-04

Tabla 4.10: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^1$ y pesos de Wendland C^2, C^4 y C^6, sobre la función test.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	9.6973e-04		1.8779e-02	1.26e-05	9.7140e-04		1.8779e-02	1.26e-05	9.7369e-04		1.8779e-02	1.26e-05
5	1089	1.6385e-04	2.5652	4.5042e-03	9.49e-06	1.6457e-04	2.5613	4.5042e-03	9.49e-06	1.6535e-04	2.5579	4.5042e-03	9.49e-06
6	4225	3.3564e-05	2.2874	1.0577e-03	6.85e-06	3.3675e-05	2.2890	1.0577e-03	6.85e-06	3.3802e-05	2.2904	1.0577e-03	6.85e-06
7	16641	8.5541e-06	1.9722	3.1480e-04	4.06e-06	8.5789e-06	1.9728	3.1480e-04	4.06e-06	8.6130e-06	1.9725	3.1480e-04	4.06e-06
8	66049	2.1935e-06	1.9634	9.7331e-05	1.09e-06	2.1852e-06	1.9730	9.7331e-05	1.09e-06	2.1849e-06	1.9790	9.7331e-05	1.09e-06
9	263169	4.8987e-07	2.1628	1.2346e-05	2.77e-06	4.9046e-07	2.1556	1.2346e-05	2.77e-06	4.9261e-07	2.1490	1.2346e-05	2.77e-06
10	1050625	1.3370e-07	1.8734	3.1107e-06	2.98e-06	1.3398e-07	1.8721	3.1107e-06	2.98e-06	1.3450e-07	1.8729	3.1107e-06	2.98e-06

Tabla 4.11: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^3$ y pesos de Wendland C^2, C^4 y C^6, sobre la función test.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	9.8877e-16		5.3291e-15	4.98e-07	9.7664e-16		7.1054e-15	4.98e-07	9.8925e-16		5.3291e-15	4.98e-07
5	1089	1.0474e-15	-0.0831	9.7700e-15	2.97e-07	1.0706e-15	-0.1326	9.7700e-15	2.97e-07	1.0682e-15	-0.1107	9.7700e-15	2.97e-07
6	4225	1.0544e-15	-0.0096	5.3291e-15	3.07e-07	1.0916e-15	-0.0280	5.3291e-15	3.07e-07	1.0943e-15	-0.0349	6.2172e-15	3.07e-07
7	16641	1.4675e-15	-0.4770	8.9706e-14	1.49e-07	1.4921e-15	-0.4509	8.9706e-14	1.49e-07	1.4906e-15	-0.4459	8.9706e-14	1.49e-07
8	66049	1.1342e-15	0.3717	1.2434e-14	2.61e-08	1.1538e-15	0.3710	1.2434e-14	2.61e-08	1.1560e-15	0.3668	1.2434e-14	2.61e-08
9	263169	1.1020e-15	0.0416	8.8818e-15	7.43e-08	1.1355e-15	0.0230	8.8818e-15	7.43e-08	1.1525e-15	0.0043	8.8818e-15	7.43e-08
10	1050625	1.1087e-15	-0.0087	1.5987e-14	9.82e-08	1.1566e-15	-0.0265	1.5987e-14	9.82e-08	1.1716e-15	-0.0237	1.5987e-14	9.82e-08

Tabla 4.12: RMSE y error máximo frente al número de nodos de Halton N para el MPU con $\Phi(x) = \|x\|^5$ y pesos de Wendland C^2, C^4 y C^6, sobre la función test.

		Wendland C^2				Wendland C^4				Wendland C^6			
k	N	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND	RMS-error	p	max-error	RCOND
4	289	4.4642e+00		9.0000e+00	1.00e+00	4.4642e+00		9.0000e+00	1.00e+00	4.4642e+00		9.0000e+00	1.00e+00
5	1089	4.2174e+00	0.0820	9.0000e+00	8.64e-01	4.2174e+00	0.0820	9.0000e+00	8.64e-01	4.2174e+00	0.0820	9.0000e+00	8.64e-01
6	4225	3.2730e+00	0.3657	8.9603e+00	1.77e-01	3.2729e+00	0.3658	8.9603e+00	1.77e-01	3.2729e+00	0.3658	8.9603e+00	1.77e-01
7	16641	1.2467e+00	1.3925	7.9106e+00	1.02e-02	1.2464e+00	1.3929	7.9106e+00	1.02e-02	1.2466e+00	1.3926	7.9106e+00	1.02e-02
8	66049	1.6655e-01	2.9041	4.2864e+00	8.20e-06	1.6736e-01	2.8967	4.2864e+00	8.20e-06	1.6797e-01	2.8917	4.2864e+00	8.20e-06
9	263169	1.8720e-02	3.1534	7.3722e-01	8.70e-10	1.8798e-02	3.1543	7.3722e-01	8.70e-10	1.8847e-02	3.1558	7.3722e-01	8.70e-10
10	1050625	1.4041e-03	3.7368	1.0869e-01	1.18e-14	1.4088e-03	3.7380	1.0869e-01	1.18e-14	1.4137e-03	3.7368	1.0869e-01	1.18e-14

Tabla 4.13: RMSE y error máximo frente al número de nodos de Halton N para el MPU con la gaussiana no estacionaria ($\varepsilon = 186.5532$) y pesos de Wendland C^2, C^4 y C^6, sobre la función test.

Conclusiones sobre los resultados para el MPU

Como la función test es un polinomio y, por tanto, C^∞, el corolario 3.22 se aplica igual que en el caso de Franke, y los órdenes garantizados son los mismos de la tabla 4.5, pues dependen solo de p y de la estabilidad de los pesos: 1/2 para $\Phi(x) = \|x\|$, 3/2 para $\Phi(x) = \|x\|^3$ y 2 o 5/2 para $\Phi(x) = \|x\|^5$, según el peso empleado.

Las tablas 4.10 y 4.11 muestran que la cota vuelve a respetarse con holgura. Para $\Phi(x) = \|x\|$ el orden observado se estabiliza en torno a 1 (entre 0.94 y 1.19 a partir de $N = 4225$), frente al 1/2 garantizado; y para $\Phi(x) = \|x\|^3$ ronda el 2 (entre 1.87 y 2.29 en ese mismo rango), frente al 3/2.

El caso $\Phi(x) = \|x\|^5$ lo comentamos por separado, pues es el motivo principal de este experimento. Así es que, en la tabla 4.12 no debemos fijarnos en los valores obtenidos para p, sino en el error constante del orden de 10^{-15} en RMS y de 10^{-14} en norma del máximo (valores muy cercanos en orden a la precisión de Matlab), para todos los valores de N y para los tres pesos. Este es precisamente el comportamiento que anticipa la proposición 4.1: como $\Phi(x) = \|x\|^5$ es ECDP de orden $m = 3$ y se aumenta con Π_2^s, luego el interpolante reproduce exactamente los polinomios de grado a lo sumo 2, y nuestra función test es justamente un polinomio de grado dos. El error que se observa no es, por tanto, error de aproximación, sino únicamente el error de redondeo de la aritmética en doble precisión: $|f|$ alcanza el valor 9 en Ω, de modo que un error absoluto de 10^{-15} corresponde a un error relativo del orden de 10^{-16}, esto es, la precisión de la máquina. Por la misma razón, la columna del orden observado carece de sentido en esta tabla: los valores oscilan en torno a cero, incluso con signo negativo, porque (2.1) está comparando cantidades que son puro ruido de redondeo.

La reproducción polinómica explica además, los errores obtenidos para los tres p. Por la proposición 4.1, $\Phi(x) = \|x\|$ reproduce solo las constantes y $\Phi(x) = \|x\|^3$ los polinomios de grado ≤ 1, de modo que al interpolar nuestro polinomio de grado dos el primero deja sin reproducir los términos lineales y cuadráticos, el segundo únicamente los cuadráticos, y el tercero no deja ninguno. Esto se refleja en los errores para $N = 1050625$: $2.42 \cdot 10^{-4}$, $1.34 \cdot 10^{-7}$ y $1.11 \cdot 10^{-15}$ respectivamente.

En cuanto al condicionamiento, los valores de `RCOND` son idénticos a los del experimento de Franke, como era de esperar, puesto que las matrices locales dependen únicamente de los nodos y de los parches, y no de la función que se interpola. Se mantienen así las conclusiones ya obtenidas: para $\Phi(x) = \|x\|^p$ el condicionamiento permanece prácticamente constante al variar N, conforme a lo demostrado en la subsección 3.5.2, y empeora al aumentar p, pero sin acercarse en ningún momento al umbral 10^{-13}.

Por último, en el caso gaussiano no disponemos de ninguna garantía teórica, pues, como señalamos al principio de esta subsección, nuestro polinomio no pertenece al espacio nativo de la gaussiana, de modo que se incumple la hipótesis $f \in N_\Phi(\Omega)$ del corolario 3.24. Resulta interesante, aun así, observar qué ocurre en la práctica. La tabla 4.13 muestra que el método converge, con un orden observado que crece de forma sostenida (0.08, 0.37, 1.39, 2.90, 3.15, 3.74) sin llegar a estabilizarse, igual que sucedía con la función de Franke. Ahora bien, los errores son con diferencia los peores de los cuatro núcleos: para los primeros valores de N el RMS vale 4.46, comparable al propio tamaño de la función (cuyo

máximo es 9), que hace que el interpolante sea inservible en ese rango. Para $N = 1050625$ el error, $1.40 \cdot 10^{-3}$, sigue siendo peor que el que $\Phi(x) = \|x\|$ alcanza ya con $N = 4225$. La explicación es la misma que dimos para Franke: el parámetro de forma ha de tomarse muy grande ($\varepsilon = 186.5532$) para que, para el mayor N (orden de 10^6), el problema no esté mal condicionado, que se fundamenta en lo que ya explicamos en 3.5.1, lo que vuelve las gaussianas de la base extremadamente estrechas. Además, `RCOND` vuelve a desplomarse hasta $1.18 \cdot 10^{-14}$, por debajo de nuestro umbral, de modo que la última fila no es fiable.

Resultados obtenidos para el RBF global

k	N	RMS-error	p	max-error	`RCOND`
1	9	4.5123e+00		9.0000e+00	1.00e+00
2	25	4.4464e+00	0.0212	9.0000e+00	1.00e+00
3	81	4.1662e+00	0.0939	8.7981e+00	9.90e-01
4	289	3.1836e+00	0.3881	8.7981e+00	4.29e-01
5	1089	1.2155e+00	1.3891	8.3524e+00	2.10e-02
6	4225	1.6265e-01	2.9017	4.5216e+00	1.71e-05
7	16641	6.4456e-03	4.6573	5.9216e-01	1.00e-13

Tabla 4.14: RMSE y error máximo frente al número de nodos de Halton N para el RBF global con la gaussiana ($\varepsilon = 47.3902$), sobre la función test.

		$\Phi(x) = \|x\|^1$				$\Phi(x) = \|x\|^3$				$\Phi(x) = \|x\|^5$			
k	N	RMS-error	p	max-error	`RCOND`	RMS-error	p	max-error	`RCOND`	RMS-error	p	max-error	`RCOND`
1	9	2.6790e-01		1.4174e+00	1.92e-02	7.6652e-01		5.2476e+00	1.88e-03	7.7054e+00		7.1299e+01	8.90e-04
2	25	1.3759e-01	0.9613	9.1775e-01	3.35e-03	8.5820e-02	3.1589	1.1043e+00	6.81e-05	2.2548e-01	5.0948	3.2778e+00	5.63e-06
3	81	2.7645e-02	2.3153	2.1339e-01	4.91e-04	4.6658e-03	4.2011	9.3040e-02	2.68e-06	5.2660e-03	5.4202	1.1963e-01	4.27e-08
4	289	1.1973e-02	1.2073	1.8965e-01	6.24e-05	1.4264e-03	1.7098	5.1036e-02	4.59e-08	4.5676e-04	3.5272	1.7457e-02	1.48e-10
5	1089	3.9818e-03	1.5882	1.0649e-01	7.40e-06	1.9488e-04	2.8717	1.1349e-02	1.01e-09	3.0369e-05	3.9108	1.7880e-03	6.76e-13
6	4225	1.2732e-03	1.6449	4.9028e-02	9.01e-07	2.2854e-05	3.0921	1.4476e-03	2.34e-11	2.1789e-06	3.8009	1.5582e-04	4.41e-15
7	16641	4.6231e-04	1.4616	2.0907e-02	1.10e-07	9.1101e-06	1.3269	8.3799e-04	5.37e-13	3.9217e-07	2.4741	2.9541e-05	2.65e-17

Tabla 4.15: RMSE y error máximo frente al número de nodos de Halton N para el RBF global con $\Phi(x) = \|x\|^1$, $\Phi(x) = \|x\|^3$ y $\Phi(x) = \|x\|^5$, sobre la función test.

Conclusiones sobre los resultados para el RBF global

Para el RBF global con $\Phi(x) = \|x\|^p$, el orden garantizado es $p/2$ por el teorema 2.30, y la tabla 4.15 vuelve a respetarlo. Para $p = 1$ el orden observado ronda el 1.5 (entre 1.21 y 1.64 a partir de $N = 289$), por encima del 1/2 garantizado. Para $p = 3$ oscila en torno a $3 > 3/2$ (2.87 y 3.09 en las filas centrales), si bien la última iteración, con `RCOND` $\approx 5 \cdot 10^{-13}$, ya no es fiable. Y para $p = 5$ el orden se sitúa en torno a 3.9, por encima de 5/2 teórico, observándose un gran mal condicionamiento, como ocurría con la función de Franke, para los dos últimos N.

Conviene subrayar un contraste con el MPU. Aunque el núcleo $\Phi(x) = \|x\|^5$ es el mismo, aquí no se observa reproducción exacta del polinomio: el error decrece hasta $3.92 \cdot 10^{-7}$, pero no alcanza la precisión de máquina. La razón es que el RBF global, tal y como lo formulamos en el capítulo

2, interpola únicamente con $P_f(x) = \sum_k c_k \, \Phi(x - x_k)$, sin el aumento polinómico ni la condición de momentos y, al no incorporarse el bloque Π^s_{m-1}, no es aplicable el corolario 3.14 y se pierde la reproducción exacta que sí garantiza la proposición 4.1 para el MPU.

En cuanto al caso gaussiano, la tabla 4.14 carece igualmente de respaldo teórico, al no pertenecer el polinomio al espacio nativo. Experimentalmente, el orden observado crece sin estabilizarse (0.02, 0.09, 0.39, 1.39, 2.90, 4.66), un comportamiento cualitativamente análogo al del régimen exponencial que sí probamos para funciones del espacio nativo, y el barrido se detiene de nuevo en $N = 16641$, donde `RCOND` alcanza el umbral 10^{-13}. Cabe notar que, a diferencia de lo que ocurría con el MPU, aquí el error final ($6.45 \cdot 10^{-3}$) es mejor que el del MPU gaussiano para ese mismo N (1.25), lo que se explica por el parámetro de forma sensiblemente menor que admite el problema global ($\varepsilon = 47.3902$ frente a 186.5532).

Capítulo 5

Conclusiones

Como se planteó en la sección 1.3, el objetivo de este Trabajo de Fin de Grado ha sido estudiar la fundamentación, la formulación, la convergencia y la estabilidad de los métodos sin malla aplicados al problema de interpolación de datos dispersos en $\mathbb{R}^s$. Frente a la rigidez geométrica que imponen los métodos basados en malla, descrita en la sección 1.1, estos métodos construyen el interpolante empleando únicamente distancias entre nodos, lo que permite trabajar sobre nubes de puntos sin conectividad estructurada y sobre dominios de geometría arbitraria. De esta forma, a lo largo de este trabajo se han analizado dos métodos sin malla: la interpolación global mediante Funciones de Base Radial (RBF) y los Métodos de Partición de la Unidad (MPU).

El primero de ellos se ha desarrollado en el capítulo 2. En él, se probó que, para $\Phi(x) = \|x\|^p$ con p impar, el teorema 2.30 garantiza un orden de convergencia algebraico $p/2$; mientras que, para el núcleo gaussiano, el teorema 2.31 proporciona un decaimiento exponencial del error. Frente a estos resultados de convergencia, el método presenta dos limitaciones en su aplicación práctica. La primera es de estabilidad: con el parámetro de forma fijo, al aumentar N los nodos se concentran en un dominio acotado y el número de condición crece sin cota, tal y como se analizó en la subsección 2.4.3. La segunda es de coste: la formulación global conduce a un único sistema denso de tamaño $N \times N$, cuya resolución tiene un coste $\Theta(N^3)$ y un consumo de memoria $\Theta(N^2)$, lo que acota el número de nodos que pueden emplearse.

Para solventar limitaciones se introdujeron, en el capítulo 3, los MPU, que sustituyen la resolución de un único sistema global por la de subsistemas más pequeños para la construcción de interpolantes locales sobre un recubrimiento $\{\Omega_j\}$, de forma que $P_f = \sum_j w_j u_j$. Tras el análisis de la sección 3.4 sabemos que no toda forma de refinar conserva esa ventaja. Para el caso de recubrimiento fijo (subsección 3.4.1), en el que se añaden nodos dentro de los parches ya existentes, se tiene $N_j \sim N/M$ y se recuperan exactamente los órdenes $\Theta(N^3)$ y $\Theta(N^2)$ del método global. Sin embargo, es el recubrimiento variable con N_j acotado uniformemente en N (subsección 3.4.2) el que fija el tamaño

máximo de los sistemas locales y reduce coste y memoria a $\Theta(N)$, cotas que además son ajustadas. Por este motivo, esta última es la variante que se conservó en el resto del trabajo.

Esta reducción del coste no se obtiene a costa del orden de aproximación, aunque aquí hay que diferenciar casos en función de Φ. Para $\Phi(x) = \|x\|^p$, el corolario 3.22 proporciona el exponente $(\kappa+1)/2$, con $\kappa = \min\{p-1, l\}$ y l el orden de estabilidad de la partición, de modo que se recupera el orden $p/2$ del método global siempre que los pesos sean suficientemente estables, es decir, $l \geq p-1$. Ahora bien, para los pesos usados en nuestros experimentos numéricos, los Wendland C^2, C^4 y C^6 son, respectivamente, 3-, 5- y 7-estables, por lo que son suficientemente estables hasta $p = 5$, salvo el caso con pesos C^2, en el que el orden garantizado desciende de 5/2 a 2. En el caso gaussiano, en cambio, el corolario 3.24 garantiza un orden $k/2$ para pesos k-estables y, únicamente para el caso de recubrimientos fijos (que no es el nuestro), un decaimiento exponencial. En consecuencia, la variante de parches variables que proporciona la escalabilidad lineal no conserva la garantía de decaimiento exponencial, sino el orden algebraico $k/2$, limitado por la suavidad de la partición. Por tanto, el MPU conserva el orden de convergencia del método global para $\Phi(x) = \|x\|^p$, pero no en el caso gaussiano con parches variables.

En cuanto a la estabilidad, la sección 3.5 condujo también a conclusiones distintas según Φ. Para $\Phi(x) = \|x\|^p$, basta emplear el núcleo reescalado $\Phi_\rho = (\|\cdot\|/\rho)^p$ y una base polinómica recentrada y reescalada al parche, cambios que, por los lemas de invariancia, no alteran el interpolante local. De esta forma, el sistema aumentado M_j deja de depender de N y el condicionamiento no empeora conforme este aumenta. Para el caso gaussiano, el lema 3.26 solo garantiza que la localización no empeora el condicionamiento del método global. Además, en la subsección 3.5.1 se demostró que, con N_j acotado, la separación local es del orden de la distancia de llenado global, por lo que con ε fijo se reproduce a escala de parche la concentración de nodos que hace que $\kappa_2(A_j) \to \infty$. La única mejora que puede observarse es de naturaleza geométrica y no admite ninguna garantía teórica, por lo que el condicionamiento del caso gaussiano ha de comprobarse numéricamente para cada N del rango de estudio.

Los experimentos numéricos del capítulo 4, realizados en dimensión $s = 2$ sobre nodos de Halton, confirmaron nuestro análisis. El orden observado respetó, en todos los casos, las cotas teóricas para el orden, dadas por la tabla 4.5, situándose por encima de ellas, y la suavidad del peso apenas alteró el error, ni siquiera para $p = 5$, donde la teoría sí distinguía entre C^2 y C^4. Los valores de `RCOND` se mantuvieron prácticamente constantes al variar N en el caso $\Phi(x) = \|x\|^p$, acorde con la teoría; mientras que empeoran con N en el gaussiano, en el que el parámetro de forma se eligió muy por encima de la cota que exige el corolario A.9 para poder alcanzar con un condicionamiento aceptable los valores para los N mayores del barrido, con la consiguiente pérdida de precisión en los menores. Respecto al coste, el RBF global dejó de ser aplicable en torno a los 10^4 nodos, al alcanzar `RCOND` el umbral de fiabilidad 10^{-13}, mientras que el MPU con $\Phi(x) = \|x\|^p$ llegó hasta el orden de 10^6 sin degradación del condicionamiento; donde se destaca que, para el mayor N que ambos métodos comparten, el global empleó algunas decenas de segundos frente al orden de un segundo del MPU.

El balance entre ambos métodos queda así invertido respecto a lo que sugieren las cotas teóricas

sobre la convergencia. Aunque el RBF global gaussiano dispone de la garantía de convergencia más rápida, su rango útil de N termina donde el sistema deja de ser estable: sobre la función de Franke, el error que alcanza en ese punto es unos dos órdenes de magnitud mayor que el del MPU con $\Phi(x) = \|x\|^5$ para ese mismo número de nodos (porque el primero aún está en un régimen de convergencia preasintótico), y unos cinco órdenes mayor que el que este último obtiene al agotar su propio barrido. Por otro lado, el experimento de la subsección 4.2.2 verifica que, de acuerdo a la proposición 4.1, el MPU con $\Phi(x) = \|x\|^5$ reproduce de forma exacta los polinomios de grado a lo sumo 2, ya que el error sobre el polinomio de prueba (de grado dos) se mantuvo, a lo lago de todo el barrido en N, en el orden de 10^{-15}, que es la precisión de la aritmética en doble precisión. El RBF global, formulado sin aumento polinómico ni condición de momentos, no presenta esta propiedad.

En conjunto, el paso de un esquema global a un ensamblaje local mediante partición de la unidad conserva, para $\Phi(x) = \|x\|^p$, las propiedades de aproximación del método de partida, transforma un algoritmo de coste cúbico en uno de coste lineal y permite acotar el número de condición de los sistemas que se resuelven con independencia de N. Es esta combinación de precisión, estabilidad y escalabilidad la que explica el papel que los MPU desempeñan hoy en día en el cálculo científico, con aplicación a la resolución numérica de Ecuaciones en Derivadas Parciales, a las simulaciones de grandes deformaciones mediante hidrodinámica de partículas, al acoplamiento de problemas multifísicos sobre nubes de puntos no coincidentes y, más recientemente, a esquemas como las Redes Neuronales Informadas por la Física (PINNs).

El trabajo deja abiertas varias líneas de continuación. En el plano teórico, la nota que sigue al corolario 3.24 restringe el decaimiento exponencial del caso gaussiano al recubrimiento fijo y a $\alpha = 0$; de modo que quedaría por determinar si dicho decaimiento se conserva teóricamente en el caso parches variables, y si la cota se extiende a las derivadas del interpolante. En esta misma dirección, el orden garantizado está limitado por la k-estabilidad de la partición, lo que sugiere estudiar familias de pesos de mayor regularidad. En el plano numérico, todos los experimentos del caso gaussiano se han realizado en aproximación no estacionaria, mientras que la estacionaria solo se abordó para el método global en la subsección 2.4.4: su adaptación al MPU permitiría abordar el compromiso entre precisión y condicionamiento que limita el caso gaussiano. Por último, la construcción explícita de recubrimiento de la subsección 3.1.1 se ha realizado sobre $\Omega = (0,1)^s$ con nodos de Halton, y los experimentos se han limitado a $s = 2$, por lo que resulta natural extender ambos a dominios y distribuciones de nodos más generales y a dimensiones superiores.

Bibliografía

[1] R. A. Adams and J. J. F. Fournier. *Sobolev Spaces*, volume 140 of *Pure and Applied Mathematics*. Academic Press, Amsterdam, 2nd edition, 2003.

[2] J. H. Ahlberg, E. N. Nilson, and J. L. Walsh. *The Theory of Splines and Their Applications*, volume 38 of *Mathematics in Science and Engineering*. Academic Press, New York, 1967.

[3] T. Belytschko, Y. Krongauz, D. Organ, M. Fleming, and P. Krysl. Meshless methods: an overview and recent developments. *Comput. Methods Appl. Mech. Eng.*, 139(1–4):3–47, 1996.

[4] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*, volume 4 of *Studies in Mathematics and its Applications*. North-Holland Publishing Company, Amsterdam, 1978. Reeditado en Classics in Applied Mathematics, SIAM, 2002.

[5] P. J. Davis. *Interpolation and Approximation*. Blaisdell Publishing Company, Waltham, MA, 1963. Reeditado por Dover Publications en 1975.

[6] C. de Boor. *A Practical Guide to Splines*, volume 27 of *Applied Mathematical Sciences*. Springer-Verlag, New York, 1978.

[7] G. E. Fasshauer. *Meshfree Approximation Methods with MATLAB*, volume 6 of *Interdisciplinary Mathematical Sciences*. World Scientific, Singapore, 2007.

[8] R. Franke. A critical comparison of some methods for interpolation of scattered data. Technical Report NPS-53-79-003, Naval Postgraduate School, Monterey, CA, 1979.

[9] P. Grisvard. *Elliptic Problems in Nonsmooth Domains*, volume 24 of *Monographs and Studies in Mathematics*. Pitman Advanced Publishing Program, Boston, MA, 1985. Reeditado por SIAM en Classics in Applied Mathematics, 2011.

[10] J. H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numer. Math.*, 2:84–90, 1960.

[11] E. J. Kansa. Multiquadrics—a scattered data approximation scheme with applications to computational fluid-dynamics—ii solutions to parabolic, hyperbolic and elliptic partial differential equations. *Comput. Math. Appl.*, 19(8-9):147–161, 1990.

[12] D. Levin. The approximation power of moving least-squares. *Math. Comp.*, 67(224):1517–1531, 1998.

[13] G.-R. Liu. *Mesh Free Methods: Moving Beyond the Finite Element Method*. CRC Press, Boca Raton, FL, 2002.

[14] W. C. H. McLean. *Strongly Elliptic Systems and Boundary Integral Equations*. Cambridge University Press, Cambridge, 2000.

[15] C. A. Micchelli. Interpolation of scattered data: Distance matrices and conditionally positive definite functions. *Constr. Approx.*, 2(1):11–22, 1986.

[16] P. K. Mishra, S. K. Nath, M. K. Sen, and G. E. Fasshauer. Hybrid Gaussian–cubic radial basis functions for scattered data interpolation. *Comput. Geosci.*, 22(5):1203–1218, 2018.

[17] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.*, 378:686–707, 2019.

[18] M. Reed and B. Simon. *Methods of Modern Mathematical Physics II: Fourier Analysis, Self-Adjointness*. Academic Press, New York, 1975.

[19] D. Schneider and B. Uekermann. Efficient partition-of-unity radial-basis-function interpolation for coupled problems. *SIAM J. Sci. Comput.*, 47(2):B558–B582, 2025.

[20] L. L. Schumaker. *Spline Functions: Basic Theory*. Cambridge University Press, Cambridge, 3rd edition, 2007.

[21] E. M. Stein. *Singular Integrals and Differentiability Properties of Functions*, volume 30 of *Princeton Mathematical Series*. Princeton University Press, Princeton, NJ, 1970.

[22] I. Steinwart, D. Hush, and C. Scovel. An explicit description of the reproducing kernel Hilbert spaces of Gaussian RBF kernels. *IEEE Trans. Inf. Theory*, 52(10):4635–4643, 2006.

[23] J. Stoer and R. Bulirsch. *Introduction to Numerical Analysis*, volume 12 of *Texts in Applied Mathematics*. Springer, New York, 3rd edition, 2002.

[24] G. Strang and G. Fix. *An Analysis of the Finite Element Method*. Prentice-Hall, Englewood Cliffs, NJ, 1973. Reeditado por Wellesley-Cambridge Press en 2008.

[25] H. Wendland. *Scattered Data Approximation*. Cambridge University Press, Cambridge, 2005.

[26] T.-T. Wong, W.-S. Luk, and P.-A. Heng. Sampling with Hammersley and Halton points. *J. Graph. Tools*, 2:9–24, 1997.

Apéndice A

Funciones en $\mathcal{N}_\Phi$, con Φ gaussiana

A.1. Algunos resultados teóricos sobre $\mathcal{N}_\Phi$, con Φ gaussiana

Con el objetivo de obtener una forma más eficiente que la estudiada en 3.2.1 para reconocer qué funciones están en el espacio nativo de la gaussiana, y poder aplicar el resultado 3.24 sobre convergencia, introducimos el siguiente teorema general.

Teorema A.1. [25, Teorema 10.12] Supongamos que $\Phi \in C(\mathbb{R}^s) \cap L_1(\mathbb{R}^s)$ es una función EDP con valores reales. Entonces el espacio nativo de Φ en $\mathbb{R}^s$ es:

$$\mathcal{N}_\Phi(\mathbb{R}^s) = \left\{ f \in L_2(\mathbb{R}^s) \cap C(\mathbb{R}^s) : \widehat{f}/\sqrt{\widehat{\Phi}} \in L_2(\mathbb{R}^s) \right\},$$

que es claramente un espacio vectorial y donde la norma $\|f\|^2_{\mathcal{N}_\Phi} = (2\pi)^{-s/2} \int_{\mathbb{R}^s} |\widehat{f}(\omega)|^2/\widehat{\Phi}(\omega)\, d\omega$.

Ahora, particularizamos el teorema A.1 al caso gaussiano.

Corolario A.2 (Caracterización del espacio nativo gaussiano)**.** Sea $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$, $\varepsilon > 0$. Entonces $\Phi \in C(\mathbb{R}^s) \cap L_1(\mathbb{R}^s)$ es real y definida positiva, con transformada de Fourier

$$\widehat{\Phi}(\omega) = \frac{1}{(2\varepsilon^2)^{s/2}}\, e^{-\|\omega\|^2/(4\varepsilon^2)},$$

que es estrictamente positiva en todo $\mathbb{R}^s$. Por el teorema A.1, una función $f \in L_2(\mathbb{R}^s) \cap C(\mathbb{R}^s)$ pertenece a $\mathcal{N}_\Phi(\mathbb{R}^s)$ si y solo si

$$\int_{\mathbb{R}^s} \left|\widehat{f}(\omega)\right|^2 e^{\|\omega\|^2/(4\varepsilon^2)}\, d\omega \;<\; \infty.$$

Demostración. El gaussiano es continuo, integrable ($\int e^{-\varepsilon^2\|x\|^2} dx = (\pi/\varepsilon^2)^{s/2}$) y EDP (corolario 2.25). Su transformada de Fourier, como vimos en la demostración de 2.25, es la gaussiana indicada en el enunciado, que es estrictamente positiva. Ahora, sustituyendo

$$\widehat{\Phi}(\omega)^{-1} = (2\varepsilon^2)^{s/2} e^{\|\omega\|^2/(4\varepsilon^2)} \text{ en la condición } \widehat{f}/\sqrt{\widehat{\Phi}} \in L_2$$

del teorema A.1, se obtiene, salvo la constante $(2\varepsilon^2)^{s/2}$, la integral del enunciado. La equivalencia con el decaimiento gaussiano de $\widehat{f}$ es inmediata: la integral converge si y solo si $|\widehat{f}(\omega)|^2 e^{\|\omega\|^2/(4\varepsilon^2)}$ es integrable, lo que exige $|\widehat{f}(\omega)| \lesssim e^{-\|\omega\|^2/(8\varepsilon^2)}$ salvo factores de crecimiento subgaussiano[1]. □

Ahora bien, la caracterización anterior se refiere al espacio nativo sobre $\mathbb{R}^s$, mientras que el corolario 3.24, que es el que queremos aplicar, exige la pertenencia al espacio nativo sobre el dominio acotado Ω, tal y como lo definimos en 3.10. El siguiente resultado clásico relaciona ambos espacios y es el que nos permitirá trasladar a Ω lo que probemos sobre $\mathbb{R}^s$.

Teorema A.3 (Restricción del espacio nativo)**.** [25, Cap. 10] Sea $\Phi \in C(\mathbb{R}^s) \cap L_1(\mathbb{R}^s)$ una función EDP y sea $\Omega \subseteq \mathbb{R}^s$ no vacío. Entonces $N_\Phi(\Omega)$ y $\mathcal{N}_\Phi(\mathbb{R}^s)$ se relacionan mediante

$$N_\Phi(\Omega) = \big\{F|_\Omega \ : \ F \in \mathcal{N}_\Phi(\mathbb{R}^s)\big\}.$$

Es decir, el espacio nativo sobre Ω está formado exactamente por las restricciones a Ω de las funciones del espacio nativo sobre $\mathbb{R}^s$. Además, se tiene que $||f||_{N_\Phi(\Omega)} \leq ||f||_{\mathcal{N}_\Phi(\Omega)}$

Necesitaremos además la siguiente propiedad, específica del núcleo gaussiano, que describe la regularidad de los elementos de su espacio nativo.

Lema A.4 (Analiticidad de los elementos del espacio nativo gaussiano)**.** [22] Sea $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$ con $\varepsilon > 0$. Toda función $f \in \mathcal{N}_\Phi(\mathbb{R}^s)$ es la restricción a $\mathbb{R}^s$ de una función entera $F : \mathbb{C}^s \to \mathbb{C}$. En particular, f es real y analítica en $\mathbb{R}^s$.

Una vez obtenida una caracterización para las funciones del espacio nativo, veamos, para justificar la afirmación que realizamos al final de la subsección 2.4.5, que toda función que admite una extensión entera de tipo exponencial está en el espacio nativo del gaussiano, para lo que primero introduciremos un resultado previo.

Teorema A.5 (Teorema de Paley–Wiener, versión L_2 con soporte compacto)**.** [18, Teorema IX.3] Sea $\gamma > 0$ y $f \in L_2(\mathbb{R}^s)$. Entonces f admite una extensión a una función entera $f : \mathbb{C}^s \to \mathbb{C}$ que satisface, para ciertas constantes $A > 0$,

$$|f(z)| \ \leq \ A\, e^{\gamma\|\operatorname{Im} z\|} \qquad \text{para todo } z \in \mathbb{C}^s,$$

si y solo si su transformada de Fourier $\widehat{f}$ tiene soporte compacto contenido en la bola cerrada $\overline{B(0,\gamma)}$.

Corolario A.6. Sea $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$. Si $f \in L_2(\mathbb{R}^s) \cap C(\mathbb{R}^s)$ admite una extensión entera de tipo exponencial, es decir $f : \mathbb{C}^s \to \mathbb{C}$ con $|f(z)| \leq A\, e^{\gamma\|\operatorname{Im} z\|}$ para ciertas $A, \gamma > 0$ (como la de la hipótesis del teorema 2.31), entonces $f \in \mathcal{N}_\Phi(\mathbb{R}^s)$.

Demostración. Por el teorema de Paley–Wiener, una función de $L_2(\mathbb{R}^s)$ que admite tal extensión entera de tipo exponencial de orden γ tiene transformada de Fourier $\widehat{f}$ de soporte compacto, contenido en la

[1] El símbolo $\lesssim$ denota desigualdad salvo por una constante multiplicativa real positiva, es decir, $A \lesssim B$ equivale a $A \leq C \cdot B$ para algún $C > 0$. La adición explícita de los *factores de crecimiento subgaussiano* indica que la acotación admite estar multiplicada por términos que dependan de ω, siempre y cuando su crecimiento asintótico sea estrictamente más lento que el de una exponencial cuadrática (como, por ejemplo, un crecimiento polinómico).

bola $\overline{B(0,\gamma)}$. Entonces

$$\int_{\mathbb{R}^s} |\widehat{f}(\omega)|^2 \, e^{\|\omega\|^2/(4\varepsilon^2)} \, d\omega = \int_{\overline{B(0,\gamma)}} |\widehat{f}(\omega)|^2 \, e^{\|\omega\|^2/(4\varepsilon^2)} \, d\omega \leq e^{\gamma^2/(4\varepsilon^2)} \int_{\overline{B(0,\gamma)}} |\widehat{f}(\omega)|^2 \, d\omega < \infty.$$

Como $f \in L_2$ se tiene que $\widehat{f} \in L_2$ y, por el corolario A.2, $f \in \mathcal{N}_\Phi(\mathbb{R}^s)$. □

Con esto concluyen los resultados teóricos sobre el espacio nativo. Ahora bien, la comprobación de que ciertas funciones que usamos a lo largo de este trabajo están en él, se realiza en la siguiente sección.

A.2. Funciones de los experimentos numéricos

Como ya hemos introducido al final del apartado anterior, veamos si, ciertas funciones a las que les hemos aplicado interpolantes, tanto locales como globales, basados en Φ gaussiana, pertenecen o no al espacio nativo.

A.2.1. Función de Franke

En primer lugar, veamos que la función de Franke sí pertenece a este espacio, cuya demostración requerirá del siguiente lema previo.

Lema A.7. Sea $g(x) = e^{-(x-b)^\top D(x-b)}$, con $x, b \in \mathbb{R}^2$ y D real, simétrica y definida positiva, se cumple

$$\widehat{g}(\omega) = \frac{1}{2\sqrt{\det D}} e^{-i\omega^T b} e^{-\frac{1}{4}\omega^T D^{-1}\omega}.$$

Demostración. Sea $g(x) = e^{-(x-b)^T D(x-b)}$ una función en $\mathbb{R}^2$, donde asumimos que D es una matriz real, simétrica y definida positiva, utilizando la convención que incluye el factor $1/(2\pi)$ en dimensión 2, la transformada de Fourier por definición es:

$$\widehat{g}(\omega) = \frac{1}{2\pi} \int_{\mathbb{R}^2} e^{-(x-b)^T D(x-b)} e^{-i\omega^T x} dx.$$

En primer lugar, realizamos el cambio de variable $y = x - b$ y, sustituyendo en la integral original, tenemos

$$\widehat{g}(\omega) = \frac{1}{2\pi} \int_{\mathbb{R}^2} e^{-y^T Dy} e^{-i\omega^T (y+b)} dy = \frac{e^{-i\omega^T b}}{2\pi} \int_{\mathbb{R}^2} e^{-y^T Dy - i\omega^T y} dy..$$

Ahora bien, dado que D es simétrica y definida positiva, podemos diagonalizarla como $D = P\Lambda P^T$, donde P es una matriz ortogonal y $\Lambda = \text{diag}(\lambda_1, \lambda_2)$ contiene los autovalores positivos de D.

Introducimos un nuevo cambio de variable vectorial $z = P^T y$. La matriz jacobiana del cambio de variable es P, que por ser ortogonal cumple que el valor absoluto de su determinante es 1, luego $dy = dz$. Para los términos del exponente ocurre lo siguiente:

- Término cuadrático: $y^T Dy = y^T(P\Lambda P^T)y = (P^T y)^T \Lambda (P^T y) = z^T \Lambda z = \lambda_1 z_1^2 + \lambda_2 z_2^2$.

- Término lineal: Dado que $y = Pz$, tenemos $\omega^T y = \omega^T P z$ y, definiendo un nuevo vector constante $\nu := P^T \omega$, el producto queda como $\nu^T z = \nu_1 z_1 + \nu_2 z_2$.

Sustituyendo esto en la integral y aplicando Fubini, se tiene

$$\int_{\mathbb{R}^2} e^{-(\lambda_1 z_1^2 + i\nu_1 z_1) - (\lambda_2 z_2^2 + i\nu_2 z_2)} dz = \left(\int_{-\infty}^{\infty} e^{-\lambda_1 z_1^2 - i\nu_1 z_1} dz_1 \right) \left(\int_{-\infty}^{\infty} e^{-\lambda_2 z_2^2 - i\nu_2 z_2} dz_2 \right).$$

Visto lo anterior, nos resultará más cómodo analizar cada integral unidimensional genérica I_j para $j \in \{1, 2\}$ por separado. Para evitar el uso de integración de contornos en el plano complejo, definimos la integral como una función dependiente del parámetro $\nu_j \in \mathbb{R}$ (asumiendo $\lambda_j > 0$):

$$I_j(\nu_j) := \int_{-\infty}^{\infty} e^{-\lambda_j z_j^2} e^{-i\nu_j z_j} dz_j = \int_{-\infty}^{\infty} e^{-\lambda_j z_j^2} \cos(\nu_j z_j) dz_j,$$

donde la parte imaginaria con el seno se anula debido a que su integrando es una función impar.

Procedemos a derivar bajo el signo integral, que se puede hacer por el Teorema de la Convergencia Dominada, ya que la derivada del integrando está acotada por una función independiente de ν_j que es integrable en $\mathbb{R}$:

$$\left| \frac{\partial}{\partial \nu_j} \left(e^{-\lambda_j z_j^2} \cos(\nu_j z_j) \right) \right| = \left| -z_j e^{-\lambda_j z_j^2} \sin(\nu_j z_j) \right| \leq |z_j| e^{-\lambda_j z_j^2} \in L^1(\mathbb{R}).$$

Por tanto, derivando obtenemos:

$$I_j'(\nu_j) = -\int_{-\infty}^{\infty} z_j e^{-\lambda_j z_j^2} \sin(\nu_j z_j) dz_j.$$

Resolvemos esta integral integrando por partes. Tomamos $u = \sin(\nu_j z_j)$ y $dv = z_j e^{-\lambda_j z_j^2} dz_j$, de modo que $du = \nu_j \cos(\nu_j z_j) dz_j$ y $v = -\frac{1}{2\lambda_j} e^{-\lambda_j z_j^2}$:

$$\int_{-\infty}^{\infty} z_j e^{-\lambda_j z_j^2} \sin(\nu_j z_j) dz_j = \underbrace{\left[-\frac{1}{2\lambda_j} e^{-\lambda_j z_j^2} \sin(\nu_j z_j) \right]_{-\infty}^{\infty}}_{=0} + \frac{\nu_j}{2\lambda_j} \int_{-\infty}^{\infty} e^{-\lambda_j z_j^2} \cos(\nu_j z_j) dz_j = \frac{\nu_j}{2\lambda_j} I_j(\nu_j).$$

Sustituyendo esto en la derivada, llegamos al siguiente problema de valores iniciales:

$$I_j'(\nu_j) = -\frac{\nu_j}{2\lambda_j} I_j(\nu_j), \quad \text{con} \quad I_j(0) = \int_{-\infty}^{\infty} e^{-\lambda_j z_j^2} dz_j = \sqrt{\frac{\pi}{\lambda_j}}.$$

Esta es una ecuación diferencial ordinaria separable cuya solución inmediata nos proporciona el valor buscado de la integral:

$$I_j(\nu_j) = \sqrt{\frac{\pi}{\lambda_j}} e^{-\frac{\nu_j^2}{4\lambda_j}}.$$

Ahora, multiplicando los resultados de las dos integrales I_1 e I_2:

$$I_1 \cdot I_2 = \left(\sqrt{\frac{\pi}{\lambda_1}} e^{-\frac{\nu_1^2}{4\lambda_1}}\right)\left(\sqrt{\frac{\pi}{\lambda_2}} e^{-\frac{\nu_2^2}{4\lambda_2}}\right) = \frac{\pi}{\sqrt{\lambda_1\lambda_2}} e^{-\frac{1}{4}\left(\frac{\nu_1^2}{\lambda_1}+\frac{\nu_2^2}{\lambda_2}\right)}.$$

Reconocemos ahora las propiedades de nuestra matriz original D:

1. El producto de los autovalores es el determinante: $\lambda_1\lambda_2 = \det D$.

2. El término en el exponente es la forma cuadrática $\nu^T\Lambda^{-1}\nu$. Como $\nu = P^T\omega$, entonces:
$$\nu^T\Lambda^{-1}\nu = (P^T\omega)^T\Lambda^{-1}(P^T\omega) = \omega^T(P\Lambda^{-1}P^T)\omega = \omega^T D^{-1}\omega.$$

Sustituyendo estas equivalencias, el valor de la integral doble es:

$$\int_{\mathbb{R}^2} e^{-y^T Dy - i\omega^T y} dy = \frac{\pi}{\sqrt{\det D}} e^{-\frac{1}{4}\omega^T D^{-1}\omega}.$$

Finalmente, llegamos a que

$$\widehat{g}(\omega) = \frac{e^{-i\omega^T b}}{2\pi}\left[\frac{\pi}{\sqrt{\det D}} e^{-\frac{1}{4}\omega^T D^{-1}\omega}\right] = \frac{1}{2\sqrt{\det D}} e^{-i\omega^T b} e^{-\frac{1}{4}\omega^T D^{-1}\omega}.$$

□

Ahora sí, veamos que la función de Franke está en el espacio nativo.

Proposición A.8. La función de Franke

$$\begin{aligned} f(x,y) = &\frac{3}{4} e^{(-1/4)((9x-2)^2+(9y-2)^2)} + \frac{3}{4} e^{(-1/49)(9x+1)^2-(1/10)(9y+1)^2} \\ &+ \frac{1}{2} e^{(-1/4)((9x-7)^2+(9y-3)^2)} - \frac{1}{5} e^{-(9x-4)^2-(9y-7)^2}. \end{aligned}$$

pertenece al espacio nativo $\mathcal{N}_\Phi(\mathbb{R}^2)$ para Φ gaussiana, $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$, para todo $\varepsilon > \varepsilon_0 := \frac{9}{\sqrt{2}}$.

Demostración. La función de Franke es una combinación lineal finita de gaussianas de la forma $g(x) = e^{-(x-b)^\top D(x-b)}$, con $b \in \mathbb{R}^2$ y D matriz diagonal definida positiva (los coeficientes $\frac{1}{4}, \frac{1}{49}, \ldots$ son positivos). Como $\mathcal{N}_\Phi(\mathbb{R}^2)$ es un espacio vectorial, basta ver que cada uno de estos sumandos gaussianos pertenece a él; el resultado se sigue entonces por combinación lineal.

Verificamos las hipótesis del teorema A.1 para un sumando genérico g. Primero, $g \in C(\mathbb{R}^2) \cap L_2(\mathbb{R}^2)$: es continua, y $|g|^2$ es a su vez una gaussiana, luego integrable. Ahora, su transformada de Fourier es también una gaussiana: para $g(x) = e^{-(x-b)^\top D(x-b)}$, por el lema A.7 tenemos

$$\widehat{g}(\omega) = \frac{1}{2\sqrt{\det D}} e^{-i\omega^T b} e^{-\frac{1}{4}\omega^T D^{-1}\omega},$$

de modo que $|\widehat{g}(\omega)| = \dfrac{1}{2\sqrt{\det D}}\, e^{-\frac{1}{4}\,\omega^\top D^{-1}\omega}$ decae como una gaussiana. Aplicando el criterio del corolario A.2, queremos ver que la siguiente integral converge:

$$\int_{\mathbb{R}^2} |\widehat{g}(\omega)|^2\, e^{\|\omega\|^2/(4\varepsilon^2)}\, d\omega = \frac{1}{4\det D}\int_{\mathbb{R}^2} e^{-\frac{1}{2}\,\omega^\top D^{-1}\omega + \frac{1}{4\varepsilon^2}\|\omega\|^2}\, d\omega.$$

Ahora bien, el exponente es la forma cuadrática $-\frac{1}{2}\,\omega^\top\big(D^{-1} - \frac{1}{2\varepsilon^2} I\big)\omega$, luego la integral gaussiana converge si y solo si esta forma es definida negativa, es decir, si la matriz

$$M := D^{-1} - \frac{1}{2\varepsilon^2} I$$

es definida positiva. Pero como D es diagonal con entradas $d_1, d_2 > 0$, M es diagonal con entradas $d_k^{-1} - \frac{1}{2\varepsilon^2}$, positivas si y solo si $d_k < 2\varepsilon^2$ para $k = 1, 2$.

Así, la restricción $d_k < 2\varepsilon^2$ se satisface para todo ε mayor que cierto umbral $\varepsilon_0 = \sqrt{\text{máx}_k\, d_k/2}$. Considerando los cuatro sumandos que teníamos al principio, el que genera el d_k más grande da $d_{max} = 81$, luego $\varepsilon_0 = \sqrt{81/2} = \frac{9}{\sqrt{2}}$. □

La proposición anterior sitúa la función de Franke en el espacio nativo sobre $\mathbb{R}^2$; concluimos trasladando el resultado al dominio acotado, que es donde lo necesitamos.

Corolario A.9 (La función de Franke pertenece a $N_\Phi(\Omega)$)**.** Sea $\Omega \subset \mathbb{R}^2$ abierto y acotado y sea $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$ con $\varepsilon > \varepsilon_0 = \frac{9}{\sqrt{2}}$. Entonces la función de Franke pertenece a $N_\Phi(\Omega)$, de modo que verifica la hipótesis del corolario 3.24.

Demostración. Por la proposición A.8, la función de Franke f pertenece a $\mathcal{N}_\Phi(\mathbb{R}^2)$ para todo $\varepsilon > \frac{9}{\sqrt{2}}$. Por el teorema A.3, toda función de $\mathcal{N}_\Phi(\mathbb{R}^2)$ restringida a Ω pertenece a $N_\Phi(\Omega)$, luego $f|_\Omega \in N_\Phi(\Omega)$, con $\|f\|_{N_\Phi(\Omega)} \le \|f\|_{\mathcal{N}_\Phi(\mathbb{R}^2)}$. □

A.2.2. Polinomio de grado dos

En este caso vamos a demostrar que un cierto polinomio de grado dos no está en el espacio nativo de Φ gaussiana.

Proposición A.10. Sea Φ la gaussiana $\Phi(x) = e^{-\varepsilon^2\|x\|^2}$, la función test

$$f(x, y) = 2 - x + 3y + x^2 - 2xy + 4y^2$$

no está en $\mathcal{N}_\Phi(\mathbb{R}^2)$.

Demostración. La caracterización del espacio nativo de la gaussiana (teorema A.1, del que parte el criterio del corolario A.2) impone como condición necesaria que todo elemento de $\mathcal{N}_\Phi(\mathbb{R}^2)$ pertenezca a $C(\mathbb{R}^2) \cap L_2(\mathbb{R}^2)$. Basta, pues, ver que $f \notin L_2(\mathbb{R}^2)$.

Descomponemos $f = Q + \ell$ en su parte cuadrática y su parte afín,

$$Q(x, y) = x^2 - 2xy + 4y^2, \qquad \ell(x, y) = 2 - x + 3y.$$

La forma cuadrática Q tiene matriz asociada $A = \left(\begin{smallmatrix} 1 & -1 \\ -1 & 4 \end{smallmatrix}\right)$, con $\det A = 3 > 0$ y traza $5 > 0$; por tanto A es definida positiva y, si $\lambda_{\text{mín}} = \frac{5-\sqrt{13}}{2} > 0$ es su menor autovalor,

$$Q(x,y) \;\geq\; \lambda_{\text{mín}}\, \|(x,y)\|^2 \qquad \text{para todo } (x,y) \in \mathbb{R}^2.$$

Como ℓ es afín, existe $C > 0$ con $|\ell(x,y)| \leq C\big(1 + \|(x,y)\|\big)$. Escribiendo $r = \|(x,y)\|$, para r suficientemente grande (digamos $r \geq R_0$) se tiene

$$|f(x,y)| \;\geq\; Q(x,y) - |\ell(x,y)| \;\geq\; \lambda_{\text{mín}}\, r^2 - C(1+r) \;\geq\; \tfrac{\lambda_{\text{mín}}}{2}\, r^2.$$

En consecuencia, pasando a coordenadas polares,

$$\int_{\mathbb{R}^2} |f(x,y)|^2\, dx\, dy \;\geq\; \int_{r \geq R_0} \Big(\tfrac{\lambda_{\text{mín}}}{2}\Big)^2 r^4\, dx\, dy = \frac{\lambda_{\text{mín}}^2}{4}\, 2\pi \int_{R_0}^{\infty} r^5\, dr = \infty.$$

Luego $f \notin L_2(\mathbb{R}^2)$ y, por la condición necesaria anterior, $f \notin \mathcal{N}_\Phi(\mathbb{R}^2)$. □

Como en el caso anterior, lo que necesitamos es la conclusión sobre Ω. Ahora bien, aquí no basta con restringir: hay que descartar que exista alguna función del espacio nativo sobre $\mathbb{R}^2$ que coincida con el polinomio en Ω. Es en este punto donde interviene la analiticidad del lema A.4.

Corolario A.11 (El polinomio no pertenece a $N_\Phi(\Omega)$)**.** Sea $\Omega \subset \mathbb{R}^2$ abierto no vacío y sea Φ gaussiana. La función test $f(x,y) = 2 - x + 3y + x^2 - 2xy + 4y^2$ no pertenece a $N_\Phi(\Omega)$, sea cual sea $\varepsilon > 0$. Por tanto, no se verifica la hipótesis del corolario 3.24 y no disponemos de garantía teórica de convergencia para este experimento.

Demostración. Procedamos por RA y supongamos que $f|_\Omega \in N_\Phi(\Omega)$. Por el teorema A.3, existiría $F \in \mathcal{N}_\Phi(\mathbb{R}^2)$ con $F|_\Omega = f|_\Omega$. Por el lema A.4, dicha F es real y analítica en $\mathbb{R}^2$, y f también lo es por ser polinómica. Así, la diferencia $F - f$ es real y analítica en $\mathbb{R}^2$, que es conexo, y se anula en el abierto no vacío Ω. Así, por el principio de identidad para funciones reales y analíticas, $F = f$ en todo $\mathbb{R}^2$. Se seguiría entonces que $f \in \mathcal{N}_\Phi(\mathbb{R}^2)$, en contradicción con la proposición A.10. □

Apéndice B

k-estabilidad de las particiones de la unidad de Shepard-Wendland

En el contexto del MPU, con N_j acotado y parches variables como los definidos en 3.1.1, elegimos las denominadas particiones de la unidad de Shepard,

$$w_j(x) = \frac{\varphi_j(x)}{\sum_{i=1}^{M} \varphi_i(x)}, \text{ donde } \varphi_j : \mathbb{R}^s \to \mathbb{R}^+ \cup \{0\}, 1 \leq j \leq M.$$

En nuestro caso, las φ_j que elegiremos generarán las llamadas particiones de la unidad de Shepard-Wendland, que definiremos más adelante.

El objetivo de este apéndice será, para estos pesos, determinar su k-estabilidad para poder aplicar a los experimentos numéricos del capítulo 4 nuestros resultados teóricos de convergencia deducidos para el MPU en función de dicha k-estabilidad.

Ahora bien, según la definición 3.6, probar que $\{w_j\}$ es k-estable exige dos cosas: que los pesos sean de clase C^k y que sus derivadas hasta orden k estén acotadas por $C_\beta \, \delta_j^{-|\beta|}$, con constantes independientes del parche y del paso de la rejilla. A cada una de ellas dedicaremos una proposición, la B.5 y la B.9, que juntas darán la $(\ell - 1)$-estabilidad exacta de los pesos; los tres corolarios finales las particularizan a las funciones de Wendland que emplearemos en los experimentos numéricos.

Para ello, deberemos introducir una serie de resultados. Los dos primeros son de naturaleza puramente topológica y los usaremos en la proposición B.5: allí necesitaremos localizar, sobre la frontera de un parche, algún punto que no pertenezca a la frontera de ningún otro, pues es en un punto así donde comprobaremos que la derivada de orden ℓ del peso salta, lo que probará que su regularidad no es mejor que $C^{\ell-1}$. Comenzamos, pues, con el siguiente lema, que nos permitirá descartar de golpe las fronteras de los $M - 1$ parches restantes.

Lema B.1. Sea $X \subseteq \mathbb{R}^s$ dotado de la topología inducida, y sean $F_1, \ldots, F_n \subseteq X$ cerrados en X con interior vacío en X. Entonces $F_1 \cup \cdots \cup F_n$ tiene interior vacío en X.

Demostración. Por inducción en n. El caso $n = 1$ es la hipótesis. Supongámoslo cierto para $n-1$, y sea $A \subseteq F_1 \cup \cdots \cup F_n$ un abierto de X, veamos que $A = \emptyset$. El conjunto $A \setminus F_n = A \cap (X \setminus F_n)$ es abierto en X (intersección de abiertos, pues F_n es cerrado en X) y está contenido en $F_1 \cup \cdots \cup F_{n-1}$. Por hipótesis de inducción, $A \setminus F_n = \emptyset$, luego $A \subseteq F_n$. Como F_n tiene interior vacío y A es abierto, $A = \emptyset$. □

El lema que sigue es el que aporta, para ese mismo argumento, que la frontera de cada parche ajeno corta a la nuestra en un conjunto suficientemente pequeño.

Lema B.2. Sean $S = \{x \in \mathbb{R}^s : \|x - c\| = \rho\}$ y $S' = \{x \in \mathbb{R}^s : \|x - c'\| = \rho'\}$ dos esferas distintas (esto es, $(c, \rho) \neq (c', \rho')$), con $s \geq 2$. Entonces $S \cap S'$ es un cerrado de interior vacío en la topología relativa de S.

Demostración. Que $S \cap S'$ es cerrado en S es inmediato: es la intersección con S del cerrado S' de $\mathbb{R}^s$.

Si $c = c'$ (y por tanto $\rho \neq \rho'$), la intersección es vacía y no hay nada que probar. En el caso contrario, en que $c \neq c'$, dejando de lado el caso trivial en que la intersección es vacía, restamos las ecuaciones $\|x - c\|^2 = \rho^2$ y $\|x - c'\|^2 = \rho'^2$, los términos $\|x\|^2$ se cancelan y queda la ecuación lineal

$$2\,x \cdot (c' - c) \;=\; \|c'\|^2 - \|c\|^2 - \rho'^2 + \rho^2,$$

de modo que $S \cap S' \subseteq S \cap P$, donde $P = \{x : x \cdot n = b\}$ es un hiperplano de vector normal $n = (c' - c)/\|c' - c\|$. Como el interior relativo de un subconjunto está contenido en el de cualquier conjunto que lo contenga, basta probar que $S \cap P$ tiene interior vacío en S; es decir, que ningún punto $y \in S \cap P$ posee un entorno relativo en S contenido en P.

Sea $y \in S \cap P$ y $u = (y - c)/\rho$, unitario. Construimos una curva sobre S que pasa por y y abandona P inmediatamente. Distinguimos dos casos según la posición de n respecto de u.

Si n no es paralelo a u, el vector $n - (n \cdot u)u$ es no nulo, luego sea $v = \big(n - (n \cdot u)u\big)/\|n - (n \cdot u)u\|$, que es unitario, ortogonal a u, y cumple $v \cdot n = \|n - (n \cdot u)u\| > 0$. La curva

$$\gamma(\theta) \;=\; c + \rho\big(\cos\theta\, u + \sin\theta\, v\big)$$

está contenida en S para todo θ (pues $\|\cos\theta\, u + \sin\theta\, v\| = 1$) y cumple $\gamma(0) = y$. Además, teniendo en cuenta que $y \cdot n = b$, se tiene

$$\gamma(\theta) \cdot n - b \;=\; \rho(\cos\theta - 1)(u \cdot n) + \rho \sin\theta\,(v \cdot n),$$

que para $\theta \to 0$ se comporta como $\rho\,\theta\,(v \cdot n) \neq 0$: luego $\gamma(\theta) \notin P$ para todo $\theta \neq 0$ suficientemente pequeño, con $\gamma(\theta) \to y$.

Si n es paralelo a u (esto es, $n = \pm u$), elegimos cualquier v unitario ortogonal a u (existe por ser $s \geq 2$) y la misma curva γ. Ahora, $v \cdot n = 0$ y

$$\gamma(\theta) \cdot n - b \;=\; \rho(\cos\theta - 1)(u \cdot n) \;=\; \mp\rho\,(1 - \cos\theta) \;\neq\; 0 \qquad \text{para } \theta \in (0, \pi),$$

con la misma conclusión.

En ambos casos, todo entorno relativo de y en S contiene puntos fuera de P, luego y no es interior a $S \cap P$ en S. Al ser y arbitrario, el interior relativo de $S \cap P$ y, por tanto, el de $S \cap S'$, es vacío. $\square$

El siguiente lema aísla todas las propiedades del perfil φ que necesitaremos: su regularidad exacta, su soporte y el salto de su derivada de orden ℓ. De este modo, las dos proposiciones posteriores no dependerán de la expresión concreta de la función de Wendland escogida, sino únicamente de que tenga la forma $t_+^\ell p(t)$, y bastará comprobar esa forma en cada caso particular.

Lema B.3. Sean $\ell \geq 2$ entero, p un polinomio con $p(0) \neq 0$, y

$$\varphi(t) \;=\; t_+^\ell\, p(t), \qquad \text{tal que} \qquad \varphi(t) > 0 \;\text{ para todo } t \in (0, 1].$$

Sean $c \in \mathbb{R}^s$, $\rho > 0$, $\Omega_c := B(c, \rho)$ y $\varphi_c(x) := \varphi\big(1 - \|x - c\|/\rho\big)$. Entonces:

(i) $p(0) > 0$, $\varphi \in C^{\ell-1}(\mathbb{R})$ y

$$\varphi^{(k)}(0) = 0 \;\; (0 \leq k \leq \ell - 1), \qquad \varphi^{(\ell)}(0^+) = \ell!\, p(0) \neq 0 = \varphi^{(\ell)}(0^-).$$

(ii) $\varphi_c > 0$ en Ω_c y $\varphi_c \equiv 0$ en $\mathbb{R}^s \setminus \Omega_c$, en particular, $\operatorname{soporte} \varphi_c = \overline{\Omega_c}$.

(iii) $\varphi_c \in C^\infty\big(\Omega_c \setminus \{c\}\big)$, $\varphi_c \in C^\infty\big(\mathbb{R}^s \setminus \overline{\Omega_c}\big)$ y $\varphi_c \in C^{\ell-1}\big(\mathbb{R}^s \setminus \{c\}\big)$. Además, $D^\beta \varphi_c \equiv 0$ sobre $\partial\Omega_c$ para todo $|\beta| \leq \ell - 1$.

(iv) Si $y \in \partial\Omega_c$ y $\nu := (y - c)/\rho$, dirección unitaria, entonces para $r < \rho$

$$\partial_\nu^k \varphi_c(c + r\nu) = \Big(-\tfrac{1}{\rho}\Big)^k \varphi^{(k)}\Big(1 - \tfrac{r}{\rho}\Big),$$

y por tanto

$$\lim_{r \to \rho^-} \partial_\nu^k \varphi_c = 0 \;\; (0 \leq k \leq \ell - 1), \qquad \lim_{r \to \rho^-} \partial_\nu^\ell \varphi_c = \frac{(-1)^\ell\, \ell!\, p(0)}{\rho^\ell} =: \kappa_\ell \neq 0,$$

mientras que, desde el exterior, $\partial_\nu^k \varphi_c \equiv 0$ para todo k.

Demostración. (i) Escribiendo $p(t) = \sum_{m=0}^d a_m t^m$ con $a_0 = p(0) \neq 0$, para $t > 0$ tenemos que $\varphi(t) = \sum_{m=0}^d a_m\, t^{\ell+m}$, luego $\varphi(t)/t^\ell \to a_0$ cuando $t \to 0^+$; como $\varphi > 0$ en $(0, 1]$, necesariamente $a_0 > 0$. Derivando esa serie finita se obtiene $\varphi^{(k)}(0^+) = 0$ para $k < \ell$ y $\varphi^{(\ell)}(0^+) = \ell!\, a_0$. Puesto que

$\varphi \equiv 0$ en $(-\infty, 0]$, las derivadas laterales izquierdas son todas nulas, de modo que las $\ell - 1$ primeras derivadas laterales coinciden y la ℓ-ésima no. Así, $\varphi \in C^{\ell-1}(\mathbb{R}) \setminus C^{\ell}(\mathbb{R})$.

(ii) Sea $x \in \Omega_c$, tenemos que $t = 1 - \|x - c\|/\rho \in (0, 1]$ y, por tanto, $\varphi(t) > 0$. Para $x \notin \Omega_c$, $t \leq 0$ y $\varphi(t) = 0$.

(iii) Observamos que la aplicación $\gamma : x \mapsto 1 - \|x - c\|/\rho$ es C^∞ en $\mathbb{R}^s \setminus \{c\}$, por lo que, si componemos con $\varphi \in C^{\ell-1}(\mathbb{R})$ se obtiene $\varphi_c \in C^{\ell-1}(\mathbb{R}^s \setminus \{c\})$. Ahora bien, para puntos tales que $||x - c|| > \rho$, γ es C^∞ y, como $\varphi|_{(-\infty,0)}$ es idénticamente nula, también es C^∞, luego la composición lo es: $\varphi_c \in C^\infty\big(\mathbb{R}^s \setminus \overline{\Omega_c}\big)$. Por su parte, sea $x \in \Omega_c - c$, entonces γ es C^∞ y $\varphi|_{(0,\infty)}$ también, luego $\varphi_c \in C^\infty\big(\Omega_c \setminus \{c\}\big)$. Como φ_c es idénticamente nula en el abierto $\mathbb{R}^s \setminus \overline{\Omega_c}$, todas sus derivadas hasta orden $\ell - 1$ se anulan allí, y por continuidad de $D^\beta \varphi_c$ ($|\beta| \leq \ell - 1$) también sobre $\partial\Omega_c$.

(iv) La restricción de φ_c a la recta $r \mapsto c + r\nu$ es $r \mapsto \varphi(1 - r/\rho)$. Por tanto, la derivada direccional iterada ∂_ν^k sobre la recta $c + r\nu$ coincide con la derivada ordinaria sobre la función restringida a dicha recta y, por la regla de la cadena sale un factor $(-1/\rho)^k$. Finalmente, concluimos la demostración por (i). □

Definición B.4 (Partición de la unidad de Shepard–Wendland)**.** Sea $\varphi(t) = t_+^\ell p(t)$ con p polinómica, $p(0) \neq 0$ y $\varphi > 0$ en $(0, 1]$, y defínase, sobre los parches $\Omega_i = B(c_i, \rho)$ del apartado 3.1.1,

$$\varphi_i(x) := \varphi\Big(1 - \frac{\|x - c_i\|}{\rho}\Big).$$

La partición de la unidad de Shepard asociada, $w_j = \varphi_j / \sum_i \varphi_i$, se denomina partición de la unidad de Shepard–Wendland.

Con lo anterior ya podemos determinar la regularidad de estos pesos, que es la primera de las dos condiciones que exige la definición 3.6. Veremos, además, que dicha regularidad es exacta, lo que nos dirá que el orden de estabilidad que obtendremos después no puede mejorarse.

Proposición B.5 (Regularidad exacta de los pesos de Shepard–Wendland en la construcción de rejilla)**.** Sea $2 \leq s \leq 3$, $\Omega = (0, 1)^s$, $H = 1/(n - 1)$ con $n \geq 3$, y considérese la construcción dada por 3.1.1: centros $\{c_j\}_{j=1}^M$ en la rejilla $H\mathbb{Z}^s \cap \overline{\Omega}$ de paso H, parches $\Omega_j = B(c_j, \rho)$ con

$$\rho = \alpha H, \qquad \frac{\sqrt{s}}{2} < \alpha < 1$$

(intervalo no vacío precisamente porque $s \leq 3$). Sea $\varphi(t) = t_+^\ell p(t)$ como en el lema B.3, y definamos

$$\varphi_j(x) = \varphi\Big(1 - \tfrac{\|x - c_j\|_2}{\rho}\Big), \qquad S(x) = \sum_{i=1}^M \varphi_i(x), \qquad w_j(x) = \frac{\varphi_j(x)}{S(x)} \quad \text{para } x \in U := \{S > 0\}.$$

Entonces U es abierto y $\overline{\Omega} \subset U$, y:

(a) (Constancia local en los centros.) Para cada j, en la bola $B(c_j,(1-\alpha)H)$ se tiene $S=\varphi_j$, y por tanto $w_j\equiv 1$ y $w_i\equiv 0$ para todo $i\neq j$. En particular, todos los pesos son C^∞ en un entorno de cada centro.

(b) (Regularidad global.) $w_j\in C^{\ell-1}(U)$ para todo j.

(c) (Exactitud.) $w_j\notin C^\ell(U)$: la derivada de orden ℓ presenta saltos no nulos tanto sobre la frontera del propio parche, $\partial\Omega_j\cap U$, como sobre las fronteras ajenas $\partial\Omega_i\cap U$ $(i\neq j)$. La regularidad de los pesos es, pues, exactamente $C^{\ell-1}$ en U.

Demostración. En primer lugar, veamos que el recubrimiento cubre todo Ω y, por tanto, $U\supset\overline{\Omega}$. Sea $x\in\overline{\Omega}$ pertenece a alguna celda de la rejilla, y su vértice más próximo c_i dista de él a lo sumo $\frac{\sqrt{s}}{2}H<\alpha H=\rho$. Luego $x\in\Omega_i$, para algún $i\in\{1,\dots,M\}$ y, por el lema B.3(ii), $S(x)\geq\varphi_i(x)>0$. Así, $\overline{\Omega}\subset U$, y U es abierto por continuidad de S.

Ahora, recordemos que, por el lema B.3(iii), cada φ_i es C^∞ en $\mathbb{R}^s\setminus(\{c_i\}\cup\partial\Omega_i)$, y es $C^{\ell-1}$, con todas sus derivadas hasta orden $\ell-1$ nulas, sobre $\partial\Omega_i$. Además, el lema B.3(iv) nos da que la derivada radial de orden ℓ salta $\kappa_\ell\neq 0$ en $\partial\Omega_i$. Además, en el centro c_i la regularidad de φ_i dependerá de φ. Por tanto, los únicos puntos a los que hay que prestar atención al determinar la suavidad de los pesos en U son los centros y las fronteras de los parches.

(a) (Constancia local en los centros.) Sea $j\in\{1,\dots,M\}$, sea $x\in B(c_j,(1-\alpha)H)$ y sea $i\neq j$ arbitrario pero fijo. Como dos centros distintos de la rejilla distan al menos H,

$$\|x-c_i\| \;\geq\; \|c_i-c_j\|-\|x-c_j\| \;>\; H-(1-\alpha)H \;=\; \alpha H \;=\; \rho,$$

luego por el lema B.3(ii) $\varphi_i(x)=0$. Además $\|x-c_j\|<(1-\alpha)H<\alpha H=\rho$ (pues $\alpha>\frac{\sqrt{s}}{2}>\frac{1}{2}$), de modo que $\varphi_j(x)>0$. Así, en toda la bola $B(c_j,(1-\alpha)H)$ se tiene $S=\varphi_j>0$, $w_j=\varphi_j/\varphi_j=1$ y $w_i\equiv 0$ para todo $i\neq j$. En un entorno de cualquier centro, pues, todos los pesos son constantes y en particular C^∞.

(b) (Regularidad global.) Sea $x\in U$, hay tres casos.

Si x pertenece a alguna bola $B(c_i,(1-\alpha)H)$, entonces por (a) w_j es constante en un entorno de x, luego C^∞.

Si x está fuera de todas esas bolas y fuera de toda frontera $\partial\Omega_i$, entonces en un entorno suficientemente pequeño toda φ_i es C^∞. Por tanto, aquí $S\in C^\infty$ y $S>0$, luego el cociente w_j es C^∞.

Si x pertenece a una o varias fronteras $\partial\Omega_i$, observemos primero que en un entorno de x no hay centros: en efecto, $\|x-c_i\|=\rho>0$ y, para $m\neq i$, $\|x-c_m\|\geq\|c_m-c_i\|-\rho\geq H-\rho=(1-\alpha)H>0$ (aquí se usa $\alpha<1$). Tomando un entorno $V\ni x$ sin centros, toda φ_m es allí al menos $C^{\ell-1}$: las que

no tienen su frontera pasando por V son C^∞, y las que sí son exactamente $C^{\ell-1}$, con derivadas nulas hasta orden $\ell-1$ sobre ella (por lo dicho al principio de la demostración). Luego $S \in C^{\ell-1}(V)$ con $S>0$, y por tanto $S^{-1} \in C^{\ell-1}(V)$. En consecuencia, $w_j = \varphi_j \cdot S^{-1}$ es producto de dos funciones de clase $C^{\ell-1}$ en V (φ_j es C^∞ si $x \notin \partial\Omega_j$, y exactamente $C^{\ell-1}$ si $x \in \partial\Omega_j$), luego $w_j \in C^{\ell-1}(V)$.

En los tres casos $w_j \in C^{\ell-1}$ localmente, luego $w_j \in C^{\ell-1}(U)$.

(c) (Exactitud.) Veamos que los pesos son exactamente $C^{\ell-1}$ sobre su propia frontera y sobre las fronteras ajenas en las que no se anulan.

(I) Frontera propia. Sea $y \in \partial\Omega_j \cap U$ un punto cubierto por algún parche vecino y que no pertenezca a ninguna frontera ajena $\partial\Omega_i$, $i \neq j$. Tales puntos existen, ya que, por el lema B.2, cada intersección $\partial\Omega_j \cap \partial\Omega_i$ es un cerrado de interior vacío en la topología relativa de $\partial\Omega_j$, y por el lema B.1, aplicado a $X = \partial\Omega_j$, la unión de las $M-1$ intersecciones sigue teniendo interior relativo vacío. Por otra parte, los puntos de $\partial\Omega_j$ cubiertos por algún parche vecino (todos ellos abiertos de $\mathbb{R}^s$) forman un abierto relativo no vacío de $\partial\Omega_j$: no vacío, porque el punto de la frontera en dirección a un centro vecino c_i dista de éste $H-\rho = (1-\alpha)H < \rho$ (pues $\alpha > \frac{1}{2}$), luego pertenece a Ω_i. Como un abierto relativo no vacío no puede estar contenido en un conjunto de interior relativo vacío, dicho abierto contiene puntos que no están en ninguna frontera más que $\partial\Omega_j$; cualquiera de ellos nos sirve como y.

Sea $\nu := (y-c_j)/\rho$ la dirección radial. Observemos primero que, en un entorno V de y suficientemente pequeño para que no contenga centros ni puntos de fronteras ajenas y para que $S > S(y)/2 > 0$, la suma S coincide, a cada lado de $\partial\Omega_j$, con una función de clase C^∞. En efecto, sea $\tilde\varphi_j(x) := \varphi\big(1-\|x-c_j\|/\rho\big)$ evaluada con la fórmula polinómica de φ sin la parte positiva (esto es, $\tilde\varphi_j(x) = \varphi_{\mathrm{pol}}\big(1-\|x-c_j\|/\rho\big)$ con $\varphi_{\mathrm{pol}}(t) = t^\ell p(t)$, sin truncar en t_+), tenemos que $\tilde\varphi_j$ es C^∞ en todo V, al ser composición de la función suave $x \mapsto 1-\|x-c_j\|/\rho$ (lo es en $\mathbb{R}^s \setminus \{c_j\}$, y V no contiene a c_j) con el polinomio φ_{pol}. Sea $T := \sum_{i\neq j} \varphi_i$, que es C^∞ en V, al no haber en V ni centros ni fronteras ajenas. Entonces

$$S = \tilde\varphi_j + T \ \text{ en } V \cap \overline{\Omega_j}, \qquad\qquad S = T \ \text{ en } V \setminus \Omega_j.$$

Ambas extensiones laterales, $\tilde\varphi_j + T$ y T, son C^∞ en V. Además son positivas: para la exterior, $T = S > S(y)/2 > 0$ en $V \setminus \Omega_j$; para la interior, como $\tilde\varphi_j(y) = 0$ y $T(y) = S(y)$, por continuidad de $\tilde\varphi_j + T$ podemos encoger V de modo que $\tilde\varphi_j + T > S(y)/2 > 0$ en todo V. De este modo, S^{-1} es C^∞ en V a cada lado de $\partial\Omega_j$. En particular, los límites laterales, tendiendo a y a lo largo de la recta radial $x = c_j + r\nu$, de $\partial_\nu^m S^{-1}$ con $m \leq \ell$, existen y son finitos.

Ahora bien, desde el exterior $w_j = 0$, luego $\partial_\nu^\ell w_j \to 0$. Desde el interior, por la regla de Leibniz a lo largo de la recta radial,

$$\partial_\nu^\ell w_j = \sum_{k=0}^{\ell} \binom{\ell}{k} \partial_\nu^k \varphi_j \cdot \partial_\nu^{\ell-k}\big(S^{-1}\big).$$

Por el lema B.3(iv), cuando $r \to \rho^-$ cada término con $0 \leq k \leq \ell-1$ es el producto de un factor que tiende a cero por un factor con límite lateral interior finito, mientras que el término $k = \ell$ tiende a

$\kappa_\ell \cdot S(y)^{-1}$. En consecuencia,

$$\lim_{r\to\rho^-} \partial_\nu^\ell w_j(c_j + r\nu) \;=\; \frac{\kappa_\ell}{S(y)} \;=\; \frac{(-1)^\ell\, \ell!\, p(0)}{\rho^\ell\, S(y)} \;\neq\; 0,$$

y por tanto $w_j \notin C^\ell$ en $y \in \partial\Omega_j$.

(II) Fronteras ajenas. Sea $i \neq j$ un índice con $\|c_i - c_j\| = H$ (existe, pues $n \geq 3$). Afirmamos que hay $y \in \partial\Omega_i \cap \Omega_j$ que no pertenece a ninguna otra frontera $\partial\Omega_m$, $m \neq i$. En efecto, $\partial\Omega_i \cap \Omega_j$ es un abierto relativo de $\partial\Omega_i$, no vacío porque $y_0 := c_i + \rho\,(c_j - c_i)/H$ verifica $\|y_0 - c_i\| = \rho$ y $\|y_0 - c_j\| = H - \rho = (1-\alpha)H < \rho$ y, exactamente como en (I), los lemas B.2 y B.1 garantizan que $\bigcup_{m\neq i}(\partial\Omega_i \cap \partial\Omega_m)$ tiene interior relativo vacío en $\partial\Omega_i$. Nótese además que $\varphi_j(y) > 0$ y que ningún centro está en $\partial\Omega_i$ (pues $\|c_m - c_i\| \geq H > \rho$).

Sea $\nu_i := (y - c_i)/\rho$ y sea V un entorno de y sin centros, sin fronteras distintas de $\partial\Omega_i$, y con $S > S(y)/2 > 0$. En V la función φ_j es C^∞ (estamos lejos de c_j y de $\partial\Omega_j$). Aplicando la regla de Leibniz sobre la recta radial $x = c_i + r\nu_i$, de manera completamente análoga al caso (I),

$$\partial_{\nu_i}^\ell w_j = \sum_{k=0}^{\ell} \binom{\ell}{k} \partial_{\nu_i}^k \varphi_j \cdot \partial_{\nu_i}^{\ell-k}\big(S^{-1}\big).$$

Como $\varphi_j \in C^\infty(V)$, todos los factores $\partial_{\nu_i}^k \varphi_j$ son continuos en y, por lo que cualquier discontinuidad provendrá exclusivamente de las derivadas de S^{-1}.

Escribamos $S = \varphi_i + \sum_{m\neq i} \varphi_m$, donde $\sum_{m\neq i} \varphi_m$ es C^∞ en V. Así, las derivadas de S heredan el comportamiento de φ_i: por el lema B.3(iv), las derivadas de S hasta orden $\ell - 1$ son continuas a través de $\partial\Omega_i$, mientras que la de orden ℓ presenta un salto. Denotando por $[\,\cdot\,]$ la diferencia entre los límites laterales (interior menos exterior) tendiendo a y por la recta $c_i + r\nu_i$,

$$\big[\partial_{\nu_i}^\ell S\big] = \lim_{r\to\rho^-} \partial_{\nu_i}^\ell \varphi_i - \lim_{r\to\rho^+} \partial_{\nu_i}^\ell \varphi_i = \kappa_\ell - 0 = \kappa_\ell.$$

Por la regla de la cadena aplicada a S^{-1}, toda contribución a $\partial_{\nu_i}^\ell(S^{-1})$ que involucre derivadas de S de orden $\leq \ell - 1$ es continua. La única discontinuidad surge del término principal $-\partial_{\nu_i}^\ell S/S^2$. Por tanto,

$$\big[\partial_{\nu_i}^\ell\big(S^{-1}\big)\big] = -\frac{\big[\partial_{\nu_i}^\ell S\big]}{S(y)^2} = -\frac{\kappa_\ell}{S(y)^2}.$$

Volviendo a la suma de Leibniz, los términos con $\ell - k \leq \ell - 1$ (es decir, $k \geq 1$) involucran derivadas continuas de S^{-1} y no aportan salto. El único salto proviene del término $k = 0$:

$$\big[\partial_{\nu_i}^\ell w_j\big] = \varphi_j(y) \cdot \big[\partial_{\nu_i}^\ell\big(S^{-1}\big)\big] = -\,\frac{\kappa_\ell\, \varphi_j(y)}{S(y)^2} \;\neq\; 0,$$

puesto que $\varphi_j(y) > 0$. El factor φ_j, al ser suave y no nulo, no puede absorber el defecto generado por el salto de S^{-1}.

Por todo lo anterior, $w_j \in C^{\ell-1}(U)$ y $w_j \notin C^\ell(U)$: la regularidad es exactamente $C^{\ell-1}$. $\square$

Una vez conocida la suavidad de los pesos, para abordar la k-estabilidad de los pesos, es necesario introducir primero los siguientes lemas. El primero acota cuántos parches pueden contener a un mismo punto, lo que será clave para que las constantes de la cota final no dependan ni del paso de la rejilla ni de N. El segundo acota las derivadas del inverso de la suma $S = \sum_i \varphi_i$, que es el denominador de los pesos de Shepard y el único factor de w_j cuyas derivadas no controlamos directamente a partir del lema B.3.

Lema B.6. Sea $C \subset \mathbb{R}^s$ un conjunto finito y separado por $q > 0$, esto es, tal que $\|c - c'\| \geq q$ para cualesquiera $c \neq c'$ en C, y sea $\rho > 0$. Entonces, para todo $x \in \mathbb{R}^s$,

$$\#\{c \in C : \ \|x - c\| < \rho\} \ \leq \ \Big(1 + \frac{2\rho}{q}\Big)^s.$$

Demostración. Sean $I(x) := \{c \in C : \ \|x - c\| < \rho\}$ y $N := \#I(x)$.

Claramente, las bolas $\{B(c, q/2)\}_{c \in C}$ son disjuntas dos a dos. En efecto, si existiese $z \in B(c, q/2) \cap B(c', q/2)$ con $c \neq c'$, la desigualdad triangular daría

$$\|c - c'\| \ \leq \ \|c - z\| + \|z - c'\| \ < \ \tfrac{q}{2} + \tfrac{q}{2} \ = \ q,$$

que contradice la hipótesis.

Además, $B(c, q/2) \subset B\big(x, \rho + \frac{q}{2}\big)$ para todo $c \in I(x)$, pues si $z \in B(c, q/2)$, entonces

$$\|z - x\| \ \leq \ \|z - c\| + \|c - x\| \ < \ \tfrac{q}{2} + \rho.$$

Ahora, si denotamos por λ_s la medida de Lebesgue de $\mathbb{R}^s$ y sea $\omega_s := \lambda_s\big(B(0,1)\big) > 0$, de modo que $\lambda_s\big(B(c,r)\big) = \omega_s\, r^s$. Además, teniendo en cuenta que la unión de las N bolas indexadas por $I(x)$ es disjunta, y que está contenida en $B\big(x, \rho + \frac{q}{2}\big)$, la aditividad de λ_s sobre conjuntos disjuntos y su monotonía dan

$$N\, \omega_s \Big(\frac{q}{2}\Big)^s \ = \ \lambda_s\Big(\bigsqcup_{c \in I(x)} B\big(c, \tfrac{q}{2}\big)\Big) \ \leq \ \lambda_s\Big(B\big(x, \rho + \tfrac{q}{2}\big)\Big) \ = \ \omega_s\Big(\rho + \frac{q}{2}\Big)^s.$$

Cancelando $\omega_s > 0$ y despejando,

$$N \ \leq \ \left(\frac{\rho + q/2}{q/2}\right)^s = \Big(1 + \frac{2\rho}{q}\Big)^s. \qquad \square$$

Veamos ahora el segundo de los lemas anunciados.

Lema B.7. Sean $V \subseteq \mathbb{R}^s$ abierto, $k \in \mathbb{N} \cup \{\infty\}$ y $S \in C^k(V)$ tal que $S \geq \sigma_0 > 0$ en V. Supongamos que existen $\rho > 0$ y constantes $B_m \geq 0$ tales que

$$\|D^\gamma S\|_{L^\infty(V)} \ \leq \ B_{|\gamma|}\, \rho^{-|\gamma|} \qquad \text{para todo multiíndice } \gamma \text{ con } 1 \leq |\gamma| \leq k.$$

Entonces $S^{-1} \in C^k(V)$ y, definiendo

$$A_0 := \sigma_0^{-1}, \qquad A_m := \sigma_0^{-1}\, 2^m \max_{1\le q\le m} \big(B_q\, A_{m-q}\big) \quad (1 \le m \le k),$$

se cumple

$$\|D^\beta\big(S^{-1}\big)\|_{L^\infty(V)} \;\le\; A_{|\beta|}\, \rho^{-|\beta|} \qquad \text{para todo } |\beta| \le k.$$

En particular, A_m depende únicamente de m, σ_0 y $B_1, \dots, B_m$.

Demostración. Como $S \in C^k(V)$ y $S > 0$ allí, se sigue que $S^{-1} \in C^k(V)$. Ahora bien, las cotas se prueban por inducción sobre $m = |\beta|$. Para $m = 0$, de $S \ge \sigma_0$ se sigue $\|S^{-1}\|_{L^\infty(V)} \le \sigma_0^{-1} = A_0$. Sea ahora $1 \le |\beta| \le k$ y supóngase la cota cierta para todo orden menor que $|\beta|$. Derivando la identidad $S \cdot S^{-1} = 1$ en V mediante la regla de Leibniz,

$$0 = D^\beta\big(S \cdot S^{-1}\big) = \sum_{\gamma \le \beta} \binom{\beta}{\gamma} D^\gamma S\; D^{\beta-\gamma}\big(S^{-1}\big),$$

y despejando el término correspondiente a $\gamma = 0$,

$$D^\beta\big(S^{-1}\big) = -\frac{1}{S} \sum_{\substack{\gamma \le \beta \\ \gamma \ne 0}} \binom{\beta}{\gamma} D^\gamma S\; D^{\beta-\gamma}\big(S^{-1}\big).$$

Usando $S \ge \sigma_0$, las hipótesis sobre S, la hipótesis de inducción, $\rho^{-|\gamma|}\rho^{-|\beta-\gamma|} = \rho^{-|\beta|}$ y $\sum_{\gamma\le\beta} \binom{\beta}{\gamma} = 2^{|\beta|}$,

$$\big|D^\beta\big(S^{-1}\big)\big| \;\le\; \frac{1}{\sigma_0}\, 2^{|\beta|} \max_{1\le q\le|\beta|} \big(B_q\, A_{|\beta|-q}\big)\, \rho^{-|\beta|} \;=\; A_{|\beta|}\, \rho^{-|\beta|}.$$

□

Nota B.8. En la definición 3.6 escribiremos los multiíndices con la letra β (y las constantes C_β), reservando α para el factor de solape $\rho = \alpha H$.

Reunidos ambos lemas, estamos ya en condiciones de probar la segunda condición de la definición 3.6, esto es, la cota sobre las derivadas de los pesos, con lo que quedará establecida su $(\ell-1)$-estabilidad.
Proposición B.9 (Estabilidad exacta de los pesos de Shepard–Wendland)**.** Bajo las hipótesis y notaciones de la proposición B.5, pongamos

$$t_0 := 1 - \frac{\sqrt{s}}{2\alpha} \in (0,1), \qquad \sigma := \min_{t\in[t_0,1]} \varphi(t) > 0, \qquad U := \Big\{x \in \mathbb{R}^s :\; S(x) > \tfrac{\sigma}{2}\Big\}.$$

Entonces U es abierto, $\overline{\Omega} \subset U$, y $\{w_j\}_{j=1}^M$ es una partición de la unidad $(\ell-1)$-estable subordinada a $\{\Omega_j\}$ en el sentido de la definición 3.6: existen constantes $C_\beta = C(s, \alpha, \varphi, |\beta|) > 0$, independientes de H y de j, tales que

$$\|D^\beta w_j\|_{L^\infty(U)} \;\le\; C_\beta\, \delta_j^{-|\beta|}, \qquad |\beta| \le \ell - 1, \qquad \delta_j = \operatorname{diam}\Omega_j = 2\rho.$$

Además, $\{w_j\}$ no es ℓ-estable: para ningún abierto $U' \supseteq \Omega$ se tiene $w_j \in C^\ell(U')$.

Demostración. Comencemos definiendo $\theta(z) := \varphi(1 - \|z\|)$, de modo que $\varphi_j(x) = \theta\big((x - c_j)/\rho\big)$ y, para $x \neq c_j$ y $|\beta| \leq \ell - 1$,

$$D^\beta \varphi_j(x) = \rho^{-|\beta|} \big(D^\beta \theta\big)\Big(\frac{x - c_j}{\rho}\Big). \tag{B.1}$$

Allí, toda la dependencia en H está en el factor $\rho^{-|\beta|}$.

Veamos ahora que $\overline{\Omega} \subset U$. Sea $x \in \overline{\Omega}$ y c_i el vértice de su celda más próximo, de modo que $\|x - c_i\| \leq \frac{H\sqrt{s}}{2}$. Entonces, para $t := 1 - \|x - c_i\|/\rho \in [t_0, 1]$ se cumple

$$S(x) \;\geq\; \varphi_i(x) = \varphi(t) \;\geq\; \sigma \;>\; \tfrac{\sigma}{2}.$$

Luego $\overline{\Omega} \subset U$, y σ no depende de H, pues depende solo de α, s a través de t_0 y de φ. Éste es el único punto donde se usa $\alpha > \sqrt{s}/2$, para ver que $t_0 \leq t$.

Nuestro objetivo ahora será encontrar una cota para el número de parches que contienen a un mismo punto y comprobaremos formalmente lo que dijimos en su momento en 3.1.1 sobre el solapamiento acotado de nuestro recubrimiento. Así, por el lema B.3(iii), para $|\beta| \leq \ell - 1$ la derivada $D^\beta \varphi_i$ se anula tanto en $\mathbb{R}^s \setminus \overline{\Omega_i}$ como sobre $\partial\Omega_i$, por lo que, en la suma $D^\beta S(x) = \sum_{i=1}^M D^\beta \varphi_i(x)$ únicamente contribuyen los índices i con $x \in \Omega_i$, es decir,

$$D^\beta S(x) = \sum_{i \in I(x)} D^\beta \varphi_i(x), \qquad I(x) := \{i : \; \|x - c_i\| < \rho\}, \qquad L(x) := \#I(x).$$

Ahora bien, dos centros distintos de la rejilla difieren en alguna coordenada en un múltiplo no nulo de H, luego $\|c_i - c_j\| \geq H$ y el conjunto de centros está separado por $q = H$, así que, aplicando el lema B.6 con $q = H$ y $\rho = \alpha H$, y usando $\alpha < 1$,

$$L := \sup_{x \in \mathbb{R}^s} L(x) \;\leq\; \Big(1 + \frac{2\rho}{H}\Big)^s = (1 + 2\alpha)^s \;<\; 3^s,$$

cota independiente de x y de H, que será importante más adelante.

Busquemos ahora la cota que queremos para los pesos. El caso sencillo (región A), si $x \in B(c_i, (1 - \alpha)H)$ para algún i, el apartado (a) de la proposición B.5 da $w_j = \text{const.}$ (igual a 1 o a 0) en toda esa bola abierta, luego $D^\beta w_j(x) = 0$ para $|\beta| \geq 1$, y las cotas son triviales.

Para el otro caso, consideramos ahora $x \in U$ con $\|x - c_i\| \geq (1 - \alpha)H$ para todo i, a lo que llamaremos región B, entonces

$$\Big\|\frac{x - c_i}{\rho}\Big\| \;\geq\; \frac{1 - \alpha}{\alpha} =: \eta \in (0, 1) \qquad (\text{pues } \alpha > \tfrac{1}{2}).$$

Definimos, para $0 \leq m \leq \ell - 1$,

$$M_m := \max_{|\beta| = m} \sup_{\|z\| \geq \eta} \big|D^\beta \theta(z)\big| \;<\; \infty,$$

finito porque $D^\beta\theta$ es continua en $\{\|z\| \geq \eta\}$ (lema B.3(iii)), idénticamente nula en $\{\|z\| > 1\}$, y $\{\eta \leq \|z\| \leq 1\}$ es compacto. Nótese que M_m depende sólo de s, α, φ, m. En la suma $D^\beta S(x) = \sum_i D^\beta \varphi_i(x)$ únicamente contribuyen los i con $x \in \Omega_i$, a lo sumo L, y cada uno está acotado por la ecuación (B.1). Por tanto,

$$\left|D^\beta S(x)\right| \;\leq\; L\, M_{|\beta|}\, \rho^{-|\beta|}, \qquad |\beta| \leq \ell - 1. \tag{B.2}$$

Queremos acotar S^{-1} en la región B. Esta no contiene ningún centro, pues todo punto suyo dista al menos $(1-\alpha)H > 0$ de cada c_i y, por el lema B.3(iii), toda φ_i es $C^{\ell-1}$ fuera de su centro, de modo que en el abierto $V := \{x \in U : \ \|x - c_i\| > (1-\alpha)H \ \ \forall i\}$ se tiene $S \in C^{\ell-1}(V)$ y $S > \sigma/2 > 0$. Por (B.2), $\|D^\gamma S\|_{L^\infty(V)} \leq L\, M_{|\gamma|}\, \rho^{-|\gamma|}$ para $1 \leq |\gamma| \leq \ell - 1$, con lo que se satisfacen las hipótesis del lema B.7, con $\sigma_0 = \sigma/2$, $B_m = LM_m$ y $k = \ell - 1$, obteniéndose constantes $A_m = A(s, \alpha, \varphi, m) > 0$, dependientes sólo de s, α, φ, m, tales que $\|D^\beta(S^{-1})\|_{L^\infty(V)} \leq A_{|\beta|}\rho^{-|\beta|}$ para $|\beta| \leq \ell - 1$. Como la región B está contenida en $\overline{V}$ y en ella $D^\beta(S^{-1})$ es continua (por estar libre de centros), la cota

$$\left|D^\beta\big(S^{-1}\big)(x)\right| \;\leq\; A_{|\beta|}\, \rho^{-|\beta|}, \qquad |\beta| \leq \ell - 1, \tag{B.3}$$

vale para todo x en la región B.

Finalmente, para acotar los pesos, $1 \leq |\beta| \leq \ell - 1$ y x en la región B, usando nuevamente (B.1) y (B.3), así como que $\left|D^\beta\theta(z)\right| \leq M_{|\beta|}$, por Leibniz se llega a que

$$\left|D^\beta w_j(x)\right| \;\leq\; \sum_{\gamma \leq \beta} \binom{\beta}{\gamma} \left|D^\gamma \varphi_j(x)\right| \left|D^{\beta-\gamma}\big(S^{-1}\big)(x)\right| \;\leq\; \Big(2^{|\beta|} \max_{0 \leq q \leq |\beta|} M_q A_{|\beta|-q}\Big)\, \rho^{-|\beta|} \;=:\; C'_{|\beta|}\, \rho^{-|\beta|},$$

con $C'_{|\beta|}$ dependiente solo de $s, \alpha, \varphi, |\beta|$.

Combinando lo anterior con que para $x \in B(c_i, (1-\alpha)H)$, $D^\beta w_j = 0, 1 \leq |\beta| \leq \ell - 1$ y usando $\rho = \delta_j/2$, obtenemos

$$\|D^\beta w_j\|_{L^\infty(U)} \;\leq\; 2^{|\beta|} C'_{|\beta|}\, \delta_j^{-|\beta|} \;=:\; C_\beta\, \delta_j^{-|\beta|}, \text{ con } C_\beta = C_\beta(s, \alpha, \varphi, |\beta|)$$

Para $|\beta| = 0$ basta observar que $0 \leq \varphi_j \leq S$, luego $0 \leq w_j \leq 1$ y $C_0 = 1$ y ya tenemos la cota que queríamos sobre los pesos.

Para concluir la k-estabilidad de los pesos, por la proposición B.5(b), $w_j \in C^{\ell-1}(U)$, por el lema B.3(ii), soporte $w_j =$ soporte $\varphi_j = \overline{\Omega_j}$, compacto y $\sum_j w_j = S/S = 1$ en $U \supset \Omega$. Junto con lo hecho a lo largo de esta demostración, concluimos que $\{w_j\}$ es $(\ell - 1)$-estable.

Además, como $n \geq 3$, existe un centro c_j con $\operatorname{dist}(c_j, \partial\Omega) \geq H > \rho$, de modo que $\partial\Omega_j \subset \Omega$. El punto $y \in \partial\Omega_j$ construido en la parte (c)(I) de la proposición B.5 pertenece entonces a Ω, y allí $w_j \notin C^\ell$. Como todo abierto $U' \supseteq \Omega$ contiene y, se sigue $w_j \notin C^\ell(U')$. □

Nota B.10 (Las funciones de Wendland en la construcción de rejilla)**.** Las funciones de Wendland de clase C^{2k} en $\mathbb{R}^s$ para $2 \le s \le 3$ y $k = 1, 2, 3$, se escriben en la forma $\varphi(t) = t_+^\ell p(t)$ del lema B.3, donde $\ell = m + k$, con $m = \left\lfloor \frac{s}{2} \right\rfloor + k + 1$, como

$$t_+^4(5 - 4t), \qquad t_+^6\big(56 - 88t + 35t^2\big), \qquad t_+^8\big(66 - 154t + 121t^2 - 32t^3\big),$$

de suavidad C^2, C^4 y C^6 respectivamente. Los tres corolarios siguientes nos dan para cada una de ellas la regularidad y la $(\ell - 1)$-estabilidad establecidas en general.

Corolario B.11 (Wendland C^2)**.** Bajo las hipótesis de la proposición B.5, tómese

$$\varphi(t) = t_+^4\,(5 - 4t), \qquad \text{es decir,} \qquad \ell = 4, \quad p(t) = 5 - 4t.$$

Entonces la regularidad de los pesos es exactamente

$$w_j \in C^3(U) \setminus C^4(U),$$

y $\{w_j\}$ es una partición de la unidad exactamente 3-estable.

Corolario B.12 (Wendland C^4)**.** Bajo las mismas hipótesis, tómese

$$\varphi(t) = t_+^6\,\big(35t^2 - 88t + 56\big), \qquad \ell = 6, \quad p(t) = 35t^2 - 88t + 56.$$

Entonces $w_j \in C^5(U) \setminus C^6(U)$ y $\{w_j\}$ es exactamente 5-estable.

Corolario B.13 (Wendland C^6)**.** Bajo las mismas hipótesis, tómese

$$\varphi(t) = t_+^8\,\big(-32t^3 + 121t^2 - 154t + 66\big), \qquad \ell = 8, \quad p(t) = -32t^3 + 121t^2 - 154t + 66.$$

Entonces $w_j \in C^7(U) \setminus C^8(U)$ y $\{w_j\}$ es exactamente 7-estable.

Demostración de los corolarios B.11–B.13. En los tres casos φ tiene ya la forma $t_+^\ell p(t)$ con p polinómica y $\ell \ge 2$, de modo que las hipótesis del lema B.3 se reducen a dos comprobaciones: que $p(0) \neq 0$, y que $\varphi > 0$ en $(0, 1]$, lo cual, por ser $t^\ell > 0$ para $t > 0$, equivale a $p > 0$ en $(0, 1]$.

La primera es inmediata: $p(0) = 5$, $p(0) = 56$ y $p(0) = 66$, respectivamente. Para la segunda:

Caso $\ell = 4$. $p(t) = 5 - 4t$ es decreciente, luego $p(t) \ge p(1) = 1 > 0$ en $(0, 1]$.

Caso $\ell = 6$. $p(t) = 35t^2 - 88t + 56$ tiene discriminante $88^2 - 4 \cdot 35 \cdot 56 = 7744 - 7840 = -96 < 0$ y coeficiente principal $35 > 0$, luego $p > 0$ en todo $\mathbb{R}$.

Caso $\ell = 8$. $p'(t) = -96t^2 + 242t - 154$ tiene discriminante $242^2 - 4 \cdot 96 \cdot 154 = 58\,564 - 59\,136 = -572 < 0$ y coeficiente principal $-96 < 0$, luego $p' < 0$ en todo $\mathbb{R}$ y p es estrictamente decreciente. En consecuencia $p(t) \ge p(1) = -32 + 121 - 154 + 66 = 1 > 0$ en $(0, 1]$.

Se aplican, pues, las proposiciones B.5 y B.9 con $\ell = 4, 6, 8$, que dan respectivamente la regularidad exacta $C^{\ell-1}$ y la $(\ell - 1)$-estabilidad exacta de los pesos. □

Estos tres últimos corolarios nos proporcionan la k-estabilidad exacta buscada para nuestros pesos Shepard-Wendland, que usamos en los experimentos numéricos del capítulo 4.

Apéndice C

Códigos de MATLAB

```
function H = haltonseq(N, s)
    % Genera N puntos de Halton de dimensión s
    p = haltonset(s);
    H = net(p, N);
end
```

Código 1: Genera la secuencia de Halton en MATLAB.

```
function gridpoints = MakeSDGrid(s, neval)
% MakeSDGrid(s,neval)
% Produce una matriz de puntos equiespaciados en el cubo de dimensión s (un
% punto por fila)

    if (s == 1)
        gridpoints = linspace(0, 1, neval)';
        return;
    end

    % 1. Creamos un vector base 1D
    v = linspace(0, 1, neval);

    % 2. Preasignamos un 'Cell Array' para recoger las 's' salidas de
    % ndgrid
    % Esto sustituye a crear [x1, x2, ..., xs]
    grid_cells = cell(1, s);

    % 3. Llamamos a ndgrid expandiendo el vector 'v' s-veces como entrada,
```

```
    % y recogemos la salida en las celdas.
    % La sintaxis {:} expande la celda automáticamente.
    inputs = repmat({v}, 1, s);
    [grid_cells{:}] = ndgrid(inputs{:});

    % 4. Preasignamos la matriz final de puntos para máxima velocidad
    gridpoints = zeros(neval^s, s);

    % 5. Volcamos los datos por columnas en la matriz final
    for d = 1:s
        gridpoints(:, d) = grid_cells{d}(:);
    end

end
```

Código 2: Genera mallado equiespaciado en el que se evalúan los errores.

```
function DM = DistanceMatrix(dsites,ctrs)
[M,s] = size(dsites); [N,s] = size(ctrs);
DM = zeros(M,N);
% Crea matriz distancia, donde los dsites son los puntos donde se evalúa
%función ||x-x_k||, mientras que los centros son los x_k. ambos suelen
%coincidir
% El comando ndgrid produce dos matrices de tamaño MxN:
%   dr: formada por N columnas idénticas (cada una contiene
%       la d-ésima coordenada de los M puntos de datos)
%   cc: formada por M filas idénticas (cada una contiene
%       la d-ésima coordenada de los N centros)
for d=1:s
    [dr,cc] = ndgrid(dsites(:,d),ctrs(:,d));
    DM = DM + (dr-cc).^2;
end
DM = sqrt(DM);
```

Código 3: Función para construir la matriz distancia.

```
function tf = testfunction(s, points)
% tf = testfunction(s,points)
% Evalúa la función test (normalizada, para que su máximo sea 1) del
% ejemplo de matrices distancia en los puntos dados (points) para la
% dimensión dada (s)

tf = 4^s * prod(points .* (1 - points), 2);

end
```

Código 4: Función de prueba en primer ejemplo de matrices distancias.

```
function PlotSurf(xe, ye, Pf, exact, maxerr, fview, caption)
    figure
    % Usamos size(xe) para que sea robusto
    Pf_plot = reshape(Pf, size(xe));
    Err_plot = reshape(abs(Pf - exact), size(xe));

    h = surf(xe, ye, Pf_plot, Err_plot);
    set(h, 'FaceColor', 'interp', 'EdgeColor', 'none');
    view(fview); colormap hsv;

    [cmin, cmax] = caxis;
    caxis([cmin - 0.25*maxerr, cmax]);
    vcb = colorbar;
    if maxerr > 0, ylim(vcb, [0, maxerr]); end
    ylabel(vcb, 'Error abs');
    title(caption);
end
```

Código 5: Genera gráficos de funciones 2D en 3D.

```
% PlotError2D(xe,ye,Pf,exact,maxerr,fview,caption)
% Genera un gráfico del error absoluto para la superficie Pf
% (abs(Pf-exact))
function PlotError2D(xe, ye, Pf, exact, maxerr, fview, caption)

    figure;

    % Reshape dinámico adaptado a la malla xe ---
    Error_2D = reshape(abs(Pf - exact), size(xe));

    errorplot = surf(xe, ye, Error_2D);
    set(errorplot, 'FaceColor', 'interp', 'EdgeColor', 'none');

    % Compatibilidad: Mantenemos caxis ---
    [cmin, cmax] = caxis;
    caxis([cmin - 0.25 * maxerr, cmax]);

    view(fview);
    colormap hsv;

    vcb = colorbar('vert');

    % Evita error si maxerr es 0 ---
    if maxerr > 0
        ylim(vcb, [0, maxerr]);
```

```
    end

    set(get(vcb, 'YLabel'), 'String', 'Error', 'FontSize', 14);

    title(caption);
end
```

Código 6: Genera gráficos del error de funciones 2D en 3D.

```
function PlotSlices(xe,ye,ze,Pf,xslice,yslice,zslice,caption)
% PlotSlices(xe,ye,ze,Pf,neval,xslice,yslice,zslice,caption)
% Genera un slice plot de volumen para Pf
% xslice,yslice,zslice define el rango y número de cortes.

figure
% Usar size(xe) hace que el reshape sea a prueba de fallos.
Pf_3D = reshape(Pf, size(xe));

pfit = slice(xe, ye, ze, Pf_3D, xslice, yslice, zslice);
set(pfit, 'FaceColor', 'interp', 'EdgeColor', 'none')
daspect([1 1 1])
view(3);
axis([0 1 0 1 0 1])

vcb = colorbar;
ylabel(vcb, 'Function value', 'FontSize', 14);
title(caption)

end
```

Código 7: Genera planos de cortes de funciones 3D.

```
% PlotErrorSlices(xe,ye,ze,Pf,exact,xslice,yslice,zslice,caption)
% Genera un slice plot de error volumétrico: abs(Pf-exact)
function PlotErrorSlices(xe, ye, ze, Pf, exact, xslice, yslice, ...
  zslice, caption)

    % Plot slices para el error
    figure;

    % Calculamos el error absoluto
    error_abs = abs(Pf - exact);

    % Usamos size(xe) para evitar un error de dimensiones
    error_3D = reshape(error_abs, size(xe));

    % Creamos los slices
```

```
    errorplot = slice(xe, ye, ze, error_3D, xslice, yslice, zslice);
    set(errorplot, 'FaceColor', 'interp', 'EdgeColor', 'none');

    daspect([1 1 1]);
    view(3);
    axis([0 1 0 1 0 1]);

    % --- Volvemos a la sintaxis con caxis ---
    [cmin, cmax] = caxis;
    caxis([cmin - 0.25 * cmax, cmax]);

    colormap hsv;

    % Barra de color
    vcb = colorbar('vert');

    % Solo aplicamos ylim si cmax es mayor que 0 para evitar fallos
    % internos
    if cmax > 0
        ylim(vcb, [0, cmax]);
    end

    set(get(vcb, 'YLabel'), 'String', 'Error', 'FontSize', 14);
    title(caption);

end
```

Código 8: Genera planos de cortes del error absoluto entre dos funciones 3D.

```
% Script ejemplo capítulo 1, con bucle for para gráficas y generación de
% tabla LaTeX
% -------------------------------------------------------------------------
clearvars; close all; clc;

%% ================= PARTE 1: GENERACIÓN DE GRÁFICAS =================
% Definimos las combinaciones de s, k y neval que queremos evaluar
valores_s = [1, 2, 3];
valores_k = [2, 4, 3];
valores_neval = [1000, 40, 10];

% Bucle que recorre las 3 combinaciones para graficar
for i = 1:length(valores_s)

    s = valores_s(i);
    k = valores_k(i);
    neval = valores_neval(i);

    fprintf(['\n[GRÁFICAS] Evaluando caso: s = %d, k = %d, neval' ...
      ' = %d\n'], s, k, neval);
```

```
21
22      N = (2^k+1)^s;
23      M = neval^s;
24
25      dsites = haltonseq(N,s);%los nodos para la interpolación (ptos de
26      % Halton)
27      ctrs = dsites;
28      epoints = MakeSDGrid(s,neval);%mallado equiespaciado en el cubo
29      rhs = testfunction(s,dsites);%valores exactos función test en nodos
30
31      IM = DistanceMatrix(dsites,ctrs);%matriz del método, donde IM_ij es
32      % ||x_i-x_j||
33      EM = DistanceMatrix(epoints,ctrs);%matriz donde EM_ij es
34  % ||z_i-x_j||, con z_i los puntos del mallado equiespaciado
35
36
37      Pf = EM * (IM\rhs); %nos da valor función interpoladora evaluada en los
38  % puntos del mallado equiespaciado, para compararlos luego con los valores
39  % de la función test original
40      exact = testfunction(s,epoints);%función test evaluada en puntos
41      % mallado equiespaciado
42
43      maxerr = norm(Pf-exact,inf);
44      rms_err = norm(Pf-exact)/sqrt(M);
45      fprintf('RMS error:      %e\n', rms_err)
46      fprintf('Error máximo: %e\n', maxerr)
47
48      switch s
49          case 1
50              figure;
51              plot(epoints, Pf)
52              title(['Interpolación (s=', num2str(s), ', k=', ...
53                num2str(k), ')']);
54              figure;
55              plot(epoints, abs(Pf-exact))
56              title(['Error absoluto (s=', num2str(s), ', k=', ...
57                num2str(k), ')']);
58          case 2
59              fview = [-30,30];
60              xe = reshape(epoints(:,2),neval,neval);
61              ye = reshape(epoints(:,1),neval,neval);
62              caption = ['Interpolación (s=', num2str(s), ') false' ...
63                ' colored by error.'];
64              PlotSurf(xe, ye, Pf, exact, maxerr, fview, caption);
65              caption = ['Error absoluto en la interpolación' ...
66                ' (s=', num2str(s), ').'];
67              PlotError2D(xe,ye,Pf,exact,maxerr,fview,caption);
68          case 3
69              xe = reshape(epoints(:,2),neval,neval,neval);
70              ye = reshape(epoints(:,1),neval,neval,neval);
```

```
            ze = reshape(epoints(:,3),neval,neval,neval);
            xslice = .25:.25:1; yslice = 1; zslice = [0,0.5];
            caption = ['Slice plot de la interpolación' ...
              ' (s=', num2str(s), ').'];
            PlotSlices(xe,ye,ze,Pf,xslice,yslice,zslice,caption);
            caption = ['Slice plot del error absoluto en la' ...
              ' interpolación(s=', num2str(s), ').'];
            PlotErrorSlices(xe,ye,ze,Pf,exact,xslice,yslice, ...
              zslice,caption);
        otherwise
            disp('-> No se muestran gráficos para s>3')
    end
end


%% ================= PARTE 2: GENERACIÓN DE LA TABLA EN LATEX ============


% Evitamos que MATLAB llene la consola de avisos de matrices mal
% condicionadas
warning('off', 'all');

% Límites máximos de 'k' para no saturar la memoria
k_max_por_dimension = [12, 6, 3, 2, 2, 1];

% Ajuste de 'neval' modificado: 4 fijo para s=4, 5 y 6
neval_tabla = [1000, 40, 10, 4, 4, 4];

fprintf('\\begin{table}[H]\n');
fprintf('  \\centering\n');
fprintf('  \\scalebox{0.8}{\n');
fprintf('  \\begin{tabular}{| c | cc | cc | cc | cc | cc | cc |}\n');
fprintf('  \\hline\n');
fprintf(['  & \\multicolumn{2}{c|}{1D} & \\multicolumn{2}{c|}{2D}' ...
  ' & \\multicolumn{2}{c|}{3D} & \\multicolumn{2}{c|}{4D} & ' ...
  '\\multicolumn{2}{c|}{5D} & \\multicolumn{2}{c|}{6D} \\\\\n']);
fprintf(['  $k$ & $N$ & RMS-error & $N$ & RMS-error & $N$ & RMS-error' ...
  ' & $N$ & RMS-error & $N$ & RMS-error & $N$ & RMS-error \\\\\n']);
fprintf('  \\hline\n');

% Bucle para las filas (k de 1 a 12)
for k_tab = 1:12
    fprintf('  %2d ', k_tab); % Imprime el valor de k

    % Bucle para las columnas (dimensiones de 1 a 6)
    for s_tab = 1:6
        if k_tab <= k_max_por_dimension(s_tab)
            N_tab = (2^k_tab + 1)^s_tab;
            neval_t = neval_tabla(s_tab);
            M_tab = neval_t^s_tab;
```

```
121
122             dsites_tab = haltonseq(N_tab, s_tab);
123             epoints_tab = MakeSDGrid(s_tab, neval_t);
124             rhs_tab = testfunction(s_tab, dsites_tab);
125
126             IM_tab = DistanceMatrix(dsites_tab, dsites_tab);
127             EM_tab = DistanceMatrix(epoints_tab, dsites_tab);
128
129             Pf_tab = EM_tab * (IM_tab \ rhs_tab);
130             exact_tab = testfunction(s_tab, epoints_tab);
131
132             rms_err_tab = norm(Pf_tab - exact_tab) / sqrt(M_tab);
133
134             % Imprime los valores separando N y Error con el ampersand (&)
135             fprintf('& %d & %1.6e ', N_tab, rms_err_tab);
136         else
137             % Si superamos el límite, dejamos las celdas de LaTeX vacías
138             % respetando las columnas
139             fprintf('& & ');
140         end
141     end
142     fprintf(' \\\\\\\\\\n'); % Imprime el salto de línea de LaTeX (\\)
143 end
144
145 fprintf('  \\hline\n');
146 fprintf('  \\end{tabular}}\n');
147 fprintf(['\\caption{Fit de la matriz distancia para los puntos de' ...
148   ' Halton para dimensiones de 1 a 6.}\n']);
149 fprintf('\\end{table}\n');
150
151 warning('on', 'all');
```

Código 9: Script del primer ejemplo de matrices distancia.

```
1  % RBFInterpolation2D
2
3  clearvars; close all; clc;
4
5  %% 1. CONFIGURACIÓN Y DEFINICIONES
6  % Parámetros de la RBF (Gaussiana)
7  rbf_gaussiana = @(e,r) exp(-(e*r).^2);
8  eps_fijo = 21.1; % Parámetro de forma (epsilon)
9
10 % Función de test de Franke
11 f1 = @(x,y) 0.75*exp(-((9*x-2).^2+(9*y-2).^2)/4);
12 f2 = @(x,y) 0.75*exp(-((9*x+1).^2/49+(9*y+1).^2/10));
13 f3 = @(x,y) 0.5*exp(-((9*x-7).^2+(9*y-3).^2)/4);
14 f4 = @(x,y) 0.2*exp(-((9*x-4).^2+(9*y-7).^2));
15 func_test_franke = @(x,y) f1(x,y)+f2(x,y)+f3(x,y)-f4(x,y);
```

```
16
17  resol_malla = 40;
18  valores_k_graficas = 1:6;
19  valores_k_tabla1 = 1:7;
20  resol_epsilon_vec = 250; % Alta resolución para capturar picos de
21  % inestabilidad
22
23  % Vistas de cámara 3D unificadas
24  view_surfaces_3d = [-40, 30];
25  view_points_3d = [-40, 30];
26
27  fprintf('\n=== INICIANDO HERRAMIENTA DE INTERPOLACIÓN RBF 2D ===\n');
28
29  %% ================= PARTE 1: GENERACIÓN DE MALLA ========
30  epoints = MakeSDGrid(2, resol_malla);
31  % Preparamos las matrices xe, ye para las funciones de dibujo.
32  % IMPORTANTE: Como MakeSDGrid nos da en cada fila las coordenadas (x,y)
33  % de los puntos del mallado equiespaciado, solo para hacer las gráficas
34  % matlab necesita los datos en forma de matrices para saber en qué orden
35  % van
36  xe = reshape(epoints(:,1), resol_malla, resol_malla);
37  ye = reshape(epoints(:,2), resol_malla, resol_malla);
38  % Evaluamos la solución exacta (función test) en la malla fina de
39  % evaluación
40  exact = func_test_franke(epoints(:,1), epoints(:,2));
41
42
43  %% ================= PARTE 2: SUPERFICIES 3D ========
44  fprintf(['\n--- PARTE 2: Generando Gráficas de Superficie y Puntos de' ...
45    ' Halton ---\n']);
46  % Bucle para k desde 1 hasta 6
47  for i = 1:length(valores_k_graficas)
48      k = valores_k_graficas(i);
49      N = (2^k+1)^2;
50      % Nodos de interpolación
51      dsites = haltonseq(N, 2);
52      ctrs = dsites;
53
54      if k == 1 || k == 2
55          figure('Name', sprintf('Nodos y Plano z=0 (k=%d)', k), ...
56            'NumberTitle', 'off');
57          % 1. Dibujamos el plano transparente en z=0
58          surf(xe, ye, zeros(size(xe)), 'FaceAlpha', 0.2, 'EdgeColor', ...
59            'none', 'FaceColor', 'interp');
60          hold on; %Para dibujar encima
61           % 2. Dibujamos los puntos en 3D fijando z=0
62          plot3(dsites(:,1), dsites(:,2), zeros(N, 1), 'ko', ...
63            'MarkerFaceColor', 'b', 'MarkerSize', 6);
64          axis([0 1 0 1 -0.2 1.2]);
65          grid on;
```

```
66          view(view_points_3d);
67          xlabel('x'); ylabel('y'); zlabel('z');
68          title(sprintf('Nodos de Halton y Plano z=0 (k = %d, N = %d)', ...
69            k, N));
70          set(gca, 'FontSize', 12);
71          hold off;
72      end
73
74      rhs = func_test_franke(dsites(:,1), dsites(:,2)); %función test
75      % evaluada en ptos Halton
76      % Calculamos las matrices de distancia una sola vez para ambos métodos
77      DM_data = DistanceMatrix(dsites, ctrs);
78      DM_eval = DistanceMatrix(epoints, ctrs);
79
80      % --- MÉTODO 1: Gaussiana ---
81      IM_gauss = rbf_gaussiana(eps_fijo, DM_data); %matriz del método
82      % (evaluada en los nodos) donde IM_ij es la gaussiana evaluada en
83      % ||x_i-x_j||
84      EM_gauss = rbf_gaussiana(eps_fijo, DM_eval); %matriz donde EM_ij es la
85      % gaussiana evaluada en ||z_i-x_j||, con z_i los puntos del mallado
86      % equiespaciado
87      warning('off', 'all');
88      Pf_gauss = EM_gauss * (IM_gauss \ rhs); %nos da valor función
89      % interpoladora de RBF Gaussiano evaluada en los puntos del mallado
90      % equiespaciado, para compararlos luego con los valores de la función
91      % test original (función de Franke)
92      warning('on', 'all');
93      maxerr_gauss = norm(Pf_gauss - exact, inf);
94      caption_gauss = sprintf('Gaussiana (\\epsilon = 21.1) N = %d', N);
95      PlotSurf(xe, ye, Pf_gauss, exact, maxerr_gauss, view_surfaces_3d, ...
96        caption_gauss);
97
98      % --- MÉTODO 2: Matriz Distancia ---
99      Pf_dm = DM_eval * (DM_data \ rhs); %nos da valor función
100     % interpoladora de matrices distancia evaluada en los puntos del
101     % mallado equiespaciado, para compararlos luego con los valores de la
102     % función test original (función de Franke)
103     maxerr_dm = norm(Pf_dm - exact, inf);
104     caption_dm = sprintf('Matriz Distancia con N = %d', N);
105     PlotSurf(xe, ye, Pf_dm, exact, maxerr_dm, view_surfaces_3d, ...
106       caption_dm);
107 end
108
109
110 %% ================= PARTE 3: TABLA LATEX primera ========
111 fprintf(['\n\n--- PARTE 3: Generando Tabla LaTeX 2.1 (Calculando k=7, ' ...
112   'tardará un poco) ---\n']);
113 warning('off', 'all');
114
115 fprintf('\n%% === TABLA 2.1 ===\n\n');
```

```
fprintf('\\begin{table}[H]\n');
fprintf('  \\centering\n');
fprintf('  \\scalebox{0.9}{\n');
fprintf('  \\begin{tabular}{cc | ccc | ccc}\n');
fprintf('  \\hline\n');
fprintf(['  & & \\multicolumn{3}{c|}{Gaussiano} & \\multicolumn{3}{c}' ...
  '{Matrices distancia} \\\\\\\n']);
fprintf(['  $k$ & $N$ & RMS-error & $p$ & max-error & RMS-error &' ...
  ' $p$ & max-error \\\\\\\n']);
fprintf('  \\hline\n');

M = resol_malla^2;

for k_tab = valores_k_tabla1
    N_tab = (2^k_tab + 1)^2;
    dsites_tab = haltonseq(N_tab, 2);
    ctrs_tab = dsites_tab;
    %Mismo proceso antes, pero ahora llega hasta k=7, para hacer la tabla
    rhs_tab = func_test_franke(dsites_tab(:,1), dsites_tab(:,2));

    DM_data_tab = DistanceMatrix(dsites_tab, ctrs_tab);
    DM_eval_tab = DistanceMatrix(epoints, ctrs_tab);

    IM_gauss = rbf_gaussiana(eps_fijo, DM_data_tab);
    EM_gauss = rbf_gaussiana(eps_fijo, DM_eval_tab);
    Pf_gauss = EM_gauss * (IM_gauss \ rhs_tab);

    rms_gauss = norm(Pf_gauss - exact) / sqrt(M);
    max_gauss = norm(Pf_gauss - exact, inf);

    Pf_dm = DM_eval_tab * (DM_data_tab \ rhs_tab);
    rms_dm = norm(Pf_dm - exact) / sqrt(M);
    max_dm = norm(Pf_dm - exact, inf);

    % CÁLCULO E IMPRESIÓN CON TASA DE CONVERGENCIA 'p'
    if k_tab == valores_k_tabla1(1)
        % Primera fila: Sin 'p' (hueco en blanco)
        fprintf('  %d & %d & %1.6e & & %1.6e & %1.6e & & %1.6e \\\\\n', ...
                k_tab, N_tab, rms_gauss, max_gauss, rms_dm, max_dm);
    else
        % Cálculo de 'p' usando el RMS anterior
        p_gauss = log(rms_gauss / rms_gauss_prev) / log(0.5);
        p_dm = log(rms_dm / rms_dm_prev) / log(0.5);
        fprintf(['  %d & %d & %1.6e & %1.4f & %1.6e & %1.6e & %1.4f & ' ...
          '%1.6e \\\\\n'], ...
                k_tab, N_tab, rms_gauss, p_gauss, max_gauss, rms_dm, ...
                p_dm, max_dm);
    end

    % Guardamos los valores para la siguiente iteración (para la primera no
```

```
166     % hacen falta porque para no hay elemento en la columna de p
167     rms_gauss_prev = rms_gauss;
168     rms_dm_prev = rms_dm;
169 end
170 fprintf('  \\hline\n  \\end{tabular}}\n\\end{table}\n');
171 warning('on', 'all');
172 
173 
174 %% ================ PARTE 4: CURVAS ERROR VS EPSILON ========
175 fprintf('\n--- PARTE 4: Generando Curvas Error vs Epsilon ---\n');
176 fprintf('    (Evaluando matrices, por favor espera un momento...)\n');
177 
178 eps_vec = linspace(0.1, 25, resol_epsilon_vec); %No empiezo en 0, porque
179 % ahí IM_ij vale 1 para todos sus elementos, luego es no singular.
180 %he dividido el intervalo en 250 ptos para tener mucha precisión a la hora
181 %de observar picos de inestabilidad en gráficas error-epsilon y a la hora
182 %de hacer la tabla
183 
184 figure('Name', 'Errores vs Epsilon (Escalado Dinamico)', 'Position', ...
185   [100, 100, 1200, 700]);
186 
187 for i = 1:length(valores_k_graficas)
188     k = valores_k_graficas(i);
189     N = (2^k+1)^2;
190     fprintf('Calculando curva para N = %d ... ', N);
191 
192     dsites = haltonseq(N, 2);
193     rhs = func_test_franke(dsites(:,1), dsites(:,2));
194     DM_data = DistanceMatrix(dsites, dsites);
195     DM_eval = DistanceMatrix(epoints, dsites);
196 
197     rms_vec = zeros(size(eps_vec));
198     max_vec = zeros(size(eps_vec));
199 
200     warning('off', 'all');
201     for j = 1:length(eps_vec)
202         ep_val = eps_vec(j);
203         IM = rbf_gaussiana(ep_val, DM_data);%Matriz donde cada componente
204         % es la Gaussiana con el epsilon en el que esté el for en ese
205         % momento, evaluada en elementos matriz distancia del método
206         % (evaluada en ptos de Halton)
207         EM = rbf_gaussiana(ep_val, DM_eval); %Matriz donde cada componente
208         % es la Gaussiana con el epsilon en el que esté el for en ese
209         % momento, evaluada en elementos matriz distancia evaluada en ptos
210         % mallado equiespaciado
211         Pf = EM * (IM \ rhs); %Función interpoladora evaluada en ptos
212         % mallado equiespaciado
213 
214         rms_vec(j) = norm(Pf - exact) / sqrt(M);
215         max_vec(j) = norm(Pf - exact, inf);
```

```
216     end
217     fprintf('Hecho.\n');
218     warning('on', 'all');
219 
220     subplot(2, 3, i);%plot con 2 filas y 3 columnas de gráficas. i indica
221     % la posición de cada gráfica conforme corre el bucle de 1 a 6
222     semilogy(eps_vec, max_vec, 'k--', 'LineWidth', 1.5); hold on;
223     semilogy(eps_vec, rms_vec, 'k-', 'LineWidth', 1.5);
224     %hemos superpuesto el error máximo y el RMS error en un gráfico, donde
225     %el eje y está en escala logarítmica
226     title(sprintf('N = %d', N));
227     xlabel('\epsilon'); ylabel('error');
228 
229 
230     y_min = min(rms_vec);
231     y_max = max(max_vec);
232     %Ajustar límite de las gráficas
233     if y_min <= 0 || isnan(y_min) || isinf(y_min), y_min = 1e-16; end
234     if isnan(y_max) || isinf(y_max), y_max = 1e2; end
235 
236     xlim([0 25]);
237     ylim([y_min / 5, y_max * 5]); % Ajuste 100% dinámico garantizado
238     % -----------------------------------
239 
240     if i == 1
241         legend('max-error', 'RMS-error', 'Location', 'southwest'); %hace
242         % salga leyenda línea continua y discontinua solo en primera
243         % gráfica (quitar si quiero)
244     end
245     grid on;
246 end
247 
248 
249 %% ================= PARTE 5: TABLA LATEX segunda ========
250 fprintf('\n\n--- PARTE 5: Generando Tabla LaTeX 2.2 ---\n');
251 
252 fprintf('\n%% === TABLA 2.2 ===\n\n');
253 fprintf('\\begin{table}[H]\n');
254 fprintf('  \\centering\n');
255 fprintf('  \\scalebox{0.85}{\n');
256 fprintf('  \\begin{tabular}{cc | cccc | cccc}\n');
257 fprintf('  \\hline\n');
258 fprintf(['  & & \\multicolumn{4}{c|}{$\\epsilon$ ``seguro'' más' ...
259   ' pequeño} & \\multicolumn{4}{c}{smallest RMS-error $\\epsilon$}' ...
260   ' \\\\\n']);
261 fprintf(['  $k$ & $N$ & $\\epsilon$ & RMS-error & $p$ & max-error &' ...
262   ' $\\epsilon$ & RMS-error & $p$ & max-error \\\\\n']);
263 fprintf('  \\hline\n');
264 
265 for i = 1:length(valores_k_graficas)
```

```
266       k_tab = valores_k_graficas(i);
267       N_tab = (2^k_tab + 1)^2;
268       dsites_tab = haltonseq(N_tab, 2);
269       ctrs_tab = dsites_tab;
270       rhs_tab = func_test_franke(dsites_tab(:,1), dsites_tab(:,2));
271       DM_data_tab = DistanceMatrix(dsites_tab, ctrs_tab);
272       DM_eval_tab = DistanceMatrix(epoints, ctrs_tab);
273
274       warning('off', 'all');
275       %-----------Busco mínimo epsilon
276       options = optimset('Display','off', 'TolX', 1e-3); %Display off es
277       % para que Matlab no diga cada paso que hace. TolX 10^(-3) es la
278       % precisión, cuando estés tan seguro de cuál es el mínimo que la duda
279       % sea menor que 0.001, deja de buscar.
280       rms_obj = @(e) norm((rbf_gaussiana(e, DM_eval_tab) * ...
281         (rbf_gaussiana(e, DM_data_tab) \ rhs_tab)) - exact) / sqrt(M);%Todo
282       %lo que antes habíamos hecho en varios pasos lo hace en una sola
283       %fórmula. Le damos el valor de epsilon y calcula el RMS-error
284       eps_rms_min = fminbnd(rms_obj, 0.05, 30, options);%Busca el mínimo de
285       %la función de la línea anterior entre 0.05 y 30 con options que es lo
286       %que le habíamos dicho antes de la toleracia y el Display off
287
288       IM_opt = rbf_gaussiana(eps_rms_min, DM_data_tab);
289       EM_opt = rbf_gaussiana(eps_rms_min, DM_eval_tab);
290       Pf_opt = EM_opt * (IM_opt \ rhs_tab);
291       rms_opt = norm(Pf_opt - exact) / sqrt(M);%calculamos RMS-error para
292       % epsilon mínimo
293       max_opt = norm(Pf_opt - exact, inf);
294       warning('on', 'all');
295
296       eps_safe = 25;
297       for test_ep = 25:-0.01:0.01%empezamos a buscar el epsilon en 25
298         % (muy estable) y vamos bajando
299           IM_test = rbf_gaussiana(test_ep, DM_data_tab);
300           if rcond(IM_test) < 1e-13 %si el rcond es menor que 10^(-13) (hemos
301             % elegido este para asegurar, aunque la precisión sea 10^(-16) ),
302             % nos quedamos con el epsilon justo mayor
303               eps_safe = test_ep + 0.01;
304               break;
305           end
306       end
307   %si llegamos hasta el final del bucle y no se ha dado rcond menor que el
308   % valor fijado, eps_safe sigue siendo 25, pero realmente el más seguro
309   % debería ser 0.01, tomamos 0.02 para asegurar, vemos que en ningún caso
310   % llegamos a ese límite
311       if eps_safe == 25 && rcond(rbf_gaussiana(0.01, DM_data_tab)) > 1e-13
312           eps_safe = 0.02;
313       end
314
315       warning('off', 'all');
```

```
    IM_safe = rbf_gaussiana(eps_safe, DM_data_tab);
    EM_safe = rbf_gaussiana(eps_safe, DM_eval_tab);
    Pf_safe = EM_safe * (IM_safe \ rhs_tab);
    %calculamos los errores para el epsilon seguro
    rms_safe = norm(Pf_safe - exact) / sqrt(M);
    max_safe = norm(Pf_safe - exact, inf);
    warning('on', 'all');

    % CÁLCULO E IMPRESIÓN CON TASA DE CONVERGENCIA 'p'
    if k_tab == valores_k_graficas(1)
        % Fila 1: Sin 'p' (hueco en blanco)
        fprintf(['  %d & %d & %.2f & %1.6e & & %1.6e & %.2f & %1.6e' ...
          ' & & %1.6e \\\\\n'], ...
                k_tab, N_tab, eps_safe, rms_safe, max_safe, ...
                eps_rms_min, rms_opt, max_opt);
    else
        % Cálculo de 'p' usando el RMS anterior
        p_safe = log(rms_safe / rms_safe_prev) / log(0.5);
        p_opt = log(rms_opt / rms_opt_prev) / log(0.5);
        fprintf(['  %d & %d & %.2f & %1.6e & %1.4f & %1.6e & %.2f &' ...
          ' %1.6e & %1.4f & %1.6e \\\\\n'], ...
                k_tab, N_tab, eps_safe, rms_safe, p_safe, max_safe, ...
                eps_rms_min, rms_opt, p_opt, max_opt);
    end

    % Guardamos los valores para la siguiente iteración
    rms_safe_prev = rms_safe;
    rms_opt_prev = rms_opt;
end

fprintf('  \\hline\n');
fprintf('  \\end{tabular}}\n');
fprintf(['\\caption{Interpolación por RBF Gaussiano a la función de' ...
  ' Frankee.}\n']);
fprintf('\\end{table}\n');
```

Código 10: Script del ejemplo de RBF Gaussiano y matrices distancia.

```
function DM = DistanceMatrixCSRBF(dsites, ctrs, ep)
% ========================================================================
% DistanceMatrixCSRBF
% ------------------------------------------------------------------------
% Construye la matriz de distancias entre un conjunto de puntos de datos y
% un conjunto de centros, pensada para funciones de base radial de SOPORTE
% COMPACTO (CSRBF). A diferencia de una matriz de distancias normal, esta
```

```
8  % se devuelve como matriz dispersa (sparse): solo se almacenan las
9  % distancias que caen DENTRO del soporte de la RBF (las demas se descartan,
10 % porque la RBF vale cero ahi). Esto es lo que hace que el metodo sea
11 % escalable en memoria y coste para conjuntos grandes de puntos.
12 %
13 % IMPORTANTE sobre el formato de la distancia:
14 %   La entrada ij-ésima dela matriz vale 1-ep*r, donde
15 % r=||dato_i-centro_j||_2 si el punto i esta dentro del parche generado por
16 % el centro j (r < 1/ep)  y 0 si esta fuera, luego si es 0, al ser sparse
17 % no se da. lo de 1-ep*r se usa para reescalar, como mucho vale casi 1, o
18 % si no menos.
19 %
20 % ENTRADAS:
21 %   dsites : matriz N x s. Cada fila es un punto de datos en R^s
22 %            (N puntos, s = dimension del espacio).
23 %   ctrs   : matriz M x s. Cada fila es un centro de una RBF
24 %            (M centros).
25 %   ep     : escalar > 0. Parametro de forma. El radio de soporte de la
26 %            RBF es 1/ep: un ep pequeño da soporte grande (funcion ancha),
27 %            un ep grande da soporte pequeño (funcion estrecha).
28 %
29 % SALIDA:
30 %   DM     : matriz DISPERSA N x M. La entrada (i,j) vale 1 - ep*r_ij, con
31 %            r_ij = distancia entre el punto de datos i y el centro j, SOLO
32 %            si r_ij < 1/ep (dentro del soporte); en caso contrario la
33 %            entrada es 0 (no se almacena, por ser sparse). Se toma 1-ep*r
34 %            para darle mayor valor a los puntos para los ptos más cercanos
35 %            al centro.
36 % ========================================================================
37
38     % Numero de puntos de datos (N) y de centros (M).
39     N = size(dsites,1);
40     M = size(ctrs,1);
41
42     % --- Radio de soporte de la RBF ---
43     % El soporte de la CSRBF es 1/ep: fuera de esa distancia la funcion
44     % vale cero. Lo usaremos como radio de busqueda para descartar pares
45     % lejanos.
46     support = 1/ep;
47
48     % --- Reserva de memoria para las listas de la matriz dispersa ---
49     % En lugar de rellenar la matriz directamente, iremos acumulando, para
50     % cada entrada no nula, su fila (rowidx), su columna (colidx) y su
51     % valor (validx); al final montaremos la matriz dispersa de golpe con
52     % sparse(). Estimamos cuántas entradas no nulas habrá para reservar
53     % memoria de una sola vez (evitar realojar memoria dentro del bucle,
54     % que es lento). La cota 25*N es una estimacion del numero de vecinos
55     % dentro del soporte; si se queda corta, MATLAB la ampliaria sola, pero
56     % conviene acertar.
57     nzmax = 25*N;
```

```
58      rowidx = zeros(1,nzmax);   % indices de fila de cada entrada no nula
59      colidx = zeros(1,nzmax);   % indices de columna de cada entrada no nula
60      validx = zeros(1,nzmax);   % valor (1-ep*r) de cada entrada no nula
61      %esos luego serán las tres primeras entradas del sparse para crear la
62      %matriz sparse dándole los valores no nulos y sus posiciones
63
64      % Punteros para ir escribiendo en las listas anteriores por tramos:
65      % istart = primera posicion libre, iend = ultima posicion escrita.
66      istart = 1;
67      iend   = 0;
68
69      % --- Eleccion de la estrategia según haya mas centros o mas datos ---
70      % El k-D tree (estructura de busqueda espacial rapida) se construye
71      % sobre uno de los dos conjuntos, y se consulta con el otro. Conviene
72      % construir el arbol sobre el conjunto más grande y recorrer en el
73      % bucle el más pequeño , porque asi se hacen menos consultas
74      % (cada consulta al arbol es barata y menos iteraciones de bucle es lo
75      % mejor). Esto lo miramos con un if
76      if M > N
77          % --- Caso: más centros que datos ---
78          % Construimos el arbol sobre los CENTROS y recorremos los DATOS.
79          Tree = createns(ctrs, 'NSMethod', 'kdtree');
80
81          % Recorremos cada punto de datos i.
82          for i = 1:N
83              % Consulta por rango: pedimos al arbol todos los CENTROS que
84              % están a distancia menor que 'support' del punto de datos i,
85              % es decir, los centros cuyo soporte cubre a este dato.
86              [idx_cell, dist_cell] = rangesearch(Tree, dsites(i,:), support);
87              idx = idx_cell{1}'; % Transpuesto a columna
88              dist = dist_cell{1}'; % Transpuesto a columna
89
90              % Cuantos centros han caido dentro del soporte de este dato.
91              newentries = length(idx);
92
93              % Avanzamos el puntero final: vamos a añadir 'newentries'
94              % entradas nuevas a las listas.
95              iend = iend + newentries;
96
97              % Rellenamos el tramo [istart:iend] de las tres listas:
98              % - fila: para un i fijo (dato fijo), todas estas entradas
99              % están en la misma fila (fila i) (este dato), por eso
100             % repmat(i,...) repite el indice i tantas veces como
101             %   centros encontrados(newentries)
102             rowidx(istart:iend) = repmat(i,1,newentries);%va recogiendo
103             % todos los índices de filas de datos no nulos cuando pasas al
104             % siguiente i, istart e iend se les suman el newentries del
105             % dato i(no del i+1), para que se vayan guardando todos los
106             % índices.
107
```

```
108             % - columna: los indices de los centros encontrados (idx). Para
109             % un mismo dato, conforme varía el vcentro, varía la columna.
110             colidx(istart:iend) = idx';
111             % - valor: la distancia en forma desplazada, 1 - ep*dist. Este
112             %   es el valor u = 1 - ep*r que meteremos en esa posición de
113             % fila y columna dadas por rowidx y colidx CSRBF espera
114             % recibir.
115             validx(istart:iend) = 1 - ep*dist';
116 
117             %inicializamos para el siguiente dato.
118             istart = istart + newentries;
119         end
120     else
121         % --- Caso: más datos que centros (o igual) ---
122         % Simétrico al anterior: construimos el arbol sobre los datos y
123         % recorremos los centros. Es el caso habitual en interpolación
124         % (hay tantos centros como datos, o el bucle sobre centros es
125         % cómodo).
126         Tree = createns(dsites, 'NSMethod', 'kdtree');
127 
128         % Recorremos cada centro j.
129         for j = 1:M
130             % Pedimos al arbol todos los DATOS dentro del soporte del
131             % centro j (los datos a los que llega la RBF centrada en j).
132             [idx_cell, dist_cell] = rangesearch(Tree, ctrs(j,:), support);
133             idx = idx_cell{1}'; % Transpuesto a columna
134             dist = dist_cell{1}'; % Transpuesto a columna
135 
136             % Numero de datos encontrados para este centro.
137             newentries = length(idx);
138             iend = iend + newentries;
139 
140             % Rellenamos el tramo correspondiente:
141             % - fila: los indices de los datos encontrados (idx). Aquí
142             %   la fila es el dato y la columna el centro j (la matriz DM
143             % es N x M = datos x centros), por eso se invierten los papeles
144             %   respecto al caso anterior.
145             rowidx(istart:iend) = idx';
146             % - columna: todas en la columna j (este centro).
147             colidx(istart:iend) = repmat(j,1,newentries);
148             % - valor: igual que antes, distancia desplazada 1 - ep*dist.
149             validx(istart:iend) = 1 - ep*dist';
150 
151             istart = istart + newentries;
152         end
153     end
154 
155     % --- Montaje de la matriz dispersa ---
156     % find(rowidx) localiza las posiciones realmente usadas de las listas
157     % (descarta el espacio reservado de mas en nzmax que quedo a cero). Sin
```

```
    % esto, las entradas sobrantes (con fila 0) ensuciarian el ensamblaje.
    idx = find(rowidx);%mira qué elementos no son 0, porque al inicializar
    % al principio pusimos un vector de 25*N ceros. No hace falta hacerlo
    % también para columnas y valores porque los ceros estrán en los mismos
    % sitios. así hacemos rowidex(idex), colidex(idex), validex(idex).

    % sparse(filas, columnas, valores, N, M) construye la matriz dispersa
    % N x M colocando 'valores' en las posiciones (filas, columnas)
    % indicadas. Tomamos solo las posiciones validas (idx).
    DM = sparse(rowidx(idx), colidx(idx), validx(idx), N, M);

end
```

Código 11: Función para construir la matriz distancia dispersa para funciones de base radial de soporte compacto.

```
function h = distancia_llenado_knnsearch(chi, Y)
% h_{chi,Omega} = max_i min_j ||Y(i,:) - chi(j,:)|| calculada de forma muy
% eficiente
%
% chi: N x s, nodos
% Y:   Neval x s, puntos de evaluacion (rejilla fina que aproxima Omega)
% Requiere: Statistics and Machine Learning Toolbox.

    [~, D] = knnsearch(chi, Y);   % D(i) = distancia de Y(i,:) a su nodo
    % más cercano en chi
    h = max(D);
end
```

Código 12: Cálculo de la distancia de llenado mediante `knnsearch`.

```
function [k_min, N_j, info] = recubrimiento_Nj(s, p_max, c)
% A partir solo de la geometria del recubrimiento (condiciones sobre n y
% Nj, que no dependen ni de f ni del valor concreto de phi), devuelve:
% k_min : primer k tal que N=(2^k+1)^s cumple la condicion (1):
% N/N_j>=n_0^s.
% N_j: menor nº (nominal) de nodos por parche con N_j >= max{Nj2, Nj3}.
% ENTRADAS:
% s: dimensión.
% p_max: mayor p impar usado, por defecto 5. Solo interviene en (3) en
% L=c(s+(p_max-1)/2,s)).
% c: holgura de (2) y (3), por defecto 1.75. Bajarlo reduce N_j.
% SALIDA: info struct con n0, lambda_s, alpha0, L, Nj2, Nj3, c, p_max. Para
% extraer una de esas variables de struct debemos hacer, por ejemplo,
% "info.n0".
    if nargin < 2 || isempty(p_max), p_max = 5;   end % si le meteemos
```

```
    % menos de dos parámetros de entrada o la segunda entrada es un vector
    % vacío, asigna p_max=5, por defecto.
    if nargin < 3 || isempty(c), c = 7/4; end %si le metemos menos de tres
    % parámetros de entrada, o el tercero es un vector vacío, asigna c=7/4.
    lambda_s = pi^(s/2) / gamma(s/2 + 1);% volumen de la bola unidad
    n0 = floor(1/(1 - sqrt(s)/2)) + 1; % condicion sobre n.
    alpha0 = (n0 - 1)/n0;
    m = (p_max + 1)/2;
    L = nchoosek(s + (m - 1), s); % dim Pi_{m-1}^s, para el mayor p.

    base =  c^s / (lambda_s*alpha0^s);
    Nj2  = base;
    Nj3  = L * base;
    N_j  = ceil(max(Nj2, Nj3));

    k_min = 1;
    while round( (2^k_min + 1) / N_j^(1/s) ) < n0
        k_min = k_min + 1;
    end

    info = struct('n0', n0, 'lambda_s', lambda_s, 'alpha0', alpha0, ...
                  'L', L, 'Nj2', Nj2, 'Nj3', Nj3, 'c', c, 'p_max', p_max);
end
```

Código 13: Construcción del recubrimiento de parches con N_j acotado.

```
function P = build_poly_basis(pts, p, center, rho)
    % Construye la base polinomial Pi_{m-1}^s evaluada en los puntos 'pts',
    % que sirve para construir el interpolante local con phi=r^p. P es una
    % matriz cuya primera columna es el primer vector base evaluado en cada
    % punto de pts, la segunda columna, el segundo vector evaluado en todos
    % los pts y así.
    % s = 2 (dimensión del espacio: x, y)
    % m = (p+1)/2. Por tanto:
    % Si p=1 -> m=1 -> Polinomios de grado 0 (constante)
    % Si p=3 -> m=2 -> Polinomios de grado <= 1 (lineales)
    % Si p=5 -> m=3 -> Polinomios de grado <= 2 (cuadráticos)
    % Si p=0 -> m=0 -> Gaussiana (No requiere polinomios)
    %
    % La base se recentra y reescala al parche: xt=(x-cx)/rho,
    % yt=(y-cy)/rho. Genera el mismo espacio Pi_{m-1}^s.
    x = (pts(:, 1) - center(1)) / rho;
    y = (pts(:, 2) - center(2)) / rho;
    N_pts = length(x);

    if p == 0
        P = [];                                          % Gaussiana
    elseif p == 1
        P = ones(N_pts, 1);                              % L = 1
```

```
    elseif p == 3
        P = [ones(N_pts, 1), x, y];                          % L = 3
    elseif p == 5
        P = [ones(N_pts, 1), x, y, x.^2, x.*y, y.^2];        % L = 6
    else
        error('Valor de p no soportado. Elige p = 1, 3, 5 o 0 (Gauss).');
    end
end
```

Código 14: Construye la base polinómica recentrada y reescalada de cada parche.

```
% ========================================================================
% SCRIPT: MPU para phi=r^p, con p = 1, 3, 5).
% Adaptado para diferentes funciones de peso (Wendland).
% ========================================================================
%   - Barrido en N=(2^k+1)^s con N_j fijo y M=M(N)
%   - eps de la gaussiana automatico (peor parche del N mayor,
% condicion 4).
%   - phi_rho= (r/rho)^p (Lema 1.60) .
%   - base polinomica reescalada en cada parche.
%   - una tabla de convergencia por combinacion (con h y min(RCOND)).
%
% Toda la geometría que solo depende de N (árbol de
% nodos, DM_eval, índices de nodos por parche y las matrices de peso SEM)
% se calculan una sola vez por N, fuera del bucle de las combinaciones.
% Además, para hacerlo lo más eficiente posible, el interpolante local se
% calcula una vez por Phi y se reutiliza para los pesos
% (no depende del peso).
% ========================================================================

clear; clc;

% --- 1. CONFIGURACIÓN DE PARÁMETROS ---
% n y wep se deducen de N y N_j.
% 'ep'(forma gaussiana) se calcula automaticamente (seccion 2c).
neval = 100;                  % Puntos de evaluación por eje (total:neval^2)
rcond_target = 1e-13;        % umbral de condicionamiento (RCOND)

% Definimos la función test: la de Franke en este caso.
f1 = @(x,y) 0.75*exp(-((9*x-2).^2+(9*y-2).^2)/4);
f2 = @(x,y) 0.75*exp(-((9*x+1).^2/49+(9*y+1).^2/10));
f3 = @(x,y) 0.5*exp(-((9*x-7).^2+(9*y-3).^2)/4);
f4 = @(x,y) 0.2*exp(-((9*x-4).^2+(9*y-7).^2));
func_test_franke = @(x,y) f1(x,y)+f2(x,y)+f3(x,y)-f4(x,y);

% --- 2. GENERACIÓN DE PUNTOS ---
% 2a. Puntos de evaluación (Malla uniforme regular)
% neval fijo => epoints y exact no dependen de N: una sola vez.
grid_eval = linspace(0, 1, neval);
```

```
[xe, ye] = meshgrid(grid_eval);
epoints = [xe(:), ye(:)]; %matriz neval^2x2 donde cada fila es un punto de
% evaluación.
exact = func_test_franke(epoints(:,1), epoints(:,2)); % Solución exacta:
% función de Franke evaluada en ptos de evaluación.

% --- 2b. ELECCIÓN DE N_j Y RANGO DEL BARRIDO ---
% N_j y k_min salen de la funcion recubrimiento_Nj, que
% aplica las condiciones del recubrimiento (solo dependen de s, del
% mayor p usado y de la constante c=1.75).
s = 2;     % dimension (Franke en [0,1]^2)
[k_min, N_j, recinfo] = recubrimiento_Nj(s, 5, 1.75); % p_max = 5, c = 1.75
% y obtenemos el kmin, que nos dará el Nmin. También nos da el Nj fijo que
% buscamos y recinfo acumula n0, L, Nj2, Nj3, c.
fprintf(['N_j (recubrimiento) = %d   (n0=%d, L=%d, Nj2=%.2f, Nj3=%.2f,' ...
  ' c=%.2f)\n'], N_j, recinfo.n0, recinfo.L, recinfo.Nj2, recinfo.Nj3, ...
  recinfo.c);%nos da el valor de N
k_max  = 10;   % N hasta (2^10+1)^s
k_list = k_min:k_max;
nK = numel(k_list); %longitud de k_list
fprintf('Barrido: k=%d..%d  ->  N=%d..%d\n', k_min, k_max, ...
  (2^k_min+1)^s, (2^k_max+1)^s);

% --- 2c. SELECCIÓN AUTOMÁTICA DE eps_gauss (peor parche real para N_max,
% la N máxima)
% El condicionamiento gaussiano lo domina el par de nodos más cercano
% dentro de un parche. Localizamos los nodos más apretados para N_max,
% miramos los parches que los contienen, tomamos el de menor rcond y
% ajustamos eps por biseccion para que rcond(peor) >= rcond_target.
% rcond crece con eps. Para N menores el condicionamiento solo mejora.
ep_native = 9/sqrt(2);  % umbral de epsilon
N_max   = (2^k_max + 1)^s;
n_max = round((N_max/N_j)^(1/s));
if n_max < recinfo.n0 %controla que el N_max sea suficientemente grande
  % para cumplir las cotas que exige el recubrimiento.
    error('MPU:sinCobertura', ...
        ['Ni N_max=%d alcanza cobertura: n_max=%d < n0=%d ' ...
         '(alpha=(n-1)/n <= sqrt(s)/2). Sube k_max o baja N_j.'], ...
        N_max, n_max, recinfo.n0);
end
rho_max = 1/n_max;
ds_max  = haltonseq(N_max, s);%nodos de Halton para N_max
tree_mx = createns(ds_max, 'NSMethod', 'kdtree');
gpu_mx  = linspace(0, 1, n_max);
[~, Dnn] = knnsearch(ds_max, ds_max, 'K', 2);%para no calcular todas las
% distancias busca los dos vecinos (nodos de Halton) más cercanos a cada
% nodo de Halton y Dnn guarda en la primera columna las distancias más
% cercanas (0, pues es un nodo con él mismo) y en la segunda columna la
% distancia más pequeño con el nodo más próximo (cuál es ese nodo que está
% cerca, es decir, su índice, no se guarda). Cada fila de Dnn da los dos
```

```
% nodos más próximos a un nodo (siguiendo un orden).
[~, ord] = sort(Dnn(:,2), 'ascend');%se queda con 2 columna de Dnn, ordena
% de menor a mayor las distancias y se con el índice de los nodos que
% ordenados de menor a mayor cercanía a otro nodo.
cand_nod = ds_max(ord(1:min(300,end)), :);%nos quedamos con los 300 nodos
% con distancias menores (guardamos sus coordenadas en matriz N_max x 2).
gi       = @(v) min(max(round(v*(n_max-1)) + 1, 1), n_max);
cand_ctr = unique([gpu_mx(gi(cand_nod(:,1))).', ...
  gpu_mx(gi(cand_nod(:,2))).'], 'rows');%la función gi lo que hace es el
% intervalo [0,1] al multplicar v*(n_max-1) lo reescala a [0,n_max-1), lo
% redondea al n más próximo y le suma uno para tomar un elemento de
% {1,...,n_max}. Este será el índice del centro más
% cercano a un nodo en esa dimensión (no distancia total, sino en esa
% dimensión). Eso se lo aplicamos coordenada a coordenada a los 300 peores
% nodos y nos dan dos columnas. Cada columna se corresponde al elemento de
% {1,...,n_max} en ese eje. Finalmente gpu_max(coordenad)  nos dice a qué
% coordenada de x (y respectivamente) de centro corresponde ese índice.
% unique por filas nos quita las filas repetidas. Así, cand_ctr nos da las
% coordenadas de los parches que estarán peor condicionados.
worst_rc = Inf; dist_rep = []; nrep = 0; %inicializamos el peor rcond en un
% número grande (conforme iteremos irá bajando), así como la matriz
% distancia asociada al parche y el número de nodos del parche.
for cc = 1:size(cand_ctr,1)
    [ic,~] = rangesearch(tree_mx, cand_ctr(cc,:), rho_max);%me guarda el
    % índice de los nodos que caen dentro del parche problemático cc-esimo.
    nod = ds_max(ic{1}, :);%nod guarda las coordenadas x,y de los nodos
    %dentro del parche cc-ésimo.
    if size(nod,1) < 2, continue; end %si solo hay un nodo en el parche
    % cc-ésimo, se ignora lo que viene ahora y y se vuelve a empezar el for
    % en el siguiente parche.
    Dloc = DistanceMatrix(nod, nod);%me calcula la matriz distancia en ese
    % parche.
    rcref = rcond(exp(-(ep_native*Dloc).^2));%me da el rcond de
    % Phi(matdistanc).
    if rcref < worst_rc, worst_rc = rcref; %si el nuevo condcionamiento es
      % peor que el anterior, guardamos este nuevo rcond (como el peor),
      % así como su matriz distancia y el número nodos en el parche.
      dist_rep = Dloc; nrep = size(nod,1);
    end
end
gr = @(ep) rcond(exp(-(ep*dist_rep).^2));
if gr(ep_native) >= rcond_target
    ep = ep_native;%si para el epsilon inicial, el sistema local peor
    %condicionado está mejor o igual condicionado que rcond=10^(-13), nos
    % quedamos ese epsilon. Entonces inicialmente la medida del intervalo
    % [ep_lo,ep_hi] es aproximadamente 10^7. si dividimos 80 veces entre 2,
    % esto es del orden de 10^(-24), que multiplicado por 10^7, nos deja
    % una ventana de medida 10^(-17), cuando la precisión en Matlab es
    % 10^(-16), luego es suficiente.
else
```

```
    ep_lo = ep_native; ep_hi = 2*ep_native;
    while gr(ep_hi) < rcond_target, ep_hi = 2*ep_hi; if ep_hi>1e7, break;
    end, end %me da un suelo, ep_lo (el epsilon que tiene rcond menor
  % 10^(-13) y un techo, ep_hi (este tiene rcond>=10^(-13).
    for it = 1:80
        em = 0.5*(ep_lo+ep_hi);
        if gr(em) >= rcond_target, ep_hi = em; else, ep_lo = em; end
    end%se realiza bisección de forma que 80 iteraciones es suficiente
        % pues, si en el caso extremo de que ep_hi=10^7, entonces,
        % inicialmente, la medida del intervalo [ep_lo,ep_hi] es
        % aproximadamente 10^7. si dividimos 80 veces entre 2,
    % esto es del orden de 10^(-24), que multiplicado por 10^7, nos deja
    % una ventana de medida 10^(-17), cuando la precisión en Matlab es
    % 10^(-16), luego es suficiente.
    ep = ep_hi;%ahora ya sí me quedo con ep_hi, como el ep más pequeño para
    % el que estará bien condicionado.
end
ep = max(ep, ep_native);  % seguro para que no sea menor que epsilon
% inicial.
fprintf(['eps_gauss = %.4f  (peor parche a N_max: %d nodos,' ...
  ' rcond=%.2e)\n\n'], ep, nrep, gr(ep));
clear ds_max tree_mx

% ========================================================================
% CONFIGURACIÓN DE BUCLES PARA EL TFG
% ========================================================================
% Convertimos lista_p a una celda (cell array) para poder mezclar números y
% texto
lista_p = {1, 3, 5, 'Gauss'};
% 3 pesos x 4 phi = 12 tablas.
lista_weights = {'WendlandC2', 'WendlandC4', 'WendlandC6'};
nP = numel(lista_p);
nW = numel(lista_weights);

% Una tabla de convergencia por combinacion: matriz nK x 6 con
% [N, M, h, RMS, MAX, min(RCOND)].
Res = cell(nP, nW);%celda 4x3
for a = 1:numel(Res), Res{a} = zeros(nK, 6); end %inicializamos cada tabla
% con 0 en filas y columnas.

warning('off', 'MATLAB:nearlySingularMatrix');
warning('off', 'MATLAB:singularMatrix'); %apagamos avisos.
fprintf('Iniciando simulaciones para el TFG...\n\n');

% 3.BARRIDO EN N: la geometria que solo depende de
% N se calcula una vez por N (no 12 veces). Dentro de cada parche, el
% interpolante local se calcula una vez por phi y se reutiliza para los
% pesos.
for ik = 1:nK
    N = (2^k_list(ik) + 1)^s;    % Número de datos (nodos de Halton)
```

```
189     n = max(round((N/N_j)^(1/s)), recinfo.n0);% centros por
190     % eje (total: n^s)
191     wep = n;
192     M = n^s;
193
194     % Nodos de datos usando la secuencia de Halton
195     dsites = haltonseq(N, 2);
196     rhs = func_test_franke(dsites(:,1), dsites(:,2)); %evalúa función
197     % Franke en
198     % nodos de Halton
199
200     % Centros de los parches
201     grid_pu = linspace(0, 1, n);
202     [xpu, ypu] = meshgrid(grid_pu);
203     cellctrs = [xpu(:), ypu(:)];
204     cellradius = 1/wep; % El radio del parche es alpha*H=1/n.
205     rho = cellradius; % usado en la normalizacion (r/rho)^p.
206
207     % distancia de llenado real h_{chi,Omega} (para las tablas y el orden)
208     h = distancia_llenado_knnsearch(dsites, epoints);
209
210     % --- 4. BÚSQUEDA ESPACIAL ---
211     % Se crean árboles para buscar rápidamente qué nodos caen dentro de
212     % cada parche.
213     % El árbol de nodos y DM_eval se construyen una vez
214     % por N (antes se rehacian en cada combinacion).
215     datatree = createns(dsites, 'NSMethod', 'kdtree');
216     DM_eval = DistanceMatrixCSRBF(epoints, cellctrs, wep); % matriz
217     % dispersa que se usa para construir los pesos
218
219     % Solo importan los parches que TOCAN algun punto de la malla; y de
220     % los nodos de datos de cada parche precalculamos sus índices una vez.
221     patchcols = find(any(DM_eval, 1));%revisa columma a columma si en cada
222     % columna (asociada a un parche), hay algún elemento distinto de cero,
223     % es decir, algún punto de evaluación en ese parche. patchcols guarda
224     % los índices (las columnas) que verifican lo anterior.
225     patch_idx = cell(numel(patchcols), 1); %crea array.
226     for jj = 1:numel(patchcols)
227         j = patchcols(jj);%hace referencia al centro j-esimo
228         [ic, ~] = rangesearch(datatree, cellctrs(j,:), cellradius);%nos da
229         % el índice de los nodos de Halton que están a distancia menor que
230         % cellradius del centro c_j.
231         patch_idx{jj} = ic{1}';%mete en el array los índices de los puntos
232         % de Halton dentro del parche j.
233         if isempty(patch_idx{jj}), DM_eval(:, j) = 0; end   % si el parche
234         % no tiene ptos de Halton, hacemos esa columna cero, para que no la
235         % consideremos luego en el MPU.
236     end
237
238     % --- 5. CONSTRUCCIÓN DE LAS MATRICES DE PESOS (SEM) SOBRE epoints ---
```

```
239     % Una matriz SEM por cada peso, calculada una vez por N (no depende
240     % de phi). SEM{iw}(i,j) = omega_j(epoints(i,:)) (peso de Shepard
241     % normalizado).
242     SEM = cell(1, nW);%es un array de una fila y número de pesos-columnas.
243     for iw = 1:nW
244         weight_type = lista_weights{iw};
245 
246         % --- DEFINICIÓN DE LAS FUNCIONES DE PESO ---
247         % La variable 'r' de estas funciones anónimas recibirá directamente
248         % la matriz dispersa que devuelve DistanceMatrixCSRBF.
249         % Es decir, 'r' es u = 1 - r/rho.
250         switch weight_type
251             case 'WendlandC2'
252                 % Fórmula: r.^4 .* (5*spones(r) - 4*r)
253                 wf = @(ep, r) r.^4 .* (5*spones(r) - 4*r);
254 
255             case 'WendlandC4'
256                 % Adaptación para C4
257                 wf = @(ep, r) r.^6 .* (35*r.^2 - 88*r + 56*spones(r));
258 
259             case 'WendlandC6'
260                 % Adaptación para C6
261                 wf = @(ep, r) r.^8 .* (-32*r.^3 + 121*r.^2 - 154*r ...
262                   + 66*spones(r));
263         end
264 
265         % Se evalúa pasando 'ep' y la matriz 'DM_eval'
266         SEM_unscaled = wf(ep, DM_eval);
267         S_inv = 1 ./ (SEM_unscaled * ones(M, 1));
268         SEM{iw} = spdiags(S_inv, 0, neval^2, neval^2) * SEM_unscaled;
269       %Matriz donde la entrada ij-ésima es (SEMiw)ij=omega_j(epoints(i,:)).
270     end
271 
272     % Inicializamos el vector interpolante global (uno por combinacion).
273     Pf = cell(nP, nW);%array de 4x3, un Pf por peso y phi.
274     for a = 1:numel(Pf), Pf{a} = zeros(neval^2, 1); end
275     min_rcond = Inf(1, nP);% vector fila de nP elementos que valen
276     % infinito.
277 
278     % --- 6. BUCLE PRINCIPAL DE LOS PARCHES LOCALES ---
279     %La geometria local (nodos, dist_j, puntos de eval) se calcula una
280     % vez por parche, el interpolante local, una vez por phi. Solo el
281     % ensamblaje final (con SEM{iw}) recorre los pesos.
282     for jj = 1:numel(patchcols)
283         j = patchcols(jj);%índice de parche con algún pto de evaluación.
284 
285         % Encontrar datos dentro de la celda j (precalculado)
286         idx = patch_idx{jj}; % índices de nodos de halton en parche j.
287         if isempty(idx), continue; end %si no hay nodos, ignorar lo de
288         % abajo.
```

```
289
290         % Nodos de datos e interpolación locales
291         local_dsites = dsites(idx, :);%coordenadas de los nodos del parche
292         % j.
293         N_j_loc = length(idx);
294
295         % Matriz A_j del método local
296         dist_j = DistanceMatrix(local_dsites, local_dsites);
297
298         % Encontrar puntos de evaluación que caen en el parche j (del
299         % patrón de la columna j de DM_eval. coincide con el soporte de
300         % SEM).
301         eidx = find(DM_eval(:, j));%da índices de columnas (parches) con
302         % algún elemento distinto de cero. Por lo que hicimos antes, estas
303         % son parches con algún nodo de Halton y algún punto de evaluación.
304         if isempty(eidx), continue; end %si es vacía ve a siguiente parche
305         % con el bucle for.
306         local_epoints = epoints(eidx, :);%ptos de evaluación en el parche
307         % j.
308
309         % Matriz B (Distancias entre puntos de evaluación y nodos de datos)
310         dist_eval = DistanceMatrix(local_epoints, local_dsites);%matriz
311         % distancia entre nodos y puntos de evaluación.
312
313         for ip = 1:nP
314             p = lista_p{ip};%itera sobre la lista de pesos.
315
316             % Implementación del caso gaussiano y Phi=r^p.
317             if isnumeric(p)%Distingue casos de p=1,3,5 del gaussiano.
318                 % Tomamos ahora Phi_rho=(r/rho)^p.
319                 A_j = (dist_j/rho).^p; % Phi(r) = (r/rho)^p
320                 % Matriz P_j definida para la nueva base local de
321                 % polinomios.
322                 P_j = build_poly_basis(local_dsites, p, ...
323                   cellctrs(j,:), rho);
324             else
325                 A_j = exp(-(ep * dist_j).^2); % Phi(r) = exp(-(ep*r)^2)
326                 % La Gaussiana no requiere base polinomial, matriz vacía
327                 P_j = build_poly_basis(local_dsites, 0, cellctrs(j,:), ...
328                   rho);
329             end
330
331             L = size(P_j, 2); % El número de columnas me da el número
332             % de vectores de la base, luego la dimensión del espacio de
333             % polinomios.
334
335             % ========================================================
336             % COMPROBACIONES TEÓRICAS
337             % --------------------------------------------------------
338             % Para garantizar que el sistema local sea no singular, el
```

```
            % conjunto de nodos debe ser Pi_{m-1}^s-unisolvente.
            % Solo aplica si hay polinomios (L > 0).
            if L > 0
                % Condición 1: Debe haber al menos tantos puntos como
                % elementos en la base polinomial.
                if N_j_loc < L
                    warning(['Parche %d: N_j (%d) < L (%d). ' ...
                      'Sistema infradeterminado para los polinomios.'], ...
                      j, N_j_loc, L);
                end
                % Condición 2: La matriz P_j debe tener rango completo.
                % Si rank(P_j) < L, significa que los puntos locales están
                % alineados sobre una curva algebraica de grado m-1 y
                % no son unisolventes. El sistema fallará.
                if rank(P_j) < L
                    warning(['Parche %d: Puntos locales NO ' ...
                      'unisolventes (rank(P_j)=%d < L=%d).'], j, ...
                      rank(P_j), L);
                end
            end
            % ============================================================

            % Ensamblaje de Matriz Local
            % IM = [A_j, P_j ; P_j^T, 0]
            IM = [A_j, P_j; P_j', zeros(L, L)];

            % Lado derecho (f_j ampliado con ceros para las
            % restricciones polinomiales).
            f_j = [rhs(idx); zeros(L, 1)];

            % Diagnóstico de condicionamiento del sistema local.
            rc = rcond(full(IM));%full IM la convierte de dispersa en
            % completa.
            if rc < min_rcond(ip), min_rcond(ip) = rc; end %para un p fijo,
            %si rc es menor que el anterior, elegimos este rc como el peor.

            % Resolución del sistema local: coefs=(lambda,c)'
            coefs = IM \ f_j;

            % --- Evaluación del interpolante local en los puntos de
            % evaluación del parche.
            % Evaluación usando Phi gaussiana o Phi=r^p
            if isnumeric(p)
                % Con la misma Phi normalizada que en A_j (coherencia).
                B = (dist_eval/rho).^p; % Phi(r) = (r/rho)^p.
                % Matriz Q (Base de polinomios evaluada en puntos de
                % evaluación)
                Q = build_poly_basis(local_epoints, p, cellctrs(j,:), rho);
            else
                B = exp(-(ep * dist_eval).^2);
```

```
                Q = build_poly_basis(local_epoints, 0, cellctrs(j,:), ...
                  rho); % Q vacía
            end

            % Ensamblaje de Matriz de Evaluación, es decir, la
            % matriz que multiplicada por coefs nos da el
            % aproximante local j, evaluado en cada uno de los
            % puntos de evaluación contenidos en el parche j.
            EM = [B, Q];

            % Interpolante local
            localfit = EM * coefs;

            % Acumulación en el interpolante global usando los
            % pesos de Shepard (SEM). El mismo localfit vale para los
            % nW tipos de pesos (no depende del peso); solo cambia SEM{iw}.
            for iw = 1:nW
                Pf{ip,iw}(eidx) = Pf{ip,iw}(eidx) + localfit .* ...
                  SEM{iw}(eidx, j);%calcula
                % Aproximante global sobre puntos evaluación, teniendo
                % en cuenta contribuciones de ptos que caen sobre
                % varios parches.
            end
        end
    end

    % --- 6. CÁLCULO DE ERRORES ---
    for ip = 1:nP
        for iw = 1:nW
            maxerr = norm(Pf{ip,iw} - exact, inf);
            rms_err = norm(Pf{ip,iw} - exact) / neval;%porque
            % sqrt{neval^s)=neval,
            % para s=2
            Res{ip,iw}(ik, :) = [N, M, h, rms_err, maxerr, min_rcond(ip)];
        end
    end
    fprintf('  k=%d  N=%d  n=%d  M=%d  h=%.3e  hecho\n', k_list(ik), N, ...
      n, M, h);
end

warning('on', 'MATLAB:nearlySingularMatrix');
warning('on', 'MATLAB:singularMatrix');

% ========================================================================
% IMPRESIÓN DE LAS 12 TABLAS. p_obs = log(RMS_ant/RMS)/log(h_ant/h).
% ========================================================================
for ip = 1:nP
    if isnumeric(lista_p{ip}), nombre_base = sprintf('r^%d', lista_p{ip});
    else, nombre_base = 'Gaussiana'; end
    for iw = 1:nW
```

```
        D = Res{ip, iw};
        pobs = NaN(nK,1);
        for r = 2:nK, pobs(r) = log2(D(r-1,4)/D(r,4)); end
        fprintf(['\n================================================' ...
          '==================\n']);
        fprintf('TABLA: Base %s | Peso: %s\n', nombre_base, ...
          lista_weights{iw});
        fprintf(['================================================' ...
          '================\n']);
        T = table(D(:,1), D(:,2), D(:,4), pobs, D(:,5), D(:,6), ...
            'VariableNames', {'N','M','RMS_error','orden_p', ...
            'max_error','min_RCOND'});
        disp(T);
    end
end

% ========================================================================
%SALIDA EN LATEX PARA OVERLEAF.
% Una tabla por phi, con los 3 pesos (C2,C4,C6) en bloques de (RMS,p,max).
% p_obs = log(RMS_ant/RMS)/log(h_ant/h). Copiar-pegar directamente.
% ========================================================================
fprintf('\n\n');
fprintf(['%%%%%%%%%%%%%%%%%  TABLAS EN LATEX (copiar a Overleaf) ' ...
  ' %%%%%%%%%%%%%%%%%\n\n']);
pesos_tex = {'C^2', 'C^4', 'C^6'};
for ip = 1:nP
    if isnumeric(lista_p{ip})
        base_lbl = sprintf('r%d', lista_p{ip});
        cap_phi  = sprintf('$\\Phi=r^{%d}$', lista_p{ip});
    else
        base_lbl = 'gauss';
        cap_phi  = sprintf(['la gaussiana no estacionaria ($\\' ...
          'varepsilon=%.4f$)'], ep);
    end

    % Orden observado por peso (una columna p por bloque)
    Pobs = cell(1, nW);
    for iw = 1:nW
        D = Res{ip, iw};
        pv = NaN(nK, 1);
        for r = 2:nK, pv(r) = log2(D(r-1,4)/D(r,4)); end
        Pobs{iw} = pv;
    end

    fprintf('\\begin{table}[H]\n');
    fprintf('  \\centering\n');
    fprintf('  \\scalebox{0.68}{\n');
    fprintf('  \\begin{tabular}{| c | c | cccc | cccc | cccc |}\n');
    fprintf('  \\hline\n');
    fprintf(['  & & \\multicolumn{4}{c|}{Wendland $%s$}' ...
```

```
            ' & \\multicolumn{4}{c|}{Wendland $%s$}' ...
            ' & \\multicolumn{4}{c|}{Wendland $%s$} \\\\\n'], ...
            pesos_tex{1}, pesos_tex{2}, pesos_tex{3});
    fprintf(['  $k$ & $N$ & RMS-error & $p$ & max-error & RCOND' ...
            ' & RMS-error & $p$ & max-error & RCOND' ...
            ' & RMS-error & $p$ & max-error & RCOND \\\\\n']);
    fprintf('  \\hline\n');
    for ik = 1:nK
        fprintf('  %d & %d', k_list(ik), Res{ip,1}(ik,1));
        for iw = 1:nW
            D = Res{ip, iw};
            if ik == 1
                fprintf(' & %.4e & & %.4e & %.2e', D(ik,4), D(ik,5), ...
                  D(ik,6));
            else
                fprintf(' & %.4e & %.4f & %.4e & %.2e', D(ik,4), ...
                  Pobs{iw}(ik), D(ik,5), D(ik,6));
            end
        end
        fprintf(' \\\\\n');
    end
    fprintf('  \\hline\n');
    fprintf('  \\end{tabular}}\n');
    fprintf(['  \\caption{RMSE y error m\\''aximo frente al ' ...
      'n\\''umero de nodos de ' ...
            'Halton $N$ para el MPU con %s y pesos de Wendland $C^2$,' ...
            ' $C^4$ y $C^6$, ' ...
            'sobre la funci\\''on de Franke.}\n'], cap_phi);
    fprintf('  \\label{tab:mpu_franke_%s}\n', base_lbl);
    fprintf('\\end{table}\n\n');
end

% ========================================================================
% RBF GLOBAL (comparativa): Gaussiano con eps fijo y r^p, p=1,3,5.
% Siguiendo el codigo 10 del anexo: interpolacion RBF global directa, sin
% aumento polinomico (para r^p se resuelve DM.^p \ rhs, como la matriz
% distancia con r^1). Barrido propio hasta N_max asequible (sistema denso).
% El eps de la gaussiana se fija como el menor con rcond >= rcond_target en
% N_max (rcond crece con eps y N_max es el peor caso: asi rcond > umbral en
% todo el barrido). p_obs = log2(RMS_ant/RMS).
% ========================================================================
k_rbf     = 1:7;          % N=(2^k+1)^2 => N=9..16641 (N_max asequible)
nK_rbf    = numel(k_rbf);
lista_rbf = {'Gauss', 1, 3, 5};
nP_rbf    = numel(lista_rbf);

% eps FIJO de la gaussiana: menor eps con rcond >= rcond_target en N_max.
N_mx_rbf  = (2^max(k_rbf) + 1)^s;
nd_mx_rbf = haltonseq(N_mx_rbf, s);
DM_mx_rbf = DistanceMatrix(nd_mx_rbf, nd_mx_rbf);
```

```
grc = @(e) rcond(exp(-(e*DM_mx_rbf).^2));
e_lo = ep_native; e_hi = 2*ep_native;
while grc(e_hi) < rcond_target, e_hi = 2*e_hi; if e_hi > 1e4, break;
end, end
for it = 1:50
    em = 0.5*(e_lo + e_hi);
    if grc(em) >= rcond_target, e_hi = em; else, e_lo = em; end
end
eps_fijo = e_hi;
fprintf(['\neps_fijo (RBF global gaussiano) = %.4f  (rcond en' ...
  ' N_max=%d: %.2e)\n'], ...
  eps_fijo, N_mx_rbf, grc(eps_fijo));

% Calculo y almacenamiento: Res_rbf{ip} = [k, N, RMS, MAX, RCOND]
Res_rbf = cell(1, nP_rbf);
for ip = 1:nP_rbf
    phi_rbf = lista_rbf{ip};
    Drbf = zeros(nK_rbf, 5);
    for ik = 1:nK_rbf
        N = (2^k_rbf(ik) + 1)^s;
        nodes = haltonseq(N, s);
        rhs_rbf = func_test_franke(nodes(:,1), nodes(:,2));
        DM_rbf  = DistanceMatrix(nodes, nodes);
        DM_ev   = DistanceMatrix(epoints, nodes);
        if isnumeric(phi_rbf)
            IM_rbf = DM_rbf.^phi_rbf;
            Pf_rbf = (DM_ev.^phi_rbf) * (IM_rbf \ rhs_rbf);
        else
            IM_rbf = exp(-(eps_fijo*DM_rbf).^2);
            Pf_rbf = exp(-(eps_fijo*DM_ev).^2) * (IM_rbf \ rhs_rbf);
        end
        rms_rbf = norm(Pf_rbf - exact) / neval;
        max_rbf = norm(Pf_rbf - exact, inf);
        Drbf(ik, :) = [k_rbf(ik), N, rms_rbf, max_rbf, rcond(IM_rbf)];
    end
    Res_rbf{ip} = Drbf;
end

% Tablas de texto (consola)
for ip = 1:nP_rbf
    if isnumeric(lista_rbf{ip}), nombre_rbf = sprintf('r^%d', ...
        lista_rbf{ip});
    else, nombre_rbf = 'Gaussiana'; end
    D = Res_rbf{ip};
    pobs = NaN(nK_rbf, 1);
    for rr = 2:nK_rbf, pobs(rr) = log2(D(rr-1,3)/D(rr,3)); end
    fprintf(['\n================================================' ...
      '=================\n']);
    fprintf('TABLA: RBF GLOBAL | Base %s\n', nombre_rbf);
    fprintf(['================================================' ...
```

```
      '=================\n']);
    T = table(D(:,1), D(:,2), D(:,3), pobs, D(:,4), D(:,5), ...
        'VariableNames', {'k','N','RMS_error','orden_p','max_error' ...
        ,'RCOND'});
    disp(T);
end

% Tablas en LaTeX (copiar a Overleaf), una por Phi, con columna RCOND
fprintf('\n\n');
fprintf(['%%%%%%%%%%%%%%%%%  TABLAS RBF GLOBAL EN LATEX (copiar' ...
  ' a Overleaf)  %%%%%%%%%%%%%%%%%\n\n']);
for ip = 1:nP_rbf
    if isnumeric(lista_rbf{ip})
        base_lbl = sprintf('r%d', lista_rbf{ip});
        cap_phi  = sprintf('$\\Phi=r^{%d}$', lista_rbf{ip});
    else
        base_lbl = 'gauss';
        cap_phi  = sprintf('la gaussiana ($\\varepsilon=%.4f$)', eps_fijo);
    end
    D = Res_rbf{ip};
    pv = NaN(nK_rbf, 1);
    for r = 2:nK_rbf, pv(r) = log2(D(r-1,3)/D(r,3)); end
    fprintf('\\begin{table}[H]\n');
    fprintf('  \\centering\n');
    fprintf('  \\scalebox{0.9}{\n');
    fprintf('  \\begin{tabular}{| c | c | c c c c |}\n');
    fprintf('  \\hline\n');
    fprintf('  $k$ & $N$ & RMS-error & $p$ & max-error & RCOND \\\\\n');
    fprintf('  \\hline\n');
    for ik = 1:nK_rbf
        if ik == 1
            fprintf('  %d & %d & %.4e & & %.4e & %.2e \\\\\n', ...
              D(ik,1), D(ik,2), D(ik,3), D(ik,4), D(ik,5));
        else
            fprintf('  %d & %d & %.4e & %.4f & %.4e & %.2e \\\\\n', ...
              D(ik,1), D(ik,2), D(ik,3), pv(ik), D(ik,4), D(ik,5));
        end
    end
    fprintf('  \\hline\n');
    fprintf('  \\end{tabular}}\n');
    fprintf(['  \\caption{RMSE y error m\\''aximo frente al n' ...
      '\\''umero de nodos de ' ...
             'Halton $N$ para el RBF global con %s, sobre la func' ...
             'i\\''on de Franke.}\n'], cap_phi);
    fprintf('  \\label{tab:rbf_franke_%s}\n', base_lbl);
    fprintf('\\end{table}\n\n');
end

% ========================================================================
% ESTUDIO DE TIEMPOS (tic/toc): MPU (Wendland C6) vs RBF global, por Phi.
```

```
% Se cronometra un solve completo por (N, Phi) [geometria + sistemas +
% evaluacion], que es lo que marca el coste real. Se toma la mediana de
% varias repeticiones. El numero de repeticiones no se fija por N (los
% tiempos de MPU y RBF no son comparables entre sí): cada caso se repite
% hasta acumular un presupuesto t_budget o alcanzar max_rep, de modo que un
%solve rápido (MPU) se promedia sobre muchas repeticiones y uno lento (RBF)
%sobre pocas (a menudo 1), automaticamente. La primera muestra de cada caso
%es de calentamiento(warm-up) y se descarta.
%Nota: dentro de cada metodo los tiempos de las 4 Phi salen casi iguales(el
% MPU domina la geometria, común a las 4; el RBF domina el solve denso NxN,
%común). Las 4 columnas sirven de comprobacion de esa independencia de Phi.
%Además, hay más filas de MPU que de RBF: el MPU barre todo k_list mientras
% que el RBF solo se cronometra hasta N_rbf_cap (sistema denso asequible).
% ========================================================================
k_tim     = k_list;                  % mismo barrido en k que el MPU
nK_tim    = numel(k_tim);
Nvec_tim  = (2.^k_tim + 1).^s;       % N por fila
N_rbf_cap = 16641;                   % RBF solo hasta N asequible(denso NxN)
t_budget  = 0.5;                    % presupuesto de tiempo por caso [s]
max_rep   = 15;               % tope de repeticiones (N pequeños y rápidos)
Tmpu = NaN(nK_tim, nP);           % tiempos MPU C6 [s], una columna por Phi
Trbf = NaN(nK_tim, nP);       % tiempos RBF global [s], una columna por Phi
fprintf('\nCronometrando MPU (C6) y RBF global...\n');
for ik = 1:nK_tim
    N = Nvec_tim(ik);
    for ip = 1:nP
        p = lista_p{ip};
        % ---------------- MPU (Wendland C6), Phi = p ----------------
        smp = zeros(1, max_rep); ns = 0; tacc = 0; warm = true;
        while true %bucle se repite infinitamente hasta un break que hay
          % más abajo.
            tt = tic;
            n = max(round((N/N_j)^(1/s)), recinfo.n0); M = n^s; rho = 1/n;
            dsites = haltonseq(N, s);
            rhs = func_test_franke(dsites(:,1), dsites(:,2));
            grid_pu = linspace(0, 1, n); [xpu, ypu] = meshgrid(grid_pu);
            cellctrs = [xpu(:), ypu(:)];
            datatree = createns(dsites, 'NSMethod', 'kdtree');
            DM_eval = DistanceMatrixCSRBF(epoints, cellctrs, n);
            SEM_u = DM_eval.^8 .* (-32*DM_eval.^3 + 121*DM_eval.^2 ...
              - 154*DM_eval + 66*spones(DM_eval));
            S_inv = 1 ./ (SEM_u * ones(M, 1));
            SEM = spdiags(S_inv, 0, neval^2, neval^2) * SEM_u;
            Pf = zeros(neval^2, 1);
            patchcols = find(any(DM_eval, 1));
            for jj = 1:numel(patchcols)
                j = patchcols(jj);
                ic = rangesearch(datatree, cellctrs(j,:), rho);
                idx = ic{1}'; if isempty(idx), continue; end
                local_dsites = dsites(idx, :);
```

```
            dist_j = DistanceMatrix(local_dsites, local_dsites);
            eidx = find(DM_eval(:, j)); if isempty(eidx), continue; end
            local_epoints = epoints(eidx, :);
            dist_eval = DistanceMatrix(local_epoints, local_dsites);
            if isnumeric(p)
                A_j = (dist_j/rho).^p;
                P_j = build_poly_basis(local_dsites, p, ...
                  cellctrs(j,:), rho);
                B = (dist_eval/rho).^p;
                Q = build_poly_basis(local_epoints, p, ...
                  cellctrs(j,:), rho);
            else
                A_j = exp(-(ep*dist_j).^2);
                P_j = build_poly_basis(local_dsites, 0, ...
                  cellctrs(j,:), rho);
                B = exp(-(ep*dist_eval).^2);
                Q = build_poly_basis(local_epoints, 0, ...
                  cellctrs(j,:), rho);
            end
            L = size(P_j, 2);
            coefs = [A_j, P_j; P_j', zeros(L, L)] \ [rhs(idx);
              zeros(L, 1)];
            localfit = [B, Q] * coefs;
            Pf(eidx) = Pf(eidx) + localfit .* SEM(eidx, j);
        end
        e = toc(tt);
        if warm
            warm = false;             % descarta la 1a (calentamiento)
        else
            ns = ns + 1; smp(ns) = e; tacc = tacc + e;
            if tacc >= t_budget || ns >= max_rep, break; end
        end
    end
    Tmpu(ik, ip) = median(smp(1:ns));
    % ---------------- RBF global, Phi = p ----------------
    if N <= N_rbf_cap
        smp = zeros(1, max_rep); ns = 0; tacc = 0; warm = true;
        while true
            tt = tic;
            nodes = haltonseq(N, s);
            rhs_r = func_test_franke(nodes(:,1), nodes(:,2));
            DMd = DistanceMatrix(nodes, nodes);
            DMe = DistanceMatrix(epoints, nodes);
            if isnumeric(p)
                Pfr = (DMe.^p) * (DMd.^p \ rhs_r);
            else
                Pfr = exp(-(eps_fijo*DMe).^2) * (exp(-(eps_fijo* ...
                  DMd).^2) \ rhs_r);
            end
            e = toc(tt);
```

```
                if warm
                    warm = false;
                else
                    ns = ns + 1; smp(ns) = e; tacc = tacc + e;
                    if tacc >= t_budget || ns >= max_rep, break; end
                end
            end
            Trbf(ik, ip) = median(smp(1:ns));
        end
    end
end

% --- cocientes normalizados: MPU por N*log(N); RBF por N^3 ---
Rmpu = NaN(nK_tim, nP); Rrbf = NaN(nK_tim, nP);
for ik = 2:nK_tim
    fN  = (Nvec_tim(ik)*log(Nvec_tim(ik))) / (Nvec_tim(ik-1)*log ...
      (Nvec_tim(ik-1)));
    fN3 = Nvec_tim(ik)^3 / Nvec_tim(ik-1)^3;
    for ip = 1:nP
        Rmpu(ik,ip) = (Tmpu(ik,ip)/Tmpu(ik-1,ip)) / fN;
        Rrbf(ik,ip) = (Trbf(ik,ip)/Trbf(ik-1,ip)) / fN3;
    end
end

% --- TABLA 1 (LaTeX): tiempos [s] ---
fprintf('\n\n');
fprintf(['%%%%%%%%%%%%%%%%%  TABLAS DE TIEMPOS EN LATEX (copiar a' ...
  ' Overleaf)  %%%%%%%%%%%%%%%%%\n\n']);
fprintf('\\begin{table}[H]\n  \\centering\n  \\scalebox{0.8}{\n');
fprintf('  \\begin{tabular}{| c | c | cccc | cccc |}\n  \\hline\n');
fprintf(['  & & \\multicolumn{4}{c|}{MPU ($C^6$) [s]}' ...
         ' & \\multicolumn{4}{c|}{RBF global [s]} \\\\\n']);
fprintf(['  $k$ & $N$ & $r^1$ & $r^3$ & $r^5$ & Gauss' ...
         ' & $r^1$ & $r^3$ & $r^5$ & Gauss \\\\\n']);
fprintf('  \\hline\n');
for ik = 1:nK_tim
    rowstr = sprintf('  %d & %d', k_tim(ik), Nvec_tim(ik));
    for ip = 1:nP, rowstr = [rowstr, sprintf(' & %.3e', Tmpu(ik,ip))]; end
    for ip = 1:nP
        if isnan(Trbf(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.3e', Trbf(ik,ip))]; end
    end
    fprintf('%s \\\\\n', rowstr);
end
fprintf('  \\hline\n  \\end{tabular}}\n');
fprintf(['  \\caption{Tiempos de computo (tic/toc) del MPU (Wendland' ...
  ' $C^6$) y' ...
         ' del RBF global, por $\\Phi$, sobre la funcion de Franke.}\n']);
fprintf('  \\label{tab:tiempos}\n\\end{table}\n\n');

```

```
% --- TABLA 2 (LaTeX): cocientes normalizados (deberian tender a ~1) ---
fprintf('\\begin{table}[H]\n  \\centering\n  \\scalebox{0.8}{\n');
fprintf('  \\begin{tabular}{| c | c | cccc | cccc |}\n  \\hline\n');
fprintf(['  & & \\multicolumn{4}{c|}{MPU: $(t_{k}/t_{k-1})/' ...
  '(N\\log N)$-ratio}' ...
         ' & \\multicolumn{4}{c|}{RBF: $(t_{k}/t_{k-1})/N^3$-ratio}' ...
         ' \\\\\n']);
fprintf(['  $k$ & $N$ & $r^1$ & $r^3$ & $r^5$ & Gauss' ...
         ' & $r^1$ & $r^3$ & $r^5$ & Gauss \\\\\n']);
fprintf('  \\hline\n');
for ik = 1:nK_tim
    rowstr = sprintf('  %d & %d', k_tim(ik), Nvec_tim(ik));
    for ip = 1:nP
        if isnan(Rmpu(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.4f', Rmpu(ik,ip))]; end
    end
    for ip = 1:nP
        if isnan(Rrbf(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.4f', Rrbf(ik,ip))]; end
    end
    fprintf('%s \\\\\n', rowstr);
end
fprintf('  \\hline\n  \\end{tabular}}\n');
fprintf(['  \\caption{Cocientes de tiempos normalizados: MPU por' ...
  ' $N\\log N$ y' ...
         ' RBF global por $N^3$. Valores $\\approx 1$ confirman el' ...
         ' orden esperado.}\n']);
fprintf('  \\label{tab:cocientes}\n\\end{table}\n\n');
```

Código 15: Script del experimento numérico de la función de Franke para RBF global y MPU.

```
% ========================================================================
% SCRIPT: MPU para phi=r^p, con p = 1, 3, 5).
% Adaptado para diferentes funciones de peso (Wendland).
% ========================================================================
%   - Barrido en N=(2^k+1)^s con N_j fijo y M=M(N)
%   - eps de la gaussiana automatico (peor parche del N mayor,
% condicion 4).
%   - phi_rho= (r/rho)^p (Lema 1.60) .
%   - base polinomica reescalada en cada parche.
%   - una tabla de convergencia por combinacion (con h y min(RCOND)).
%
% Toda la geometría que solo depende de N (árbol de
% nodos, DM_eval, índices de nodos por parche y las matrices de peso SEM)
% se calculan una sola vez por N, fuera del bucle de las combinaciones.
% Además, para hacerlo lo más eficiente posible, el interpolante local se
% calcula una vez por Phi y se reutiliza para los pesos
% (no depende del peso).
% ========================================================================
```

```
19
20  clear; clc;
21
22  % --- 1. CONFIGURACIÓN DE PARÁMETROS ---
23  % n y wep se deducen de N y N_j.
24  % 'ep'(forma gaussiana) se calcula automaticamente (seccion 2c).
25  neval = 100;                 % Puntos de evaluación por eje (total:neval^2)
26  rcond_target = 1e-13;       % umbral de condicionamiento (RCOND)
27
28  % Definimos la función test: un polinomio.
29  testfunction = @(x,y) 2 - x + 3*y + x.^2 - 2*x.*y + 4*y.^2;
30
31  % --- 2. GENERACIÓN DE PUNTOS ---
32  % 2a. Puntos de evaluación (Malla uniforme regular)
33  % neval fijo => epoints y exact no dependen de N: una sola vez.
34  grid_eval = linspace(0, 1, neval);
35  [xe, ye] = meshgrid(grid_eval);
36  epoints = [xe(:), ye(:)]; %matriz neval^2x2 donde cada fila es un punto de
37  % evaluación.
38  exact = testfunction(epoints(:,1), epoints(:,2)); % Solución exacta:
39  % función test evaluada en ptos de evaluación.
40
41  % --- 2b. ELECCIÓN DE N_j Y RANGO DEL BARRIDO ---
42  % N_j y k_min salen de la funcion recubrimiento_Nj, que
43  % aplica las condiciones del recubrimiento (solo dependen de s, del
44  % mayor p usado y de la constante c=1.75).
45  s = 2;     % dimensión.
46  [k_min, N_j, recinfo] = recubrimiento_Nj(s, 5, 1.75); % p_max = 5, c = 1.75
47  % y obtenemos el kmin, que nos dará el Nmin. También nos da el Nj fijo que
48  % buscamos y recinfo acumula n0, L, Nj2, Nj3, c.
49  fprintf(['N_j (recubrimiento) = %d   (n0=%d, L=%d, Nj2=%.2f, Nj3=%.2f,' ...
50    ' c=%.2f)\n'], N_j, recinfo.n0, recinfo.L, recinfo.Nj2, recinfo.Nj3, ...
51    recinfo.c);%nos da el valor de N
52  k_max  = 10;   % N hasta (2^10+1)^s
53  k_list = k_min:k_max;
54  nK = numel(k_list); %longitud de k_list
55  fprintf('Barrido: k=%d..%d  ->  N=%d..%d\n', k_min, k_max, ...
56    (2^k_min+1)^s, (2^k_max+1)^s);
57
58  % --- 2c. SELECCIÓN AUTOMÁTICA DE eps_gauss (peor parche real para N_max,
59  % la N máxima)
60  % El condicionamiento gaussiano lo domina el par de nodos más cercano
61  % dentro de un parche. Localizamos los nodos más apretados para N_max,
62  % miramos los parches que los contienen, tomamos el de menor rcond y
63  % ajustamos eps por biseccion para que rcond(peor) >= rcond_target.
64  % rcond crece con eps. Para N menores el condicionamiento solo mejora.
65  ep_native = 9/sqrt(2);  % umbral de epsilon
66  N_max   = (2^k_max + 1)^s;
67  n_max = round((N_max/N_j)^(1/s));
68  if n_max < recinfo.n0 %controla que el N_max sea suficientemente grande
```

```
  % para cumplir las cotas que exige el recubrimiento.
    error('MPU:sinCobertura', ...
        ['Ni N_max=%d alcanza cobertura: n_max=%d < n0=%d ' ...
         '(alpha=(n-1)/n <= sqrt(s)/2). Sube k_max o baja N_j.'], ...
        N_max, n_max, recinfo.n0);
end
rho_max = 1/n_max;
ds_max  = haltonseq(N_max, s);%nodos de Halton para N_max
tree_mx = createns(ds_max, 'NSMethod', 'kdtree');
gpu_mx  = linspace(0, 1, n_max);
[~, Dnn] = knnsearch(ds_max, ds_max, 'K', 2);%para no calcular todas las
% distancias busca los dos vecinos (nodos de Halton) más cercanos a cada
% nodo de Halton y Dnn guarda en la primera columna las distancias más
% cercanas (0, pues es un nodo con él mismo) y en la segunda columna la
% distancia más pequeño con el nodo más próximo (cuál es ese nodo que está
% cerca, es decir, su índice, no se guarda). Cada fila de Dnn da los dos
% nodos más próximos a un nodo (siguiendo un orden).
[~, ord] = sort(Dnn(:,2), 'ascend');%se queda con 2 columna de Dnn, ordena
% de menor a mayor las distancias y se con el índice de los nodos que
% ordenados de menor a mayor cercanía a otro nodo.
cand_nod = ds_max(ord(1:min(300,end)), :);%nos quedamos con los 300 nodos
% con distancias menores (guardamos sus coordenadas en matriz N_max x 2).
gi       = @(v) min(max(round(v*(n_max-1)) + 1, 1), n_max);
cand_ctr = unique([gpu_mx(gi(cand_nod(:,1))).', ...
  gpu_mx(gi(cand_nod(:,2))).'], 'rows');%la función gi lo que hace es el
% intervalo [0,1] al multplicar v*(n_max-1) lo reescala a [0,n_max-1), lo
% redondea al n más próximo y le suma uno para tomar un elemento de
% {1,...,n_max}. Este será el índice del centro más
% cercano a un nodo en esa dimensión (no distancia total, sino en esa
% dimensión). Eso se lo aplicamos coordenada a coordenada a los 300 peores
% nodos y nos dan dos columnas. Cada columna se corresponde al elemento de
% {1,...,n_max} en ese eje. Finalmente gpu_max(coordenad)  nos dice a qué
% coordenada de x (y respectivamente) de centro corresponde ese índice.
% unique por filas nos quita las filas repetidas. Así, cand_ctr nos da las
% coordenadas de los parches que estarán peor condicionados.
worst_rc = Inf; dist_rep = []; nrep = 0; %inicializamos el peor rcond en un
% número grande (conforme iteremos irá bajando), así como la matriz
% distancia asociada al parche y el número de nodos del parche.
for cc = 1:size(cand_ctr,1)
    [ic,~] = rangesearch(tree_mx, cand_ctr(cc,:), rho_max);%me guarda el
    % índice de los nodos que caen dentro del parche problemático cc-esimo.
    nod = ds_max(ic{1}, :);%nod guarda las coordenadas x,y de los nodos
    %dentro del parche cc-ésimo.
    if size(nod,1) < 2, continue; end %si solo hay un nodo en el parche
    % cc-ésimo, se ignora lo que viene ahora y y se vuelve a empezar el for
    % en el siguiente parche.
    Dloc = DistanceMatrix(nod, nod);%me calcula la matriz distancia en ese
    % parche.
    rcref = rcond(exp(-(ep_native*Dloc).^2));%me da el rcond de
    % Phi(matdistanc).
```

```
    if rcref < worst_rc, worst_rc = rcref; %si el nuevo condcionamiento es
      % peor que el anterior, guardamos este nuevo rcond (como el peor),
      % así como su matriz distancia y el número nodos en el parche.
      dist_rep = Dloc; nrep = size(nod,1);
    end
end
gr = @(ep) rcond(exp(-(ep*dist_rep).^2));
if gr(ep_native) >= rcond_target
    ep = ep_native;%si para el epsilon inicial, el sistema local peor
    %condicionado está mejor o igual condicionado que rcond=10^(-13), nos
    % quedamos ese epsilon. Entonces inicialmente la medida del intervalo
    % [ep_lo,ep_hi] es aproximadamente 10^7. si dividimos 80 veces entre 2,
    % esto es del orden de 10^(-24), que multiplicado por 10^7, nos deja
    % una ventana de medida 10^(-17), cuando la precisión en Matlab es
    % 10^(-16), luego es suficiente.
else
    ep_lo = ep_native; ep_hi = 2*ep_native;
    while gr(ep_hi) < rcond_target, ep_hi = 2*ep_hi; if ep_hi>1e7, break;
    end, end %me da un suelo, ep_lo (el epsilon que tiene rcond menor
  % 10^(-13) y un techo, ep_hi (este tiene rcond>=10^(-13).
    for it = 1:80
        em = 0.5*(ep_lo+ep_hi);
        if gr(em) >= rcond_target, ep_hi = em; else, ep_lo = em; end
    end%se realiza bisección de forma que 80 iteraciones es suficiente
        % pues, si en el caso extremo de que ep_hi=10^7, entonces,
        % inicialmente, la medida del intervalo [ep_lo,ep_hi] es
        % aproximadamente 10^7. si dividimos 80 veces entre 2,
    % esto es del orden de 10^(-24), que multiplicado por 10^7, nos deja
    % una ventana de medida 10^(-17), cuando la precisión en Matlab es
    % 10^(-16), luego es suficiente.
    ep = ep_hi;%ahora ya sí me quedo con ep_hi, como el ep más pequeño para
    % el que estará bien condicionado.
end
ep = max(ep, ep_native);  % seguro para que no sea menor que epsilon
% inicial.
fprintf(['eps_gauss = %.4f  (peor parche a N_max: %d nodos,' ...
  ' rcond=%.2e)\n\n'], ep, nrep, gr(ep));
clear ds_max tree_mx

% ========================================================================
% CONFIGURACIÓN DE BUCLES PARA EL TFG
% ========================================================================
% Convertimos lista_p a una celda (cell array) para poder mezclar números y
% texto
lista_p = {1, 3, 5, 'Gauss'};
% 3 pesos x 4 phi = 12 tablas.
lista_weights = {'WendlandC2', 'WendlandC4', 'WendlandC6'};
nP = numel(lista_p);
nW = numel(lista_weights);

```

```
% Una tabla de convergencia por combinacion: matriz nK x 6 con
% [N, M, h, RMS, MAX, min(RCOND)].
Res = cell(nP, nW);%celda 4x3
for a = 1:numel(Res), Res{a} = zeros(nK, 6); end %inicializamos cada tabla
% con 0 en filas y columnas.

warning('off', 'MATLAB:nearlySingularMatrix');
warning('off', 'MATLAB:singularMatrix'); %apagamos avisos.
fprintf('Iniciando simulaciones para el TFG...\n\n');

% 3.BARRIDO EN N: la geometria que solo depende de
% N se calcula una vez por N (no 12 veces). Dentro de cada parche, el
% interpolante local se calcula una vez por phi y se reutiliza para los
% pesos.
for ik = 1:nK
    N = (2^k_list(ik) + 1)^s;    % Número de datos (nodos de Halton)
    n = max(round((N/N_j)^(1/s)), recinfo.n0);% centros por
    % eje (total: n^s)
    wep = n;
    M = n^s;

    % Nodos de datos usando la secuencia de Halton
    dsites = haltonseq(N, 2);
    rhs = testfunction(dsites(:,1), dsites(:,2)); %evalúa función test en
    % nodos de Halton

    % Centros de los parches
    grid_pu = linspace(0, 1, n);
    [xpu, ypu] = meshgrid(grid_pu);
    cellctrs = [xpu(:), ypu(:)];
    cellradius = 1/wep; % El radio del parche es alpha*H=1/n.
    rho = cellradius; % usado en la normalizacion (r/rho)^p.

    % distancia de llenado real h_{chi,Omega} (para las tablas y el orden)
    h = distancia_llenado_knnsearch(dsites, epoints);

    % --- 4. BÚSQUEDA ESPACIAL ---
    % Se crean árboles para buscar rápidamente qué nodos caen dentro de
    % cada parche.
    % El árbol de nodos y DM_eval se construyen una vez
    % por N (antes se rehacian en cada combinacion).
    datatree = createns(dsites, 'NSMethod', 'kdtree');
    DM_eval = DistanceMatrixCSRBF(epoints, cellctrs, wep); % matriz
    % dispersa que se usa para construir los pesos

    % Solo importan los parches que TOCAN algun punto de la malla; y de
    % los nodos de datos de cada parche precalculamos sus índices una vez.
    patchcols = find(any(DM_eval, 1));%revisa columna a columna si en cada
    % columna (asociada a un parche), hay algún elemento distinto de cero,
    % es decir, algún punto de evaluación en ese parche. patchcols guarda
```

```
    % los índices (las columnas) que verifican lo anterior.
    patch_idx = cell(numel(patchcols), 1); %crea array.
    for jj = 1:numel(patchcols)
        j = patchcols(jj);%hace referencia al centro j-esimo
        [ic, ~] = rangesearch(datatree, cellctrs(j,:), cellradius);%nos da
        % el índice de los nodos de Halton que están a distancia menor que
        % cellradius del centro c_j.
        patch_idx{jj} = ic{1}';%mete en el array los índices de los puntos
        % de Halton dentro del parche j.
        if isempty(patch_idx{jj}), DM_eval(:, j) = 0; end   % si el parche
        % no tiene ptos de Halton, hacemos esa columna cero, para que no la
        % consideremos luego en el MPU.
    end

    % --- 5. CONSTRUCCIÓN DE LAS MATRICES DE PESOS (SEM) SOBRE epoints ---
    % Una matriz SEM por cada peso, calculada una vez por N (no depende
    % de phi). SEM{iw}(i,j) = omega_j(epoints(i,:)) (peso de Shepard
    % normalizado).
    SEM = cell(1, nW);%es un array de una fila y número de pesos-columnas.
    for iw = 1:nW
        weight_type = lista_weights{iw};

        % --- DEFINICIÓN DE LAS FUNCIONES DE PESO ---
        % La variable 'r' de estas funciones anónimas recibirá directamente
        % la matriz dispersa que devuelve DistanceMatrixCSRBF.
        % Es decir, 'r' es u = 1 - r/rho.
        switch weight_type
            case 'WendlandC2'
                % Fórmula: r.^4 .* (5*spones(r) - 4*r)
                wf = @(ep, r) r.^4 .* (5*spones(r) - 4*r);

            case 'WendlandC4'
                % Adaptación para C4
                wf = @(ep, r) r.^6 .* (35*r.^2 - 88*r + 56*spones(r));

            case 'WendlandC6'
                % Adaptación para C6
                wf = @(ep, r) r.^8 .* (-32*r.^3 + 121*r.^2 - 154*r ...
                  + 66*spones(r));
        end

        % Se evalúa pasando 'ep' y la matriz 'DM_eval'
        SEM_unscaled = wf(ep, DM_eval);
        S_inv = 1 ./ (SEM_unscaled * ones(M, 1));
        SEM{iw} = spdiags(S_inv, 0, neval^2, neval^2) * SEM_unscaled;
      %Matriz donde la entrada ij-ésima es (SEMiw)ij=omega_j(epoints(i,:)).
    end

    % Inicializamos el vector interpolante global (uno por combinacion).
    Pf = cell(nP, nW);%array de 4x3, un Pf por peso y phi.
```

```
    for a = 1:numel(Pf), Pf{a} = zeros(neval^2, 1); end
    min_rcond = Inf(1, nP);% vector fila de nP elementos que valen
    % infinito.

    % --- 6. BUCLE PRINCIPAL DE LOS PARCHES LOCALES ---
    %La geometria local (nodos, dist_j, puntos de eval) se calcula una
    % vez por parche, el interpolante local, una vez por phi. Solo el
    % ensamblaje final (con SEM{iw}) recorre los pesos.
    for jj = 1:numel(patchcols)
        j = patchcols(jj);%índice de parche con algún pto de evaluación.

        % Encontrar datos dentro de la celda j (precalculado)
        idx = patch_idx{jj}; % índices de nodos de halton en parche j.
        if isempty(idx), continue; end %si no hay nodos, ignorar lo de
        % abajo.

        % Nodos de datos e interpolación locales
        local_dsites = dsites(idx, :);%coordenadas de los nodos del parche
        % j.
        N_j_loc = length(idx);

        % Matriz A_j del método local
        dist_j = DistanceMatrix(local_dsites, local_dsites);

        % Encontrar puntos de evaluación que caen en el parche j (del
        % patrón de la columna j de DM_eval. coincide con el soporte de
        % SEM).
        eidx = find(DM_eval(:, j));%da índices de columnas (parches) con
        % algún elemento distinto de cero. Por lo que hicimos antes, estas
        % son parches con algún nodo de Halton y algún punto de evaluación.
        if isempty(eidx), continue; end %si es vacía ve a siguiente parche
        % con el bucle for.
        local_epoints = epoints(eidx, :);%ptos de evaluación en el parche
        % j.

        % Matriz B (Distancias entre puntos de evaluación y nodos de datos)
        dist_eval = DistanceMatrix(local_epoints, local_dsites);%matriz
        % distancia entre nodos y puntos de evaluación.

        for ip = 1:nP
            p = lista_p{ip};%itera sobre la lista de pesos.

            % Implementación del caso gaussiano y Phi=r^p.
            if isnumeric(p)%Distingue casos de p=1,3,5 del gaussiano.
                % Tomamos ahora Phi_rho=(r/rho)^p.
                A_j = (dist_j/rho).^p; % Phi(r) = (r/rho)^p
                % Matriz P_j definida para la nueva base local de
                % polinomios.
                P_j = build_poly_basis(local_dsites, p, ...
                  cellctrs(j,:), rho);
```

```
319             else
320                 A_j = exp(-(ep * dist_j).^2); % Phi(r) = exp(-(ep*r)^2)
321                 % La Gaussiana no requiere base polinomial, matriz vacía
322                 P_j = build_poly_basis(local_dsites, 0, cellctrs(j,:), ...
323                   rho);
324             end
325
326             L = size(P_j, 2); % El número de columnas me da el número
327             % de vectores de la base, luego la dimensión del espacio de
328             % polinomios.
329
330             % ========================================================
331             % COMPROBACIONES TEÓRICAS
332             % --------------------------------------------------------
333             % Para garantizar que el sistema local sea no singular, el
334             % conjunto de nodos debe ser Pi_{m-1}^s-unisolvente.
335             % Solo aplica si hay polinomios (L > 0).
336             if L > 0
337                 % Condición 1: Debe haber al menos tantos puntos como
338                 % elementos en la base polinomial.
339                 if N_j_loc < L
340                     warning(['Parche %d: N_j (%d) < L (%d). ' ...
341                       'Sistema infradeterminado para los polinomios.'], ...
342                       j, N_j_loc, L);
343                 end
344                 % Condición 2: La matriz P_j debe tener rango completo.
345                 % Si rank(P_j) < L, significa que los puntos locales están
346                 % alineados sobre una curva algebraica de grado m-1 y
347                 % no son unisolventes. El sistema fallará.
348                 if rank(P_j) < L
349                     warning(['Parche %d: Puntos locales NO ' ...
350                       'unisolventes (rank(P_j)=%d < L=%d).'], j, ...
351                       rank(P_j), L);
352                 end
353             end
354             % ========================================================
355
356             % Ensamblaje de Matriz Local
357             % IM = [A_j, P_j ; P_j^T, 0]
358             IM = [A_j, P_j; P_j', zeros(L, L)];
359
360             % Lado derecho (f_j ampliado con ceros para las
361             % restricciones polinomiales).
362             f_j = [rhs(idx); zeros(L, 1)];
363
364             % Diagnóstico de condicionamiento del sistema local.
365             rc = rcond(full(IM));%full IM la convierte de dispersa en
366             % completa.
367             if rc < min_rcond(ip), min_rcond(ip) = rc; end %para un p fijo,
368             %si rc es menor que el anterior, elegimos este rc como el peor.
```

```
369
370             % Resolución del sistema local: coefs=(lambda,c)'
371             coefs = IM \ f_j;
372
373             % --- Evaluación del interpolante local en los puntos de
374             % evaluación del parche.
375             % Evaluación usando Phi gaussiana o Phi=r^p
376             if isnumeric(p)
377                 % Con la misma Phi normalizada que en A_j (coherencia).
378                 B = (dist_eval/rho).^p; % Phi(r) = (r/rho)^p.
379                 % Matriz Q (Base de polinomios evaluada en puntos de
380                 % evaluación)
381                 Q = build_poly_basis(local_epoints, p, cellctrs(j,:), rho);
382             else
383                 B = exp(-(ep * dist_eval).^2);
384                 Q = build_poly_basis(local_epoints, 0, cellctrs(j,:), ...
385                   rho); % Q vacía
386             end
387
388             % Ensamblaje de Matriz de Evaluación, es decir, la
389             % matriz que multiplicada por coefs nos da el
390             % aproximante local j, evaluado en cada uno de los
391             % puntos de evaluación contenidos en el parche j.
392             EM = [B, Q];
393
394             % Interpolante local
395             localfit = EM * coefs;
396
397             % Acumulación en el interpolante global usando los
398             % pesos de Shepard (SEM). El mismo localfit vale para los
399             % nW tipos de pesos (no depende del peso); solo cambia SEM{iw}.
400             for iw = 1:nW
401                 Pf{ip,iw}(eidx) = Pf{ip,iw}(eidx) + localfit .* ...
402                   SEM{iw}(eidx, j);%calcula
403                 % Aproximante global sobre puntos evaluación, teniendo
404                 % en cuenta contribuciones de ptos que caen sobre
405                 % varios parches.
406             end
407         end
408     end
409
410     % --- 6. CÁLCULO DE ERRORES ---
411     for ip = 1:nP
412         for iw = 1:nW
413             maxerr = norm(Pf{ip,iw} - exact, inf);
414             rms_err = norm(Pf{ip,iw} - exact) / neval;%porque
415             % sqrt{neval^s)=neval,
416             % para s=2
417             Res{ip,iw}(ik, :) = [N, M, h, rms_err, maxerr, min_rcond(ip)];
418         end
```

```
    end
    fprintf('  k=%d  N=%d  n=%d  M=%d  h=%.3e  hecho\n', k_list(ik), N, ...
      n, M, h);
end

warning('on', 'MATLAB:nearlySingularMatrix');
warning('on', 'MATLAB:singularMatrix');

% ========================================================================
% IMPRESIÓN DE LAS 12 TABLAS. p_obs = log(RMS_ant/RMS)/log(h_ant/h).
% ========================================================================
for ip = 1:nP
    if isnumeric(lista_p{ip}), nombre_base = sprintf('r^%d', lista_p{ip});
    else, nombre_base = 'Gaussiana'; end
    for iw = 1:nW
        D = Res{ip, iw};
        pobs = NaN(nK,1);
        for r = 2:nK, pobs(r) = log2(D(r-1,4)/D(r,4)); end
        fprintf(['\n================================================' ...
          '==================\n']);
        fprintf('TABLA: Base %s | Peso: %s\n', nombre_base, ...
          lista_weights{iw});
        fprintf(['================================================' ...
          '==================\n']);
        T = table(D(:,1), D(:,2), D(:,4), pobs, D(:,5), D(:,6), ...
            'VariableNames', {'N','M','RMS_error','orden_p', ...
            'max_error','min_RCOND'});
        disp(T);
    end
end

% ========================================================================
%SALIDA EN LATEX PARA OVERLEAF.
% Una tabla por phi, con los 3 pesos (C2,C4,C6) en bloques de (RMS,p,max).
% p_obs = log(RMS_ant/RMS)/log(h_ant/h). Copiar-pegar directamente.
% ========================================================================
fprintf('\n\n');
fprintf(['%%%%%%%%%%%%%%%%%  TABLAS EN LATEX (copiar a Overleaf) ' ...
  ' %%%%%%%%%%%%%%%%%\n\n']);
pesos_tex = {'C^2', 'C^4', 'C^6'};
for ip = 1:nP
    if isnumeric(lista_p{ip})
        base_lbl = sprintf('r%d', lista_p{ip});
        cap_phi  = sprintf('$\\Phi=r^{%d}$', lista_p{ip});
    else
        base_lbl = 'gauss';
        cap_phi  = sprintf(['la gaussiana no estacionaria ($\\' ...
          'varepsilon=%.4f$)'], ep);
    end

```

```
    % Orden observado por peso (una columna p por bloque)
    Pobs = cell(1, nW);
    for iw = 1:nW
        D = Res{ip, iw};
        pv = NaN(nK, 1);
        for r = 2:nK, pv(r) = log2(D(r-1,4)/D(r,4)); end
        Pobs{iw} = pv;
    end

    fprintf('\\begin{table}[H]\n');
    fprintf('  \\centering\n');
    fprintf('  \\scalebox{0.68}{\n');
    fprintf('  \\begin{tabular}{| c | c | cccc | cccc | cccc |}\n');
    fprintf('  \\hline\n');
    fprintf(['  & & \\multicolumn{4}{c|}{Wendland $%s$}' ...
             ' & \\multicolumn{4}{c|}{Wendland $%s$}' ...
             ' & \\multicolumn{4}{c|}{Wendland $%s$} \\\\\n'], ...
             pesos_tex{1}, pesos_tex{2}, pesos_tex{3});
    fprintf(['  $k$ & $N$ & RMS-error & $p$ & max-error & RCOND' ...
             ' & RMS-error & $p$ & max-error & RCOND' ...
             ' & RMS-error & $p$ & max-error & RCOND \\\\\n']);
    fprintf('  \\hline\n');
    for ik = 1:nK
        fprintf('  %d & %d', k_list(ik), Res{ip,1}(ik,1));
        for iw = 1:nW
            D = Res{ip, iw};
            if ik == 1
                fprintf(' & %.4e & & %.4e & %.2e', D(ik,4), D(ik,5), ...
                  D(ik,6));
            else
                fprintf(' & %.4e & %.4f & %.4e & %.2e', D(ik,4), ...
                  Pobs{iw}(ik), D(ik,5), D(ik,6));
            end
        end
        fprintf(' \\\\\n');
    end
    fprintf('  \\hline\n');
    fprintf('  \\end{tabular}}\n');
    fprintf(['  \\caption{RMSE y error m\\''aximo frente al ' ...
      'n\\''umero de nodos de ' ...
             'Halton $N$ para el MPU con %s y pesos de Wendland $C^2$,' ...
             ' $C^4$ y $C^6$, ' ...
             'sobre la funci\\''on test.}\n'], cap_phi);
    fprintf('  \\label{tab:mpu_test_%s}\n', base_lbl);
    fprintf('\\end{table}\n\n');
end

% ========================================================================
% RBF GLOBAL (comparativa): Gaussiano con eps fijo y r^p, p=1,3,5.
% Siguiendo el codigo 10 del anexo: interpolacion RBF global directa, sin
```

```
% aumento polinomico (para r^p se resuelve DM.^p \ rhs, como la matriz
% distancia con r^1). Barrido propio hasta N_max asequible (sistema denso).
% El eps de la gaussiana se fija como el menor con rcond >= rcond_target en
% N_max (rcond crece con eps y N_max es el peor caso: asi rcond > umbral en
% todo el barrido). p_obs = log2(RMS_ant/RMS).
% ========================================================================
k_rbf     = 1:7;          % N=(2^k+1)^2 => N=9..16641 (N_max asequible)
nK_rbf    = numel(k_rbf);
lista_rbf = {'Gauss', 1, 3, 5};
nP_rbf    = numel(lista_rbf);

% eps FIJO de la gaussiana: menor eps con rcond >= rcond_target en N_max.
N_mx_rbf  = (2^max(k_rbf) + 1)^s;
nd_mx_rbf = haltonseq(N_mx_rbf, s);
DM_mx_rbf = DistanceMatrix(nd_mx_rbf, nd_mx_rbf);
grc = @(e) rcond(exp(-(e*DM_mx_rbf).^2));
e_lo = ep_native; e_hi = 2*ep_native;
while grc(e_hi) < rcond_target, e_hi = 2*e_hi; if e_hi > 1e4, break;
end, end
for it = 1:50
    em = 0.5*(e_lo + e_hi);
    if grc(em) >= rcond_target, e_hi = em; else, e_lo = em; end
end
eps_fijo = e_hi;
fprintf(['\neps_fijo (RBF global gaussiano) = %.4f  (rcond en' ...
  ' N_max=%d: %.2e)\n'], ...
  eps_fijo, N_mx_rbf, grc(eps_fijo));

% Calculo y almacenamiento: Res_rbf{ip} = [k, N, RMS, MAX, RCOND]
Res_rbf = cell(1, nP_rbf);
for ip = 1:nP_rbf
    phi_rbf = lista_rbf{ip};
    Drbf = zeros(nK_rbf, 5);
    for ik = 1:nK_rbf
        N = (2^k_rbf(ik) + 1)^s;
        nodes = haltonseq(N, s);
        rhs_rbf = testfunction(nodes(:,1), nodes(:,2));
        DM_rbf  = DistanceMatrix(nodes, nodes);
        DM_ev   = DistanceMatrix(epoints, nodes);
        if isnumeric(phi_rbf)
            IM_rbf = DM_rbf.^phi_rbf;
            Pf_rbf = (DM_ev.^phi_rbf) * (IM_rbf \ rhs_rbf);
        else
            IM_rbf = exp(-(eps_fijo*DM_rbf).^2);
            Pf_rbf = exp(-(eps_fijo*DM_ev).^2) * (IM_rbf \ rhs_rbf);
        end
        rms_rbf = norm(Pf_rbf - exact) / neval;
        max_rbf = norm(Pf_rbf - exact, inf);
        Drbf(ik, :) = [k_rbf(ik), N, rms_rbf, max_rbf, rcond(IM_rbf)];
    end
```

```
    Res_rbf{ip} = Drbf;
end

% Tablas de texto (consola)
for ip = 1:nP_rbf
    if isnumeric(lista_rbf{ip}), nombre_rbf = sprintf('r^%d', ...
        lista_rbf{ip});
    else, nombre_rbf = 'Gaussiana'; end
    D = Res_rbf{ip};
    pobs = NaN(nK_rbf, 1);
    for rr = 2:nK_rbf, pobs(rr) = log2(D(rr-1,3)/D(rr,3)); end
    fprintf(['\n================================================' ...
      '==================\n']);
    fprintf('TABLA: RBF GLOBAL | Base %s\n', nombre_rbf);
    fprintf(['================================================' ...
      '==================\n']);
    T = table(D(:,1), D(:,2), D(:,3), pobs, D(:,4), D(:,5), ...
        'VariableNames', {'k','N','RMS_error','orden_p','max_error' ...
        ,'RCOND'});
    disp(T);
end

% Tablas en LaTeX (copiar a Overleaf), una por Phi, con columna RCOND
fprintf('\n\n');
fprintf(['%%%%%%%%%%%%%%%%  TABLAS RBF GLOBAL EN LATEX (copiar' ...
  ' a Overleaf)  %%%%%%%%%%%%%%%%\n\n']);
for ip = 1:nP_rbf
    if isnumeric(lista_rbf{ip})
        base_lbl = sprintf('r%d', lista_rbf{ip});
        cap_phi  = sprintf('$\\Phi=r^{%d}$', lista_rbf{ip});
    else
        base_lbl = 'gauss';
        cap_phi  = sprintf('la gaussiana ($\\varepsilon=%.4f$)', eps_fijo);
    end
    D = Res_rbf{ip};
    pv = NaN(nK_rbf, 1);
    for r = 2:nK_rbf, pv(r) = log2(D(r-1,3)/D(r,3)); end
    fprintf('\\begin{table}[H]\n');
    fprintf('  \\centering\n');
    fprintf('  \\scalebox{0.9}{\n');
    fprintf('  \\begin{tabular}{| c | c | c c c c |}\n');
    fprintf('  \\hline\n');
    fprintf('  $k$ & $N$ & RMS-error & $p$ & max-error & RCOND \\\\\n');
    fprintf('  \\hline\n');
    for ik = 1:nK_rbf
        if ik == 1
            fprintf('  %d & %d & %.4e & & %.4e & %.2e \\\\\n', ...
              D(ik,1), D(ik,2), D(ik,3), D(ik,4), D(ik,5));
        else
            fprintf('  %d & %d & %.4e & %.4f & %.4e & %.2e \\\\\n', ...
```

```
                D(ik,1), D(ik,2), D(ik,3), pv(ik), D(ik,4), D(ik,5));
        end
    end
    fprintf('  \\hline\n');
    fprintf('  \\end{tabular}}\n');
    fprintf(['  \\caption{RMSE y error m\\''aximo frente al n' ...
      '\\''umero de nodos de ' ...
             'Halton $N$ para el RBF global con %s, sobre la func' ...
             'i\\''on test.}\n'], cap_phi);
    fprintf('  \\label{tab:rbf_test_%s}\n', base_lbl);
    fprintf('\\end{table}\n\n');
end

% ========================================================================
% ESTUDIO DE TIEMPOS (tic/toc): MPU (Wendland C6) vs RBF global, por Phi.
% Se cronometra un solve completo por (N, Phi) [geometria + sistemas +
% evaluacion], que es lo que marca el coste real. Se toma la mediana de
% varias repeticiones. El numero de repeticiones no se fija por N (los
% tiempos de MPU y RBF no son comparables entre sí): cada caso se repite
% hasta acumular un presupuesto t_budget o alcanzar max_rep, de modo que un
%solve rápido (MPU) se promedia sobre muchas repeticiones y uno lento (RBF)
%sobre pocas (a menudo 1), automaticamente. La primera muestra de cada caso
%es de calentamiento(warm-up) y se descarta.
%Nota: dentro de cada metodo los tiempos de las 4 Phi salen casi iguales(el
% MPU domina la geometria, común a las 4; el RBF domina el solve denso NxN,
%común). Las 4 columnas sirven de comprobacion de esa independencia de Phi.
%Además, hay más filas de MPU que de RBF: el MPU barre todo k_list mientras
% que el RBF solo se cronometra hasta N_rbf_cap (sistema denso asequible).
% ========================================================================
k_tim     = k_list;                  % mismo barrido en k que el MPU
nK_tim    = numel(k_tim);
Nvec_tim  = (2.^k_tim + 1).^s;       % N por fila
N_rbf_cap = 16641;                   % RBF solo hasta N asequible(denso NxN)
t_budget  = 0.5;                    % presupuesto de tiempo por caso [s]
max_rep   = 15;              % tope de repeticiones (N pequeños y rápidos)
Tmpu = NaN(nK_tim, nP);          % tiempos MPU C6 [s], una columna por Phi
Trbf = NaN(nK_tim, nP);      % tiempos RBF global [s], una columna por Phi
fprintf('\nCronometrando MPU (C6) y RBF global...\n');
for ik = 1:nK_tim
    N = Nvec_tim(ik);
    for ip = 1:nP
        p = lista_p{ip};
        % ---------------- MPU (Wendland C6), Phi = p ----------------
        smp = zeros(1, max_rep); ns = 0; tacc = 0; warm = true;
        while true %bucle se repite infinitamente hasta un break que hay
          % más abajo.
            tt = tic;
            n = max(round((N/N_j)^(1/s)), recinfo.n0); M = n^s; rho = 1/n;
            dsites = haltonseq(N, s);
            rhs = testfunction(dsites(:,1), dsites(:,2));
```

```
            grid_pu = linspace(0, 1, n); [xpu, ypu] = meshgrid(grid_pu);
            cellctrs = [xpu(:), ypu(:)];
            datatree = createns(dsites, 'NSMethod', 'kdtree');
            DM_eval = DistanceMatrixCSRBF(epoints, cellctrs, n);
            SEM_u = DM_eval.^8 .* (-32*DM_eval.^3 + 121*DM_eval.^2 ...
              - 154*DM_eval + 66*spones(DM_eval));
            S_inv = 1 ./ (SEM_u * ones(M, 1));
            SEM = spdiags(S_inv, 0, neval^2, neval^2) * SEM_u;
            Pf = zeros(neval^2, 1);
            patchcols = find(any(DM_eval, 1));
            for jj = 1:numel(patchcols)
                j = patchcols(jj);
                ic = rangesearch(datatree, cellctrs(j,:), rho);
                idx = ic{1}'; if isempty(idx), continue; end
                local_dsites = dsites(idx, :);
                dist_j = DistanceMatrix(local_dsites, local_dsites);
                eidx = find(DM_eval(:, j)); if isempty(eidx), continue; end
                local_epoints = epoints(eidx, :);
                dist_eval = DistanceMatrix(local_epoints, local_dsites);
                if isnumeric(p)
                    A_j = (dist_j/rho).^p;
                    P_j = build_poly_basis(local_dsites, p, ...
                      cellctrs(j,:), rho);
                    B = (dist_eval/rho).^p;
                    Q = build_poly_basis(local_epoints, p, ...
                      cellctrs(j,:), rho);
                else
                    A_j = exp(-(ep*dist_j).^2);
                    P_j = build_poly_basis(local_dsites, 0, ...
                      cellctrs(j,:), rho);
                    B = exp(-(ep*dist_eval).^2);
                    Q = build_poly_basis(local_epoints, 0, ...
                      cellctrs(j,:), rho);
                end
                L = size(P_j, 2);
                coefs = [A_j, P_j; P_j', zeros(L, L)] \ [rhs(idx);
                  zeros(L, 1)];
                localfit = [B, Q] * coefs;
                Pf(eidx) = Pf(eidx) + localfit .* SEM(eidx, j);
            end
            e = toc(tt);
            if warm
                warm = false;              % descarta la 1a (calentamiento)
            else
                ns = ns + 1; smp(ns) = e; tacc = tacc + e;
                if tacc >= t_budget || ns >= max_rep, break; end
            end
        end
        Tmpu(ik, ip) = median(smp(1:ns));
        % ---------------- RBF global, Phi = p ----------------
```

```
719         if N <= N_rbf_cap
720             smp = zeros(1, max_rep); ns = 0; tacc = 0; warm = true;
721             while true
722                 tt = tic;
723                 nodes = haltonseq(N, s);
724                 rhs_r = testfunction(nodes(:,1), nodes(:,2));
725                 DMd = DistanceMatrix(nodes, nodes);
726                 DMe = DistanceMatrix(epoints, nodes);
727                 if isnumeric(p)
728                     Pfr = (DMe.^p) * (DMd.^p \ rhs_r);
729                 else
730                     Pfr = exp(-(eps_fijo*DMe).^2) * (exp(-(eps_fijo* ...
731                       DMd).^2) \ rhs_r);
732                 end
733                 e = toc(tt);
734                 if warm
735                     warm = false;
736                 else
737                     ns = ns + 1; smp(ns) = e; tacc = tacc + e;
738                     if tacc >= t_budget || ns >= max_rep, break; end
739                 end
740             end
741             Trbf(ik, ip) = median(smp(1:ns));
742         end
743     end
744 end
745
746 % --- cocientes normalizados: MPU por N*log(N); RBF por N^3 ---
747 Rmpu = NaN(nK_tim, nP); Rrbf = NaN(nK_tim, nP);
748 for ik = 2:nK_tim
749     fN  = (Nvec_tim(ik)*log(Nvec_tim(ik))) / (Nvec_tim(ik-1)*log ...
750       (Nvec_tim(ik-1)));
751     fN3 = Nvec_tim(ik)^3 / Nvec_tim(ik-1)^3;
752     for ip = 1:nP
753         Rmpu(ik,ip) = (Tmpu(ik,ip)/Tmpu(ik-1,ip)) / fN;
754         Rrbf(ik,ip) = (Trbf(ik,ip)/Trbf(ik-1,ip)) / fN3;
755     end
756 end
757
758 % --- TABLA 1 (LaTeX): tiempos [s] ---
759 fprintf('\n\n');
760 fprintf(['%%%%%%%%%%%%%%%%%  TABLAS DE TIEMPOS EN LATEX (copiar a' ...
761   ' Overleaf)  %%%%%%%%%%%%%%%%%\n\n']);
762 fprintf('\\begin{table}[H]\n  \\centering\n  \\scalebox{0.8}{\n');
763 fprintf('  \\begin{tabular}{| c | c | cccc | cccc |}\n  \\hline\n');
764 fprintf(['  & & \\multicolumn{4}{c|}{MPU ($C^6$) [s]}' ...
765          ' & \\multicolumn{4}{c|}{RBF global [s]} \\\\\n']);
766 fprintf(['  $k$ & $N$ & $r^1$ & $r^3$ & $r^5$ & Gauss' ...
767          ' & $r^1$ & $r^3$ & $r^5$ & Gauss \\\\\n']);
768 fprintf('  \\hline\n');
```

```
for ik = 1:nK_tim
    rowstr = sprintf('  %d & %d', k_tim(ik), Nvec_tim(ik));
    for ip = 1:nP, rowstr = [rowstr, sprintf(' & %.3e', Tmpu(ik,ip))]; end
    for ip = 1:nP
        if isnan(Trbf(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.3e', Trbf(ik,ip))]; end
    end
    fprintf('%s \\\\\n', rowstr);
end
fprintf('  \\hline\n  \\end{tabular}}\n');
fprintf(['  \\caption{Tiempos de computo (tic/toc) del MPU (Wendland' ...
  ' $C^6$) y' ...
         ' del RBF global, por $\\Phi$, sobre la funcion test.}\n']);
fprintf('  \\label{tab:tiempos}\n\\end{table}\n\n');

% --- TABLA 2 (LaTeX): cocientes normalizados (deberian tender a ~1) ---
fprintf('\\begin{table}[H]\n  \\centering\n  \\scalebox{0.8}{\n');
fprintf('  \\begin{tabular}{| c | c | cccc | cccc |}\n  \\hline\n');
fprintf(['  & & \\multicolumn{4}{c|}{MPU: $(t_{k}/t_{k-1})/' ...
  '(N\\log N)$-ratio}' ...
         ' & \\multicolumn{4}{c|}{RBF: $(t_{k}/t_{k-1})/N^3$-ratio}' ...
         ' \\\\\n']);
fprintf(['  $k$ & $N$ & $r^1$ & $r^3$ & $r^5$ & Gauss' ...
         ' & $r^1$ & $r^3$ & $r^5$ & Gauss \\\\\n']);
fprintf('  \\hline\n');
for ik = 1:nK_tim
    rowstr = sprintf('  %d & %d', k_tim(ik), Nvec_tim(ik));
    for ip = 1:nP
        if isnan(Rmpu(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.4f', Rmpu(ik,ip))]; end
    end
    for ip = 1:nP
        if isnan(Rrbf(ik,ip)), rowstr = [rowstr, ' & '];
        else, rowstr = [rowstr, sprintf(' & %.4f', Rrbf(ik,ip))]; end
    end
    fprintf('%s \\\\\n', rowstr);
end
fprintf('  \\hline\n  \\end{tabular}}\n');
fprintf(['  \\caption{Cocientes de tiempos normalizados: MPU por' ...
  ' $N\\log N$ y' ...
         ' RBF global por $N^3$. Valores $\\approx 1$ confirman el' ...
         ' orden esperado.}\n']);
fprintf('  \\label{tab:cocientes}\n\\end{table}\n\n');
```

Código 16: Script del experimento de RBF global y MPU sobre la función test polinómica.